

TeacherSupporter

Trọn bộ công nghệ

Hướng dẫn tự học

Từ Spring Boot đến Kubernetes và CI/CD —
cẩm nang trọn vẹn, dạy từ một dự án thật

Vuong Dang

viết cùng Claude

Bản tiếng Việt — tháng 9 năm 2026

Cách đọc cuốn sách này

Cuốn sách này dạy mọi công nghệ có trong dự án TeacherSupporter — Spring Boot và Spring Cloud, Postgres và MongoDB, Kafka, Docker, Kubernetes, và pipeline Jenkins gắn kết tất cả lại với nhau — bằng chính các file của dự án làm ví dụ. Không có gì được bịa ra cho trang sách: mọi cấu hình bạn sẽ đọc đều đang được triển khai, hoặc sắp được triển khai, trên một máy chủ gia đình thật hai nhân tên là ts-server.

Những quy tắc cuốn sách tuân theo

Khái niệm trước, cấu hình sau. Mỗi chương trước hết dạy ý tưởng bằng ngôn ngữ trung lập, không gắn với nhà cung cấp nào, rồi đọc file thật của dự án từng dòng một, rồi đưa một hai ví dụ từ *bên ngoài* dự án để bạn phân biệt được cái gì là quy luật chung, cái gì chỉ là thói quen cục bộ.

Quyết định là nội dung chính. Ở bất cứ đâu dự án phải chọn giữa các phương án (Jib thay vì docker build, Kustomize thay vì Helm, một server Postgres thứ hai thay vì một cơ sở dữ liệu thứ hai), lý do nằm trong hộp DECISION. Đó chính là những câu hỏi “tại sao bạn lại...” mà các buổi phỏng vấn được dựng nên từ đó.

Lỗi được trình bày từ triệu chứng. Các hộp PITFALL bắt đầu bằng thứ bạn thực sự nhìn thấy (CrashLoopBackOff, một lỗi TLS, một 404 lạng lẽ) rồi lần ngược về nguyên nhân, vì đó là hướng mà việc gỡ lỗi thật diễn ra.

Đào sâu đầy một thiết kế tới khi gãy. Phần VIII quay lại mỗi chương một công nghệ và hỏi những câu các chương chính còn để dành: vì sao chọn nó, nó giải quyết gì, tốn gì, cái gì gãy lúc 3 giờ sáng, và bản nâng cấp cụ thể trông ra sao — với code trọn vẹn, không phải phác thảo. Đọc sau chương chính tương ứng, hoặc trước một buổi phỏng vấn.

Chú giải các hộp

QUYẾT ĐỊNH — VÍ DỤ

Vì sao dự án chọn thiết kế này thay vì thiết kế kia, và điều gì phải thay đổi để lựa chọn còn lại thắng thế.

CẠM BẦY — VÍ DỤ

Một triệu chứng sớm muộn bạn cũng sẽ gặp, kèm nguyên nhân và cách sửa.

THỰC HÀNH — VÍ DỤ

Một lệnh để chạy trên stack thật, cùng kết quả bạn nên mong đợi. Cuốn sách này được viết để đọc bên cạnh một terminal.

TRÊN PHẦN CỨNG NÀY — VÍ DỤ

Một lựa chọn chỉ đúng trên chiếc máy này (2 nhân, 19.4 GB RAM, SSD SATA, Wi-Fi). Đừng bê những thứ này vào trung tâm dữ liệu.

TỰ KIỂM TRA (đáp án: Phụ lục A)

Mỗi chương kết thúc bằng vài câu hỏi trong hộp này. Đáp án ở Phụ lục A. Hãy viết câu trả lời ra giấy trước khi xem đáp án — nhận ra không phải là nhớ được.

Ký hiệu

Code inline và tên file trông như thế này: `application.yml`. Các listing có chú thích mang những dấu khoanh tròn — ①, ② — được giải thích ngay bên dưới listing. Lệnh shell được hiển thị với dấu nhắc `$` cho server và `PS>` cho Windows PowerShell; đừng gõ dấu nhắc.

Điều gì được giả định, điều gì không

Giả định: bạn đọc được Java, và đã dùng Maven và git. Không giả định: bất kỳ kiến thức nào trước đó về Kubernetes, Jenkins, Kafka, bên trong Docker, hay

frontend. Ở những chỗ tài liệu của dự án đã trình bày việc cài đặt máy một lần (`docs/RUNBOOK-SERVER-SETUP.md`), cuốn sách chỉ tóm tắt rồi đi tiếp.

Bản đồ dự án

TeacherSupporter là một nền tảng quản lý khóa học được xây dựng dưới dạng các microservice Spring Cloud: giáo viên quản lý khóa học, học sinh, bài tập và bài nộp; một dictionary service phục vụ tra cứu từ vựng; đăng ký và đăng nhập (mật khẩu, Google OAuth2, và xác thực hai lớp TOTP) nằm trong một auth service riêng; thông báo e-mail chảy qua Kafka. Dự án cổ tình dùng stack Spring Cloud “cổ điển” — Eureka, Config Server, Spring Cloud Gateway — vì stack đó vẫn đang chạy ở rất nhiều doanh nghiệp, và hiểu nó khiến các lựa chọn thay thế cloud-native (Chương 27) trở nên có ý nghĩa thay vì huyền bí.

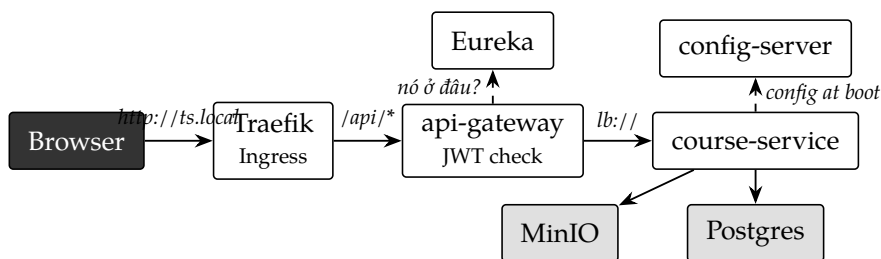
Dàn nhân vật

Service	Cổng	Kho lưu trữ	Nhiệm vụ
config-server	8888	— (classpath)	phục vụ cấu hình cho mọi service
discovery-server	8761	—	registry Eureka
api-gateway	8080	—	điểm vào duy nhất, kiểm tra JWT
auth-service	8081	Postgres ts_auth	tài khoản, JWT, OAuth2, 2FA
course-service	8082	Postgres ts_course, MinIO	khóa học, học sinh, file
dictionary-service	8083	MongoDB	tra cứu từ vựng
notification-service	8084	— (Kafka vào)	sự kiện → e-mail
frontend	3000	—	giao diện web (React)

Bảng 1: Tám đơn vị triển khai và những gì mỗi cái sở hữu.

Một request, từ đầu đến cuối

Mỗi chương soi sáng một mắt xích của chuỗi này. Phần I là các hộp bên phải; Phần II là các kho dữ liệu; Phần III là đường Kafka (không vẽ — nó chờ sự kiện, không chờ request); các Phần IV–VI là cách tất cả những thứ đó được đóng gói, chạy, và giao đi.



Hình 1: Đường đi của request mà cuốn sách này sẽ quay lại nhiều lần.

Mọi thứ nằm ở đâu trong repository

TeacherSupporter/	
pom.xml	parent: modules, shared plugin config (Jib!)
common/	shared DTOs and events
config-server/	+ src/main/resources/configurations/*.yaml
discovery-server/	api-gateway/ auth-service/
course-service/	dictionary-service/ notification-service/
frontend/	
docker-compose.yml	local dev stack
Jenkinsfile	the CI/CD pipeline (Chapter 23)
deploy/	
jenkins/	Jenkins controller image + run script
k8s/	
create-secrets.sh	secrets are made, not committed
jenkins-rbac.yaml	what Jenkins may touch in the cluster
base/	every Kubernetes manifest (Part V)
docs/	design docs, runbooks, and this book

Cỗ máy bên dưới

Mọi thứ được triển khai lên ts-server: một chiếc laptop tận dụng lại chạy Ubuntu 24.04 với 2 nhân vật lý, 19.4GB RAM và một SSD SATA, truy cập được từ Windows qua Tailscale. Nó chạy k3s (Kubernetes một node), một Jenkins chạy trong Docker, và một image registry cục bộ. Ràng buộc định hình gần như mọi quyết định vận hành trong cuốn sách này là *CPU*, chứ không phải bộ nhớ: hai nhân nghĩa là build và demo không được chồng lên nhau.

Mục lục

Cách đọc cuốn sách này	ii
Bản đồ dự án	v
I Ứng dụng	1
1 Spring Boot: Cỗ máy bên dưới mọi service	2
1.1 Vấn đề Spring Boot giải quyết	2
1.1.1 Starter	2
1.1.2 Auto-configuration	3
1.2 Cấu hình ngoài: thứ tự phân giải thuộc tính	3
1.2.1 Ràng buộc nới lỏng (relaxed binding)	4
1.2.2 Placeholder với giá trị mặc định	4
1.3 File của dự án, có chú thích	4
1.4 Profile	5
1.5 Actuator	5
1.6 Bên ngoài dự án	5
2 Cấu hình tập trung: Config Server	7
2.1 Vấn đề	7
2.2 Spring Cloud Config hoạt động thế nào	7
2.3 Server của dự án, có chú thích	7
2.4 Thứ tự khởi động và điều nhảy bootstrap	9
2.5 Làm mới cấu hình mà không khởi động lại	9
2.6 Bên ngoài dự án	9
3 Khám phá dịch vụ: Eureka	11
3.1 Vấn đề	11
3.2 Eureka hoạt động thế nào	11
3.3 Server, và thiết lập sống còn duy nhất trên k8s	11
3.4 Bên ngoài dự án	12

4	API Gateway	14
4.1	Vấn đề	14
4.2	Mô hình của Spring Cloud Gateway	14
4.3	Các route của dự án, có chú giải	14
4.4	Hai bộ định tuyến: gateway và Ingress	16
4.5	Những góc nâng cao	16
4.6	Ngoài dự án	16
5	Bảo mật: JWT, OAuth2 và biên tin cậy	18
5.1	Spring Security trong một trang	18
5.2	JWT: danh tính phi trạng thái	19
5.3	Mẫu biên tin cậy	19
5.4	Đăng nhập OAuth2 với Google	20
5.5	Xác thực hai lớp TOTP	20
5.6	Ngoài dự án	21
6	Các service nghiệp vụ và module common	22
6.1	course-service: con ngựa thồ	22
6.2	auth-service: danh tính	22
6.3	dictionary-service: anh nhỏ	22
6.4	notification-service: consumer thuần túy	22
6.5	Đồng bộ và bất đồng bộ, trong codebase này	23
6.6	Module common	23
II	Dữ liệu	25
7	Dữ liệu quan hệ: Postgres, JPA, Flyway	26
7.1	Mỗi service một cơ sở dữ liệu	26
7.2	JPA và Hibernate	26
7.3	Vì sao ddl-auto: none	27
7.4	Flyway: migration dưới dạng code	27
7.5	Connection pool	28
7.6	Ngoài dự án	28
8	Dữ liệu dạng document: MongoDB	30
8.1	Mô hình	30
8.2	Spring Data MongoDB	30
8.3	Cấu hình — toàn bộ	31
8.4	Không schema là con dao hai lưỡi	31
8.5	Vận hành trong cụm này	32

8.6	Ngoài phạm vi dự án	32
9	Lưu trữ đối tượng: MinIO và mô hình S3	33
9.1	Bài toán	33
9.2	Cấu hình, có chú giải	33
9.3	Presigned URL: mẫu đáng để biết	34
9.4	Ngoài phạm vi dự án	34
III	Truyền thông điệp	36
10	Kafka: Mô hình lõi	37
10.1	Một log, không phải một hàng đợi	37
10.2	Consumer group	38
10.3	Broker, replication, KRaft	38
10.4	Mê cung advertised listener	38
10.5	Ngữ nghĩa chuyển giao	39
10.6	Ngoài phạm vi dự án	39
11	Spring Kafka: Producer, Listener, Poison Pill	41
11.1	Phát sự kiện	41
11.2	Tiêu thụ	41
11.3	Poison pill	42
11.4	Những góc nâng cao	43
IV	Container	44
12	Docker: Image, Container, Compose	45
12.1	Container thực chất là gì	45
12.2	Dockerfile của dự án, có chú giải	45
12.3	Mạng và Compose	46
13	Image không cần daemon: Jib và registry cục bộ	48
13.1	Vì sao không cứ docker build trong CI?	48
13.2	Cấu hình, có chú giải	48
13.3	Registry, và vì sao lại là localhost:5000	49
13.4	Tag, digest, và cái bẫy SNAPSHOT	50

V	Kubernetes	51
14	Kubernetes: Giải phẫu cụm và vòng lặp điều hòa	52
14.1	Ý tưởng duy nhất mà mọi thứ khác treo lên	52
14.2	Các bộ phận chuyển động	52
14.3	Object, label, namespace	53
14.4	Đưa kubecttl tới cụm	53
15	Workload: Pod, Deployment, StatefulSet	55
15.1	Pod	55
15.2	Deployment và rollout	55
15.3	StatefulSet: khi danh tính quan trọng	56
15.4	Các loại workload khác	57
16	Cấu hình và Secret trong cụm	58
16.1	Hai cơ chế tiêm cấu hình	58
16.2	Secret: là gì và không là gì	58
16.3	create-secrets.sh, có chú giải	58
16.4	ConfigMap	59
16.5	Ngoài phạm vi dự án	60
17	Probe sức khỏe và quản lý tài nguyên	61
17.1	Ba loại probe	61
17.2	Requests, limits và JVM	63
18	Mạng: Service, DNS, Ingress	65
18.1	Vấn đề mà pod tạo ra	65
18.2	Ingress và Traefik	65
18.3	Đường đi trọn vẹn của một request	66
18.4	Ngoài phạm vi dự án	67
19	Lưu trữ: Volume, Claim và trạng thái trên một node	68
19.1	Chồng trừu tượng	68
19.2	local-path trên k3s	68
19.3	Hai kiểu claim trong manifest	69
19.4	Ngoài phạm vi dự án	70
20	Kustomize: Manifest theo bộ	71
20.1	Vấn đề	71
20.2	Kustomization, có chú giải	71
20.3	Base và overlay	72

VI CI/CD	75
21 CI/CD: Các khái niệm	76
21.1 Hai thực hành, một pipeline	76
21.2 Từ vựng	76
22 Jenkins: CI server	79
22.1 Kiến trúc	79
22.2 Image của controller, có chú giải	79
22.3 Credential	80
22.4 Danh tính triển khai: một ServiceAccount, không phải admin	80
23 Pipeline, từng dòng một	83
23.1 Phần mở đầu và environment	83
23.2 Ba stage	84
23.3 Xử lý thất bại	86
23.4 Cùng pipeline đó, ở nơi khác	86
VII Vận hành và chặng đường phía trước	88
24 Vận hành stack: Cẩm nang gỡ lỗi thực địa	89
24.1 Bốn câu hỏi	89
24.2 Bảng phân loại trạng thái	89
24.3 Đọc thể giới của một service từ bên trong	90
24.4 Thói quen giữ cho cuối tuần được rảnh	91
25 Kafka trong cụm: Strimzi và luồng gửi mail	93
25.1 Operator và custom resource	93
25.2 Cài operator: một việc của admin	93
25.3 Cụm, được khai báo	94
25.4 Topic dưới dạng object	95
25.5 Service cuối cùng, và hộp thư của nó	96
25.6 Ngoài dự án	98
26 Frontend và một host duy nhất	99
26.1 Frontend là gì, dưới mắt người vận hành	99
26.2 Hai giai đoạn, một Dockerfile	99
26.3 nginx: fallback giúp SPA hoạt động	100
26.4 Giành lấy /: một origin	101
26.5 Manifest	102

26.6	Pipeline mọc thêm một stage	103
26.7	Kiểm chứng cột mốc	104
26.8	Truy cập từ bất cứ đâu: tên trên tailnet	105
26.9	Ngoài dự án	106
27	Chặng đường phía trước	107
27.1	Khả năng quan sát	107
27.2	GitOps: Argo CD	107
27.3	Cho stack cổ điển về hưu	108
27.4	Jenkins vào cùm	108
27.5	OpenShift, nói ngắn	108
27.6	Bạn đang đứng ở đâu	109
VIII	Đào sâu	110
28	Đào sâu: JWT ở quy mô lớn — xác minh ở đâu, token béo phì và khóa bất đối xứng	111
28.1	Xác minh ở đâu: gateway hay từng service?	111
28.1.1	Mô hình 1: xác minh tập trung (điều dự án đang làm)	111
28.1.2	Ngụy biện “mạng nội bộ đáng tin”	112
28.1.3	Mô hình 2: zero trust — service nào cũng xác minh	114
28.2	Chữ ký bảo đảm điều gì	114
28.3	HS256 so với RS256	115
28.3.1	HS256: một khóa, hai vai	115
28.3.2	RS256: hai khóa, mỗi khóa một vai	115
28.3.3	Chi phí	116
28.3.4	Chuyển đổi dự án	117
28.3.5	Xoay khóa, và kid để làm gì	119
28.4	Token béo phì (payload bloat)	119
28.5	Mẫu phantom token	120
28.6	Chuyện gì xảy ra lúc 3 giờ sáng	121
28.7	Khi JWT là công cụ sai	122
28.8	Ngoài dự án	122
28.9	Câu hỏi từ phỏng vấn thật	122
29	Đào sâu: Cấu hình — Config Server, Secret và refresh ở quy mô lớn	125
29.1	Vì sao tập trung, và nó giải quyết gì	125
29.2	Đọc đường đi thật	126
29.2.1	Ưu tiên, chính xác	127

29.3	Ưu và nhược, thành thật	129
29.4	Cái gì gây	129
29.5	Chuyện gì gây lúc 3 giờ sáng	131
29.6	Nâng cấp giải pháp	131
29.6.1	Nâng cấp 1: backend git với giá trị mã hóa	132
29.6.2	Nâng cấp 2: cho server nghỉ hưu	133
29.7	Ngoài dự án	134
29.8	Câu hỏi từ phỏng vấn thật	135
30	Đào sâu: Khám phá dịch vụ — bên trong Eureka, các kiểu lỗi và phương án Kubernetes	138
30.1	Vì sao cần registry, và vì sao ở client	138
30.2	Đọc cấu hình thật	139
30.2.1	Các bộ đếm thời gian	140
30.3	Ưu và nhược, thành thật	142
30.4	Cái gì gây	142
30.5	Chuyện gì gây lúc 3 giờ sáng	143
30.6	Nâng cấp giải pháp	144
30.6.1	Bước 1: tài liệu profile cho gateway	144
30.6.2	Bước 2: Feign không cần registry	145
30.6.3	Bước 3: các manifest	146
30.7	Ngoài dự án	147
30.8	Câu hỏi từ phỏng vấn thật	147
31	Đào sâu: Gateway dưới tải — bên trong stack reactive, rate limit và việc của biên	150
31.1	Vì sao cần gateway, và nó giải quyết gì	150
31.2	Stack reactive, đọc từ filter	151
31.2.1	Thread: bốn, không phải bốn trăm	152
31.2.2	Chặn làm gì một event loop	152
31.2.3	Predicate, thứ tự, và phép tính StripPrefix	153
31.3	CORS ở biên	153
31.4	Việc của biên, và việc không phải	154
31.5	Ưu và nhược, thẳng thắn	155
31.6	Cái gì gây	155
31.7	Chuyện gì gây lúc 3 giờ sáng	156
31.8	Nâng cấp giải pháp	156
31.9	Ngoài dự án	160
31.10	Câu hỏi từ phỏng vấn thật	160

32 Đào sâu: Gọi đồng bộ — OpenFeign, Resilience4j và lỗi dây chuyền	163
32.1 Vì sao Feign, và nó giải quyết gì	163
32.2 Hợp đồng trong common, và rủi ro phiên bản của nó	164
32.3 Circuit breaker, với các con số của dự án	165
32.4 Timeout xếp chồng	166
32.5 Retry: chỉ cho lời gọi lặp lại được	167
32.6 Bulkhead: giới hạn vùng nổ	167
32.7 Fallback trả về gì	167
32.8 Ưu và nhược, thẳng thắn	168
32.9 Cái gì gãy	168
32.10 Chuyện gì gãy lúc 3 giờ sáng	169
32.11 Nâng cấp giải pháp	170
32.12 Ngoài dự án	174
32.13 Câu hỏi từ phòng vấn thật	175
33 Đào sâu: Dữ liệu xuyên service — mỗi service một cơ sở dữ liệu, dữ liệu trùng lặp và outbox	178
33.1 Vì sao chọn nó, và nó giải quyết gì	178
33.2 Một học viên tới course-service bằng cách nào	179
33.2.1 Thứ hoàn toàn không được lan truyền	181
33.3 Ưu và nhược, thẳng thắn	181
33.4 Bài toán ghi kép	181
33.5 Nâng cấp giải pháp: transactional outbox	182
33.5.1 Change data capture: relay cấp công nghiệp	186
33.6 Saga: khi hai service đều phải thay đổi	186
33.7 Đọc xuyên service	187
33.8 Chuyện gì gãy lúc 3 giờ sáng	187
33.9 Ngoài dự án	188
33.10 Câu hỏi từ phòng vấn thật	189
34 Đào sâu: Bảo đảm giao nhận Kafka — ít nhất một lần, consumer lữ đấng và dead letter	192
34.1 Vì sao chọn nó, và nó giải quyết gì	192
34.2 Những thiết lập dự án chưa từng viết	193
34.2.1 Producer	193
34.2.2 Consumer	194
34.3 Chính xác khi nào xảy ra bản sao	195
34.4 Ưu và nhược, thẳng thắn	196
34.5 Nâng cấp giải pháp	196
34.5.1 Một consumer lữ đấng có sổ ghi những gì đã thấy	196

34.5.2	Topic thử lại và dead-letter topic	198
34.5.3	Đúng một lần, và những gì nó không bao phủ	199
34.6	Chuyện gì gây lúc 3 giờ sáng	200
34.7	Ngoài dự án	200
34.8	Câu hỏi từ phỏng vấn thật	201
35	Đào sâu: File ở quy mô lớn — presigned URL, endpoint, và vì sao byte không thuộc về Postgres	204
35.1	Vì sao kho đối tượng, và nó giải quyết gì	204
35.2	Đọc code của dự án	205
35.2.1	Hai client, hai endpoint	205
35.2.2	Key, upload, download	206
35.3	Signature Version 4, và vì sao host được hàn chết vào	208
35.4	Ái chuyển byte	209
35.5	Ưu và nhược, thắng thắn	210
35.6	Chuyện gì gây	211
35.7	Chuyện gì gây lúc 3 giờ sáng	211
35.8	Nâng cấp giải pháp: presigned PUT	212
35.9	Ngoài dự án	215
35.10	Câu hỏi từ phỏng vấn thật	216
36	Đào sâu: Từ điển trên MongoDB — mô hình document, tìm kiếm regex và những đánh đổi thắng thắn	219
36.1	Vì sao MongoDB ở đây, và nó giải quyết gì	219
36.2	Đọc đường tìm kiếm	221
36.2.1	Server thực thi nó thế nào	222
36.2.2	Điều kiện trên mảng	222
36.3	Đồ thị	223
36.4	Transaction và nhất quán, thật thà	224
36.5	Ưu và nhược, thắng thắn	224
36.6	Chuyện gì gây	225
36.7	Chuyện gì gây lúc 3 giờ sáng	225
36.8	Nâng cấp giải pháp	226
36.8.1	Phân trang trong cơ sở dữ liệu	226
36.8.2	Gộp lô các liên kết	227
36.8.3	Index cho sort, và tìm kiếm có neo	227
36.8.4	Text index cho tìm-từ-ở-bất-kỳ-đâu	228
36.9	Ngoài dự án	229
36.10	Câu hỏi từ phỏng vấn thật	229

37 Đào sâu: Khả năng quan sát — trace xuyên Feign và Kafka, lấy mẫu, và những cảnh báo đáng giá	232
37.1 Vì sao chọn nó, và nó giải quyết gì	232
37.2 Một trace đi qua dự án này như thế nào	233
37.2.1 Cấu hình, trong mọi service	233
37.2.2 Từng chặng một	234
37.2.3 Đối chiếu log	236
37.3 Lấy mẫu	236
37.4 Metric: cái gì có và cái gì chưa	237
37.5 Ưu và nhược	237
37.6 Cái gì gây	238
37.7 Chuyện gì gây lúc 3 giờ sáng	239
37.8 Nâng cấp giải pháp	239
37.9 Ngoài dự án	243
37.10 Câu hỏi từ phòng vận thật	243
38 Đào sâu: Trình duyệt gặp gateway — nơi cất token, luồng refresh và CORS	247
38.1 Vì sao chọn nó, và nó giải quyết gì	247
38.2 Luồng refresh, như đã viết	248
38.2.1 Interceptor	248
38.2.2 Bài toán single-flight	250
38.2.3 Đăng xuất và trang callback OAuth	250
38.3 CORS: chính sách bạn không đang dùng	251
38.4 Ưu và nhược	251
38.5 Cái gì gây	252
38.6 Chuyện gì gây lúc 3 giờ sáng	253
38.7 Nâng cấp giải pháp	254
38.8 Ngoài dự án	257
38.9 Câu hỏi từ phòng vận thật	258
A Đáp án tự kiểm tra	261
B Thuật ngữ	282
C Bảng tra nhanh	288
D Các file cấu hình	291

Phần I

Ứng dụng

Spring Boot: Cỗ máy bên dưới mọi service

Bây trong tám đơn vị triển khai của dự án này là ứng dụng Spring Boot. Trước khi bất kỳ file YAML nào có thể có nghĩa, bạn cần ba ý tưởng: một *starter* kéo theo những gì, *auto-configuration* quyết định những gì thay cho bạn, và — quan trọng nhất cho mọi thứ trong Phần V và VI — chính xác *Spring phân giải một thuộc tính cấu hình như thế nào* khi cùng một key được định nghĩa ở nhiều nơi cùng lúc.

1.1 Vấn đề Spring Boot giải quyết

Spring cổ điển bắt bạn nối dây mọi thứ: một servlet container để triển khai vào, cấu hình XML hoặc Java khai báo từng bean, và một danh sách dài các dependency tương thích phiên bản với nhau. Spring Boot đảo ngược điều đó. Một ứng dụng chỉ là một phương thức `main()` bình thường:

```
@SpringBootApplication
public class CourseServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(CourseServiceApplication.class, args);
    }
}
```

`@SpringBootApplication` là ba annotation núp chung trong một chiếc áo choàng: `@Configuration` (lớp này có thể định nghĩa bean), `@ComponentScan` (tìm `@Service`, `@RestController`, ... trong package này và bên dưới), và `@EnableAutoConfiguration` — cái thú vị nhất.

1.1.1 Starter

Một starter như `spring-boot-starter-web` là một jar rỗng mà công việc duy nhất là mang một danh sách dependency được tuyển chọn: Spring MVC, Jackson, và một Tomcat *nhúng*. “Nhúng” quan trọng về mặt vận hành: server nằm bên trong jar, nên `java -jar course-service.jar` là một web server hoàn chỉnh. Riêng sự thật đó là điều khiến các container image ở Chương 13 đơn giản đến vậy — không có Tomcat nào để cài, chỉ cần cung cấp một JVM.

1.1.2 Auto-configuration

Lúc khởi động, Spring Boot duyệt qua một danh sách các cấu hình ứng viên và áp dụng từng cái mà điều kiện của nó thỏa mãn. Các điều kiện đọc lên kiểu như:

- *lớp này có trên classpath không?* (`@ConditionalOnClass` — bạn đã thêm driver Postgres, nên một `DataSource` được cấu hình)
- *người dùng đã tự định nghĩa bean này chưa?* (`@ConditionalOnMissingBean` — bean tường minh của bạn luôn thắng)
- *một thuộc tính có được đặt không?* (`@ConditionalOnProperty`)

Auto-configuration là lý do course-service không chứa dòng code nào tạo connection pool, Kafka producer, hay nối dây S3 client — thêm dependency cộng vài thuộc tính *chính là* cấu hình. Khi hành vi dường như xuất hiện từ hư không, hãy chạy ứng dụng với `--debug` một lần: “condition evaluation report” liệt kê mọi auto-configuration được áp dụng và bị bỏ qua, kèm lý do.

1.2 Cấu hình ngoài: thứ tự phân giải thuộc tính

Đây là cơ chế chịu lực nhiều nhất trong cả cuốn sách. Mọi mẹo mà các manifest Kubernetes ở Phần V giở ra đều hoạt động nhờ nó.

Một thuộc tính Spring như `spring.datasource.url` có thể được định nghĩa trong nhiều *property source* (nguồn thuộc tính). Spring gộp tất cả vào một `Environment`, và khi hai nguồn định nghĩa cùng một key, nguồn cụ thể hơn thắng. Rút gọn về những nguồn dự án này thực sự dùng, từ yếu nhất đến mạnh nhất:

1. giá trị mặc định trong `application.yml` đóng gói trong jar
2. cấu hình lấy từ `config-server` (Chương 2)
3. biến môi trường của hệ điều hành
4. tham số dòng lệnh

QUYẾT ĐỊNH — VÌ SAO CỤM GHI ĐỀ BẰNG BIẾN MÔI TRƯỜNG

Cùng một image phải chạy không đổi trên laptop (Compose) lẫn trong k3s. Nướng sự khác biệt giữa các môi trường vào jar nghĩa là mỗi môi trường một image — đúng thứ container sinh ra để tránh. Biến môi trường đúng trên cả YAML đóng gói lẫn `config-server`, nên Deployment của Kubernetes có thể trở lại một service sang `postgres-course:5432` mà không đụng đến một file nào thuộc về lập trình viên. Một artifact, nhiều môi trường, khác biệt được tiêm vào ở biên.

1.2.1 Ràng buộc nới lỏng (relaxed binding)

Tên biến môi trường không thể chứa dấu chấm. Vì vậy Spring ánh xạ `SPRING_DATASOURCE_URL` → `spring.datasource.url`: viết hoa, dấu chấm thành gạch dưới. Đọc bất kỳ Deployment nào trong `deploy/k8s/base/` và giờ bạn có thể giải mã mọi mục `env`: thành “cái này ghi đè key YAML kia.” Ví dụ `SPRING_DATA_MONGODB_URI` trong `dictionary-service.yaml` ghi đè `spring.data.mongodb.uri` từ `config-server`.

1.2.2 Placeholder với giá trị mặc định

Các file cấu hình của dự án dùng thêm một thành ngữ nữa:

```
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIs...}
```

`${NAME:default}` nghĩa là “dùng biến môi trường nếu có, không thì dùng giá trị mặc định.” Đây là lý do một lập trình viên có thể chạy `auth-service` mà không cần cài đặt gì, trong khi `production` tiêm `secret` thật — và là lý do ở Chương 16 `Secret` của Kubernetes chỉ cần cung cấp `JWT_SECRET` là nắm quyền kiểm soát giá trị này.

1.3 File của dự án, có chú thích

Mỗi service chỉ mang một `application.yml` tí hon; phần cấu hình thú vị nằm trên `config-server`. Đây là file của `course-service`, đầy đủ:

```
spring:
  application:
    name: course-service ①
  config:
    import: optional:configserver:http://localhost:8888 ②
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/ ③
```

- ① Danh tính của service. `Config-server` dùng nó để chọn file cần phục vụ (`configurations/course-service.yml`); `Eureka` dùng nó làm tên đăng ký; gateway định tuyến tới nó bằng `lb://course-service`. Một chuỗi ký tự buộc cả ba hệ thống lại với nhau.
- ② Import cấu hình từ xa lúc bootstrap. `optional`: nghĩa là “cứ khởi động dù `config-server` có sập” — dễ chịu khi dev, và đáng bàn ở `production` (một service

khởi động với nửa cấu hình có thể hỏng theo những cách lạ lùng hơn một service từ chối khởi động). Giá trị mặc định `localhost:8888` được ghi đè bởi `SPRING_CONFIG_IMPORT` trong cả Compose lẫn Kubernetes.

- ③ Eureka ở đâu — lại một giá trị mặc định `localhost`, lại được ghi đè theo từng môi trường bằng `EUREKA_CLIENT_SERVICEURL_DEFAULTZONE`.

1.4 Profile

Một *profile* Spring là một biến thể cấu hình có tên. `spring.profiles.active=native` trên `config-server` (Chương 2) là chỗ dùng profile duy nhất của dự án cho đến giờ, nhưng cơ chế này tổng quát hơn: `application-prod.yml` được nạp đề lên `application.yml` khi profile `prod` đang kích hoạt, cho phép một jar mang nhiều tính cách. Kubernetes làm giảm nhu cầu đó — thay vào đó môi trường tiềm sự khác biệt vào — đó là lý do dự án này dựa vào biến môi trường, không phải file profile, cho thay đổi theo môi trường.

1.5 Actuator

`spring-boot-starter-actuator` thêm các endpoint vận hành dưới `/actuator: /health` (readiness tổng hợp của DB, đĩa, các kiểm tra tùy chỉnh), `/info`, `/metrics`, `/prometheus`. Hai sự thật quan trọng lúc này:

- Starter phải có trên classpath. Cấu hình exposure mà không có `spring-boot-starter-actuator` thì vô tác dụng — dự án đã mắc đúng lỗi đó ở bốn service và phát hiện ra qua một probe Kubernetes thất bại (Chương 17).
- Endpoint phải được *expose* (`management.endpoints.web.exposure.include`) — dự án expose `health, info, metrics, prometheus`.
- Expose không phải là *phân quyền*: nếu Spring Security bảo vệ `/actuator/**`, endpoint tồn tại nhưng trả lời 401. Chính tương tác này quyết định service nào dùng được probe Kubernetes nào ở Chương 17 — `dictionary-service` cho phép actuator và được probe qua `/actuator/health`; `auth-service` thì không, và phải được probe qua `/v3/api-docs` thay thế.

1.6 Bên ngoài dự án

Hai ví dụ tương phản của cùng một bộ máy thuộc tính:

Một ứng dụng *twelve-factor* kiểu *Heroku* cấu hình mọi thứ bằng biến môi trường — `DATABASE_URL`, `PORT` — và mang một `application.yml` chỉ toàn placeholder `${...}`. Thứ tự phân giải của Spring khiến phong cách đó trở thành bản địa.

Một ngân hàng chạy một artifact qua năm môi trường (dev/test/uat/pre-prod/prod) thường ghép một config-server dựa trên git repository với profile cho từng môi trường: `course-service-uat.yml` nằm trong một repo được kiểm toán, và việc thăng cấp là một lần merge git. Config-server classpath native của dự án (Chương 2) là phiên bản một-lập-trình-viên của cùng hình dạng đó.

CẠM BẦY — GIÁ TRỊ GHI ĐỀ CỦA TÔI BỊ BỎ QUA

Triệu chứng: bạn đặt một biến môi trường nhưng service vẫn dùng giá trị cũ. Các nguyên nhân thường gặp, theo thứ tự tần suất: tên biến không ánh xạ tới key bạn nghĩ (kiểm tra cách viết theo relaxed binding — `SERVICEURL` trong key Eureka là một từ, một cái tên bất quy tắc phải chép chính xác); giá trị cũng được đặt bởi một nguồn *mạnh hơn* (một tham số dòng lệnh); hoặc service đọc key lúc build thay vì từ Environment. Chẩn đoán bằng endpoint actuator `/actuator/env` (khi được expose), nó liệt kê mọi property source và nguồn nào đã thắng.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Một key được định nghĩa trong `application.yml` của jar, trong file của config-server cho service đó, và dưới dạng biến môi trường trong Deployment. Giá trị nào thắng, và vì sao thứ tự đó là đúng cho container?
2. `@SpringBootApplication` gộp ba annotation nào, và annotation nào trong số đó giải thích vì sao xóa một dependency không dùng đến có thể làm thay đổi hành vi lúc chạy?
3. `${GOOGLE_CLIENT_ID:placeholder}` — biểu thức này phân giải thành gì khi biến chưa được đặt, và điều đó cho auth-service tính chất vận hành nào lúc khởi động?
4. Vì sao expose một endpoint actuator không đảm bảo rằng một probe Kubernetes dùng được nó?

Cấu hình tập trung: Config Server

2.1 Vấn đề

Bây service, mỗi cái đều cần URL database, địa chỉ Kafka, thiết lập JWT, host mail. Chép những thứ đó vào bầy file `application.yml` và mỗi thay đổi trở thành bầy lần sửa, bầy lần build lại, cùng lời đảm bảo rằng sớm muộn hai bản sao sẽ lệch nhau. Câu trả lời của Spring Cloud cổ điển: một HTTP service nhỏ mà công việc duy nhất là trao cho từng ứng dụng cấu hình của nó lúc khởi động.

2.2 Spring Cloud Config hoạt động thế nào

Server expose cấu hình qua một hợp đồng HTTP đơn giản:

```
GET /{application}/{profile}
GET /course-service/default    -> the config for course-service
```

Phía client là dòng `spring.config.import: configserver:...` bạn đã thấy ở Chương 1. Trong lúc khởi động — *trước* khi hầu hết bean được tạo — client lấy tài liệu của mình về và gộp vào Environment như một property source đứng trên YAML đóng gói nhưng thua biến môi trường. Thứ tự phân giải ở Mục 1.2 làm toàn bộ công việc; config-server chỉ là thêm một nguồn nữa trong chồng.

2.3 Server của dự án, có chú thích

```
server:
  port: 8888                                ①
spring:
  application:
    name: config-server
  profiles:
    active: native                            ②
  cloud:
    config:
      server:
        native:
          search-locations: classpath:/configurations ③
```

```
management:
  endpoints:
    web:
      exposure:
        include: health
```

④

- ① 8888 là cổng config-server theo quy ước, cũng như 8761 là của Eureka. Quy ước tiết kiệm tài liệu.
- ② Profile native nghĩa là “phục vụ file từ một vị trí trên filesystem/classpath” thay vì backend git mặc định.
- ③ Các file được phục vụ nằm *ngay trong jar của chính config-server*: configurations/course-service-auth-service.yml, v.v. — được chọn bằng cách khớp tên file với spring.application.name của service gửi yêu cầu.
- ④ Chỉ health được expose — và đây là endpoint mà healthcheck của Compose và các probe của Kubernetes dựa vào.

QUYẾT ĐỊNH — CLASSPATH NATIVE VS. BACKEND GIT

Backend chuẩn doanh nghiệp là một git repository: thay đổi cấu hình là các commit — được kiểm toán, được review, hoàn tác được mà không cần build lại gì cả. Cái giá là một repository thứ hai và việc quản lý thông tin đăng nhập. Với một dự án một-lập-trình-viên mà thay đổi cấu hình dù sao cũng đi cùng thay đổi code, classpath native giữ mọi thứ trong một repo và một pipeline. Thứ bị đánh đổi: thay một giá trị giờ đòi hỏi build lại image config-server. Hãy biết nhà tuyển dụng của bạn đứng ở phía nào của sự đánh đổi đó.

CẠM BÃY — SERVICE KHỞI ĐỘNG VỚI GIÁ TRỊ MẶC ĐỊNH RỒI HỎNG MỘT CÁCH KỲ LẠ

Triệu chứng: auth-service khởi động được nhưng mọi request đều thất bại với lỗi kết nối tới localhost:5432. Nguyên nhân: config-server không liên lạc được lúc khởi động, và optional:configserver: để service khởi động từ giá trị mặc định đóng gói sẵn — vốn trở về localhost. Vì thất bại xuất hiện vài phút sau và cách xa nguyên nhân, nó trông như một vấn đề database. Kiểm tra: log khởi động in ra cấu hình từ xa có được lấy về hay không. File Compose ngăn vấn đề thứ tự này bằng depends_on: condition: service_healthy; Kubernetes không có thứ tự khởi động (Chương 15), nên các service phải chịu được một config-server đến muộn bằng cách crash rồi khởi động lại cho đến khi nó xuất hiện.

2.4 Thứ tự khởi động và điều nhảy bootstrap

Config-server tạo ra một ràng buộc trình tự ngầm trên toàn hệ thống: mọi service khác đều muốn nó lên trước. Bạn sẽ gặp hệ quả của điều này ba lần trong cuốn sách:

- Compose mã hóa thứ tự một cách tường minh bằng `depends_on` và một healthcheck dựa trên `curl` (Chương 12).
- Kubernetes từ chối mã hóa thứ tự khởi động; hệ thống hội tụ bằng `retry` (Chương 15).
- Pipeline Jenkins triển khai mọi manifest trong một lệnh `kubectl apply -k` và để các rollout tự ổn định đồng thời (Chương 23).

2.5 Làm mới cấu hình mà không khởi động lại

Mặc định, một client đọc cấu hình của nó một lần lúc khởi động. Spring Cloud cung cấp `/actuator/refresh` (lấy lại theo yêu cầu, từng instance, áp dụng cho các bean @RefreshScope) và Spring Cloud Bus (phát tán lệnh refresh qua một message broker). Dự án này không dùng cả hai: trên Kubernetes một thay đổi cấu hình được giao đi dưới dạng image mới hoặc thay đổi env, và rolling update của Deployment (Chương 15) dù sao cũng khởi động lại các pod. Đó là câu trả lời hiện đại cho câu hỏi mà config-server ban đầu được thiết kế để giải quyết.

2.6 Bên ngoài dự án

Một thiết lập doanh nghiệp điển hình dựa trên git, để đối chiếu:

```
spring:
  cloud:
    config:
      server:
        git:
          uri: https://git.company.com/platform/config-repo
          default-label: main
          search-paths: '{application}'
```

Cấu hình nằm theo từng service trong các thư mục của một repo riêng; label là một branch, nên `default-label: release-42` ghim một môi trường vào một bản phát hành. Mẫu phổ biến thứ hai bỏ hẳn config-server: ConfigMap và Secret của Kubernetes được gắn vào dưới dạng biến môi trường — chính là hướng dự án

này đang đi tới (Chương 16 và 26); config-server tồn tại ở đây chủ yếu như một ví dụ sống của stack cổ điển.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Các nguồn này thắng theo thứ tự nào cho `spring.datasource.url`: YAML đóng gói, tài liệu config-server, biến môi trường của Deployment?
2. `profiles.active`: native thay đổi gì về nơi server tìm tài liệu, và backend git mua được gì mà native không thể?
3. Vì sao `optional`: trong `spring.config.import` là con dao hai lưỡi? Mô tả kiểu thất bại mà nó cho phép.
4. Cấu hình của chính config-server không thể đến từ một config-server. Nó phải nằm ở đâu, và đó là nguyên lý bootstrap tổng quát nào?

Khám phá dịch vụ: Eureka

3.1 Vấn đề

Gateway phải chuyển tiếp `/api/courses` tới `course-service`. Tới địa chỉ nào? Trong Compose nó có thể ghi cứng `course-service:8082` — nhưng khi đó mỗi replica mới, mỗi service dời chỗ, mỗi môi trường đều là một thay đổi cấu hình. Khám phá dịch vụ (service discovery) thay địa chỉ bằng *tên được phân giải lúc chạy*: các service tự đăng ký vào một registry; client hỏi registry.

3.2 Eureka hoạt động thế nào

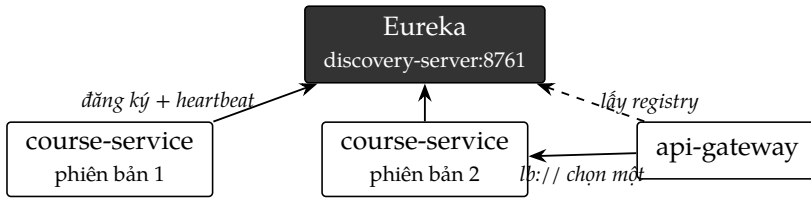
Eureka (từ Netflix, tổ tiên của stack Spring Cloud) là một REST service giữ một registry các instance:

- **Đăng ký.** Lúc khởi động mỗi client POST danh tính của mình: tên ứng dụng (`spring.application.name`), host, cổng, URL health.
- **Heartbeat.** Mỗi 30 s (mặc định) client gia hạn lease của mình. Bỏ lỡ vài lần gia hạn và instance bị loại bỏ (evict).
- **Cache phía client.** Bên tiêu thụ lấy registry về và cache lại (mặc định làm mới mỗi 30 s), rồi cân bằng tải *cục bộ* giữa các instance mà nó biết. Registry sập trong chốc lát không làm dừng lưu lượng — bên gọi tiếp tục dùng cache của mình.
- **Tự bảo toàn.** Nếu quá nhiều heartbeat biến mất cùng lúc, Eureka giả định đó là sự cố mạng chứ không phải chết hàng loạt và ngừng loại bỏ. Ít bất ngờ hơn khi mạng chập chờn; dữ liệu cũ hơn khi sự cố là thật.

Phía tiêu thụ trong dự án này là URI `lb://course-service` của gateway (Chương 4): “tra tên này trong registry, chọn một instance.”

3.3 Server, và thiết lập sống còn duy nhất trên k8s

Module `discovery-server` là Eureka với gần như không có cấu hình — một annotation `@EnableEurekaServer` và một cổng. Cấu hình thú vị nằm trên mỗi *client*, và



Hình 3.1: Đăng ký, heartbeat, và cân bằng tải phía client.

một dòng trong đó xứng đáng có mục riêng vì nó quyết định cả hệ thống có chạy được trên Kubernetes hay không:

```
- name: EUREKA_INSTANCE_PREFER_IP_ADDRESS
  value: "true"
```

Mặc định một client đăng ký *hostname* của nó. Trên laptop của bạn thế là ổn. Bên trong một pod Kubernetes, *hostname* là *tên pod* — `course-service-7d9f6c-x2v1q` — thứ các pod khác không phân giải được. Mọi đăng ký sẽ là một địa chỉ không ai gọi tới được. *IP* của pod, ngược lại, định tuyến được trên toàn cụm. Vì thế mọi Deployment trong `deploy/k8s/base/` đều đặt cờ này: đăng ký IP, không phải tên.

CẠM BÃY — MỌI THỨ ĐỀU ĐĂNG KÝ, KHÔNG GÌ GỌI ĐƯỢC GÌ

Triệu chứng: dashboard Eureka hiện mọi service đều UP, nhưng mọi request đi qua gateway đều thất bại với `UnknownHostException`. Nguyên nhân: *hostname* được đăng ký là tên pod (xem ở trên). Đây là lỗi Eureka-trên-Kubernetes phổ biến nhất, và nó vô hình cho đến lần gọi chéo service đầu tiên.

QUYẾT ĐỊNH — VÌ SAO VẪN DỪNG EUREKA, KHI KUBERNETES ĐÃ CÓ SERVICE

Kubernetes đã cho mỗi Service một tên DNS (`course-service:8082`) với cân bằng tải tích hợp sẵn — khám phá dịch vụ là bản địa của nền tảng. Eureka ở đây là sự dư thừa có chủ đích vì CV: stack cổ điển vẫn phổ biến trong ngành, và việc di cư *khỏi* nó (Chương 27: thay `lb://` bằng DNS Service thuần, xóa hai đơn vị triển khai) tự thân là loại công việc hiện đại hóa mà doanh nghiệp trả tiền để có. Xây cả hai stack là để học cuộc di cư đó.

3.4 Bên ngoài dự án

Consul (HashiCorp) là lựa chọn thay thế chính ngoài JVM: dựa trên agent, không phụ thuộc ngôn ngữ, với health check do agent thực hiện thay vì heartbeat tự báo

cáo, cộng một kho key-value có thể thay luôn cả config-server. *Khám phá dịch vụ bản địa của Kubernetes* là cực còn lại: không có tiến trình registry nào cả — các controller của nền tảng theo dõi readiness của pod và cập nhật endpoint của Service; “đăng ký” đơn giản là một pod trở thành Ready (Chương 17 cho thấy bộ máy đó). Hãy để ý mẫu hình: nền tảng cung cấp càng nhiều, ứng dụng càng phải mang ít.

TRÊN PHẦN CỨNG NÀY

Các giá trị mặc định bảo thủ của Eureka (heartbeat 30 s, cache client 30 s, loại bỏ sau 90 s) nghĩa là một instance đã crash vẫn có thể nhận lưu lượng tới một phút. Với một replica cho mỗi service thì điều đó hầu như không quan trọng — bên gọi đăng nào cũng thất bại — nhưng nó giải thích vì sao các đợt triển khai cuốn chiếu ở Chương 23 chờ readiness của Kubernetes, chứ không chờ trạng thái Eureka.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Lần theo những gì xảy ra từ lúc course-service khởi động đến lúc gateway định tuyến tới nó thành công: những lần đăng ký, lấy về, và cache nào có liên quan, và độ trễ trong trường hợp xấu nhất là bao nhiêu?
2. Vì sao prefer-ip-address quan trọng trên Kubernetes nhưng không quan trọng trên Compose? (Mỗi nền tảng làm cho cái gì phân giải được?)
3. Chế độ tự bảo toàn đánh đổi tính chất nào lấy tính chất nào? Cho một kịch bản nó giúp ích và một kịch bản nó gây hại.
4. Nếu bỏ Eureka và trở gateway thẳng tới `http://course-service:8082`, bộ máy Kubernetes nào thay thế từng tính năng của Eureka?

API Gateway

4.1 Vấn đề

Không có gateway, frontend phải biết địa chỉ của sáu service, mỗi service phải tự cài đặt xác thực, và những mối quan tâm xuyên suốt (CORS, rate limit, header truy vết) bị làm sáu lần hoặc không lần nào. Gateway cho client bên ngoài *một* địa chỉ duy nhất và gom mọi mối quan tâm ở biên về một chỗ; phía sau nó, các service có thể tin tưởng bên gọi mình.

4.2 Mô hình của Spring Cloud Gateway

Ba khái niệm tạo nên mọi route:

Route một URI đích cộng với các điều kiện và phép viết lại bên dưới.

Predicates điều kiện quyết định request có khớp hay không: path, method, header, host. Route khớp đầu tiên thắng.

Filters các phép biến đổi áp dụng trên đường đi: bỏ hoặc thêm đoạn path, thêm header, rate limit, retry.

Gateway được xây trên stack *reactive* (Project Reactor + Netty, web-application-type: reactive trong application.yml của nó) thay vì Tomcat servlet. Một proxy dành cả đời để chờ các server khác; mô hình event-loop giữ được hàng nghìn request đang bay mà không cần mỗi request một thread. Về vận hành, đây là lý do Deployment của gateway được cấp memory request nhỏ hơn các service MVC (384 Mi so với 512 Mi) và khởi động nhanh hơn — điều Chương 17 tận dụng bằng một startup probe ngắn hơn.

4.3 Các route của dự án, có chú giải

Từ configurations/api-gateway.yml (rút gọn — khối course-service lặp lại cho students, assignments, enrollments, submissions):

```
spring:
  cloud:
    gateway:
      routes:
```

```

- id: auth-service
  uri: lb://auth-service      ①
  predicates:
    - Path=/api/auth/**      ②
  filters:
    - StripPrefix=2          ③
- id: auth-oauth2
  uri: lb://auth-service
  predicates:
    - Path=/oauth2/**        ④
- id: course-service-courses
  uri: lb://course-service
  predicates:
    - Path=/api/courses/**
  filters:
    - StripPrefix=2
app:
  jwt:
    secret: ${JWT_SECRET:mySecret...} ⑤

```

- ① lb:// = “phân giải tên này qua discovery client rồi cân bằng tải.” Tên đó là `spring.application.name` của đích. Đổi lb:// thành http:// là bạn có một proxy tĩnh, không dính gì tới Eureka.
- ② Predicate theo path. ** khớp mọi độ sâu.
- ③ StripPrefix=2 bỏ hai đoạn: `/api/auth/login` tới auth-service dưới dạng `/login`. Đây là lý do `SecurityConfig` của auth-service (Chương 5) cho phép `/login` chứ không phải `/api/auth/login` — service không hề biết tiền tố công khai của mình.
- ④ Không có StripPrefix ở đây: redirect URI OAuth2 của Google được đăng ký với đường dẫn đầy đủ, nên nó phải tới auth-service nguyên vẹn.
- ⑤ Gateway giữ khóa *xác minh* JWT. Cùng secret với auth-service, bên ký — một secret dùng chung, hai vai trò, và trên Kubernetes là một object Secret mount vào cả hai (Chương 16).

CẠM BẤY — SERVICE TRẢ 404, CURL TRÊN POD TRẢ 200

Triệu chứng: gọi `/api/courses/list` qua gateway nhận 404, nhưng cũng endpoint đó lại trả lời khi gọi thẳng vào service. Nguyên nhân: lệch tiền tố — hoặc StripPrefix đã bỏ đoạn mà controller mong đợi, hoặc mapping của controller chứa tiền tố mà gateway đã bỏ rồi. Gỡ lỗi bằng cách log đường dẫn *sau khi viết lại*: bật debug logging cho `spring.cloud.gateway` và so với `@RequestMapping` của controller.

4.4 Hai bộ định tuyến: gateway và Ingress

Trên Kubernetes dự án này có *hai* bộ định tuyến L7 nối tiếp nhau: Traefik (Ingress controller, Chương 18) ở biên cụm, rồi tới gateway này. Sự phân công là có chủ ý:

- Traefik sở hữu *host*: TLS về sau, một cửa vào công khai, tách nhánh theo tiền tố path. Nó gửi /api, /oauth2, /login/oauth2 tới gateway và sẽ gửi / tới frontend.
- Gateway sở hữu *biên ứng dụng*: phân giải qua Eureka, xác minh JWT, viết lại theo từng route.

QUYẾT ĐỊNH — KHÔNG CÓ CỬA HỒNG

Cột mốc 3 từng có một Ingress gửi /courses thẳng tới course-service — giản giáo hữu ích, nhưng nó đi vòng qua bước xác minh JWT: một lối vào không xác thực tới dữ liệu được bảo vệ. Nó bị xóa ngay ngày gateway được triển khai. Quy tắc rút ra: *mọi* route bên ngoài phải đi qua thành phần thực thi xác thực, nếu không xác thực chỉ là đồ trang trí.

4.5 Những góc nâng cao

Đáng biết, nhưng chưa dùng ở đây: filter RequestRateLimiter (token bucket dựa trên Redis), Retry có backoff cho các GET lũy đẳng, filter CircuitBreaker (Resilience4j) trả về fallback khi một service sập, và các filter mặc định áp cho mọi route. Mỗi thứ chỉ vài dòng YAML — gateway là nơi những chính sách như vậy thuộc về, vì đó là điểm duy nhất mọi request bên ngoài đều đi qua.

4.6 Ngoài dự án

Cùng ý tưởng ba khái niệm trên route đó trong một gateway thuần Kubernetes (không Eureka, dùng DNS của Service thay thế) — kiểu annotation của nginx-ingress:

```
nginx.ingress.kubernetes.io/rewrite-target: /$2
# path: /api/courses(/|$)(.*) -> service: course-service
```

Và phiên bản cloud quản lý: bảng route của AWS API Gateway hay GCP API Gateway làm predicate/filter dưới dạng cấu hình của nhà cung cấp, với việc xác minh JWT giao cho nhà cung cấp (Cognito, IAP). Khái niệm chuyển 1:1; chỉ cú pháp và hóa đơn là khác.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Lần theo POST /api/auth/login từ trình duyệt tới phương thức controller trên Kubernetes: kể tên mọi chặng và mọi lần viết lại đường dẫn.
2. Vì sao các route OAuth2 không được bỏ tiền tố trong khi các route API thì phải?
3. Gateway xác minh JWT; auth-service ký chúng. Nếu hai bên có lúc giữ secret khác nhau thì cái gì hỏng, với triệu chứng gì?
4. Vì sao stack reactive là lựa chọn đúng cho proxy nhưng không đương nhiên đúng cho course-service?

Bảo mật: JWT, OAuth2 và biên tin cậy

Bảo mật là nơi dự án này dạy được nhiều nhất, vì nó dùng ba cơ chế mà ngành dùng liên tục — JWT phi trạng thái, đăng nhập OAuth2, xác thực hai lớp TOTP — và nối chúng qua một gateway theo mẫu “biên tin cậy” (trusted edge) chuẩn mực.

5.1 Spring Security trong một trang

Spring Security là một chuỗi filter servlet đứng trước các controller của bạn. Mỗi request đi qua chuỗi đó; các filter xác thực (bạn là ai?) và các quy tắc phân quyền quyết định (bạn được phép không?). Chuỗi của auth-service trong dự án, từ SecurityConfig của nó:

```
http
    .csrf(AbstractHttpConfigurer::disable) ①
    .sessionManagement(s ->
        s.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) ②
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/register", "/login", "/activate",
            "/refresh", "/verify-2fa", "/change-password",
            "/oauth2/**", "/login/oauth2/**",
            "/v3/api-docs/**", "/swagger-ui/**").permitAll() ③
        .anyRequest().authenticated())
    .oauth2Login(o -> o.successHandler(oAuth2LoginSuccessHandler))
    .addFilterBefore(gatewayHeaderAuthFilter,
        UsernamePasswordAuthenticationFilter.class); ④
```

- ① Bảo vệ CSRF che chắn cho phiên cookie của trình duyệt; một API JWT phi trạng thái không có phiên cookie nào để lợi dụng, nên nó bị tắt. Tắt vì lý do đó — không phải vì một tutorial bảo thế.
- ② STATELESS: không tạo HTTP session. Mỗi request phải tự mang bằng chứng danh tính. Đây là tính chất cho phép bất kỳ replica nào trả lời bất kỳ request nào — điều kiện tiên quyết để mở rộng.
- ③ Các đường dẫn công khai. Để ý đó là các đường dẫn *đã bỏ tiền tố* (/login, không phải /api/auth/login) — Chương 4 giải thích vì sao. Cũng để ý thứ *vắng mặt*: /actuator/**. Sự vắng mặt đó quyết định chiến lược probe Kubernetes của service này ở Chương 17.

- ④ Filter tùy biến tin tưởng các header danh tính từ gateway — chủ đề của Mục 5.3.

5.2 JWT: danh tính phi trạng thái

Một JSON Web Token gồm ba phần `base64url`, `header.payload.signature`. Payload mang các *claim*: `subject` (id người dùng), `vai trò`, `hạn dùng`. Chữ ký là HMAC-SHA256 trên header và payload với một `secret` dùng chung — chuỗi tối thiểu 256 bit mà cả `auth-service` lẫn `gateway` đều giữ.

Các tính chất quan trọng:

- **Tự chứa.** Xác minh không cần gọi cơ sở dữ liệu: kiểm tra chữ ký, kiểm tra hạn, đọc *claim*. Chính điều này cho phép *gateway* xác thực mà không sở hữu dữ liệu người dùng.
- **Đọc được.** Base64 là encoding, không phải mã hóa (encryption) — ai cũng giải mã được payload. Đừng bao giờ đặt bí mật trong *claim*.
- **Không thu hồi được cho tới khi hết hạn.** Token đã ký thì không thể “hủy ký”. Vì thế mới có mẫu hai token: một *access token* sống ngắn (ở đây 15 phút) đi cùng một *refresh token* sống dài (7 ngày) mà có được đối chiếu với cơ sở dữ liệu tại `/refresh` — việc thu hồi diễn ra ở đó.

5.3 Mẫu biên tin cậy

`JwtAuthenticationFilter` của `gateway` xác minh token, rồi chuyển tiếp request với danh tính dưới dạng header thuần:

```
ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
    .header(JwtConstants.HEADER_USER_ID, userId)
    .header(JwtConstants.HEADER_USER_ROLE, role)
    .build();
```

Ở phía sau, `GatewayHeaderAuthFilter` của mỗi service đọc các header đó và dựng Spring Security context từ chúng — không xác minh lại. Service giữ được sự đơn giản; mật mã chỉ chạy một lần.

Nghĩa vụ đi kèm mẫu này: các header đó chỉ đáng tin nếu *mọi* đường tới một service đều đi qua `gateway`. Một client với được thẳng tới `course-service` có thể giả mạo `X-User-Id`. Đây là lý do sâu xa khiến Ingress `/courses` trực tiếp bị xóa (Chương 4), và là lý do lưu lượng nội cụm vẫn là điểm yếu còn lại — pod nào cũng vẫn có thể gọi `course-service:8082` với header bịa ra. `NetworkPolicy` hay

mTLS (service mesh) là cách gia cố trong công nghiệp; Chương 27 bàn khi nào chúng xứng đáng với độ phức tạp của mình.

5.4 Đăng nhập OAuth2 với Google

Luồng *authorization code* của OAuth2, như nó chạy ở đây:

1. Trình duyệt gọi `/oauth2/authorization/google` (qua gateway); Spring chuyển hướng tới Google kèm `client-id` của ứng dụng và một `redirect_uri`.
2. Người dùng đồng ý tại Google; Google chuyển hướng trình duyệt về `redirect_uri` — `/login/oauth2/code/google` — mang theo một *code* dùng một lần.
3. Auth-service đổi `code` + `client-secret` lấy token theo đường server-tới-server, đọc e-mail/hồ sơ người dùng, rồi `OAuth2LoginSuccessHandler` của dự án tiếp quản: tìm hoặc tạo tài khoản cục bộ và phát hành JWT *của chính chúng ta*. Google xác thực; ứng dụng vẫn phân quyền bằng token của riêng mình.

CẠM BẤY — REDIRECT_URI_MISMATCH

Triệu chứng: Google hiện “Error 400: redirect_uri_mismatch” thay vì màn hình đồng ý. Nguyên nhân: URI ứng dụng gửi đi không trùng từng byte với URI đã đăng ký trong Google Cloud console. Nó thay đổi theo môi trường (`localhost:8080` khi dev, `ts.local` trong cụm — đó là lý do Deployment ghi đề `...GOOGLE_REDIRECT_URI`), và mọi biến thể đều phải được đăng ký. URI phải tới auth-service *chưa bị bỏ tiền tố*, đó là lý do các route OAuth2 trên gateway không có `StripPrefix`.

5.5 Xác thực hai lớp TOTP

Mật khẩu dùng một lần theo thời gian (Time-based One-Time Password, RFC 6238): khi đăng ký, server sinh một secret, hiện nó dưới dạng mã QR, và ứng dụng authenticator suy ra mã 6 chữ số từ `HMAC(secret, time/30s)`. Đăng nhập thành hai bước — kiểm tra mật khẩu phát ra một trạng thái chờ sống ngắn, rồi `/verify-2fa` kiểm tra mã và phát hành token thật. Server và điện thoại không bao giờ liên lạc với nhau; chúng chỉ chia sẻ secret và một chiếc đồng hồ.

5.6 Ngoài dự án

Lựa chọn thay thế chính trong công nghiệp đưa việc phát hành token ra khỏi code của bạn hoàn toàn: một identity provider (Keycloak tự host, hoặc Auth0/Cognito được quản lý) phát hành token ký bằng cặp khóa *bất đối xứng* (RS256). Service xác minh bằng khóa *công khai* lấy từ một URL JWKS — không có secret dùng chung phải phân phối, xoay khóa có sẵn:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: https://idp.company.com/realms/app/protocol/openid-connect/certs
```

Hiểu phiên bản HS256 tự tay làm — thứ dự án này xây — chính là điều làm cho cấu hình kia có ý nghĩa: cùng claim, cùng cách xác minh, chỉ khác nghi thức khóa. Chương 28 tháo rời thiết kế này dưới tải: xác minh thuộc về đâu, khóa bất đối xứng thay đổi điều gì, và làm sao giữ token mỏng.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao có thể tắt bảo vệ CSRF cho một API JWT phi trạng thái nhưng không thể cho một webapp dùng phiên cookie?
2. Kẻ tấn công đánh cắp được một access token. Nó dùng được bao lâu, trong lúc đó server không thể làm gì, và endpoint nào cuối cùng chặn được phiên?
3. Trong mẫu biên tin cậy, phát biểu chính xác bất biến phải giữ để X-User-Id đáng tin, và nêu hai sai lầm triển khai sẽ phá vỡ nó.
4. Trong luồng Google, secret nào không bao giờ rời server, giá trị nào đi qua trình duyệt, và vì sao giá trị trình duyệt thấy được lại an toàn?
5. HS256 so với RS256: ai xác minh được, ai ký được, và vì sao các identity provider chọn RS256?

Các service nghiệp vụ và module common

Các chương trước bàn về những service hạ tầng. Chương này vẽ bản đồ ba service làm việc thật, cộng với module common dùng chung — và qua chúng, hai cách microservice nói chuyện với nhau.

6.1 course-service: con ngựa thồ

Khóa học, học viên, bài tập, ghi danh, bài nộp — CRUD kinh điển trên Postgres (Chương 7), cộng thêm file đính kèm lưu trong MinIO (Chương 9). Nó *phát* sự kiện `ts.assignment.created` và *lắng nghe* `ts.user.activated` — mẫu đáng để ý là nó phản ứng với sự kiện nghiệp vụ của service khác mà không bao giờ gọi service đó.

6.2 auth-service: danh tính

Mọi thứ Chương 5 đã mô tả: đăng ký kèm kích hoạt qua e-mail, đăng nhập bằng mật khẩu và Google, 2FA TOTP, phát hành JWT, refresh token. Nó phát ba sự kiện: `ts.user.registered`, `ts.user.activated`, `ts.user.admin-provisioned`.

6.3 dictionary-service: anh nhỏ

Tra cứu từ vựng trên MongoDB (Chương 8). Đơn giản có chủ ý — nó tồn tại một phần để chứng minh kiến trúc chứa được một document store, một phần làm service ít rủi ro để thử mọi thứ trước.

6.4 notification-service: consumer thuần túy

Không có REST API đáng kể, không cơ sở dữ liệu. Nó đăng ký ba topic và biến sự kiện thành e-mail (Maildev khi dev, một SMTP relay thật về sau):

Topic	Producer	Mail gửi đi
ts.user.registered	auth-service	link kích hoạt
ts.user.admin-provisioned	auth-service	thông báo tài khoản
ts.assignment.created	course-service	thông báo bài tập mới

Vì đầu vào *duy nhất* của nó là Kafka, đây là service duy nhất không thể chạy hữu ích trước khi có broker — đó là lý do manifest Kubernetes của nó đi cùng cột mốc Kafka (Chương 25) chứ không cùng các service khác.

6.5 Đồng bộ và bất đồng bộ, trong codebase này

Dự án dùng cả hai kiểu tương tác, mỗi kiểu đúng chỗ của nó:

Đồng bộ (HTTP qua gateway) khi bên gọi cần câu trả lời để đi tiếp: frontend lấy danh sách khóa học. Ràng buộc về thời gian — cả hai bên phải đang chạy.

Bất đồng bộ (sự kiện qua Kafka) khi bên gọi chỉ cần ghi nhận một sự thật: một đăng ký vừa xảy ra; ai quan tâm thì hành động. auth-service không biết notification-service tồn tại. Tách rời về thời gian — nếu consumer sập, mail đi muộn chứ không mất.

Quy tắc ngón tay cái codebase tuân theo: *lệnh* (command) cần câu trả lời thì dùng HTTP; *sự thật* mà người khác có thể phản ứng thì là sự kiện, đặt tên ở thì quá khứ (UserRegistered, AssignmentCreated).

6.6 Module common

```
com/ts/common/
dto/           CourseDto, StudentDto, UserDto, ...
               UserRegisteredEvent, UserActivatedEvent,
               AssignmentCreatedEvent, AdminProvisionedUserEvent
exception/     ApiException, ErrorResponse
security/      JwtConstants      <- header names, claim names
```

Ba kiểu chia sẻ, với ba hồ sơ rủi ro khác nhau:

- **Các record sự kiện** là *hợp đồng* giữa producer và consumer. Chia sẻ class bảo đảm hai bên nhất trí — với cái giá là phải triển khai đồng bộ khi hình dạng thay đổi. (Lựa chọn thay thế trong công nghiệp — schema registry — xuất hiện ở Chương 11.)

- **JwtConstants** chia sẻ *tên* header và claim giữa gateway và các service. Một tên header gõ sai sẽ hỏng âm thầm — request đơn giản là tới nơi dưới dạng vô danh — nên gom về một chỗ là khoản bảo hiểm rẻ.
- **DTO và exception** là chia sẻ cho tiện: dễ chịu trong monorepo, và là thứ đầu tiên nên nhân-bản-thay-vì-chia-sẻ nếu các service có ngày tách ra thành các repository riêng, vì các class tiện ích dùng chung chính là cách những service “độc lập” rất cuộc lại phải phát hành cùng nhau.

QUYẾT ĐỊNH — CÓ NÊN CÓ MODULE DÙNG CHUNG KHÔNG?

Những người theo chủ nghĩa thuần túy microservice cấm chia sẻ code giữa các service. Lựa chọn thuần túy — mỗi service tự định nghĩa bản sao của mọi sự kiện — lại cho phép trôi lệch đúng ở chỗ sự nhất trí là bắt buộc. Dự án này vạch ranh giới ở: chia sẻ *hợp đồng* (sự kiện, tên header), nghĩ kỹ trước khi chia sẻ bất cứ thứ gì có hành vi. `jib.skip=true` trong POM của common đánh dấu nó là module duy nhất là thư viện, không phải thứ để triển khai.

CẠM BÃY — BÃY TRIỂN KHAI ĐỒNG BỘ (LOCKSTEP)

Triệu chứng: sau khi thêm một trường vào `UserRegisteredEvent` và chỉ triển khai `auth-service`, `notification-service` bắt đầu ném lỗi giải tuần tự trên mọi thông điệp — và vì nó dùng chung consumer group, nó cứ thử lại mãi cùng một thông điệp hỏng. Nguyên nhân: hợp đồng thay đổi chỉ ở một phía, và topic giờ chứa sự kiện ở hai hình dạng. Các quy tắc ngăn chuyện đó: chỉ *thêm* trường tùy chọn, không bao giờ đổi tên hay xóa; triển khai consumer trước producer; và cấu hình xử lý lỗi cho consumer (Chương 11) để một thông điệp độc không làm nghẽn cả topic.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Với mỗi trường hợp, nói HTTP hay sự kiện là phù hợp và vì sao: (a) frontend cần danh sách học viên; (b) một bài nộp vừa được chấm và học viên cần được báo; (c) `course-service` phải xác minh một JWT.
2. Vì sao `notification-service` là service duy nhất có manifest Kubernetes phải chờ cột mốc Kafka, trong khi `auth-service` cũng dùng Kafka?
3. Nếu một thứ thuộc về `common` và một thứ không bao giờ nên ở đó, kèm lập luận.
4. Quy tắc thứ tự triển khai nào làm cho thay đổi schema sự kiện an toàn, và bản thân thay đổi đó phải có tính chất gì?

Phần II

Dữ liệu

Dữ liệu quan hệ: Postgres, JPA, Flyway

Hai trong số các service, mỗi service sở hữu một cơ sở dữ liệu PostgreSQL: auth-service sở hữu `ts_auth`, course-service sở hữu `ts_course`. Chương này bàn ba tầng nằm giữa một phương thức @Service và các byte trên đĩa — JPA/Hibernate, connection pool, và Flyway — cùng quy tắc kiến trúc giải thích vì sao lại có tới *hai* cơ sở dữ liệu.

7.1 Mỗi service một cơ sở dữ liệu

QUYẾT ĐỊNH — HAI SERVER POSTGRES, KHÔNG PHẢI HAI SCHEMA TRONG MỘT

Mỗi service sở hữu dữ liệu của mình một cách độc quyền; đường duy nhất tới dữ liệu của service khác là qua API hoặc sự kiện của nó. Điều này cấm kiểu thất bại kinh điển của cơ sở dữ liệu dùng chung — service A lặng lẽ join vào bảng của service B, trói schema hai bên với nhau mãi mãi. Dự án đi xa hơn mức cần thiết: không chỉ cơ sở dữ liệu riêng mà cả tiến trình server riêng, nên ngay cả kết nối chéo vô tình cũng bất khả, và mỗi bên có thể được tinh chỉnh, sao lưu, hay làm hỏng độc lập. Chi phí trên máy này: khoảng 256 MiB cho mỗi server thêm. Trong một tài khoản cloud, cùng mức cô lập đó thường là hai cơ sở dữ liệu trên một instance RDS được quản lý — cô lập bằng thông tin đăng nhập thay vì bằng tiến trình — vì instance được quản lý tính tiền theo server.

7.2 JPA và Hibernate

JPA là đặc tả (các annotation như @Entity, @OneToMany, EntityManager); Hibernate là bản cài đặt mà Spring Boot tự cấu hình. Spring Data JPA nằm bên trên, suy ra truy vấn từ tên phương thức repository (findByCourseIdAndStudentId) và sinh phần mã lập.

Cấu hình từ configurations/course-service.yml:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_course ①
```

```

username: postgres
password: root
jpa:
  hibernate:
    ddl-auto: none
    show-sql: true
flyway:
  enabled: true

```

- ① Mặc định khi dev. Compose ghi đè host thành postgres-course; Kubernetes ghi đè y hệt qua `SPRING_DATASOURCE_URL`. Để ý Service k8s được cố ý đặt tên postgres-course để chuỗi ghi đè giống nhau ở cả hai môi trường.
- ② Thông tin đăng nhập dạng văn bản thuần chỉ chấp nhận được vì file này là *mặc định khi dev*; trong cụm, `SPRING_DATASOURCE_USERNAME/PASSWORD` tới từ một Secret Kubernetes (Chương 16) và có ưu tiên cao hơn.
- ③ Dòng quan trọng nhất trong khối — Mục 7.3.
- ④ Mọi câu SQL đều được log — một công cụ dạy học và một máy dò truy vấn N+1 khi dev; tắt đi ở nơi dung lượng log tốn tiền.
- ⑤ Quản lý schema thuộc về Flyway, không phải Hibernate.

7.3 Vì sao `ddl-auto: none`

Hibernate có thể sinh schema từ entity (`ddl-auto: update`), và tutorial nào cũng dùng nó. Hệ thống production thì không, vì update là cái bánh cóc một chiều không có lịch sử: nó thêm cột nhưng không bao giờ xóa hay đổi tên một cách an toàn; cơ sở dữ liệu của hai lập trình viên lệch nhau âm thầm; không có bản ghi nào về cái gì đổi khi nào, và không có cách nào xem xét một thay đổi schema trước khi nó chạm tới production.

7.4 Flyway: migration dưới dạng code

Flyway coi schema là một chuỗi có thứ tự các script SQL bất biến, mỗi script áp dụng đúng một lần, được theo dõi trong một bảng (`flyway_schema_history`) ngay trong cơ sở dữ liệu đích:

```

auth-service/src/main/resources/db/migration/
V1__create_users.sql
V2__create_refresh_tokens.sql
V3__add_admin_user_invitations.sql
V4__seed_default_admin.sql

```

Lúc khởi động, Flyway so sánh lịch sử với các script trên classpath và áp dụng những script còn thiếu, theo thứ tự phiên bản, trước khi Hibernate kịp khởi tạo. Các quy tắc làm nó hoạt động: không bao giờ sửa một migration đã áp dụng (mỗi dòng lưu một checksum — sửa sẽ bị phát hiện và từ chối); chỉ thêm phiên bản mới; và viết migration tương thích với phiên bản ứng dụng *trước đó* khi bạn triển khai cuốn chiếu (Chương 15): thêm-rồi-chuyển-rồi-xóa, không bao giờ đổi tên trong một bước.

CẠM BẦY — CHẠY TỐT Ở MÁY MÌNH, CRASHLOOPBACKOFF TRONG CỤM

Triệu chứng: service khởi động được trên máy bạn nhưng pod của nó khởi động lại không ngừng; log cho thấy Flyway lệch checksum hoặc một migration thất bại. Nguyên nhân: cơ sở dữ liệu cục bộ đã được migrate bằng một script cũ hơn (hoặc đã bị sửa) — bảng lịch sử trên laptop của bạn và của cụm không khớp nhau. Đây cũng là lý do startup probe của Kubernetes (Chương 17) nhắm vào một endpoint chỉ trả lời sau khi Spring context đã lên: “migration đã áp dụng” là một phần của nghĩa *sẵn sàng*. Sửa bằng cách thêm một migration hiệu chỉnh, không bao giờ bằng cách sửa lịch sử.

7.5 Connection pool

Mở một kết nối Postgres tốn một bắt tay TCP, xác thực, và một lần fork tiến trình server — quá đắt cho mỗi request. HikariCP (pool mặc định của Spring Boot) giữ một nhóm nhỏ kết nối đang mở để request mượn rồi trả. Tối đa mặc định: 10.

TRÊN PHẦN CỨNG NÀY

Truyền miệng về kích thước pool nói “nhiều kết nối hơn, thông lượng cao hơn”; thực tế trên server 2 nhân thì ngược lại — Postgres làm việc *theo từng kết nối*, và công thức thực dụng ($\approx$ số nhân $\times 2$) cho thấy mặc định 10 đã là hào phóng với hai service chia nhau hai nhân. Nếu Postgres có lúc báo too many clients, hãy hạ pool xuống trước khi nâng max_connections.

7.6 Ngoài dự án

Phiên bản cloud được quản lý tương đương với thiết lập của chương này — AWS RDS Postgres với cùng luồng Flyway — chỉ thay đổi URL và mô hình tin cậy:

```
spring:
```

```
datasource:  
  url: jdbc:postgresql://app-db.xxxxx.eu-central-1.rds.amazonaws.com:5432/ts_course  
  username: ${DB_USER}      # from AWS Secrets Manager  
  password: ${DB_PASSWORD}
```

Sao lưu, sao chép (replication) và failover thành việc của nhà cung cấp — đó là lý do cảnh báo của Chương 19 về trạng thái tự host lại quan trọng. Biến thể công nghiệp thứ hai đáng biết: chạy Flyway như một bước CI riêng (flyway migrate vào đích) thay vì lúc ứng dụng khởi động, để thay đổi schema triển khai độc lập với code và một migration chậm không ăn hết ngân sách của startup probe.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Nêu hai kiểu thất bại cụ thể của ddl-auto: update trong môi trường làm việc nhóm mà none + Flyway ngăn được.
2. Vì sao không bao giờ được sửa một migration đã áp dụng, và Flyway dùng gì để phát hiện?
3. Bạn cần đổi tên một cột mà không có downtime dưới triển khai cuốn chiếu. Phác thảo chuỗi nhiều bản phát hành.
4. Bảo vệ và phản bác quyết định hai server: mỗi phía một lập luận, và bối cảnh nào thì phía nào thắng.

Dữ liệu dạng document: MongoDB

8.1 Mô hình

MongoDB lưu các *document* — cấu trúc giống JSON (BSON) — trong các *collection*, không có schema cố định. Ở chỗ Postgres chuẩn hóa một mục từ điển thành các hàng rải trên các bảng *words*, *definitions*, *examples* nối với nhau bằng khóa, Mongo lưu cả mục đó thành một *document* lồng nhau:

```
{ "word": "ubiquitous",
  "phonetic": "yoo-BIK-wi-tuhs",
  "meanings": [
    { "partOfSpeech": "adjective",
      "definitions": [
        { "definition": "present everywhere",
          "example": "smartphones are ubiquitous" } ] } ] }
```

QUYẾT ĐỊNH — VÌ SAO TỪ ĐIỂN LÀ SERVICE DÙNG MONGO

Một mục từ điển được đọc như một khối, ghi như một khối, và lồng nhau một cách tự nhiên — trường hợp đẹp nhất của mô hình *document*. Không có *transaction* xuyên mục, không có *join* nào để mất. Ngược lại, khóa học và ghi danh thực sự có tính quan hệ (một bài nộp tham chiếu một học viên và một bài tập; tính nhất quán giữa chúng là quan trọng), nên chúng ở lại Postgres. “Polyglot persistence” chính là thể này: chọn theo từng *service*, theo hình dạng dữ liệu — không phải một cơ sở dữ liệu được sùng ái cho mọi thứ, cũng không phải chạy theo cái mới cho vui.

8.2 Spring Data MongoDB

Mô hình lập trình giống JPA đủ gần để *dictionary-service* làm một phiên đá Rosetta tốt: `@Document` thay cho `@Entity`, `MongoRepository` thay cho `JpaRepository`, các phương thức truy vấn suy diễn giống hệt về tinh thần. Những gì biến mất: *Flyway* và *DDL schema* (*collection* tự xuất hiện ở lần ghi đầu tiên), *dialect*, và việc tinh chỉnh *pool* (*driver* tự quản lý *pool* kết nối của nó). Những gì xuất hiện: một *URI* kết nối duy nhất.

8.3 Cấu hình — toàn bộ

Toàn bộ cấu hình dữ liệu của dictionary-service là một dòng, cộng với bản ghi đề cho cụm:

```
# configurations/dictionary-service.yml (dev default)
spring:
  data:
    mongodb:
      uri: mongodb://localhost:27017/ts_dictionary

# deploy/k8s/base/dictionary-service.yaml (the override)
- name: SPRING_DATA_MONGODB_URI
  value: "mongodb://mongodb:27017/ts_dictionary"
```

URI mang host, cổng, tên database, và (khi bật) thông tin đăng nhập cùng các tùy chọn — một chuỗi duy nhất để ghi đề cho mỗi môi trường, đúng việc mà cơ chế relaxed binding ở Chương 1 sinh ra để làm.

8.4 Không schema là con dao hai lưỡi

Không có migration không có nghĩa là không có schema — nghĩa là schema sống *ngầm trong code*. Hai document do hai phiên bản ứng dụng ghi ra có thể có hình dạng khác nhau trong cùng một collection, và không gì ngoài code đọc của bạn nhận ra điều đó. Các kỷ luật thay thế Flyway: giữ các lớp document đọc ngược tương thích (trường mới là tùy chọn), viết script sửa dữ liệu khi hình dạng thật sự thay đổi, và kiểm tra hợp lệ ở biên. Mongo cũng có thể áp JSON Schema cho từng collection — tùy chọn bật thêm, và không dùng ở đây.

CẠM BẦY — COLLECTION CHƯA BAO GIỜ ĐƯỢC TẠO

Triệu chứng: service khởi động, truy vấn chạy, mọi thứ trả về rỗng — không có lỗi ở đâu cả. Nguyên nhân: URI trở vào một database mới tinh (gõ nhầm, hoặc một PVC mới trong cụm); Mongo lặng lẽ tạo một ts_dictionary rỗng ở lần chạm đầu tiên. Postgres sẽ thất bại ầm ĩ (database does not exist); cơ chế tạo-khi-ghi của Mongo biến một địa chỉ sai thành kết quả rỗng. Khi một service dùng Mongo “mất” dữ liệu, hãy kiểm tra *nó đang nói chuyện với database nào* trước mọi thứ khác: `kubect1 exec vào node của pod và chạy mongosh --eval 'show dbs'`.

8.5 Vận hành trong cụm này

Manifest (`deploy/k8s/base/mongodb.yaml`) theo cùng mẫu `StatefulSet` như `Postgres` (Chương 15 giải thích loại object này; Chương 19 nói về lưu trữ), với hai lưu ý riêng của `Mongo`, cả hai đều lấy từ comment trong file: `readiness probe exec mongosh --eval "db.adminCommand('ping')"` — “đang nhận lệnh,” chứ không chỉ “cổng đã mở” — và nó chạy không có xác thực, chấp nhận được chỉ vì đây là một `Service ClusterIP` không có `Ingress`: không thể chạm tới từ ngoài cụm. Khoảng khắc giả định đó thay đổi (một `NodePort` lộ ra ngoài, nhiều `tenant`), xác thực phải là việc đầu tiên.

8.6 Ngoài phạm vi dự án

Phiên bản được quản lý là `MongoDB Atlas`; `URI` có thêm thông tin đăng nhập và `TLS`, ngoài ra không gì thay đổi:

```
spring:
  data:
    mongodb:
      uri: mongodb+srv://app:${MONGO_PWD}@cluster0.abc.mongodb.net/ts_dictionary?
      ↪ retryWrites=true
```

Để đối chiếu, `DynamoDB` là một kho document với triết lý ngược lại: không schema như `Mongo`, nhưng các mẫu truy cập phải được thiết kế sẵn vào bảng (`partition key`, không có truy vấn tùy hứng) — một lời nhắc rằng “`NoSQL`” là một họ các đánh đổi, không phải một thứ duy nhất.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Những tính chất nào của dữ liệu từ điển khiến nó hợp với mô hình document, và những tính chất nào của dữ liệu ghi danh khiến nó không hợp?
2. “Không schema” chuyển việc áp đặt schema từ database sang đâu? Nêu hai kỷ luật thay thế cho migration.
3. Vì sao tên database sai thất bại âm ỉ trong `Postgres` nhưng lặng lẽ trong `Mongo`, và bạn thích kiểu thất bại nào hơn ở production?
4. Pod `Mongo` không có mật khẩu trong khi `Postgres` có, trong cùng một cụm. Dừng lại lập luận và nêu điều kiện làm lập luận đó mất hiệu lực.

Lưu trữ đối tượng: MinIO và mô hình S3

9.1 Bài toán

Bài nộp của học viên là file — PDF, ảnh, file nén. File trong cơ sở dữ liệu quan hệ làm phình bản sao lưu và stream rất tệ; file trên đĩa cục bộ của pod biến mất cùng pod. Câu trả lời của ngành là *lưu trữ đối tượng* (object storage): một dịch vụ HTTP lưu các blob bất biến trong các *bucket* có tên, dưới các *key* dạng chuỗi, kèm metadata, không có thư mục (dấu / trong key là quy ước đặt tên, không phải cấu trúc). Amazon S3 định nghĩa API; MinIO là một server tự host nói cùng API đó — nên course-service dùng AWS SDK chuẩn và không phân biệt được sự khác nhau.

9.2 Cấu hình, có chú giải

Từ configurations/course-service.yml:

```
app:
  s3:
    endpoint: ${APP_S3_ENDPOINT:http://localhost:9000} ①
    public-endpoint: ${APP_S3_PUBLIC_ENDPOINT:http://localhost:9000} ②
    access-key: ${APP_S3_ACCESS_KEY:minioadmin} ③
    secret-key: ${APP_S3_SECRET_KEY:minioadmin}
    bucket: ${APP_S3_BUCKET:ts-submissions}
    presigned-url-expiry-minutes: ${APP_S3_PREIGNED_EXPIRY_MIN:15} ④
```

- ① Nơi *service* nói chuyện với MinIO. Trong cụm: `http://minio:9000`.
- ② Nơi *trình duyệt* nói chuyện với MinIO — chủ đề của cái bẫy bên dưới.
- ③ Thông tin đăng nhập với giá trị mặc định cho dev; cụm bơm giá trị thật từ `Secret minio-app.app.s3.*` là cấu hình tùy biến (`@ConfigurationProperties`), không do Spring quản lý — cơ chế ghi đè vẫn áp dụng như nhau.
- ④ Một link `presigned` còn hiệu lực trong bao lâu.

9.3 Presigned URL: mẫu đáng để biết

Cách tải file ngay thơ là dẫn byte *xuyên qua* service: trình duyệt → service → MinIO và ngược lại, chiếm một thread ứng dụng cho mỗi lượt truyền. Cách làm chuẩn của S3: service dùng thông tin đăng nhập của mình để *ký một URL* — bucket, key, hạn hiệu lực, chữ ký HMAC — rồi đưa URL đó cho trình duyệt, và trình duyệt tải *trực tiếp* từ kho lưu trữ. Service tồn vài micro giây; kho lưu trữ làm việc stream; link tự hết hạn. Thủ thuật này cũng dùng được cho upload.

CẠM BẦY — BÀI TOÁN HAI ENDPOINT

Triệu chứng: upload chạy tốt, nhưng bấm vào link tải thì hỏng với ERR_NAME_NOT_RESOLVED trên minio:9000. Nguyên nhân: một presigned URL nhúng sẵn endpoint *mà nó được ký với*. Service ký với minio:9000 — một cái tên chỉ phân giải được bên trong mạng của cụm — còn trình duyệt sống bên ngoài. Vì thế mới có hai endpoint cấu hình: ký link tải với public-endpoint, nói chuyện với kho lưu trữ qua endpoint. Mọi hệ thống S3 đóng container đều gặp lỗi này một lần; một số gặp lại lần nữa với chính chữ ký, vì chữ ký bao trùm cả host — bạn không thể ký với host này rồi sau đó viết lại URL sang host khác.

TRÊN PHẦN CỨNG NÀY

MinIO ở đây là một container đơn với một PVC đơn — ổn cho bài nộp trong một homelab, chẳng giống gì MinIO production (nhiều node, erasure coding) hay độ bền mười một số chín của S3. Điểm thú vị là course-service không phân biệt được: cùng những lời gọi SDK đó sẽ chạm tới S3 thật chỉ với endpoint và thông tin đăng nhập thay đổi. Đó chính là ý nghĩa của việc nói một API chuẩn.

9.4 Ngoài phạm vi dự án

Cùng service này trên AWS thật: đặt endpoint và public-endpoint thành URL S3 thật (hoặc để mặc định của SDK), thay key tính bằng một IAM role mà pod đảm nhận (IRSA trên EKS — hoàn toàn không có thông tin đăng nhập sống lâu), và bật versioning cho bucket cùng các quy tắc vòng đời (tự động chuyển bài nộp cũ sang tầng lưu trữ rẻ hơn). Các khái niệm — bucket, key, ký trước, danh tính thực hiện ký — chuyển sang nguyên vẹn; thứ thay đổi là ai quản lý độ bền và danh tính được chứng minh ra sao.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao presigned URL mở rộng tốt hơn việc proxy tải file qua service, và chúng đưa vào kiểu lỗi mới nào?
2. Giải thích vì sao endpoint và public-endpoint phải khác nhau bên trong cụm, và vì sao viết lại URL sau khi ký không phải là cách sửa.
3. Một presigned URL bị lộ trong log chat. Điều gì giới hạn thiệt hại, theo thiết kế?
4. Điều gì thay đổi — và điều gì không — khi code này chuyển từ MinIO sang S3 thật?

Phần III

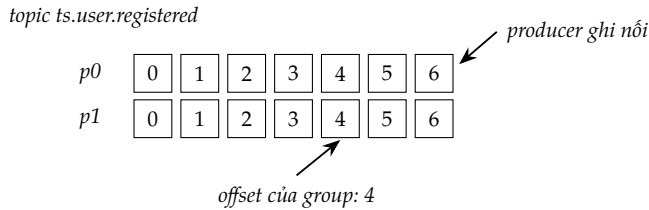
Truyền thông điệp

Kafka: Mô hình lỗi

Kafka là công nghệ đáng học sâu nhất trong stack này: nó xuất hiện trong một phần lớn các mô tả công việc backend, và mô hình của nó thực sự khác với một hàng đợi thông điệp (message queue). Chương này là Kafka độc lập với nền tảng; Chương 11 là phía Spring.

10.1 Một log, không phải một hàng đợi

Trừu tượng lỗi của Kafka là một *log chỉ ghi nối thêm* (append-only log). Một *topic* (ví dụ `ts.user.registered`) được chia thành các *phân vùng* (partition); mỗi partition là một dãy *record* có thứ tự và bất biến, mỗi record ở một *offset* được đánh số. Record không bị xóa khi đọc — chúng hết hạn theo chính sách retention (dự án này: 168 giờ). Đọc không phá hủy; consumer chỉ nhớ *mình đã đọc tới đâu*.



Hình 10.1: Hai partition; producer ghi nối thêm ở đầu, consumer group nhớ một offset cho mỗi partition.

Các hệ quả, mà một hàng đợi không có cái nào:

- **Phát lại được.** Một consumer mới có thể bắt đầu từ offset 0 và thấy lịch sử. Sang năm triển khai một service phân tích mới; nó có thể xử lý các lượt đăng ký của tuần trước.
- **Nhiều người đọc.** Notification-service và course-service cùng đọc sự kiện auth một cách độc lập, theo nhịp riêng, không giành của nhau — mỗi *consumer group* giữ offset riêng của mình.
- **Có thứ tự trong từng partition.** Các record có cùng *key* bám về cùng một partition và giữ nguyên thứ tự. Đây là lý do các publisher trong dự án này dùng `userId` làm key: mọi sự kiện về một người dùng đến theo đúng trình

tự. Giữa các partition thì không có thứ tự — chỉ thiết kế dựa trên thứ tự theo key.

10.2 Consumer group

Một consumer group là một đội có tên chia nhau việc: mỗi partition được gán cho đúng một consumer *trong group*. Hai consumer trong group notification-group trên một topic 2 partition nhận mỗi bên một partition; consumer thứ ba ngồi không — *partition là đơn vị của tính song song*. Khi một consumer tham gia hoặc chết, group *rebalance*: partition được gán lại và các thành viên tạm dừng trong chốc lát. Các group khác nhau vô hình với nhau, và đó là cách cùng một sự kiện tỏa ra cả notification-group lẫn course-group.

10.3 Broker, replication, KRaft

Một cụm production chạy nhiều *broker*; mỗi partition có một leader trên một broker và các *replica* trên những broker khác (replication-factor, RF). Producer ghi vào leader; `acks=all` chờ cho tới khi các replica đồng bộ (ISR) đã có bản ghi. Việc điều phối metadata trước đây cần ZooKeeper, một hệ phân tán thứ hai; Kafka hiện đại thay nó bằng *KRaft* — các broker tự chạy một quorum Raft nội bộ. Dự án này chạy KRaft với một node đóng cả hai vai broker và controller, RF = 1: ổn cho homelab, khả năng chịu lỗi bằng không — một đĩa hỏng là mất log.

10.4 Mê cung advertised listener

Lỗi Kafka được Google nhiều nhất. Từ `docker-compose.yml`:

```
KAFKA_LISTENERS: PLAINTEXT://:19092,CONTROLLER://:9093,EXTERNAL://:9092 ①
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:19092,EXTERNAL://localhost:9092 ②
KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093 ③
```

- ① *Listener*: các socket mà broker mở. Ba cái: một cho client bên trong mạng Docker, một cho controller KRaft, một công bố ra máy host.
- ② *Advertised listener*: các địa chỉ mà broker *bảo client hãy dùng*. Đây là cái bẫy: yêu cầu đầu tiên của một Kafka client là “leader của các partition ở đâu?”, và broker trả lời bằng những chuỗi này. Container phải được bảo là `kafka:19092` (phân giải được trên mạng Docker); công cụ trên laptop của bạn phải được bảo là `localhost:9092`. Một broker, hai đối tượng, hai địa chỉ advertised.

- ③ Quorum KRaft gồm một thành viên: node 1 là toàn bộ cuộc bầu cử.

CẠM BẦY — KẾT NỐI ĐƯỢC, RỒI TIMEOUT

Triệu chứng: client kết nối tới địa chỉ bootstrap thành công, rồi mọi send/poll đều thất bại với Connection to node -1 could not be established hoặc treo. Nguyên nhân: bootstrap chạy được, nhưng *câu trả lời metadata advertise* một địa chỉ mà client không với tới (ví dụ laptop của bạn bị bảo là kafka:19092). Địa chỉ bootstrap chỉ mở đầu cuộc trò chuyện; advertised listener mang toàn bộ lưu lượng sau đó. Chẩn đoán bằng cách hỏi broker advertise cái gì — ví dụ kcat -L -b localhost:9092 — và kiểm tra đúng những chuỗi đó có phân giải được từ nơi client chạy hay không.

10.5 Ngữ nghĩa chuyển giao

Mặc định của Kafka là *ít nhất một lần* (at-least-once): producer gửi lại sau khi mất ack, hoặc consumer sập sau khi xử lý nhưng trước khi commit offset, cả hai đều gây trùng lặp — chứ không mất. Hãy thiết kế consumer *lũy đẳng* (idempotent: xử lý cùng một sự kiện hai lần là vô hại): notification-service gửi trùng một email là chấp nhận được; một service thanh toán thì sẽ phải theo dõi ID các sự kiện đã xử lý. Đúng một lần (exactly-once) có tồn tại (producer lũy đẳng + transaction) nhưng trả giá bằng độ phức tạp; hãy biết rằng ít nhất một lần + consumer lũy đẳng là món cơm bữa của cả ngành.

10.6 Ngoài phạm vi dự án

Kafka được quản lý (AWS MSK, Confluent Cloud) bỏ đi việc vận hành broker và bán cùng giao thức đó — client của bạn đổi một chuỗi bootstrap và có thêm SASL/TLS. Những cái tên khác trong mảng này giải quyết những bài toán khác: RabbitMQ là một *hàng đợi* định tuyến thông minh (phân phối công việc, ack theo từng thông điệp, không phát lại); Redpanda hiện thực lại giao thức Kafka mà không cần JVM. Khi Chương 25 chuyển broker này vào cụm dưới Strimzi, không gì trong chương này thay đổi — Strimzi là *vận hành* cho cùng một Kafka.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Các sự kiện đăng ký của một người dùng phải được xử lý theo thứ tự, nhưng khả năng mở rộng cũng quan trọng. Bạn dùng gì làm key, bao nhiêu partition, và giới hạn song song là gì?

2. Hai group, notification-group và course-group, cùng đọc ts.user.activated. Giải thích cách cả hai nhận được mọi sự kiện.
3. Phân biệt LISTENERS với ADVERTISED_LISTENERS, và giải thích vì sao lỗi chúng gây ra xuất hiện *sau* một kết nối thành công.
4. Vì sao chuyển giao ít nhất một lần đẩy tính lũy đăng về phía consumer, và chiến lược lũy đăng hợp lý của notification-service sẽ là gì?

Spring Kafka: Producer, Listener, Poison Pill

11.1 Phát sự kiện

Phát một sự kiện trong dự án này gồm hai bộ phận: một `KafkaTemplate` và cấu hình chọn serializer. Từ `auth-service`:

```
kafkaTemplate.send("ts.user.registered", event.userId(), event);
//                topic                key                value
```

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
```

Key (`userId`) là một chuỗi; value là record sự kiện được tuần tự hóa thành JSON. Dùng `user id` làm key là quyết định về thứ tự ở Chương 10 được cụ thể hóa. `send()` là bất đồng bộ — nó trả về một *future*; thông điệp nằm trong bộ đệm phía client cho tới khi broker *ack*. Hai hệ quả vận hành: một service có thể *khởi động* mà không có broker (các lần gửi thất bại về sau, theo từng lời gọi — đó là lý do `auth-service` có thể triển khai lên cụm trước Kafka), và một service sập ngay sau khi `send()` trả về có thể mất thông điệp đang nằm trong bộ đệm — mẫu *outbox* bên dưới tồn tại cho khi điều đó quan trọng.

11.2 Tiêu thụ

```
@KafkaListener(topics = "ts.user.registered",
                groupId = "notification-group")
public void on(UserRegisteredEvent event) {
    mailService.sendActivation(event);
}
```

Spring chạy một vòng lặp poll cho mỗi listener container, giải tuần tự từng record, gọi phương thức, và commit offset sau khi xử lý thành công — ít nhất một lần ngay từ đầu. Nửa consumer của file YAML chứa những dòng liên quan tới production nhất của chương này:

```
spring:
  kafka:
    consumer:
      group-id: course-group
      auto-offset-reset: earliest ①
      key-deserializer: ...ErrorHandlingDeserializer ②
      value-deserializer: ...ErrorHandlingDeserializer
      properties:
        spring.deserializer.value.delegate.class: ...JsonDeserializer
        spring.json.trusted.packages: com.ts.common.dto ③
```

- ① Nơi một group *hoàn toàn mới* bắt đầu khi chưa có offset đã commit: earliest = từ đầu khoảng retention; mặc định latest = chỉ thông điệp mới. earliest nghĩa là một consumer triển khai muộn vẫn xử lý các sự kiện đã phát trước khi nó tồn tại — lựa chọn đúng khi sự kiện là những sự thật cần hành động. Không còn ý nghĩa một khi group đã có offset.
- ② Lá chắn chống poison pill — mục tiếp theo.
- ③ Giải tuần tự JSON khởi tạo các lớp được nêu tên trong header thông điệp; chỉ tin cậy com.ts.common.dto ngăn một producer độc hại hoặc có lỗi khiến consumer khởi tạo lớp tùy ý. Một ranh giới bảo mật, không phải mã lập.

11.3 Poison pill

Một *poison pill* (viên thuốc độc) là thông điệp mãi mãi không giải tuần tự được — lỗi ở producer, một trường bị đổi tên, payload hỏng. Không có phòng thủ, listener ném lỗi *trước khi* thông điệp có thể bị bỏ qua, offset không bao giờ tiến lên, và container thử lại cùng một record không ngừng: partition bị kẹt và log đầy lên với tốc độ tối đa.

ErrorHandlingDeserializer bọc deserializer thật: khi thất bại nó phát ra một lỗi có kiểu thay vì ném ngoại lệ bên trong vòng lặp poll, để error handler của Spring bỏ qua record đó (tùy chọn sau vài lần thử lại, tùy chọn phát nó sang một dead-letter topic). Cách ghép — error handler bên ngoài, JSON delegate bên trong — là mẫu production chuẩn, và là lý do cấu hình nêu tên *hai* lớp deserializer cho mỗi bên.

CẠM BẦY — MỘT THÔNG điệp HỒNG, IM LẶNG HOÀN TOÀN

Triệu chứng: email thông báo dừng hẳn; consumer ghi cùng một stack trace giải tuần tự hàng nghìn lần. Nguyên nhân: một poison pill trên một partition mà không có deserializer xử lý lỗi — vòng lặp không xử lý được cũng không bỏ qua được. Lỗi này ẩn mình trong những hệ thống chưa bao giờ thử với một thông điệp sai định dạng. Hãy kiểm chứng lá chắn của bạn một lần, có chủ đích: phát rác vào một topic dev và xem nó bị bỏ qua.

11.4 Những góc nâng cao

Dead-letter topic. `DefaultErrorHandler` + `DeadLetterPublishingRecordRecoverer`: sau N lần thử thất bại, record rơi vào `<topic>.DLT` để xem xét và phát lại — xử lý lỗi có dấu vết kiểm toán.

Mẫu outbox. “Lưu người dùng và phát sự kiện” là hai hệ thống và không thể nguyên tử một cách trực tiếp. Outbox: ghi sự kiện vào một bảng outbox trong cùng *transaction cơ sở dữ liệu* với hàng người dùng; một relay (hoặc Debezium đọc WAL) phát từ bảng đó. Nếu một ngày auth-service không được phép mất sự kiện đăng ký, đây là câu trả lời.

Schema registry. Các lớp sự kiện dùng chung ở Chương 6 giải quyết việc producer/consumer thống nhất với nhau bên trong một repo. Ở quy mô tổ chức, hợp đồng chuyển vào một schema registry (schema Avro/Protobuf, được kiểm tra tương thích lúc phát) — cùng ý tưởng, nhưng do hạ tầng áp đặt thay vì một jar dùng chung.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao auth-service khởi động được khi Kafka đang sập, và đến thời điểm nào thì một sự cố Kafka trở nên hữu hình với nó?
2. Một consumer group mới được triển khai hôm nay. Giải thích `auto-offset-reset: earliest` khiến nó làm gì, và khi nào thiết lập này không còn tác dụng với group đó.
3. Đi qua kịch bản poison pill khi không có `ErrorHandlingDeserializer`, rồi khi có. Trong một hệ thống production đầy đủ, record hồng cuối cùng nằm ở đâu?
4. Vì sao `spring.json.trusted.packages` là một biện pháp kiểm soát bảo mật?
5. Khi nào mẫu outbox là cần thiết, và nó làm cho cặp thao tác ghi nào trở nên nguyên tử một cách hiệu quả?

Phần IV

Container

Docker: Image, Container, Compose

12.1 Container thực chất là gì

Container không phải là một máy ảo thu nhỏ. Nó là một tiến trình Linux bình thường, được cô lập bởi hai tính năng của kernel: *namespace* (góc nhìn riêng về tiến trình, mạng, các mount của hệ thống file, hostname) và *cgroups* (giới hạn CPU và bộ nhớ). Không có hệ điều hành khách, không có hypervisor — đó là lý do container khởi động trong vài mili giây, và cũng là lý do JVM bên trong phải được báo về giới hạn bộ nhớ của nó (MaxRAMPercentage của Chương 17) — vì nhìn từ bên trong, nó thấy RAM của *máy chủ*.

Image là hệ thống file đã đóng gói cộng với metadata (entrypoint, biến môi trường, cổng khai báo), được build thành một chồng các *layer* bất biến. Layer được định danh theo nội dung và dùng chung: năm mươi image FROM eclipse-temurin:25-jre-alpine chỉ lưu layer JRE một lần. *Registry* (Docker Hub, GHCR, registry:2 của dự án) lưu trữ image; docker push/pull chỉ chuyển những layer mà phía bên kia còn thiếu.

12.2 Dockerfile của dự án, có chú giải

Service nào cũng mang đúng năm dòng này:

```
FROM eclipse-temurin:25-jre-alpine ①
WORKDIR /app
COPY target/*.jar app.jar ②
EXPOSE 8082 ③
ENTRYPOINT ["java", "-jar", "app.jar"]④
```

- ① Image gốc: một JRE (không phải JDK — lúc chạy không cần trình biên dịch) trên Alpine Linux. Image gốc nhỏ hơn = bề mặt tấn công nhỏ hơn và pull nhanh hơn.
- ② Sao chép fat jar do Maven build ra. Điểm tinh tế: Dockerfile này đòi hỏi mvn package phải chạy *trước* — việc build diễn ra bên ngoài. Hệ quả: một lệnh COPY cho jar cỡ ~60 MB tạo ra một layer to, nên *mọi* lần build đều push lại toàn bộ dù chỉ sửa một dòng. Jib (Chương 13) sinh ra để sửa đúng chuyện này.

- ③ Chỉ là tài liệu, không phải hành động — nó không mở gì cả. Việc công bố cổng diễn ra lúc chạy.
- ④ Entrypoint dạng exec: java là PID 1, nhận tín hiệu trực tiếp, nên docker stop kích hoạt quá trình tắt Spring sạch sẽ.

12.3 Mạng và Compose

Các container trên một *bridge network* của Docker tìm thấy nhau bằng *tên service* — Docker chạy một DNS server trả lời cho postgres-course, kafka, minio. Còn localhost, bên trong một container, chính là *container đó* — cái bấy số một với người mới, và là lý do mọi URL mặc định cho dev trong các file cấu hình đều cần ghi đề theo từng môi trường. Cổng chỉ với tới được từ máy chủ khi đã được *công bố* (-p 5434:5432).

Compose biến một loạt lệnh docker run thành một file khai báo duy nhất — docker-compose.yml của dự án là cả stack dev: bốn kho dữ liệu, Kafka, Zipkin, Maildev và bảy service, với ba thành ngữ đáng gọi tên:

```
discovery-server:
  depends_on:
    config-server:
      condition: service_healthy ①
config-server:
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"] ②
postgres-course:
  ports:
    - "5434:5432" ③
  volumes:
    - postgres-course-data:/var/lib/postgresql/data
```

①+② mã hóa thứ tự khởi động — và không chỉ “đã chạy” mà là “đã trả lời endpoint sức khỏe,” điều làm cho thứ tự này có ý nghĩa thật. ③ ánh xạ hai cổng máy chủ khác nhau (5433, 5434) vào cùng cổng nội bộ 5432 của hai container Postgres — tên thì riêng theo từng mạng, cổng máy chủ thì toàn cục. Volume có tên giữ dữ liệu qua những lần tạo lại container; Chương 19 là phiên bản Kubernetes của cùng nhu cầu này.

QUYẾT ĐỊNH — VÌ SAO COMPOSE VẪN SỐNG CẠNH KUBERNETES

Cụm (cluster) là đích triển khai, nhưng Compose vẫn là vòng lặp bên trong: một lệnh, chạy ngay trên laptop, cổng debug đã map sẵn (mọi service đều mở một cổng JDWP trong Compose — gắn debugger của IDE vào 5082 và bước từng dòng qua course-service). Kubernetes mua được tự phục hồi và triển

khai cuốn chiếu bằng cái giá là nghi thức rườm rà; một lập trình viên đang lặt đi lặt lại trên logic nghiệp vụ không muốn trả cả hai. Hai môi trường, một hợp đồng image chung giữa chúng.

CẠM BẦY — CHẠY TỐT TRONG COMPOSE, CHẾT TRONG KUBERNETES

Triệu chứng: stack xanh ở máy local; cùng image đó lại crash-loop trong cụm. Những nghi phạm quen thuộc, đều thuộc về môi trường: một URL vẫn trỏ tới tên service của Compose trong khi tên Service bên k8s khác đi; một `depends_on` mà cụm không tôn trọng (Kubernetes không có thứ tự khởi động — Chương 15); hoặc giới hạn bộ nhớ có trong cụm nhưng không có ở local, biến một JVM béo thành OOMKill. Image là y hệt — hãy so *môi trường*, không phải code: `docker inspect` với `kubectl describe pod`.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Container và VM đều cô lập. Nêu hai cơ chế container dùng, và hai hệ quả của việc không có kernel khách.
2. Vì sao `localhost:5432` hỏng bên trong container dù chạy được trên laptop của bạn, và Compose với Kubernetes đưa ra hai cách sửa khác nhau nào?
3. Cả hai container Postgres đều lắng nghe cổng 5432 nội bộ mà không xung đột — giải thích. Xung đột lại có thể xảy ra ở đâu?
4. Điều gì khiến `condition: service_healthy` mạnh hơn `depends_on` thường, và file nào của dự án cung cấp tín hiệu sức khỏe mà nó chờ?

Image không cần daemon: Jib và registry cục bộ

13.1 Vì sao không cứ `docker build` trong CI?

Các Dockerfile vẫn chạy tốt. Ba lý do khiến pipeline build image theo cách khác:

1. **CI không cần Docker daemon.** `docker build` đòi hỏi một daemon đang chạy; cấp cho Jenkins một daemon nghĩa là `docker-in-docker` hoặc mount socket Docker của máy chủ — tức là quyền root trên máy chủ, trao cho mọi build. Jib build và push image bằng Java thuần: không daemon, không đặc quyền.
2. **Phân layer khớp với cách code thay đổi.** Dockerfile sao chép một fat jar: một layer khổng lồ, push lại toàn bộ mỗi lần build. Jib tách ứng dụng thành các layer theo mức độ biến động — dependency / resource / class — nên sửa một dòng code chỉ push lại layer class nhỏ. Trên hai nhân CPU và Wi-Fi, đó là khác biệt giữa vài giây và vài phút cho mỗi service.
3. **Sẵn sàng cho OpenShift một cách tình cờ.** Image do Jib tạo chạy bằng người dùng không phải root với quyền file thuộc group 0 — đúng điều mô hình bảo mật UID tùy ý của OpenShift đòi hỏi. Tính khả chuyển miễn phí.

13.2 Cấu hình, có chú giải

Jib được cấu hình một lần trong `pluginManagement` của `pom.xml` cha; mỗi module chạy được tham gia bằng một tham chiếu `<plugin>` tron, còn common *rút lui* bằng `jib.skip=true`:

```
<plugin>
  <groupId>com.google.cloud.tools</groupId>
  <artifactId>jib-maven-plugin</artifactId>
  <version>3.4.4</version>
  <configuration>
    <from><image>eclipse-temurin:25-jre-alpine</image></from> ①
    <to><image>${docker.registry}/${project.artifactId}:${project.version}</image></to> ②
    <allowInsecureRegistries>true</allowInsecureRegistries> ③
    <container>
      <mainClass>${start-class}</mainClass> ④
```



```
<creationTime>USE_CURRENT_TIMESTAMP</creationTime> ⑤
</container>
</configuration>
</plugin>
```

- ① Cùng image gốc với các Dockerfile — hành vi lúc chạy không đổi, chỉ khác đường đóng gói.
- ② Phân giải thành localhost:5000/course-service:1.0.0-SNAPSHOT. Tên được viết từ *góc nhìn của cụm* — Mục 13.3.
- ③ Registry cục bộ dùng HTTP thường; Jib từ chối push không an toàn trừ khi được bảo. Mọi công cụ trong chuỗi này (Jib, containerd) đều chọn an-toàn-mặc-định như nhau và đều cần cùng một lời cho phép tường minh.
- ④ Đặt tường minh vì bộ quét bytecode đi kèm Jib chưa đọc được file class của Java 25 — một ví dụ thật về việc Java quá mới làm khổ bộ công cụ.
- ⑤ Jib mặc định dùng mốc thời gian epoch-0 để build tái lập được; ở đây ta đánh đổi điều đó để docker images hiển thị thời gian build thật.

13.3 Registry, và vì sao lại là localhost:5000

```
docker run -d --restart=unless-stopped --name registry \
-p 5000:5000 -v registry-data:/var/lib/registry registry:2
```

Sự thật không hiển nhiên buộc phải có kiến trúc này: **k3s không nhìn thấy image của Docker**. k3s chạy containerd riêng với kho image riêng; docker build trên máy chủ đặt image vào nơi k3s không bao giờ ngó tới. Registry là giao diện trung thực giữa bên build và cụm — thứ mà pipeline đăng nào cũng cần.

Tên image không chỉ là cái tên — nó là *pull từ đâu*: localhost:5000/course-service nghĩa là “pull từ localhost:5000.” Vì bên pull chính là node k3s, và Jenkins (bên push) dùng chung network namespace với máy chủ (Chương 22), cùng một chuỗi này hợp lệ cho cả hai — và localhost không bao giờ trôi như IP LAN hay hostname có thể. Hai điểm cấu hình làm nó chạy được:

```
# /etc/rancher/k3s/registries.yaml -- containerd assumes HTTPS otherwise
mirrors:
  "localhost:5000":
    endpoint: ["http://localhost:5000"]
```

CẠM BÃY — HTTP: SERVER GAVE HTTP RESPONSE TO HTTPS CLIENT

Lỗi đặc trưng của chương này. Cả Jib (push) lẫn containerd (pull) đều mặc định TLS; registry:2 chỉ nói HTTP thường. Nếu một pod kẹt ở ImagePullBackOff với dòng chữ này trong `kubectl describe pod`, thì mục trong `registries.yaml` đang thiếu — và lưu ý containerd *chỉ đọc nó lúc khởi động*: `systemctl restart k3s` sau khi sửa.

QUYẾT ĐỊNH — REGISTRY CỤC BỘ VS. GHCR

GitHub Container Registry sẽ cho phép pull image từ bất cứ đâu và thêm quét lỗ hổng — với cái giá là đẩy hàng trăm megabyte lên Wi-Fi gia đình mỗi lần build. Registry trong LAN giữ vòng lặp build ở tốc độ ổ đĩa. Quyết định này đảo chiều ngay khi có máy thứ hai cần pull image; khi đó trong manifest không gì thay đổi ngoài tiền tố image.

13.4 Tag, digest, và cái bẫy SNAPSHOT

Tag (:1.0.0-SNAPSHOT, :a1b2c3d) là con trỏ có thể dời; *digest* (@sha256:...) là mã băm nội dung bất biến. Pipeline push mỗi build với hai nhãn: tag SNAPSHOT (mà manifest tham chiếu) và tag git-SHA (mà các lần deploy ghim vào — Chương 23). Kubernetes cache image theo tag; một SNAPSHOT push lại sẽ bị node lặng lẽ bỏ qua — đó là lý do mọi Deployment đều đặt `imagePullPolicy: Always`: kiểm tra lại registry mỗi lần khởi động; nếu digest khớp, phép kiểm tra chỉ tốn một lần tải manifest.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Nếu ba vấn đề CI mà Jib giải quyết so với build dựa trên daemon, và vấn đề nào là chuyện bảo mật.
2. Vì sao tên image phải viết từ góc nhìn của *cụm*, và điều gì hỏng nếu Jenkins và k3s hiểu `localhost:5000` khác nhau?
3. Một đồng nghiệp đề xuất `docker save | k3s ctr images import` thay cho registry. Cách đó tốn gì mỗi lần deploy, và stage nào của pipeline tự nhiên ngừng hoạt động?
4. Giải thích cách một :1.0.0-SNAPSHOT push lại cộng với pull policy mặc định tạo ra tình huống “tôi đã deploy mà chẳng có gì đổi,” và cả hai cách sửa.

Phần V

Kubernetes

Kubernetes: Giải phẫu cụm và vòng lặp điều hòa

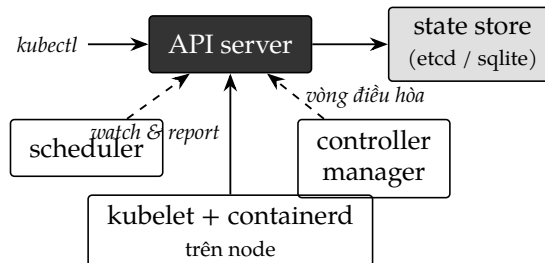
14.1 Ý tưởng duy nhất mà mọi thứ khác treo lên

Bạn không bảo Kubernetes *làm* gì; bạn bảo nó điều gì phải *đúng*. Bạn ghi *trạng thái mong muốn* (“một Deployment tên course-service với 1 replica của image X”) vào cơ sở dữ liệu của nó qua API server; các *controller* chạy vòng lặp điều hòa (reconciliation) mãi mãi, so sánh mong muốn với thực tế và hành động trên phần chênh lệch. Xóa một pod: controller của nó nhận ra thực tế $\neq$ mong muốn và tạo cái khác. Không ai phát lại lệnh của bạn — trạng thái tự hội tụ.

Điều này giải thích những hành vi nếu không sẽ có vẻ lạ:

- `kubectl apply` là một phép gộp vào trạng thái mong muốn, không phải chạy script — apply cùng một file hai lần, lần thứ hai chẳng có gì xảy ra (tính lũy đẳng — thuộc tính mà pipeline dựa vào ở Chương 23).
- Không có “thứ tự khởi động.” Mọi thứ khởi động cùng lúc; service nào cần dependency chưa có sẽ crash và được khởi động lại cho tới khi thể giới sẵn sàng. Crash-loop *trong lúc* hội tụ là vận hành bình thường, không phải lỗi.
- Xóa khỏi file manifest không xóa khỏi cụm — apply chỉ thêm và cập nhật, nó không dọn (prune). Gỡ Ingress course-service cũ đã cần một lệnh `kubectl delete` tường minh.

14.2 Các bộ phận chuyển động



Hình 14.1: Mọi thứ chỉ nói chuyện với API server.

API server cửa trước duy nhất; `kubectl` là một REST client của nó. Xác thực, phân quyền (RBAC của Chương 22) và kiểm tra hợp lệ diễn ra ở đây.

Kho trạng thái (state store) cơ sở dữ liệu trạng thái mong muốn (`etcd` trong cụm chuẩn; `k3s` nhúng `sqlite` khi chỉ có một node).

Bộ lập lịch (scheduler) gán pod chưa được lập lịch vào node, tôn trọng *requests* tài nguyên của Chương 17.

Controller manager các vòng lặp điều hòa: Deployment → ReplicaSet → pod, endpoints, và hàng chục vòng khác.

kubelet agent trên node: theo dõi các pod được gán cho node mình, điều khiển `containerd` để chạy chúng, thực thi probe, báo cáo trạng thái.

`k3s` đóng gói toàn bộ những thứ này vào một binary — một Kubernetes được CNCF chứng nhận, trừ đi chi phí của một VM, cộng thêm pin sẵn: Traefik làm Ingress controller, lưu trữ local-path, `containerd`. Mọi manifest trong sách này áp dụng nguyên xi lên EKS hay OpenShift; chính chứng nhận đó là lý do học trên `k3s` chuyển giao được.

14.3 Object, label, namespace

Mọi object là một tài liệu YAML với cùng bộ khung — `apiVersion`, `kind`, `metadata` (tên, namespace, label), `spec` (mong muốn), `status` (quan sát được, do controller ghi). Hai cơ chế tổ chức:

Namespace phân vùng cụm; dự án này đặt mọi thứ trong `ts`, giữ nó tách biệt với `kube-system` và cho phép giới hạn phạm vi RBAC — quyền của Jenkins dừng lại ở ranh giới namespace (Chương 22).

Label cặp khóa/giá trị tùy ý; *selector* truy vấn chúng. Mọi mối nối trong Phần V đều là một label selector: Service tìm pod theo label, Deployment sở hữu pod theo label. Kustomization đóng dấu `app.kubernetes.io/part-of: teacher-supporter` lên mọi thứ để một selector có thể liệt kê — hoặc xóa — cả ứng dụng.

14.4 Đưa kubectl tới cụm

`kubectl` đọc `~/.kube/config`: URL của cụm, chứng chỉ CA và một thông tin đăng nhập. Từ Windows, dự án này với tới `https://100.85.66.43:6443` qua Tailscale — điều đòi hỏi cài `k3s` với `--tls-san 100.85.66.43`, vì chứng chỉ của API server chỉ hợp lệ cho những tên được nướng vào lúc cài đặt.

THỰC HÀNH — CẢM NHẬN VÒNG LẶP

```
kubectl -n ts get pods
kubectl -n ts delete pod -l app=course-service
kubectl -n ts get pods -w
```

Xóa pod, rồi quan sát: một pod thay thế xuất hiện trong vài giây, được tạo không phải bởi lệnh của bạn mà bởi ReplicaSet controller nhận ra thiếu một replica. Bạn chưa hề ra lệnh khởi động lại — bạn làm trạng thái thực tế lệch khỏi mong muốn, và vòng lặp đã khép lại khoảng cách đó.

CẠM BÃY — CERTIFICATE IS VALID FOR... NOT 100.85.66.43

Triệu chứng: kubectl từ Windows thất bại khi kiểm tra TLS trong khi chạy tốt trên server. Nguyên nhân: chứng chỉ API server thiếu địa chỉ tailnet trong SAN — nó được cài mà không có --tls-san. Sửa lúc cài đặt chỉ là một cớ; sửa sau đó nghĩa là xóa thư mục TLS của server và khởi động lại k3s. Một trường hợp cụ thể của quy tắc chung: chứng chỉ gắn với *tên*, nên hãy quyết định mọi tên mà server sẽ được gọi *trước* khi cấp.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Giải thích “khai báo”: bạn viết gì, ai hành động trên nó, và vì sao apply cùng một manifest hai lần không thay đổi gì?
2. Vì sao Kubernetes cố ý không có depends_on, và điều gì thay cho thứ tự khởi động trong lúc hội tụ?
3. Lần theo kubectl -n ts delete pod ... qua các thành phần: ai nhận, ai nhận ra, ai hành động, trên máy nào?
4. Vì sao gỡ Ingress cũ khỏi git không gỡ nó khỏi cụm, và hai lệnh nào đều có thể gỡ được nó?

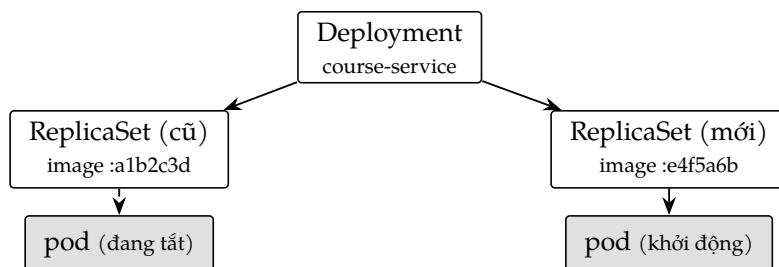
Workload: Pod, Deployment, StatefulSet

15.1 Pod

Pod là đơn vị mà Kubernetes lập lịch: một hoặc nhiều container dùng chung network namespace (một IP, localhost giữa chúng) và volume. Hầu như mọi pod ở đây chỉ có một container; các mẫu nhiều container (sidecar làm proxy hay đẩy log) mới quan trọng khi bạn gặp service mesh. Pod *sinh ra để chết* — chúng không bao giờ được chữa, chỉ được thay, và IP của chúng chết theo. Mọi thứ còn lại trong Phần V — controller, Service, PVC — tồn tại để xây hành vi bền vững từ những pod dùng xong bỏ.

15.2 Deployment và rollout

Bạn không bao giờ tạo pod trần. Một *Deployment* khai báo “N replica của mẫu pod này”; nó quản lý một *ReplicaSet* (bộ đếm), và *ReplicaSet* quản lý các pod. Ba tầng gián tiếp là thứ làm cho việc cập nhật trở nên thanh lịch: đổi mẫu (một tag image mới), *Deployment* tạo một *ReplicaSet mới*, tăng dần nó lên trong khi cái cũ giảm xuống:



Hình 15.1: Một rolling update đang giữa chừng: hai *ReplicaSet*, lưu lượng đang dịch chuyển.

Rollout chỉ tiến tới khi pod mới trở nên *Ready* (probe của Chương 17 — đây là nơi chúng chứng tỏ giá trị), nên một image hỏng làm rollout dừng lại thay vì kéo sập service; và `kubectl rollout undo` trở ngược về *ReplicaSet* trước, thứ được giữ lại chính là vì mục đích này. `kubectl rollout status` của pipeline (Chương 23) là CI đang chờ bộ máy này kết thúc.

Với một replica — cụm này — rolling update vẫn nghĩa là khởi-động-mới-rồi-tắt-cũ, một khoảng chổng lẩn hai pod ngăn ngửi tổn một thoáng gấp đôi bộ nhớ. `maxUnavailable/maxSurge` tinh chỉnh điều nhảy đó ở quy mô lớn.

15.3 StatefulSet: khi danh tính quan trọng

Với cơ sở dữ liệu, tính hoán đổi được là hoàn toàn sai: pod nào lên cũng phải là *đúng Postgres đó với đúng dữ liệu đó*. StatefulSet cung cấp thứ mà Deployment từ chối: tên ổn định (`postgres-course-0`, không phải hậu tố ngẫu nhiên), PVC riêng cho từng replica qua `volumeClaimTemplates` (Chương 19) sống lâu hơn pod và gắn lại vào pod kế nhiệm, và thay thế có thứ tự, từng cái một. Từ `postgres-course.yaml`:

```
kind: StatefulSet
spec:
  serviceName: postgres-course
  replicas: 1
  volumeClaimTemplates:      # a PVC per replica, not per manifest
  - metadata: { name: data }
    spec:
      accessModes: [ReadWriteOnce]
      resources: { requests: { storage: 5Gi } }
```

Quy tắc quyết định: *tiến trình* có giữ trạng thái mà tiến trình kế tiếp phải thừa kế không? Cơ sở dữ liệu thì có; Spring service thì không — trạng thái của chúng nằm trong các cơ sở dữ liệu kia, điều làm chúng thành Deployment, và điều khiến chúng mở rộng và rollout được. (Tính phi trạng thái của JWT ở Chương 5 chính là thuộc tính này ở tầng trên.)

CẠM BÃY — MẤT DỮ LIỆU KIỂU CƠ-SỞ-DỮ-LIỆU-TRONG-DEPLOYMENT

Triệu chứng: ai đó chạy cơ sở dữ liệu như một Deployment với volume thường; chạy tốt hàng tháng; rồi một sự cố node tạo ra replica *thứ hai* trở vào cùng bộ file, và cơ sở dữ liệu hỏng. Nguyên nhân: hợp đồng của Deployment là “các replica hoán đổi được” và rolling update của nó chạy cũ và mới *đồng thời* trong chốc lát — chí mạng khi cả hai cùng mở một thư mục dữ liệu. Thứ tự của StatefulSet + claim riêng từng replica tồn tại để điều này bất khả. Bài học khái quát: chọn controller theo *hợp đồng* của nó, không phải theo file YAML nào bạn thấy trước.

15.4 Các loại workload khác

Cho đầy đủ, các controller còn lại, chưa cái nào được dùng ở đây: **DaemonSet** (một pod mỗi node — bộ gom log, metric của node), **Job** (chạy tới khi xong — Flyway-như-bước-CI của Chương 7 sẽ là một Job), **CronJob** (Job theo lịch — sao lưu, mảnh ghép còn thiếu của Chương 19).

THỰC HÀNH — XEM MỘT ROLLOUT DIỄN RA

```
kubectl -n ts get rs -l app=course-service          # one ReplicaSet
kubectl -n ts set image deployment/course-service \
  course-service=localhost:5000/course-service:1.0.0-SNAPSHOT
kubectl -n ts rollout status deployment/course-service
kubectl -n ts get rs -l app=course-service          # two -- old kept at 0
kubectl -n ts rollout undo deployment/course-service
```

Lệnh `get rs` thứ hai là điểm mấu chốt: ReplicaSet cũ vẫn tồn tại, scale về không — đích của `undo`, mặc định được giữ cho mười bản sửa đổi gần nhất.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao Deployment quản lý ReplicaSet thay vì quản lý pod trực tiếp? Tính năng nào rơi ra từ tầng gián tiếp đó?
2. Phát biểu khác biệt về hợp đồng giữa Deployment và StatefulSet, mỗi cái một câu, rồi phân loại: Eureka server, một cache Redis, một Kafka broker, api-gateway.
3. Trong rolling update, điều kiện nào quyết định việc tắt pod cũ, và bộ máy của chương nào cung cấp điều kiện đó?
4. Giải thích chính xác cơ chế khiến chạy Postgres dưới Deployment rước lấy hỏng dữ liệu trong khi StatefulSet thì không.

Cấu hình và Secret trong cụm

16.1 Hai cơ chế tiêm cấu hình

Phần I đã xác lập rằng biến môi trường thắng mọi file. Kubernetes cung cấp hai cách để đặt chúng, và các manifest dùng cả hai một cách có chủ đích:

```
env:                                     # literal, visible in git
- { name: SPRING_DATASOURCE_URL,
    value: "jdbc:postgresql://postgres-course:5432/ts_course" }
envFrom:                                # every key of a Secret
- secretRef: { name: postgres-course-app }
- secretRef: { name: jwt }
```

Chính sự tách bạch này là ranh giới bảo mật: topology (hostname, cổng, feature flag) không phải bí mật và thuộc về git, nơi có thể review; thông tin đăng nhập (credential) không bao giờ xuất hiện trong git. Khi đọc một Deployment ở đây, mọi mục `env:` là cấu hình công khai và mọi dòng `envFrom:` là con trỏ tới thứ được tạo ngoài luồng.

16.2 Secret: là gì và không là gì

Secret là một object thuộc namespace, chứa các cặp key/value được encode base64. Hai điều thắng thắn về nó:

- Base64 là *encoding*, không phải mã hóa. `echo cm9vdA== | base64 -d` lộ ngay mật khẩu. Thứ bảo vệ nó là kiểm soát truy cập (RBAC quyết định ai được đọc Secret — để ý Chương 22 không cấp cho Jenkins verb nào như vậy), vòng đời tách khỏi code, và tùy chọn mã hóa khi lưu (encryption at rest).
- Nó vẫn là chỗ chứa đúng: mọi cơ chế cao hơn nó (Sealed Secrets, Vault, secret manager của cloud) cuối cùng đều vật chất hóa thành một Secret để pod tiêu thụ. Học object thuần trước là học chính giao diện.

16.3 `create-secrets.sh`, có chú giải

Secret ở đây được tạo bằng một script lũy đẳng, không phải manifest:

```
kubectl -n "$NS" create secret generic postgres-course \
  --from-literal=POSTGRES_USER="$PG_USER" \
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_course \
  --dry-run=client -o yaml | kubectl apply -f - ①

kubectl -n "$NS" create secret generic postgres-course-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f - ②

kubectl -n "$NS" create secret generic jwt \
  --from-literal=JWT_SECRET="$JWT_SECRET" ③
```

- ① Thành ngữ tạo-lũy-đăng: create đơn thuần sẽ thất bại nếu Secret đã tồn tại; render bằng `--dry-run` rồi pipe sang `apply` khiến script chạy lại an toàn — tạo mới hay cập nhật, đăng nào cũng hội tụ. Một bài học nhỏ về khai báo-hơn-mệnh-lệnh, dưới dạng shell.
- ② Cùng một credential viết hai lần, theo cách mỗi bên tiêu thụ muốn: image Postgres cần `POSTGRES_USER`; Spring cần `SPRING_DATASOURCE_USERNAME`. Hai Secret nghĩa là `envFrom` của mỗi pod chỉ nhập những tên nó hiểu — không pod nào phải ôm một mớ biến lạ.
- ③ Một Secret được hai Deployment tiêu thụ — `auth-service` dùng nó để ký JWT, `api-gateway` dùng để xác minh. Hợp đồng khóa-chia-sẻ của Chương 5, thể hiện dưới dạng hạ tầng: xoay vòng (rotate) khóa là một lần chạy script cộng hai lần khởi động lại pod, và hai bên tiêu thụ *không thể* lệch nhau.

QUYẾT ĐỊNH — SCRIPT, KHÔNG PHẢI MANIFEST — VÀ KHÔNG NẪM TRONG KUSTOMIZATION

Manifest Secret trong git sẽ đặt credential đã base64 chỉ cách bất kỳ ai có quyền truy cập repo một lệnh `base64 -d`, mãi mãi, trong lịch sử. Script giữ giá trị trong môi trường của người vận hành (`JWT_SECRET=... ./create-secrets.sh`), giá trị mặc định khớp với dev nên hành vi giống hệt, và git chỉ giữ *hình dạng*. Comment trong `kustomization` nói cùng một điều từ phía bên kia: secret cố tình vắng mặt trong `resources`:

16.4 ConfigMap

Người anh em không-bí-mật của Secret — cùng cơ chế, không giả vờ nhạy cảm, cộng thêm một tài năng mà Secret cũng có nhưng ConfigMap dùng nhiều hơn:

mount thành *file*. Dự án này chưa có ConfigMap nào, vì một lý do thẳng thắn: cấu hình không-bí-mật của nó đã chảy qua config-server và biến môi trường. ConfigMap đầu tiên sẽ đến một cách tự nhiên cùng việc cho config-server nghỉ hưu ở Chương 27 — các file configurations/*.yaml được mount vào pod, để Spring đọc trực tiếp và xóa hẳn một service.

CẠM BẦY — ĐỔI SECRET RỒI, CHẴNG CÓ GÌ XẢY RA

Triệu chứng: bạn chạy lại create-secrets.sh với mật khẩu mới; các service vẫn dùng mật khẩu cũ, cho tới khi vài ngày sau một pod nào đó khởi động lại và bắt đầu lỗi xác thực — một sự cố hên giờ. Nguyên nhân: biến môi trường được *tiêm lúc container khởi động*; pod đang chạy không bao giờ thấy Secret cập nhật. Vì vậy xoay vòng theo thiết kế là hai bước: cập nhật Secret, rồi `kubectl -n ts rollout restart deployment ...` cho mọi bên tiêu thụ, ngay lập tức — trong lúc credential cũ và mới đều còn dùng được, nếu hệ thống phía sau cho phép.

16.5 Ngoài phạm vi dự án

Khoảng cách giữa thiết lập này và thực hành công nghiệp là một khái niệm: làm cho secret *giao được qua pipeline* mà không đọc được trong git. Ba nấc trên chiếc thang đó: **Sealed Secrets** (mã hóa bằng khóa công khai của cụm; commit khối đã niêm phong; một controller giải mã trong cụm — tương thích GitOps với một bộ phận chuyển động); **External Secrets Operator** (một manifest Secret là *con trỏ* vào Vault/AWS Secrets Manager, được controller đồng bộ); và **Vault injection** (secret không bao giờ trở thành object Kubernetes). Mỗi cách đều thay thế script; không cách nào thay đổi cách pod tiêu thụ — `envFrom` sống sót qua mọi cuộc di cư, và đó là lý do các manifest đã sẵn sàng cho tương lai.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Base64 không phải mã hóa — vậy hãy kể ba thứ thực sự bảo vệ một Secret, và dự án này đứng ở đâu với từng thứ.
2. Vì sao hai Secret cho một credential Postgres? Việc tách này ngăn được lỗi gì?
3. Mô tả trọn vẹn việc xoay vòng JWT secret trên cụm này: lệnh, thứ tự, và kiểu lỗi khi chỉ làm một nửa.
4. Mục cấu hình nào trong dự án sẽ chuyển sang ConfigMap trước tiên, và điều đó cho service nào nghỉ hưu?

Probe sức khỏe và quản lý tài nguyên

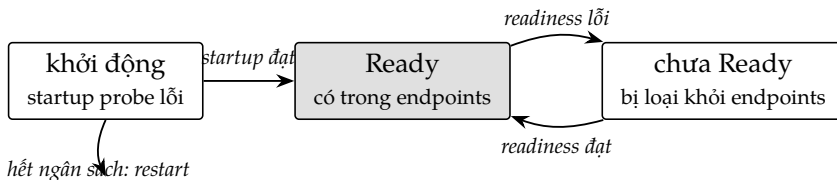
Chương này là trái tim vận hành của Phần V: kubelet quyết định một pod còn sống và sẵn sàng như thế nào, và bộ lập lịch cùng kernel chia hai nhân CPU cho một tá JVM ra sao. Gần như mỗi dòng ở đây đều học được từ việc nhìn một pod thật cứ xử sai.

17.1 Ba loại probe

startupProbe “vẫn đang boot — kiên nhẫn.” Khi nó còn thất bại, các probe khác bị tạm hoãn. Cạn ngân sách thì container bị khởi động lại.

readinessProbe “có thể nhận traffic *ngay bây giờ* không?” Thất bại thì pod bị rút khỏi endpoints của Service — không restart, không kích tính, chỉ là không nhận request mới cho tới khi qua lại.

livenessProbe “tiến trình đã hết cứu được chưa?” Thất bại thì container bị *khởi động lại*. Cái nguy hiểm.



Hình 17.1: Vòng đời của pod dưới con mắt kubelet.

Từ `course-service.yaml`:

```
startupProbe:
  httpGet: { path: /v3/api-docs, port: http } ①
  periodSeconds: 10
  failureThreshold: 12 ②
readinessProbe:
  httpGet: { path: /v3/api-docs, port: http }
  periodSeconds: 5
  failureThreshold: 3
# no livenessProbe ③
```

- ① Quyết định về đường dẫn probe. `/actuator/health` là endpoint sinh ra cho việc này — nhưng `course-service` và `auth-service` không cho nó qua Spring Security, nên nó trả 401 và kubelet tính là thất bại. Đường dẫn không cần xác thực duy nhất chỉ trả 200 *sau khi toàn bộ context đã lên* — Flyway đã migrate, bean đã dựng — là `/v3/api-docs` của `springdoc`. `Dictionary-service` và `gateway` có cho phép `actuator` và probe nó đúng cách; còn `notification-service` không có Spring Security nào cả, nên endpoint `health` của nó mặc định mở — ba service, ba câu trả lời cho cùng một câu hỏi. Quy tắc: đường dẫn probe phải không cần xác thực *và* là tín hiệu readiness thật; một kiểm tra `tcpSocket` qua ngay khi Tomcat bind cổng thì chẳng đạt điều nào.
- ② $12 \times 10\text{ s} =$ hai phút ân hạn khởi động — đo cho một JVM khởi động nguội trên hai nhân bị tranh chấp.
- ③ Cố tình vắng mặt. Liveness giết; thứ duy nhất nó bắt được thêm so với readiness là một JVM treo cứng thật sự — hiếm — trong khi một đợt dừng GC dài trên máy 2 nhân đang tải nặng có thể làm probe thất bại và giết một pod *khỏe mạnh*. Hãy thêm liveness khi bạn đã quan sát được cái treo mà nó sinh ra để bắt.

CẠM BẦY — STARTUP PROBE FAILED: STATUSCODE 404 — HOẶC 403

Triệu chứng (milestone 6, build #6): log của `notification-service` cho thấy Kafka đã gán partition và ứng dụng khởi động hoàn toàn, vậy mà pod không bao giờ Ready; probe báo 404. `dictionary-service` cũng vậy, nhưng 403. Nguyên nhân: các file `config-server` `expose health, info, metrics, prometheus` cho mọi service — nhưng chỉ `config-server`, `discovery-server` và `api-gateway` có `spring-boot-starter-actuator` trên classpath. Cấu hình `expose` cho một thư viện không tồn tại thì lặng lẽ vô hiệu; URL đơn giản là 404. Biến thể 403 là đóng góp của Spring Security: 404 forward sang `/error`, `/error` không nằm trong danh sách `permitAll`, và một chuỗi filter `stateless` không có entry point thì trả 403 — nên triệu chứng chỉ về phân quyền trong khi nguyên nhân là thiếu jar. Sửa: thêm starter vào cả bốn POM của service. Bài học: đường dẫn probe phải được kiểm chứng bằng `curl` từ trong pod *trước khi* tin nó trong manifest — khẳng định trước đó của sách rằng `dictionary-service` “probe health đúng cách” được viết từ `SecurityConfig` của nó, không phải từ một request thật.

CẠM BẦY — VÒNG LẶP RESTART TỰ SINH RA CHÍNH NÓ

Triệu chứng: một service khởi động lại vài phút một lần khi tải nặng, mỗi lần restart làm tải nặng thêm. Nguyên nhân: một liveness probe quá gắt bị timeout trong lúc GC dừng hoặc gọi downstream chậm — probe *gây ra* chính

sự cố nó sinh ra để phát hiện. Kiểu lỗi này là lý do “không có liveness probe” là mặc định có thể biện hộ, và là lý do liveness, khi được thêm, chỉ nên kiểm tra “event loop có còn phản hồi không,” không bao giờ kiểm tra phụ thuộc downstream.

17.2 Requests, limits và JVM

```
resources:
  requests: { memory: 512Mi, cpu: 200m } ①
  limits: { memory: 768Mi } ②
env:
  - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" } ③
```

- ① *Requests* là sổ sách của bộ lập lịch: pod chỉ được đặt vào nơi còn 512 Mi và 0,2 CPU chưa ai nhận. Requests *giữ chỗ*, không giới hạn.
- ② *Limits* là trần do kernel thực thi. Vượt limit bộ nhớ là tiến trình bị OOM-kill — trạng thái `OOMKilled`, exit code 137, không Java exception, không dòng log. Để ý sự bất đối xứng: có limit bộ nhớ, nhưng *không có limit CPU* — xem hộp quyết định tiếp theo.
- ③ JVM nhận biết container định cỡ heap theo tỷ lệ của limit bộ nhớ *của container*: 75% của 768 Mi cho heap, phần còn lại cho metaspace, thread và off-heap. Không có dòng này, giá trị mặc định để lại hoặc lãng phí, hoặc biên an toàn OOM-kill bằng không.

QUYẾT ĐỊNH — VÌ SAO KHÔNG CÓ LIMIT CPU

Limit CPU bóp nghẹt (throttle): container vượt hạn mức bị tạm dừng tới chu kỳ lập lịch tiếp theo — và JVM đang khởi động chính là lúc service muốn từng chu kỳ CPU (nạp class, JIT). Khởi động bị bóp biến 60 giây thành 200 và làm startup probe thất bại: chính limit chế tạo ra thất bại. Khác bộ nhớ, CPU là tài nguyên nén được — tranh chấp làm mọi thứ chậm lại thay vì giết — nên mẫu dùng ở đây là lời khuyên chuẩn: **luôn request CPU, hiếm khi limit**. Bộ nhớ có limit vì kiểu lỗi của bộ nhớ không giới hạn là kernel giết *một ai đó*, và bạn muốn ranh giới ấy đoán trước được.

TRÊN PHẦN CỨNG NÀY

Tổng requests của mọi service xấp xỉ phần còn lại của 19 GB sau khi trừ Jenkins, k3s và các datastore — một cách có chủ đích. Requests là lời hứa với bộ lập lịch; hứa quá tay trên một node duy nhất nghĩa là pod không bao giờ được lập lịch (Pending mãi mãi), hứa thiếu nghĩa là chơi roulette OOM. Trên một node, bạn đang hoạch định dung lượng bằng tay; autoscaler trên cloud biến cùng phép tính đó thành định cỡ node pool.

THỰC HÀNH — XEM MỘT LẦN KHỞI ĐỘNG QUA CON MẮT CỦA PROBE

```
kubectl -n ts delete pod -l app=course-service
kubectl -n ts get pods -w
# Running 0/1 <- startup probe still failing; no traffic yet
# Running 1/1 <- /v3/api-docs answered; endpoints updated
kubectl -n ts describe pod -l app=course-service | tail -n 12
```

Khoảng cách giữa 0/1 và 1/1 là Flyway cộng khởi động context — cửa sổ mà, nếu không có probe, request sẽ đập vào một service mới lên được một nửa.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Với từng probe: thất bại thì chuyện gì xảy ra, và probe nào an toàn để đặt gắt?
2. Vì sao /v3/api-docs là tín hiệu readiness trung thực cho course-service còn tcpSocket: 8082 là lời nói dối? Điều gì khiến /actuator/health không dùng được ở đó, mà dùng được trên dictionary-service?
3. Một pod hiện OOMKilled, exit 137, log sạch. Giải thích cơ chế và vì sao không có Java exception nào. Hai thứ đầu tiên cần kiểm tra?
4. Requests và limits: bộ lập lịch đọc cái nào, kernel thực thi cái nào, và vì sao dự án này giới hạn bộ nhớ mà không giới hạn CPU?

Mạng: Service, DNS, Ingress

18.1 Vấn đề mà pod tạo ra

Pod có IP định tuyến được trong cụm — và mất nó mỗi lần bị thay thế. Chương 15 đã biến pod thành thứ dùng xong bỏ; phải có gì đó ổn định đứng phía trước. Thứ đó là *Service*: một IP ảo và tên DNS cố định, phía sau là danh sách sống các endpoint pod được chọn theo label:

```
apiVersion: v1
kind: Service
metadata: { name: course-service }
spec:
  selector: { app: course-service }    ①
  ports:
    - { port: 8082, targetPort: 8082 } ②
```

① Endpoint controller liên tục liệt kê các pod *Ready* khớp selector này — readiness (Chương 17) là bài kiểm tra tư cách thành viên. ② Traffic tới port của Service được chuyển tiếp tới targetPort của một endpoint được chọn.

DNS của cụm cho mỗi Service một cái tên: `course-service` trong namespace, `course-service.ts.svc.cluster.local` từ bất cứ đâu. Đây là lý do các override biến môi trường trên Kubernetes trông giống hệt bên Compose — cả hai nền tảng đều phân giải tên service trên một mạng riêng; dự án cố tình đặt tên Service k8s theo tên service Compose để các override khớp nhau.

Kiểu Service mặc định, *ClusterIP*, chỉ với tới được từ bên trong cụm — một thuộc tính bảo mật mà Phần II đã dựa vào cho Mongo không mật khẩu. *NodePort* và *LoadBalancer* tồn tại để expose Service trực tiếp; dự án này thay vào đó dồn mọi thứ qua một Ingress.

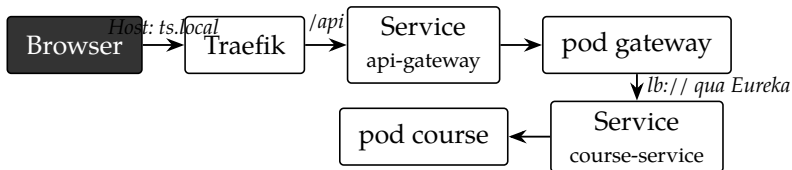
18.2 Ingress và Traefik

Một Service expose một thứ; *Ingress* là cửa trước ở tầng L7: định tuyến HTTP theo host và path, sau này là kết thúc TLS, được khai báo dưới dạng dữ liệu và thực thi bởi một *Ingress controller* — trong k3s là Traefik đi kèm, theo dõi các object Ingress. Từ `api-gateway.yaml`:

```
kind: Ingress
spec:
  rules:
    - host: ts.local                                ①
      http:
        paths:
          - path: /api                              ②
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
# /oauth2 and /login/oauth2 likewise -> api-gateway
```

- ① Quy tắc theo host: áp dụng cho request có header Host là ts.local — một cái tên chỉ tồn tại trong file hosts của Windows, trỏ tới IP tailnet của server. Nhiều host có thể dùng chung cặp cổng 80/443 duy nhất của cụm.
- ② Tỏa theo path. Chỉ ba prefix được vào, tất cả tới gateway; / để trống cho Ingress riêng của frontend (Chương 26), mà Traefik gộp vào bảng định tuyến của cùng host.

18.3 Đường đi trọn vẹn của một request



Hình 18.1: Hai bộ định tuyến, hai hệ thống khám phá, một request.

Đáng dừng lại một chút: tra cứu Eureka của gateway trả về IP pod (prefer-ip-address của Chương 3), nên chẳng thứ tư về kỹ thuật đi vòng qua Service course-service. Hai hệ thống khám phá chạy song song — Eureka cho gateway-tới-service, DNS của Service cho mọi thứ còn lại (config-server, cơ sở dữ liệu, Kafka). Bước đơn giản hóa ở Chương 27 gộp chúng thành một.

CẠM BẦY — SERVICE KHÔNG CHỌN ĐƯỢC GÌ

Triệu chứng: curl tới tên Service bị timeout; pod đang Running và khỏe mạnh. Nguyên nhân #1: lệch label — selector của Service và label trong pod template khác nhau bởi một lỗi gõ; danh sách endpoint rỗng và không ai kêu

ca. Kiểm tra: `kubectl -n ts get endpoints course-service` — cột ENDPOINTS trống chính là lỗi này. Nguyên nhân #2: endpoint có nhưng pod chưa Ready — cùng lệnh đó hiện chúng dưới `NotReadyAddresses`. Lớp trừu tượng Service thất bại *trong im lặng*; object endpoints là nơi nó lộ ra.

QUYẾT ĐỊNH — MỘT HOST INGRESS + FILE HOSTS, KHÔNG DÙNG NODEPORT

NodePort (mỗi service một cổng lạ lẫm) sẽ chạy được qua Tailscale mà không cần trò DNS nào — và tạo ra URL kiểu `100.85.66.43:30082` vừa lộ topology, vừa né gateway, vừa không làm TLS tử tế được. Một host Ingress nghĩa là một địa chỉ cần bảo vệ và trải nghiệm trình duyệt giống hết production; dòng `ts.local` trong file hosts là chi phí duy nhất phía client. Đường nâng cấp — một domain thật cộng cert-manager, hoặc Cloudflare Tunnel — chỉ đổi host của Ingress và không gì khác.

18.4 Ngoài phạm vi dự án

Trên EKS, cùng object Ingress đó, thêm annotation cho controller của AWS, sẽ cấp phát một ALB thật; trên OpenShift nó trở thành Route — cùng ý tưởng, khác danh từ. Mảnh ghép cụm này bỏ qua: **NetworkPolicy** — quy tắc tường lửa cho traffic pod-tới-pod (“chỉ gateway được chạm tới `course-service:8082`”), tức là sự thực thi trong cụm mà mẫu biên-tin-cây của Chương 5 gọi là điểm mềm còn lại của nó. Tin cây mạng phẳng là bình thường với homelab và là cẩu thả với một ngân hàng.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. IP pod đổi mỗi lần thay thế, vậy mà `course-service:8082` luôn chạy. Kể tên từng cơ chế nằm giữa cái tên và socket của pod.
2. Readiness tương tác với endpoints của Service ra sao, và điều đó nghĩa là gì trong một rolling update?
3. Chẩn đoán: Service tồn tại, pod Ready, `get endpoints` hiện danh sách rỗng. Sai ở đâu, và vì sao không có gì báo lỗi?
4. Vì sao `ts.local` cần một dòng trong file hosts, và thứ gì thay thế dòng đó trong hai đường nâng cấp lên production?

Lưu trữ: Volume, Claim và trạng thái trên một node

19.1 Chồng trừu tượng

Hệ thống file của container chết cùng nó. Kubernetes tách *yêu cầu* lưu trữ khỏi việc *cung cấp* nó qua ba object:

PersistentVolume (PV) một mẫu lưu trữ thật — một thư mục, một volume EBS, một export NFS.

PersistentVolumeClaim (PVC) yêu cầu của workload: “5 Gi, ReadWriteOnce.” Claim là thứ pod mount; nó gắn (bind) vào một PV.

StorageClass nhà máy: cách tạo PV theo nhu cầu khi một claim xuất hiện (*cấp phát động*).

Sự gián tiếp này là tính khả chuyển: `postgres-course.yaml` chỉ nói “tôi cần 5 Gi”; còn *làm thế nào* nó thành đĩa là việc của cụm. Cùng manifest đó nhận một thư mục cục bộ trên k3s và một volume EBS trên EKS.

19.2 local-path trên k3s

k3s đi kèm StorageClass `local-path`: một PVC trở thành một thư mục dưới `/var/lib/rancher/k3s/storage/` trên SSD của node. Đơn giản, nhanh, và thẳng thắn về giới hạn — dữ liệu nằm trên *đĩa của node đó*. Trên cụm nhiều node, điều đó ghim pod vào node; trên cụm một node này, nó chỉ có nghĩa đúng như nó nói.

TRÊN PHẦN CỨNG NÀY

`local-path` cung cấp *bền vững* (persistence: dữ liệu sống sót qua khởi động lại pod và node), không phải *bền chắc* (durability: một SSD hỏng là mất tất cả) và không phải *sao lưu* (một lệnh `DELETE` sai lầm không có bản sao nào để quay về). Mảnh còn thiếu, nói thẳng, là một CronJob chạy `pg_dump` lên MinIO hoặc ra ngoài máy. Quy tắc ngón tay cái đáng ghi nhớ: cơ sở dữ liệu không có khôi phục đã kiểm chứng chỉ là một cái cache.

19.3 Hai kiểu claim trong manifest

MinIO (một Deployment với một replica) dùng PVC độc lập; các cơ sở dữ liệu dùng volumeClaimTemplates bên trong StatefulSet — một PVC *đúc riêng cho từng replica* và, quan trọng nhất, *gắn lại vào pod thay thế* chứ không phải vào một pod sống thứ hai. Sự phân biệt đó là nửa lưu trữ của cạm bẫy hồng dữ liệu ở Chương 15.

```
volumeClaimTemplates:
- metadata: { name: data }
  spec:
    accessModes: [ReadWriteOnce] ①
    resources: { requests: { storage: 5Gi } }
# in the pod template:
volumeMounts:
- { name: data, mountPath: /var/lib/postgresql/data }
env:
- { name: PGDATA, value: /var/lib/postgresql/data/pgdata } ②
```

- ① ReadWriteOnce = mount đọc-ghi bởi một *node* tại một thời điểm — vừa khít cho cơ sở dữ liệu. (RWX — nhiều node — cần lưu trữ kiểu NFS và là chế độ người ta muốn cho upload dùng chung, trước khi phát hiện lưu trữ đối tượng, Chương 9, là câu trả lời tốt hơn.)
- ② Mẹo PGDATA, một kinh điển thực thụ: image Postgres từ chối khởi tạo vào thư mục không rỗng, mà volume ext4 sinh ra đã có lost+found. Trỏ PGDATA sâu thêm một thư mục là né được. Bạn sẽ gặp đúng mẫu này trong hàng năm trời StackOverflow về Postgres-trên-Kubernetes.

CẠM BÃY — XÓA STATEFULSET, DỮ LIỆU VẪN CÒN — HOẶC MẤT

Hai bất ngờ trong một. Thứ nhất: xóa StatefulSet *không* xóa PVC của nó — theo thiết kế, dữ liệu sống lâu hơn sai lầm điều phối; tạo lại StatefulSet là nó gắn lại. Thứ hai, bẫy ngược: xóa PVC khi đang thử nghiệm *có* xóa PV và thư mục của nó (reclaimPolicy Delete, mặc định của cấp phát động). Sự bất đối xứng này đáng thuộc lòng trước khi bạn dọn một namespace: object workload là thứ dùng xong bỏ; claim mới là nơi dữ liệu nằm.

THỰC HÀNH — NHÌN XUYÊN QUA LỚP TRỮ TƯỢNG

```
kubectl -n ts get pvc           # claims and their PVs
kubectl get pv                  # cluster-scoped, note the paths
ssh ts-server 'sudo ls /var/lib/rancher/k3s/storage/'
```

```
# pvc-... one directory per PV -- your databases, as plain files
```

Giải ảo chính là mục đích: một PVC trên cụm này là một thư mục. Trên EKS, cùng hai lệnh kubectl đó sẽ hiện ID volume EBS — cùng object, khác nhà máy.

19.4 Ngoài phạm vi dự án

Trên Kubernetes có quản lý, StorageClass ánh xạ sang đĩa cloud (EBS/gp3 trên EKS, Persistent Disk trên GKE) với snapshot, mã hóa và đổi cỡ — object VolumeSnapshot đưa sao lưu vào cùng mô hình khai báo. Nhưng mẫu công nghiệp mà chương này hướng tới lại là phép trừ: các đội ngày càng giữ *cơ sở dữ liệu ngoài cụm* (RDS, Cloud SQL) và để Kubernetes chỉ chạy những thứ phi trạng thái — đổi sự chậm chạp StatefulSet ở Chương 15 lấy một hóa đơn dịch vụ có quản lý. Giờ bạn đã thấy cả hai mặt của cuộc đánh đổi đó.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. PV, PVC, StorageClass: pod mount cái nào, cái nào thuộc phạm vi cụm, và sự gián tiếp mua được gì cho manifest của dự án trên một EKS tương lai?
2. Vì sao ReadWriteOnce là đủ cho Postgres, và loại workload nào đẩy người ta về phía RWX — thường là sai?
3. Giải thích cả hai chiều của sự bất đối xứng khi xóa: StatefulSet mất/PVC còn, PVC mất/dữ liệu mất.
4. Bền vững, bền chắc và sao lưu mỗi thứ nghĩa là gì, và local-path cung cấp thứ nào?

Kustomize: Manifest theo bộ

20.1 Vấn đề

Phần V sinh ra cả tá file YAML. Apply từng file một thì dễ quên mất một file; và ngay khi có môi trường thứ hai (“prod, nhưng 2 replica và secret thật”), copy-paste manifest cho từng môi trường bảo đảm sẽ trôi (drift) cấu hình. Hai công cụ thống trị mảng này; dự án dùng công cụ được tích hợp sẵn trong kubectl.

20.2 Kustomization, có chú giải

`kubectl apply -k deploy/k8s/base` đọc một file duy nhất:

```
# deploy/k8s/base/kustomization.yaml
namespace: ts                                ①
resources:                                    ②
- postgres-course.yaml
- postgres-auth.yaml
- mongodb.yaml
- minio.yaml
# kafka.yaml requires the Strimzi operator (one-time admin step:
# ../install-strimzi.sh) -- without its CRDs, apply fails on kind Kafka.
- kafka.yaml
- kafka-topics.yaml
- maildev.yaml
- config-server.yaml
- discovery-server.yaml
- course-service.yaml
- auth-service.yaml
- dictionary-service.yaml
- notification-service.yaml
- api-gateway.yaml
commonLabels:                                ③
  app.kubernetes.io/part-of: teacher-supporter
```

- ① Đóng dấu lên mọi resource — không manifest nào cần dòng `namespace:` riêng, và chuyển cả ứng dụng sang namespace khác chỉ là sửa một dòng.
- ② Danh sách chính thức. Hai sự vắng mặt có chủ ý dạy nhiều hơn những gì có mặt: `namespace.yaml` bị loại vì nó thuộc phạm vi cluster và Role theo names-

pace của Jenkins không thể apply nó (Chương 22); các Secret bị loại vì những lý do của Chương 16. Một đối tượng thứ ba cũng ở diện chỉ-admin thì thậm chí không phải là file: Strimzi operator và các CRD của nó, được cài một lần bằng `install-strimzi.sh` (Chương 25) — `kafka.yaml` có trong danh sách nhưng không thể apply chừng nào chúng chưa tồn tại. Thứ tự liệt kê chỉ mang tính tài liệu — API server nhận tất cả, còn quá trình reconcile lo phần thực tế.

- ③ Một label đóng lên mọi object, biến cả ứng dụng thành một selector: `kubectl get all -l app.kubernetes.io/part-of=teacher-supporter` — hoặc, trong một buổi diễn tập thảm họa, một lệnh `delete`.

20.3 Base và overlay

Tính năng thật sự của Kustomize xuất hiện cùng môi trường thứ hai. *Base* giữ sự thật; các *overlay* vá lên nó:

```
# deploy/k8s/overlays/prod/kustomization.yaml
resources:
- ../../base
patches:
- target: { kind: Deployment, name: course-service }
  patch: |-
    - op: replace
      path: /spec/replicas
      value: 2
images:
- name: localhost:5000/course-service
  newTag: v1.4.2
```

Không template, không biến — overlay là *phép gộp YAML thuần*. Mỗi lớp đều là YAML Kubernetes hợp lệ mà bạn đọc được, diff được, và đẩy qua `kubectl kustomize` để xem kết quả cuối cùng trước khi bất cứ thứ gì chạm vào cluster. Bộ cục thư mục đã sẵn dạng overlay (*base/*), nên ngày cần overlay *prod*, đó là một lệnh `mkdir`, không phải một cuộc di dời.

QUYẾT ĐỊNH — KUSTOMIZE, KHÔNG PHẢI HELM

Helm là câu trả lời còn lại: manifest dưới dạng template Go (`replicas: {{ .Values.replicaCount }}`) với file giá trị cho từng môi trường, đóng gói thành các *chart* có phiên bản, chia sẻ được. Template mạnh hơn hẳn — vòng lặp, điều kiện, một chart triển khai được ứng dụng của bất kỳ ai — và đó chính là điểm rõ: Helm tỏa sáng khi bạn *phân phối* phần mềm lên cluster của người

lạ; còn để triển khai ứng dụng của chính mình, template khiến mọi manifest không đọc nổi cho đến khi được render. Kustomize giữ YAML thuần đúng là YAML thuần, đi kèm trong kubectl, và phủ hết việc vá replica/image/label — toàn bộ nhu cầu ở đây. Bạn vẫn sẽ dùng Helm chart cho phần mềm bên thứ ba (Strimzi, kube-prometheus-stack); còn viết một chart thì có thể đợi đến khi bạn có người dùng.

CẠM BÃY — SPEC.SELECTOR: FIELD IS IMMUTABLE

Triệu chứng: kubectl apply -k trong pipeline thất bại ở một Deployment với lỗi này, dù cùng file đó apply bằng tay thì ổn. Nguyên nhân: commonLabels đóng label của nó không chỉ vào metadata mà vào cả *selector* của mọi Deployment — và selector là bất biến sau khi tạo. Một object được tạo lần đầu *bên ngoài* Kustomize (kubectl apply -f maildev.yaml lúc thiết lập cột mốc 6, selector {app: maildev}) sẽ không bao giờ được Kustomize cập nhật về sau (selector {app: maildev, part-of: teacher-supporter}). Cách sửa: xóa object tạo bằng tay và để pipeline tạo lại. Quy tắc: object do một kustomization quản lý thì *chỉ* apply thông qua nó — muốn thử một lần, hãy render bằng kubectl kustomize rồi pipe kết quả đó, không bao giờ dùng file thô. (labels: với includeSelectors: false của Kustomize hiện đại tránh được việc chèn vào selector — nhưng đổi sang lúc này sẽ phá mọi selector đã tạo theo cách cũ.)

CẠM BÃY — HÔM QUA CÒN CHẠY: ẢO GIÁC PRUNE

Triệu chứng: bạn xóa một manifest khỏi resources:, apply lại, và object vẫn đang chạy — hàng tháng trời, cho đến khi nó xung đột với thứ gì đó. Nguyên nhân: apply-cả-bộ vẫn là apply — quy tắc của Chương 14 rằng apply không bao giờ xóa vẫn đúng với Kustomize. Ingress bị gỡ ở Chương 4 đã cần một lệnh kubectl delete thủ công. Các cách sửa có hệ thống: kubectl apply -k --prune (vẫn bị chặn sau cờ, và có lý do chính đáng — một selector gắn nhầm label có thể xóa hàng loạt), hoặc các controller GitOps (Chương 27) mà toàn bộ công việc là làm git và cluster hội tụ, kể cả phần xóa.

THỰC HÀNH — RENDER TRƯỚC KHI TIN

```
kubectl kustomize deploy/k8s/base | less      # the final YAML, merged
kubectl kustomize deploy/k8s/base | grep -c 'kind:' # object count
kubectl apply -k deploy/k8s/base --dry-run=server # validate, no changes
```

kubectl kustomize thuần túy là biến đổi văn bản — đọc đầu ra của nó là cách

bạn xem xét một overlay thật sự thay đổi gì, và `--dry-run=server` là thứ gần nhất với một unit test cho manifest.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Một kustomization giải quyết hai vấn đề gì so với `kubectl apply -f` từng file, ngay cả khi không có overlay nào?
2. Vì sao namespace và các Secret cố ý vắng mặt trong `resources:`? (Hai lý do khác nhau.)
3. Helm so với Kustomize: nêu sân nhà của mỗi công cụ trong một câu, và kể tên phần mềm bên thứ ba trong tương lai của dự án sẽ đến dưới dạng chart.
4. Một đồng nghiệp gỡ `mongodb.yaml` khỏi danh sách để “cho nghỉ hưu” nó. Điều gì thật sự xảy ra, và hai quy trình đúng là gì?

Phần VI

CI/CD

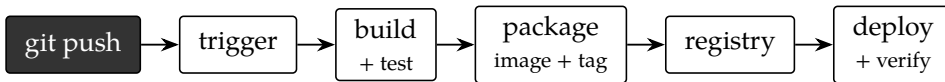
CI/CD: Các khái niệm

21.1 Hai thực hành, một pipeline

Continuous Integration mọi thay đổi được merge vào branch chung thường xuyên và được *kiểm chứng tự động* — biên dịch, test, đóng gói. Sản phẩm của CI là sự tự tin: branch chính luôn ở trạng thái được biết là tốt, và một hỏng hóc bị bắt vài phút sau commit gây ra nó, khi tác giả vẫn còn nhớ ngữ cảnh.

Continuous Delivery/Deployment mọi thay đổi đã kiểm chứng được tự động chuẩn bị để phát hành (Delivery) hoặc thật sự được phát hành (Deployment — không có cổng chặn bởi con người). Dự án này thực hành dạng mạnh hơn: một lần push lên `main` kết thúc bằng các pod đang chạy, không ai bấm nút.

Pipeline là cỗ máy cưỡng chế cả hai:



Hình 21.1: Hình dạng pipeline phổ quát. Tên công cụ thay đổi; các stage thì không.

Dù là Jenkins+Jib+k3s (dự án này), GitHub Actions+Docker+EKS, hay GitLab CI+Kaniko+OpenShift — mã nguồn đi vào, artifact đã test đi vào registry, bộ điều phối được bảo hộ tự về nó, rồi kiểm chứng rằng nó đã làm vậy. Học hình dạng này một lần và pipeline của mọi nhà tuyển dụng chỉ còn là bài tập đổi tên.

21.2 Từ vựng

Trigger thứ khởi động một lần chạy. *Webhook*: máy chủ git gọi CI server khi có push — tức thì, nhưng đòi hỏi CI server phải truy cập được từ internet. *Polling*: CI hỏi repo “có gì mới không?” theo lịch — không cần ai truy cập được, đổi lại là độ trễ. Dự án này dùng polling (H/5 * * * *: khoảng năm phút một lần) chính vì Jenkins nằm sau NAT ở nhà, không có địa chỉ công khai — topology quyết định trigger, một mẫu hình bạn sẽ gặp ở mọi mạng doanh nghiệp bị khóa chặt.

Stage / step các đơn vị của pipeline: stage là các pha có tên (hiện trên UI, chặn tiến trình); step là các lệnh bên trong.

Artifact bất cứ thứ gì build sinh ra mà đáng giữ lại: file jar, image, báo cáo test. Artifact phải *bất biến và truy vết được* — vì thế image được tag bằng SHA của git, để một pod đang chạy có thể truy ngược về đúng commit mà nó đang chạy.

Test gate tính chất rằng các stage sau chỉ chạy nếu các stage trước đã qua. Một pipeline triển khai dù test thất bại chỉ là một script triển khai với vài bước thừa.

Environment promotion cùng một artifact đi từ dev → staging → prod. Quy tắc sắt: *thăng cấp artifact, không bao giờ build lại* — một lần build lại “cho prod” là một binary khác với cái bạn đã test. Bộ máy biến môi trường của Chương 1 tồn tại chính xác để một image vừa với mọi môi trường.

Rollback lối thoát hiểm. Vì registry giữ các image tag theo SHA và Kubernetes giữ các ReplicaSet cũ (Chương 15), rollback là trở lại một artifact trước đó — vài giây, không phải một lần build lại.

QUYẾT ĐỊNH — TRIỂN KHAI Ở MỌI LẦN PUSH LÊN MAIN — CHO MỘT LẬP TRÌNH VIÊN

Continuous Deployment không cần phê duyệt là lựa chọn đúng ở đây: một lập trình viên, bán kính ảnh hưởng cỡ homelab, và vòng phản hồi nhanh nhất có thể. Cùng lựa chọn đó ở một ngân hàng sẽ sai, không phải vì tự động hóa rủi ro mà vì các thay đổi *chịu quản lý* cần bằng chứng đã được xem xét. Con đường lên cấp giữ nguyên pipeline và thêm cổng: một stage phê duyệt thủ công trước prod, hoặc — Chương 27 — GitOps, nơi “deploy” trở thành một pull request có thể xem xét vào repo cấu hình. Hình dạng pipeline sống sót; chỉ chính sách cổng chặn thay đổi.

CẠM BẦY — PIPELINE XANH MÀ NÓI DỐI

Triệu chứng: lần chạy nào cũng xanh; production vẫn hồng. Các nguyên nhân kinh điển: test bị bỏ qua trong lệnh build (`-DskipTests` — điều mà pipeline *này* làm ở stage build, chấp nhận một cách có ý thức vì `mvn verify` chạy trong CI trên GitHub Actions — hãy biết rõ các cổng của chính mình); stage deploy không kiểm chứng rollout (pipeline này có — `rollout status` làm build thất bại nếu pod không bao giờ Ready); hoặc thành công được đo bằng “các lệnh thoát với mã 0” thay vì “service trả lời request.” Một pipeline xanh là một lời khẳng định; hãy biết chính xác nó khẳng định điều gì.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. CI và CD đưa ra những lời hứa khác nhau. Nêu mỗi lời hứa trong một câu, và chỉ ra stage nào trong pipeline của dự án này thực hiện lời hứa nào.
2. Webhook so với polling: nêu ràng buộc quyết định trong dự án này, và một bối cảnh doanh nghiệp có cùng ràng buộc đó.
3. Vì sao một artifact được thăng cấp không bao giờ được build lại cho từng môi trường, và cơ chế nào từ Chương 1 làm cho việc thăng cấp một-artifact trở nên khả thi?
4. Pipeline của bạn xanh và prod sập. Liệt kê ba cách khác nhau mà một pipeline có thể nói dối, và cách sửa cho mỗi cách.

Jenkins: CI server

22.1 Kiến trúc

Jenkins là một ứng dụng web Java — *controller* — lưu job, credential, plugin và lịch sử build trong một thư mục, JENKINS_HOME, và thực thi build hoặc trên chính nó (qua các *executor*, các khe build song song của nó) hoặc trên các *agent* từ xa. Mẫu hình doanh nghiệp là một controller không build gì cả, đẩy việc ra các agent dùng xong bỏ (thường là pod, qua plugin Kubernetes).

QUYẾT ĐỊNH — BUILD TRÊN CONTROLLER, JENKINS NGOÀI CLUSTER

Hai sự đảo ngược thực hành doanh nghiệp kiểu homelab, cả hai đều là phép tính CPU. Agent pod sẽ tốn chi phí lập lịch và kéo image cho mỗi lần build trên cùng hai nhân mà chính build cần; hai executor của controller dành những chu kỳ đó cho Maven thay vì thế. Và Jenkins chạy trong Docker trên máy chủ, không phải trong k3s: nếu bạn làm hỏng cluster — và trong lúc học, bạn sẽ làm hỏng — thứ có thể dựng lại cluster không được chết cùng nó. Cả hai đều đảo ngược gọn gàng ở quy mô doanh nghiệp: Jenkins trong cluster với pod agent là nội dung của Chương 27, và bản thân cuộc di dời chính là bài học.

22.2 Image của controller, có chú giải

Thay vì bấm chuột cài công cụ vào một Jenkins nguyên bản, dự án nướng toolchain vào một image — `deploy/jenkins/Dockerfile`:

```
FROM jenkins/jenkins:lts-jdk21      ①
USER root
RUN ... temurin-25-jdk ...           ②
RUN ... apache-maven-3.9.9 ...      ③
RUN wget -qO /usr/local/bin/kubect1 ... ④
ENV JDK25_HOME=/usr/lib/jvm/temurin-25-jdk-amd64 ⑤
USER jenkins
RUN jenkins-plugin-cli --plugins \
    git workflow-aggregator ... credentials-binding ⑥
```

① Jenkins LTS chạy trên JDK 21 — phiên bản mà *chính Jenkins* chạy trên đó.

- ② Dự án nhắm tới Java 25, nên một JDK thứ hai được cài cho việc *build*. Lưu ý JAVA_HOME *không* bị đổi toàn cục ⑤ — Jenkinsfile đặt nó cho từng pipeline. Runtime của controller và toolchain build là hai mối quan tâm độc lập; Dockerfile này giữ chúng tách bạch rõ ràng.
- ③ Maven ghim ở 3.9.9 — gói của distro thường lạc hậu.
- ④ kubectl: stage deploy chỉ là binary này cộng với một kubeconfig.
- ⑥ Plugin ghim trong image, không bấm chọn trên UI: hỗ trợ pipeline, git, báo cáo JUnit, và credentials-binding (dùng bởi withCredentials ở Chương 23). Toàn bộ ý nghĩa: *bản thân CI server giờ tái tạo được* — hủy container, build lại từ Dockerfile, và chỉ JENKINS_HOME (một volume có tên) mang trạng thái.

Một cờ lúc chạy trong run.sh xứng đáng có đoạn riêng: --network host. Jib bên trong Jenkins push tới localhost:5000 — và bên trong một container dùng mạng bridge, localhost sẽ là *container*. Mạng host làm Jenkins dùng chung namespace mạng của máy chủ, nên localhost:5000 của nó chính là registry mà k3s kéo image từ đó (hợp đồng đặt tên của Chương 13). Không phải đường tắt — mà là yêu cầu của cách đặt tên image.

22.3 Credential

Jenkins lưu secret trong một kho mã hóa, tham chiếu bằng ID; pipeline gắn chúng vào biến môi trường chỉ trong phạm vi một khối (withCredentials), và bộ che log xóa giá trị của chúng khỏi đầu ra console. Dự án này lưu một credential: kubeconfig-ts, một credential dạng file chứa kubeconfig mà Jenkins dùng để triển khai — và đó là phần thú vị.

22.4 Danh tính triển khai: một ServiceAccount, không phải admin

Con đường lười biếng là đưa cho Jenkins kubeconfig admin của k3s — cluster-admin, được phép xóa mọi namespace. Thay vào đó, deploy/k8s/jenkins-rbac.yaml dựng một danh tính tối thiểu:

```
kind: ServiceAccount          # the identity
metadata: { name: jenkins, namespace: ts }
---
kind: Role                   # namespaced permissions
metadata: { name: jenkins-deployer, namespace: ts }
rules:
  - apiGroups: ["apps"]
```



```

resources: ["deployments", "statefulsets"]
verbs: ["get", "list", "watch", "create", "patch", "update"] ①
- apiGroups: ["apps"]
  resources: ["replicasets"]
  verbs: ["get", "list", "watch"] ②
# + services, PVCs, secrets, configmaps, ingresses, pods/log ...
- apiGroups: ["kafka.strimzi.io"] ③
  resources: ["kafkas", "kafkanodepools", "kafkatopics"]
  verbs: ["get", "list", "create", "patch", "update"]
---
kind: RoleBinding                                # identity <- permissions

```

- ① Các verb được suy ra từ chính các lệnh của pipeline: `kubectl apply` cần `get+create+patch`; `set image` cần `get+patch`. *Không có delete ở bất cứ đâu*: CI chỉ tiến tới; con người mới xóa. Một pipeline bị chiếm quyền vì thế có thể làm hỏng các Deployment trong ts nhưng không thể quét sạch chúng, và không thể nhìn ra ngoài namespace — Role, không phải ClusterRole, là quyết định về bán kính ảnh hưởng.
- ② Phát hiện theo cách trung thực: `rollout status` theo dõi các ReplicaSet của Deployment, nên cần quyền đọc chúng — loại quyền bạn tìm ra bằng cách chạy pipeline với Role đó và đọc lỗi Forbidden đầu tiên.
- ③ Thêm cùng cột mốc 6: pipeline được phép khai báo cluster và topic Kafka (custom resource theo namespace), nhưng cài operator và các CRD của nó — phạm vi cluster — vẫn là việc của admin (Chương 25). RBAC lớn thêm một rule cho mỗi API group mà pipeline chạm tới; file Role là nhật ký thay đổi của những gì CI đã được tin cậy.

Kubeconfig lưu trong Jenkins đơn giản ghép địa chỉ cluster với token của ServiceAccount này thay vì chứng chỉ admin.

CẠM BÃY — FORBIDDEN: CANNOT CREATE RESOURCE "NAMESPACES"

Triệu chứng: stage deploy chết với lỗi Forbidden trên namespaces dù file RBAC trông đã đầy đủ. Nguyên nhân: namespace thuộc *phạm vi cluster*, và một Role theo namespace không thể cấp quyền đó — chính xác là lý do namespace.yaml bị loại khỏi kustomization (Chương 20) và namespace được admin tạo một lần. Gỡ lỗi RBAC nói chung: `kubectl auth can-i patch deployments -n ts --as system:serviceaccount:ts:jenkins` trả lời “việc này có được phép không” mà không cần chạy pipeline.

THỰC HÀNH — CHỨNG MINH RANH GIỚI TỒN TẠI

```
kubectl auth can-i patch deployments -n ts \
  --as system:serviceaccount:ts:jenkins      # yes
kubectl auth can-i delete deployments -n ts \
  --as system:serviceaccount:ts:jenkins      # no
kubectl auth can-i list pods -n kube-system \
  --as system:serviceaccount:ts:jenkins      # no
```

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Những gì nằm trong JENKINS_HOME, và vì sao nướng plugin vào image cộng với một volume cho home làm CI server trở thành thứ dùng xong bỏ được?
2. Vì sao controller chạy hai JDK, và mỗi JDK được chọn ở đâu?
3. Dựng lại danh sách verb RBAC cho một pipeline chỉ chạy kubectl apply -k và rollout status — verb nào trong Role của dự án khi đó có thể bỏ đi?
4. Giải thích --network host trên container Jenkins theo hợp đồng đặt tên image của Chương 13.
5. ServiceAccount, Role, RoleBinding: cái nào là danh tính, cái nào là năng lực, cái nào là mối nối — và vì sao Role-chứ-không-phải-ClusterRole là tiêu đề bảo mật?

Pipeline, từng dòng một

Mọi thứ trong cuốn sách này hội tụ về một file Groovy 70 dòng. Một Jenkinsfile dạng khai báo sống trong chính repository mà nó build — pipeline dưới dạng mã, có phiên bản và được xem xét như mọi thứ khác.

23.1 Phần mở đầu và environment

```

pipeline {
  agent any                                ❶
  options {
    timestamps()
    disableConcurrentBuilds()              ❷
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }
  triggers { pollSCM('H/5 * * * *') }    ❸
  environment {
    JAVA_HOME = "${env.JDK25_HOME}"        ❹
    PATH       = "${env.JDK25_HOME}/bin:${env.PATH}"
    TAG        = "${env.GIT_COMMIT.take(7)}"  ❺
    NS         = 'ts'
    REGISTRY   = 'localhost:5000'
    SERVICES   = 'config-server discovery-server course-service api-gateway auth-service
    ↪ dictionary-service notification-service'
    MODULES    = 'common,config-server,discovery-server,course-service,api-gateway,auth-
    ↪ service,dictionary-service,notification-service'
  }
}

```

- ❶ Bất kỳ executor nào trên controller — thỏa hiệp homelab của Chương 22 gói trong một từ.
- ❷ Hai build cùng lúc trên hai nhân sẽ giẫm đạp lên nhau; các lần chạy xếp hàng cũng được gộp lại (một commit mới trong lúc đang build thay thế những lần đang chờ).
- ❸ Polling vì NAT chặn webhook. Chữ H là phân bố theo hash: “một phút cố định nào đó trong mỗi 5 phút,” để nhiều job không cùng poll vào phút :00.
- ❹ Công tắc JDK theo từng pipeline mà image của controller đã chuẩn bị: build thấy Java 25, bản thân Jenkins vẫn ở 21.
- ❺ Tag triển khai là SHA git rút gọn — mọi image truy vết được về commit của

nó, mọi lần deploy ghim vào một tag bất biến (bài học tag-so-với-digest của Chương 13 được áp dụng).

SERVICES/MODULES sổ đăng ký của pipeline về những gì tồn tại: **MODULES** cấp cho Maven (phân cách bằng dấu phẩy, gồm cả **common** được build nhưng không sinh image); **SERVICES** cấp cho kubectl (phân cách bằng dấu cách, chỉ những thứ triển khai được). Thêm một service vào cluster nghĩa là mở rộng cả hai — bước thủ công duy nhất cho mỗi service mới.

23.2 Ba stage

```
stage('Build') {
  steps {
    dir('Project/TeacherSupporter') {           ①
      sh 'mvn -B -DskipTests -pl ${MODULES} -am verify' ②
    } } }
stage('Push images') {
  steps {
    dir('Project/TeacherSupporter') {
      sh 'mvn -B -DskipTests -pl ${MODULES} -am package jib:build -Djib.to.tags=${TAG}'
      ↪ ③
    } } }
stage('Deploy') {
  steps {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubecfg-ts',
        variable: 'KUBECONFIG')]) {           ④
        sh '''
          set -eu                               ⑤
          kubectl apply -k deploy/k8s/base      ⑥
          for s in ${SERVICES}; do
            kubectl -n ${NS} set image deployment/${s} \
              ${s}=${REGISTRY}/${s}:${TAG}      ⑦
          done
          for s in ${SERVICES}; do
            kubectl -n ${NS} rollout status \
              deployment/${s} --timeout=300s    ⑧
          done
        '''
      } } } }
```

- ① Gốc repository là một monorepo học tập; dự án nằm trong một thư mục con, nên mọi stage đều `dir()` vào đó.
- ② `-pl` (project list) + `-am` (also make dependencies): chỉ build các module được liệt kê và những gì chúng cần — kinh tế học monorepo trên hai nhân. `-B` là

- chế độ batch (không có đầu ra tương tác). -DskipTests là sự làm yếu cồng kềnh được chấp nhận có ý thức từ hộp cạm bẫy của Chương 21.
- ③ Một lệnh build và push cả bảy image — Chương 13 gói trong một dòng. -Djib.to.tags thêm tag SHA bên cạnh tag SNAPSHOT. Chữ package đứng trước jib:build không phải để trang trí — xem hộp cạm bẫy bên dưới; nó đã tồn tại hai lần build thất bại trong ngày cột mốc 5 và 6 ra lò.
 - ④ Credential dạng file hiện hình thành một file tạm; KUBECONFIG là biến môi trường mà kubectl đọc. Ngoài khối này file không tồn tại; trong log nội dung của nó không bao giờ xuất hiện.
 - ⑤ Thất bại khi gặp bất kỳ lỗi nào và khi gặp biến chưa đặt — thiếu nó, một lần apply thất bại sẽ bị lướt qua và “đã triển khai” được báo cáo.
 - ⑥ Hội tụ cluster về các manifest — lũy đẳng (Chương 14), nên chạy lại an toàn ở mỗi build dù có gì thay đổi hay không.
 - ⑦ Trỏ lại từng Deployment vào image tag theo SHA. Đây mới là thứ thật sự kích hoạt rollout — và lưu ý bản thân manifest vẫn giữ tag SNAPSHOT, nên chỉ *apply* thôi sẽ không triển khai lại; việc ghim của pipeline và giá trị mặc định của manifest cùng tồn tại.
 - ⑧ Cổng kiểm chứng: chặn cho đến khi mỗi rollout hoàn tất; làm build thất bại nếu có pod nào không bao giờ Ready trong vòng năm phút. Dòng này là thứ khiến màu xanh của pipeline có nghĩa “đang chạy,” chứ không phải “các lệnh đã thực thi” — nó ủy thác sự thật cho các probe của Chương 17.

CẠM BÃY — COULD NOT FIND ARTIFACT COM.TS:COMMON:JAR:1.0.0-SNAPSHOT

Triệu chứng: stage Build qua, rồi stage Push thất bại ở module đầu tiên phụ thuộc vào common, không phân giải được một file jar mà stage trước vừa build xong. Nguyên nhân: mvn jib:build là một *goal trần* — Maven chạy thẳng nó, không có pha vòng đời nào, nên không có gì trong reactor được biên dịch hay đóng gói trong phiên đó, và các phụ thuộc anh em quay về kho cục bộ, nơi common chưa bao giờ được cài (verify dừng trước install). Các module không có phụ thuộc anh em (config-server, discovery-server) qua trót lọt, khiến lỗi trông như đặc thù của từng module. Cách sửa: đặt một pha lên trước — package jib:build — để reactor đóng gói common trước khi Jib hỏi đến nó. Quy tắc chung: một goal của plugin cần artifact của module khác phải đi cùng một pha vòng đời, không được gọi một mình.

Hai vòng lặp thay vì một là một điểm đúng đắn nhỏ: mọi rollout khởi động trước, rồi mới đợi tất cả — nếu không, việc chờ service N sẽ tuần tự hóa việc khởi động service N+1, kéo dài tổng thời gian triển khai mà chẳng được gì.

23.3 Xử lý thất bại

```
post {
  success { echo "Deployed ${TAG} to namespace ${NS}." }
  failure {
    dir('Project/TeacherSupporter') {
      withCredentials([...]) {
        sh 'kubectl -n ${NS} get pods || true'
      } } } }
```

Khi thất bại, log build kết thúc bằng một danh sách pod — lệnh đầu tiên mà một con người sẽ chạy, đã được chạy sẵn. Phần `|| true` giữ cho chính bước chẩn đoán không che mất trạng thái thoát thật của build.

23.4 Cùng pipeline đó, ở nơi khác

Để hiệu chỉnh, đây là hình dạng y hệt dưới dạng GitHub Actions:

```
on: { push: { branches: [main] } }      # webhook, not polling
jobs:
  deploy:
    runs-on: ubuntu-latest                # disposable agent, not controller
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 25 }
      - run: mvn -B -pl $MODULES -am verify
      - run: mvn -B -pl $MODULES -am jib:build -Djib.to.tags=${GITHUB_SHA::7}
      - run: kubectl apply -k deploy/k8s/base && ...
      env: { KUBECONFIG_DATA: ${ secrets.KUBECONFIG } }
```

Cùng các stage, cùng cách đánh tag, cùng ý tưởng gắn secret. Khác biệt nằm ở sự đánh đổi hạ tầng: runner của GitHub là các VM dùng xong bỏ (agent làm đúng cách, miễn phí) nhưng chúng sống trên internet — nên biến thể này sẽ cần registry và cluster truy cập được từ bên ngoài, đúng rằng buộc đã định hình thiết kế homelab. Pipeline thì khả chuyển; *topology mạng mới là thứ thật sự khác nhau giữa các nơi.*

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao tag các lần deploy bằng SHA git khi tag SNAPSHOT cũng tồn tại? Nếu chế độ lỗi mà deploy chỉ-SNAPSHOT mời gọi (Chương 13) và khả năng truy vết mà SHA bổ sung.

2. Giải thích `-pl ... -am` và nó tiết kiệm gì trong một monorepo. Điều gì hỏng nếu bạn quên mở rộng `MODULES` cho một service mới?
3. Dòng đơn lẻ nào khiến thành công của pipeline này có nghĩa “các pod đã Ready,” và nó dựa vào bộ máy của chương nào?
4. Vì sao dùng hai vòng lặp trong stage `deploy` thay vì `set-image-rồi-đợi` cho từng service?
5. Liệt kê ba thứ thay đổi và ba thứ không đổi khi Jenkinsfile này được chuyển sang GitHub Actions.

Phần VII

Vận hành và chặng đường phía trước

Vận hành stack: Cẩm nang gỡ lỗi thực địa

Đây là chương để đọc lại khi có một cluster hỏng trước mặt bạn. Nó tập hợp quy trình gỡ lỗi mà các chương trước đã tích lũy, sắp xếp theo cách sự cố thật diễn ra.

24.1 Bốn câu hỏi

Gần như mọi sự cố Kubernetes đều được định vị bằng cách hỏi, theo thứ tự:

1. **Pod có đang chạy không?** `kubectl -n ts get pods` — đọc cột STATUS và RESTARTS; bảng phân loại bên dưới.
2. **Nền tảng nói gì về nó?** `kubectl -n ts describe pod <pod>` — mục Events ở cuối là kubelet và scheduler kể lại các quyết định của chúng: kéo image, probe thất bại, OOM kill, từ chối lập lịch.
3. **Ứng dụng nói gì?** `kubectl -n ts logs <pod>` (thêm `--previous` cho container vừa chết — cờ hay bị quên nhất trong ứng phó sự cố).
4. **Tôi có tái hiện được góc nhìn của nó về thế giới không?** `kubectl -n ts exec -it <pod> -- sh` rồi thử kết nối mà ứng dụng khẳng định là đang thất bại.

24.2 Bảng phân loại trạng thái

THỰC HÀNH — CỐ Ý LÀM HỎNG — BA BÀI DIỄN TẬP

Phá hoại có chủ đích, vào một ngày yên tĩnh, dạy nhiều hơn bất kỳ sự cố nào:

```
# 1. wrong image tag -> watch ImagePullBackOff, then roll back
kubectl -n ts set image deployment/course-service \
    course-service=localhost:5000/course-service:nope
kubectl -n ts rollout undo deployment/course-service

# 2. scale the database down -> watch dependents degrade honestly
kubectl -n ts scale statefulset/postgres-course --replicas=0
kubectl -n ts get pods -w      # course-service readiness reacts
kubectl -n ts scale statefulset/postgres-course --replicas=1
```

Trạng thái	Ý nghĩa và bước đi đầu tiên
Pending	không lập lịch được: requests vượt quá tài nguyên còn trống, hoặc PVC chưa được gắn. describe cho biết là gì.
ImagePullBackOff	registry không truy cập được, gõ sai tên, hoặc bẫy HTTPS-so-với-HTTP của Chương 13.
CrashLoopBackOff	tiến trình thoát ngay sau khi khởi động; thời gian chờ nhân đôi giữa các lần thử. logs --previous.
OOMKilled (exit 137)	chạm giới hạn bộ nhớ; không có trace Java nào — cú kill từ kernel của Chương 17. Tăng limit hoặc thu nhỏ heap.
Running 0/1	còn sống nhưng readiness thất bại: bình thường khi khởi động; là vấn đề khi kéo dài — thử chính URL của probe.
Completed/Error	cho Job; với pod service, Error nghĩa là tiến trình đã thoát với mã khác 0 một lần.

Bảng 24.1: Cột STATUS đang cố nói với bạn điều gì.

```
# 3. kill Eureka -> verify existing routes survive on client caches
kubectl -n ts delete pod -l app=discovery-server
```

Mỗi bài diễn tập ghép lý thuyết của một chương với hành vi quan sát được của nó: 13 (đặt tên), 17–18 (probe và endpoint), 3 (cache phía client).

24.3 Đọc thể giới của một service từ bên trong

Khi log khẳng định kết nối thất bại, hãy kiểm chứng lời khẳng định đó từ chính vị trí mạng của pod — DNS và khả năng truy cập khác nhau giữa máy bạn và pod:

```
kubectl -n ts exec -it deploy/course-service -- sh
wget -qO- http://config-server:8888/actuator/health # DNS + service up?
nc -zv postgres-course 5432 # TCP reachable?
env | grep SPRING # what was injected?
```

Lệnh cuối cùng đó dần xếp tranh cãi rất nhanh: nó in ra cấu hình mà pod *thật sự* nhận được, hơn hẳn việc đọc lại manifest.

24.4 Thói quen giữ cho cuối tuần được rảnh

Sau mỗi lần deploy pipeline đã kiểm chứng rollout rồi; dù vậy vẫn liếc qua `kubectl -n ts get pods — RESTARTS` nhích dần lên là sự cố ngày mai đang tự báo trước.

Hằng tuần `kubectl top pods -n ts` (cần `metrics-server`, đi kèm trong `k3s`): so sánh mức dùng thực tế với requests của Chương 17 và sửa lại phần `hư cấu`. Kiểm tra đĩa: `df -h` trên máy chủ — `registry` và file log lớn dần; `registry:2` thỉnh thoảng cần dọn rác.

Trước buổi demo *đừng build* — hộp phần cứng của Chương 5: hai nhân nghĩa là một lần build Maven và buổi demo tranh nhau đúng phần CPU đang giữ cho `readiness probe` trả lời.

TRÊN PHẦN CỨNG NÀY

`ufw` trên máy chủ phải miễn trừ các CIDR của pod và service (`10.42.0.0/16`, `10.43.0.0/16`) — chặn tường lửa chúng sẽ phá DNS và lưu lượng pod-tới-pod theo những cách giả dạng lỗi ứng dụng: timeout chấp chờn, tên lúc phân giải được lúc không. Khi mạng trên `k3s` xảy ra chuyện *kỳ lạ*, hãy nghi ngờ tường lửa máy chủ trước ứng dụng. Và `k3s` chỉ đọc `registries.yaml` lúc khởi động — sau khi sửa, `systemctl restart k3s`, lệnh này khởi động lại mọi pod trong chốc lát: đừng làm giữa buổi demo.

CẠM BẦY — GỠ LỖI SAI TẦNG

Thói quen đắt giá nhất khi gỡ lỗi Kubernetes là bắt đầu từ nơi triệu chứng xuất hiện thay vì nơi nguyên nhân sống. Một lỗi 502 trên trình duyệt có thể là Traefik (không có endpoint), gateway (Eureka lạc hậu), service (readiness thất bại), cơ sở dữ liệu (pod bị đuổi), hay đĩa (đầy). Bốn câu hỏi tồn tại để đi qua các tầng theo thứ tự; hãy cưỡng lại cám dỗ mở log ứng dụng trước chỉ vì đó là tầng quen thuộc nhất. `describe` trước `logs` — nền tảng thường đã biết rồi.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Sắp xếp bốn câu hỏi và nêu lệnh cho mỗi câu. Câu nào dùng một cờ mà mọi người hay quên, và khi nào cờ đó quan trọng?
2. `CrashLoopBackOff` SO VỚI `Running` 0/1: các tầng khác nhau đang thất bại — khái niệm probe nào phân biệt chúng?
3. Một service khẳng định không tới được Postgres; từ laptop của bạn Post-

gres trả lời. Bước chẩn đoán tiếp theo là gì và vì sao thành công trên laptop của bạn không liên quan?

4. Vì sao một thay đổi tường lửa máy chủ có thể giả dạng lỗi ứng dụng trên k3s, và hai CIDR nào phải luôn được miễn trừ?

Kafka trong cụm: Strimzi và luồng gửi mail

Cột mốc 6 chuyển broker của Chương 10 từ Compose vào k3s và triển khai service cuối cùng — notification-service — để luồng *sự kiện* đầu tiên chạy trơn vẹn trong cụm: đăng ký một người dùng, nhận một lá mail. Đây cũng là chương mà ý tưởng điều hòa (reconciliation) của Chương 14 mọc thêm phần mở rộng quan trọng nhất của nó: *operator*.

25.1 Operator và custom resource

Kubernetes có sẵn controller cho các kind dựng sẵn — Deployment, Service. Một *operator* là controller cho những kind bạn tự thêm: một *CustomResourceDefinition* (CRD) dạy API server một danh từ mới (Kafka, KafkaTopic), và vòng lặp điều hòa của operator biến các thể hiện của nó thành những object bình thường — pod, Service, ConfigMap — rồi giữ chúng hội tụ. Kiến thức vận hành trước đây nằm trong runbook (cách roll một broker an toàn, cách tính advertised listener, cách thêm partition) nay được mã hóa trong code của operator. Strimzi là operator cho Kafka; mẫu hình này là phổ quát (operator cho Postgres, Prometheus operator, cert-manager).

25.2 Cài operator: một việc của admin

```
# deploy/k8s/install-strimzi.sh (abridged)
kubectl apply --server-side --force-conflicts \
  -f "https://strimzi.io/install/latest?namespace=${NS}" -n "$NS" ①
kubectl -n "$NS" rollout status deployment/strimzi-cluster-operator
```

- ① Gói này chứa các CRD, Deployment của operator và RBAC của nó — phần lớn ở *phạm vi cụm* (cluster-scoped). Hai hệ quả: nó do admin apply chứ không phải Jenkins (Role của Jenkins, Chương 22, dừng ở namespace), và nó được apply *server-side* vì các tài liệu CRD quá lớn cho annotation last-applied phía client. Lũy đẳng, nên cũng chính là lệnh nâng cấp. Tham số `?namespace=` giới hạn operator chỉ theo dõi ts.

Kustomization (Chương 20) liệt kê `kafka.yaml`, nhưng file đó không thể apply cho đến khi script này đã chạy một lần — thành viên thứ ba của bộ “điều kiện tiên quyết do admin làm”, sau namespace và các secret.

CẠM BẦY — NO MATCHES FOR KIND “KAFKA” IN VERSION v1BETA2

Triệu chứng: apply một manifest Kafka chép từ bất kỳ tutorial nào cũng thất bại với lỗi này dù operator đã cài và khỏe mạnh. Nguyên nhân: Strimzi 1.x chỉ phục vụ `kafka.strimzi.io/v1`; phiên bản `v1beta2` mà cả thập kỷ bài blog dùng đã biến mất. Hãy kiểm tra CRD thật sự phục vụ phiên bản nào trước khi tin một ví dụ: `kubectl get crd kafkas.kafka.strimzi.io -o jsonpath={.spec.versions[*].name}`. Dự án này dính lỗi đó ngay ngày cài đặt.

25.3 Cụm, được khai báo

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool                                ①
metadata:
  name: dual-role
  labels: { strimzi.io/cluster: ts-kafka } ②
spec:
  replicas: 1
  roles: [controller, broker]
  storage: { type: persistent-claim, size: 5Gi, deleteClaim: false } ③
  resources:
    requests: { memory: 768Mi, cpu: 200m }
    limits: { memory: 1Gi }
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: ts-kafka
spec:
  kafka:
    listeners:
      - { name: plain, port: 9092, type: internal, tls: false } ④
    config:
      offsets.topic.replication.factor: 1 ⑤
      default.replication.factor: 1
      min.insync.replicas: 1
      log.retention.hours: 168
      jvmOptions: { -Xms: 256m, -Xmx: 512m } ⑥
  entityOperator:
    topicOperator: {} ⑦
```

- ① Hai object: *pool* nói Kafka chạy ở đâu (replica, vai trò, lưu trữ, tài nguyên); *resource* Kafka nói nó hành xử thế nào. Một node KRaft hai vai trò là cách Kubernetes viết `process.roles=broker,controller` của Compose.
- ② Label này nối *pool* với *cụm*; operator khớp theo nó. Sai label thì im lặng — *pool* đơn giản là không bao giờ thành hình.
- ③ Một PVC qua *local-path*, y hệt các cơ sở dữ liệu (Chương 19). `deleteClaim:false`: xóa *resource* Kafka vẫn giữ lại log — cùng sự bất đối xứng mà *StatefulSet* có theo mặc định.
- ④ So với khối ba *listener*, hai *advertised address* trong `docker-compose.yml`: biến mất. Bạn khai báo *một listener nội bộ*; operator tính ra *Service bootstrap* và *Service* cho từng broker, các *advertised address*, và *listener* của KRaft controller. Mê cung của Chương 10 là kiến thức vận hành; giờ nó nằm trong operator.
- ⑤ Một broker duy nhất nghĩa là mọi *replication factor* phải bằng 1, nếu không lần tạo topic đầu tiên sẽ thất bại vì đòi những broker không tồn tại.
- ⑥ Heap mặc định của Kafka giả định một máy chủ; nửa GiB đủ cho lưu lượng nhỏ giọt của homelab và để dành bộ nhớ cho các service JVM.
- ⑦ Nửa topic của *entity operator* theo dõi các object *KafkaTopic* và điều hòa chúng thành topic thật. Nửa user bị bỏ qua: không xác thực, không có user nào để quản lý.

Strimzi đặt tên *Service bootstrap* là `<cluster>-kafka-bootstrap`; vì thế mọi *producer* và *consumer* nhận `SPRING_KAFKA_BOOTSTRAP_SERVERS=ts-kafka-kafka-bootstrap:9092` trong *Deployment* của nó — mặc định `localhost:9092` trong cấu hình của Phần III bị ghi đè theo đúng cách mọi địa chỉ khác đã bị ghi đè.

QUYẾT ĐỊNH — OPERATOR, KHÔNG PHẢI STATEFULSET VIẾT TAY

Một *StatefulSet* KRaft thuần (một pod, một PVC, khối *env* của *compose* chép vào) sẽ tốn ít bộ nhớ hơn — không operator, không *entity operator* — và đó là phương án dự phòng thắng thắn nếu tranh chấp CPU có lúc gây khó. Operator thắng vì đó là thứ các công ty dùng Kubernetes chạy (Red Hat bán nó dưới tên AMQ Streams), vì topic trở thành object khai báo, và vì nâng cấp broker cùng khởi động lại cuốn chiếu trở thành code đã được người khác kiểm thử. Học nó chính là mục đích; phương án dự phòng vẫn được ghi lại trong `STACK-K3S-JENKINS.md`.

25.4 Topic dưới dạng object

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.registered
  labels: { strimzi.io/cluster: ts-kafka }
spec: { partitions: 1, replicas: 1 }
```

Kafka sẽ tự tạo các topic này ở lần dùng đầu tiên; khai báo chúng biến số partition thành một quyết định được review, loại bỏ cuộc đua ngay đầu giữa producer và topic, và biến `kubectl get kafkatopics` thành bản kiểm kê mọi hợp đồng sự kiện trong hệ thống (bốn, khớp với bảng của Chương 6). Mỗi topic một partition: thứ tự toàn cục, nên đương nhiên giữ được thứ tự theo từng người dùng mà các publisher đặt key cho. Partition có thể thêm về sau, không bao giờ bớt được.

25.5 Service cuối cùng, và hộp thư của nó

Manifest của notification-service là Deployment tiêu chuẩn của Chương 15 với ba điểm riêng: địa chỉ bootstrap ở trên; `SPRING_MAIL_HOST=maildev / PORT=1025` ghi đè mặc định `#{MAIL_HOST:localhost}` của cấu hình qua relaxed binding; và các probe trên `/actuator/health` — service này hoàn toàn không có Spring Security, nên endpoint mở theo mặc định (câu trả lời thứ ba cho câu hỏi về probe của Chương 17). Maildev là một Deployment nhỏ xíu kèm Service (cổng 1025 SMTP, 1080 web); nó không gửi đi đâu cả và hiển thị mọi thứ nó bắt được. Không có Ingress: đây là công cụ của người vận hành, truy cập qua port-forward. Nó cũng cung cấp bài học thực chiến thứ hai của cột mốc này: được apply bằng tay trước khi pipeline chạy, selector của nó thiếu label mà Kustomize thêm vào, và lần deploy đầu tiên của pipeline chết với “field is immutable” — cạm bẫy của Chương 20, học theo cách đặt giá.

CẠM BÃY — NOTREADY — FATALPROBLEM, TRONG KHI POD ĐANG RUNNING

Triệu chứng: vài phút sau khi apply, `kubectl get kafka` cho thấy READY trống và một condition ghi “Pod is unschedulable or is not starting” — thế nhưng `get pods` cho thấy broker 1/1 Running. Nguyên nhân: vòng điều hòa của operator hết timeout trong lúc image Kafka 600 MB vẫn đang tải qua Wi-Fi, và ghi lại một trạng thái thất bại; vòng lặp định kỳ *tiếp theo* (mỗi hai phút) quan sát thấy pod đang chạy và chuyển nó sang Ready. Trạng thái của operator chỉ trung thực về cuối cùng. Trước khi phản ứng với một condition đỏ, hãy kiểm tra pod mà nó đang mô tả — và kiểm tra log của operator, nơi các dòng

“reconciled” cho thấy vòng lặp vẫn đang quay.

TRÊN PHẦN CỨNG NÀY

Operator + broker + entity operator tốn khoảng 1,5 GB ở đây, với heap của broker giới hạn 512 MiB. Lần kéo image đầu tiên là phần đắt nhất — hãy chuẩn bị tinh thần mười phút trên Wi-Fi gia đình, và cạm bẫy trạng thái cũ ở trên. Mọi thứ sau đó đều nhanh: image đã được cache trên node, điều mà bài kiểm tra khối bên dưới tận dụng.

THỰC HÀNH — CHỨNG MINH BROKER, RỒI CHỨNG MINH LUỒNG

Một pod dùng xong vút với Kafka CLI, ở cùng vị trí mạng với các service, dùng image node đã có sẵn:

```
kubectl -n ts run kafka-smoke --rm -i --restart=Never \
  --image=quay.io/strimzi/kafka:1.2.0-kafka-4.3.1 -- sh -c '
  B=ts-kafka-kafka-bootstrap:9092
  bin/kafka-topics.sh --bootstrap-server $B --create --topic smoke \
    --partitions 1 --replication-factor 1
  echo hello | bin/kafka-console-producer.sh --bootstrap-server $B --topic smoke
  bin/kafka-console-consumer.sh --bootstrap-server $B --topic smoke \
    --from-beginning --max-messages 1 --timeout-ms 20000
  bin/kafka-topics.sh --bootstrap-server $B --delete --topic smoke'
```

Dùng một topic nháp, không phải topic thật: một chuỗi “hello” thuần văn bản trên `ts.user.registered` là poison pill cho consumer JSON (Chương 11) — được bỏ qua êm thấm nhờ `ErrorHandlingDeserializer`, nhưng việc gì phải phát hiện ra trong production. Rồi đến luồng thật, khi pipeline đã triển khai xong các service:

```
curl -X POST http://ts.local/api/auth/register -H 'Content-Type: application/json' \
  -d '{"email": "t@example.com", "password": "Passw0rd!", "firstName": "Test",
    "lastName": "User", "role": "TEACHER", "activationMethod": "EMAIL"}'
# -> 201 {"message": "Registration successful. Check your email ..."}
kubectl -n ts logs deploy/notification-service | grep -i registered
# maildev's REST API, from inside the cluster (or port-forward 1080 for the UI):
kubectl -n ts run mailcheck --rm -i --restart=Never \
  --image=curlimages/curl:8.10.1 -- -s http://maildev:1080/email
# -> [{"subject": "Activate your TeacherSupporter account", "to": "t@example.com", ...}
```

Request đó đi qua mọi phần của cuốn sách này: Ingress → gateway → auth-service → Postgres → Kafka → notification-service → SMTP. Nó chạy lần đầu ngày 2026-08-25, pipeline build #7 — sau khi các build #3–#6 mỗi lần dạy một

cạm bẫy của chương này và của các Chương 17, 20 và 23. Một đầu mỗi còn lỏng mà nó phơi ra: link kích hoạt trong mail vẫn ghi `localhost:8080`, mặc định dev của URL gốc cho link — một ghi đề env dành cho cột mốc 7, khi frontend cho cụm một địa chỉ công khai thật sự.

25.6 Ngoài dự án

Kafka được quản lý — AWS MSK, Confluent Cloud, endpoint Kafka của Azure Event Hubs — là operator của operator: cùng một khai báo hình dạng Kafka nhưng thực hiện trong console của cloud, với những broker bạn không bao giờ thấy. Chuyển từ đây sang đó là một chuỗi bootstrap cộng SASL/TLS trên listener; các object `KafkaTopic` sẽ nhường chỗ cho resource Terraform nói đúng điều tương tự. Bài học còn lại để mang theo: mẫu hình operator là cách Kubernetes lưu trữ *mọi* thành phần nền tảng có trạng thái — Postgres (CloudNativePG), Redis, Elasticsearch — và khi đã cài, đã dùng sai phiên bản, đã debug trạng thái của một cái, bạn biết hình dạng của tất cả chúng.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Định nghĩa operator theo cách nói về controller của Chương 14, và nêu hai kind object mà Strimzi đã thêm vào cụm này.
2. Vì sao `install-strimzi.sh` là bước của admin trong khi `kafka.yaml` do Jenkins apply? Khái niệm RBAC nào vạch ra ranh giới đó?
3. Liệt kê ba thứ mà khối Kafka của Compose cấu hình bằng tay nhưng resource Kafka không hề nhắc tới, và nói ai tính chúng bây giờ.
4. Bạn thấy `NotReady` / `FatalProblem` trên resource Kafka. Hai thứ bạn kiểm tra trước khi hành động là gì, và vì sao trạng thái đó có thể sai?
5. Vì sao khai báo topic dưới dạng object khi Kafka tự tạo được chúng, và vì sao mỗi topic chỉ một partition?

Frontend và một host duy nhất

Cột mốc 7 đưa thứ có thể triển khai cuối cùng — frontend React — vào cụm và giành lấy đường dẫn / trên `ts.local` mà Ingress của Chương 18 đã cố ý để trống. Đây là một cột mốc ngắn tính theo số dòng YAML và dài tính theo khái niệm: đó là image duy nhất mà pipeline build từ Dockerfile, pod duy nhất không phải JVM, và là thay đổi biến “các service gọi được bằng `curl`” thành “một ứng dụng dùng được trong trình duyệt”.

26.1 Frontend là gì, dưới mắt người vận hành

Một ứng dụng React đã build không phải là một server. `npm run build` (kiểm tra TypeScript, rồi đóng gói bằng Vite) sinh ra một thư mục file tĩnh: một `index.html`, một bundle JavaScript có mã băm nội dung trong tên (`index-CqZING8N.js`), một file CSS, một favicon. Không có gì trong đó chạy trên server; từng dòng của nó chạy trong trình duyệt của khách, rồi trình duyệt gọi `/api/...` qua HTTP như bất kỳ client nào khác. Vậy “triển khai frontend” nghĩa là triển khai một *file server* có chứa những file đó — ở đây là nginx — và bảo router biên biết request nào là file, request nào là API.

Cách nhìn đó quyết định mọi thứ còn lại trong chương này. Không config server (ứng dụng không có thuộc tính nào để import), không Eureka (không ai gọi nó theo tên), không secret (trình duyệt đọc được mọi thứ frontend nắm giữ, nên không được có gì bí mật ở đó), và một dấu chân tài nguyên nhỏ hơn cả chi phí nạp class của JVM nhỏ nhất.

26.2 Hai giai đoạn, một Dockerfile

Jib (Chương 13) build image JVM từ `target/classes`; nó không chạy được `npm`. Vì thế frontend dùng Dockerfile nhiều giai đoạn (multi-stage) kinh điển — và lý do nó có hai giai đoạn cũng chính là lý do các image JVM dùng image gốc JRE thay vì JDK:

```
FROM node:24-alpine AS build           ①
WORKDIR /app
COPY package.json package-lock.json ./ ②
RUN npm ci --no-audit --no-fund
```

```
COPY . .
RUN npm run build

FROM nginx:1.27-alpine ③
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html ④
EXPOSE 80
```

- ① Giai đoạn build có Node, trình biên dịch TypeScript, và sau npm ci là chừng 300 MB node_modules. Không thứ nào trong đó cần cho việc *phục vụ* kết quả.
- ② Lockfile trước, mã nguồn sau — quy tắc cache theo layer của Chương 12. npm ci cài đúng các phiên bản đã ghim trong package-lock.json và từ chối sửa nó; một chỉnh sửa mã nguồn chỉ làm mất hiệu lực các layer từ COPY . . trở xuống, nên việc tải dependency được bỏ qua ở mọi lần rebuild không đụng tới package.json.
- ③ Một image gốc mới tinh. Image cuối cùng là nginx cộng file tĩnh: khoảng 15 MB, không Node, không trình biên dịch, không trình quản lý gói — bề mặt tấn công nhỏ hơn và kéo nhanh hơn, miễn phí.
- ④ --from=build là phép sao chép multi-stage: chỉ dist/ bằng từ giai đoạn một sang giai đoạn hai. Mọi thứ khác ở giai đoạn một bị vứt bỏ khi build kết thúc.

QUYẾT ĐỊNH — SAO KHÔNG THÊM NODE VÀO IMAGE JENKINS RỒI DỪNG JIB?

Jib có thể gói bất kỳ thư mục nào thành image, nên con đường “Node trong Jenkins, Jib để đóng gói” là có. Nó bị loại vì che giấu kỹ thuật build image phổ biến nhất sau kỹ thuật ít phổ biến nhất. Mọi đội ngũ phát hành frontend đều có Dockerfile này; đọc và viết được nó tự thân đã là một kỹ năng thị trường tuyển dụng cần, và có một image không-Jib trong pipeline cũng chứng minh pipeline không âm thầm mang hình dáng của Jib. Cái giá là một Docker CLI trong image Jenkins và một socket daemon được mount — Mục 26.6.

26.3 nginx: fallback giúp SPA hoạt động

```
server {
    listen 80;
    root /usr/share/nginx/html;
    location / {
        try_files $uri $uri/ /index.html; ①
    }
    location /assets/ {
        add_header Cache-Control "public, max-age=31536000, immutable"; ②
    }
}
```

```

    }
    location = /healthz { return 200 "ok\n"; }
  }
}

```

- ① Dòng mà mọi ứng dụng một trang (SPA) đều cần và mọi lần triển khai đầu tiên đều quên. React Router (BrowserRouter trong `main.tsx`) sở hữu những URL như `/courses/42` hoàn toàn trong trình duyệt: bấm một link không bao giờ hỏi server. Nhưng một lần refresh, một bookmark hay một link dán vào có hỏi server về `/courses/42`, file đó không tồn tại, và một file server thuần trả 404. `try_files` thử đường dẫn nguyên văn, rồi thư mục, rồi rơi về `index.html` — ứng dụng khởi động và router của nó đọc URL. OAuth callback (`/oauth/callback`) cũng dựa vào điều này: redirect của Google là một request lạnh tới một route chỉ tồn tại trong JavaScript.
- ② Vite ghi mã băm nội dung vào tên mọi asset, nên file đổi thì luôn có tên mới. Điều đó khiến “cache vĩnh viễn” an toàn cho `/assets/` — và *không an toàn* cho `index.html`, thứ phải được tải lại để biết các tên mới sau mỗi lần deploy. Cache file HTML là cách người dùng rơi vào cảnh ứng dụng cũ đi đòi những bundle mà server không còn nữa.
- ③ Một đích cho probe không tốn gì và không bao giờ chạm đĩa (các probe của Chương 17 cần thứ gì đó để gọi).

Hãy để ý điều file này *không* làm: proxy `/api`. Trên laptop, `vite.config.ts` proxy `/api` tới gateway ở `localhost:8080` để dev server và API trông như một origin. Trong cụm, việc đó thuộc về Traefik — nginx không bao giờ thấy một request API nào cả.

26.4 Giành lấy /: một origin

```

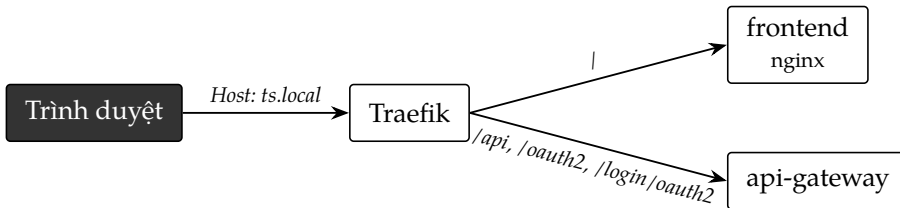
kind: Ingress
metadata: { name: frontend }
spec:
  rules:
    - host: ts.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: { service: { name: frontend, port: { number: 80 } } }

```

- ① Một quy tắc bắt-tất-cả, trong một object Ingress *thứ hai* đặt cạnh Ingress của gateway. Traefik gộp mọi quy tắc Ingress của một host và định tuyến theo

tiền tố khớp dài nhất: `/api/courses` khớp cả `/` lẫn `/api`, và `/api` thắng. Ba tiền tố của gateway vì thế tiếp tục hoạt động không đổi, còn mọi thứ khác — `/`, `/login`, `/courses/42`, `/assets/...` — rơi xuống nginx. Hai object, một bảng định tuyến gộp; quy tắc “apply không bao giờ xóa” của Chương 20 vẫn áp dụng cho từng cái.

Hệ quả chính là lý do cột mốc này tồn tại: **một origin**. Trình duyệt tải trang từ `http://ts.local` và gọi `http://ts.local/api/...` — cùng scheme, host và cổng. So với Compose, nơi trang đến từ `localhost:3000` còn API từ `localhost:8080`: khác cổng là khác origin, nên mọi lời gọi API đều có một CORS preflight đi trước và `CorsConfig` của gateway phải liệt kê rõ `localhost:3000`. Trong cụm, `CorsConfig` đơn giản là không bao giờ được hỏi tới — chẳng có request cross-origin nào để kiểm soát. Cookie, redirect của OAuth, và bất kỳ chính sách `SameSite` nào về sau đều đơn giản đi vì cùng lý do.



Hình 26.1: Tiền tố dài nhất thắng: một host, hai backend.

Một thuộc tính đi theo việc đổi host. `OAuth2LoginSuccessHandler` kết thúc đăng nhập Google bằng cách chuyển hướng trình duyệt tới `app.oauth2.success-redirect-uri`, mà `auth-service.yml` của `config-server` đặt là `localhost:3000/oauth/callback` — đúng trên laptop, sai trong cụm. `auth-service.yml` ghi đè nó bằng biến môi trường `APP_OAUTH2_SUCCESS_REDIRECT_URI=http://ts.local/oauth/callback` — quy tắc ưu tiên của Chương 16 (biến môi trường thắng config server) làm đúng việc nó đã làm cho URL datasource.

26.5 Manifest

Deployment mang hình dáng của Chương 15 với gần như mọi thứ bị lược bỏ: không `envFrom`, không biến `SPRING_*`, không `startupProbe` (nginx lên trong vài mili giây), và `requests: { memory: 32Mi, cpu: 10m }` — một phần mười hai của JVM nhỏ nhất. `imagePullPolicy: Always` với tag `:latest` làm mặc định của manifest, được pipeline ghi đè bằng tag commit y hệt các service JVM.

26.6 Pipeline mộc thêm một stage

```
stage('Frontend image') {
  steps {
    dir('Project/TeacherSupporter') {
      sh '''
        docker build -t ${REGISTRY}/frontend:${TAG} \
          -t ${REGISTRY}/frontend:latest frontend ①
        docker push ${REGISTRY}/frontend:${TAG}
        docker push ${REGISTRY}/frontend:latest
      '''
    }
  }
}
```

- ① Build context là frontend/, được cắt gọn bởi một .dockerignore loại trừ node_modules và dist: các module native của laptop Windows không được gửi lên daemon, và image tự build lấy của mình. frontend gia nhập SERVICES (danh sách của kubectl) nhưng không vào MODULES (của Maven) — các vòng lặp set image / rollout status của stage Deploy xử lý nó mà không cần đổi gì.

Để docker build chạy được bên trong container Jenkins, hai thứ thay đổi trong thiết lập của Chương 22. Image có thêm Docker CLI (docker-ce-cli — chỉ phần client), và run.sh mount /var/run/docker.sock của host vào container. CLI nói chuyện qua socket đó với daemon của host; không có daemon nào bên trong Jenkins và không có Docker-in-Docker. Image tạo ra nằm trong kho image của host, và docker push localhost:5000/... tới được registry qua mạng host mà container vốn đã dùng chung.

CẠM BẤY — PERMISSION DENIED TRÊN /VAR/RUN/DOCKER.SOCK

Triệu chứng: CLI đã cài, socket đã mount, và mọi lệnh docker đều thất bại với lỗi quyền. Nguyên nhân: socket thuộc sở hữu của nhóm docker trên host — GID 984 trên ts-server — và con số đó chẳng có ý nghĩa gì bên trong image, nơi user jenkins thuộc một nhóm cùng tên nhưng GID khác (hoặc không có). Cách sửa trong run.sh: --group-add "\$(stat -c %g /var/run/docker.sock)" thêm GID số của host vào user của container lúc chạy, không phải nướng một con số đặc thù của máy vào Dockerfile.

QUYẾT ĐỊNH — SOCKET LÀ ROOT

Mount socket Docker trao quyền kiểm soát host tương đương root: ai chạy được `docker run -v /:/host` là sở hữu cả máy. Với một homelab một người dùng mà Jenkins chỉ truy cập được qua tailnet, đây là sự đánh đổi được chấp nhận, nói rõ thành lời. Production build image mà không cần daemon nào cả — Kaniko hoặc BuildKit chạy như một pod không đặc quyền, hoặc Jib cho JVM — và việc chuyển Jenkins vào cụm ở Chương 27 là nơi khoản thỏa hiệp này sẽ được trả lại.

TRÊN PHẦN CỨNG NÀY

Pod frontend rẻ nhất namespace (yêu cầu 32 Mi), nhưng *build* thì không: npm ci cộng tsc cộng Vite chạy gần một CPU-phút và một gigabyte RAM trên controller, trong khi Maven có thể vẫn đang chạy trong cùng container 2 GB đó. Jenkinsfile giữ các stage tuần tự vì lý do đó; trên cái máy này, stage song song chỉ tranh giành nhau.

26.7 Kiểm chứng cột mốc

THỰC HÀNH — DANH SÁCH KIỂM TRA CỘT MỐC 7

1. `kubectl -n ts get ingress` — hai object trên `ts.local`; `kubectl -n ts get pods -l app=frontend` Ready.
2. Mở `http://ts.local/` — trang đăng nhập. Rồi dán thẳng `http://ts.local/courses` vào thanh địa chỉ: một trang, không phải 404 của nginx — fallback SPA hoạt động.
3. Đăng ký, đăng nhập, tạo một khóa học: tab network của trình duyệt cho thấy các request `/api/...` không có preflight OPTIONS nào — cùng origin.
4. `curl -sI http://ts.local/assets/index-*.js | grep Cache-Control` — `immutable`; làm tương tự với `/` — không có header cache.
5. Đổi một đoạn chữ bất kỳ trong một trang, push, và xem pipeline: stage Frontend image dùng lại layer npm ci, và rollout thay vào một image 15 MB.

Build #9 là lần chạy đầu tiên của stage mới: `docker build` hoàn tất trong 90 giây trên controller, rollout thay frontend:f5d9213 vào, và danh sách kiểm tra ở trên đạt đúng như đã viết — `/courses` trả về `index.html`, `/api/courses` trả về JSON 401 của gateway, asset mang `immutable` và `/` không mang header cache nào.

CẠM BẦY — INGRESS SỐNG LÂU HƠN MANIFEST CỦA NÓ

kubectl get ingress hôm đó liệt kê *ba* object trên ts.local: của gateway, của frontend — và course-service, tám ngày tuổi, định tuyến /courses thẳng tới service mà không kiểm tra JWT. Manifest của nó đã bị gỡ khỏi course-service.yaml từ Chương 18 kèm một comment nói rõ điều đó, nhưng quy tắc của Chương 20 vẫn đúng: apply không bao giờ xóa thứ nó không còn thấy. Phải cần một lệnh tay kubectl -n ts delete ingress course-service. Bài học không phải là câu lệnh mà là việc kiểm toán: sau mỗi cột mốc, hãy liệt kê thứ cụm có, không chỉ thứ git *nói* — cho đến khi việc dọn (prune) của GitOps (Chương 27) biến hai thứ đó thành một.

26.8 Truy cập từ bất cứ đâu: tên trên tailnet

Mọi thứ ở trên trả lời trên ts.local — một cái tên chỉ tồn tại trong file hosts của một chiếc laptop, trên một LAN. “Live” nghĩa là một URL phân giải được trên mọi thiết bị bạn có, và cái máy đã có sẵn một cái: Tailscale cho mỗi node một tên MagicDNS, ts-server.tailbfe002.ts.net, mà mọi peer trên tailnet phân giải được mà không cần sửa file hosts. Traefik định tuyến theo header Host (Chương 18), nên một cái tên nó không liệt kê sẽ nhận 404 dù gói tin có tới nơi. Cách sửa là thêm một mục rules nữa với cùng danh sách đường dẫn trong cả hai object Ingress (Phụ lục D có chúng) — apply bằng tay với một bộ lọc chỉ trích ra các tài liệu Ingress từ đầu ra của kubectl kustomize, vì một lệnh apply -k đầy đủ sẽ đặt lại tag image của mọi Deployment về 1.0.0-SNAPSHOT của manifest và roll cả namespace (set image của Chương 23 nằm *sau* apply chính vì lý do này).

Cái tên đó rồi phơi ra một lỗi mà LAN đã che giấu. Một lần đăng ký qua host mới sinh ra mail kích hoạt (chuỗi của Chương 25, tròn vện qua tailnet), nhưng link bên trong ghi http://localhost:8080/api/auth/activate?code=...: một host không điện thoại nào tới được, và một endpoint POST mà GET của trình đọc mail đăng nào cũng không dùng nổi. EmailService dựng mọi link — kích hoạt, mời, đăng nhập — từ chuỗi literal. Sửa chữa là quy tắc của Chương 16 áp dụng cho URL: origin công khai là *cấu hình*. app.frontend.base-url mặc định là dev server của Vite trong notification-service.yml của config-server; manifest ghi đề nó bằng APP_FRONTEND_BASE_URL đặt thành tên tailnet, cùng thủ thuật relaxed binding như SPRING_MAIL_HOST; và các link giờ trở tới trang /activate của frontend, nơi thực hiện POST. Hai biến OAuth chuyển sang cùng host, đồng nghĩa với thêm một mục nữa dưới “Authorized redirect URIs” của Google — việc đăng ký mà Chương 5 gọi là thứ duy nhất không manifest nào tự động hóa được.

CẠM BẦY — MỘT LINK LÀ MỘT HỢP ĐỒNG VỚI BÊN NGOÀI

Bất cứ thứ gì rời khỏi hệ thống — một email, một payload webhook, một mã QR — đều mang một URL phải hoạt động từ *bên ngoài*, nên nó không bao giờ được là tên Service và không bao giờ nên là literal. Bài kiểm tra rất rẻ: đăng ký từ một thiết bị không phải laptop build và bấm vào. Trên stack này, chính request đầu tiên qua hostname mới đã phơi ra một lỗi có mặt từ commit đầu tiên.

26.9 Ngoai dự án

Mẫu hình — router biên tách tĩnh khỏi API theo đường dẫn, phần tĩnh đến từ một file server ngớ ngếch — là phổ quát, với file server thường được thay bằng lưu trữ đối tượng cộng CDN (S3 + CloudFront, hoặc một static host như Netlify) và việc tách do quy tắc origin của CDN làm. Giai đoạn build vẫn y nguyên nửa đầu của Dockerfile này. Hai thói quen chuyển sang trực tiếp: bấm-và-cache-vĩnh-viễn cho asset kèm `index.html` không cache, và fallback SPA — theo cách nói của CloudFront là “ánh xạ 403/404 sang `/index.html` với mã 200”, cùng ý tưởng nhưng dùng vùng hơn.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao image frontend có hai dòng `FROM`, và chính xác thứ gì bằng qua giữa chúng?
2. Một người dùng bookmark `http://ts.local/courses/7` và mở nó sáng hôm sau. Lần theo request qua Traefik và nginx, và nói dòng nào khiến nó thành công.
3. Cả hai object Ingress đều khai báo host: `ts.local`. Traefik quyết định thế nào để `/api/courses` đi tới gateway và `/assets/x.js` đi tới nginx?
4. Giải thích vì sao `CorsConfig` của gateway trở nên không liên quan trong cụm trong khi vẫn cần thiết trên laptop.
5. `--group-add "$(stat -c %g /var/run/docker.sock)"` sửa điều gì, và vì sao giá trị được tính ra thay vì viết sẵn?
6. Link trong mail kích hoạt đã sai hàng tháng mà không ai để ý. Nêu hai khiếm khuyết riêng biệt trong `http://localhost:8080/api/auth/activate?code=...`, và nói mỗi cái sẽ hỏng ở môi trường nào (laptop, compose, cụm).

Chặng đường phía trước

Với frontend được phục vụ từ cụm (Chương 26), mọi thứ có thể triển khai của dự án đều đã được triển khai và ứng dụng dùng được trong trình duyệt. Các cột mốc còn lại, mỗi cái thay một thứ tự chế bằng cơ chế công nghiệp — và mỗi cái đều có ý nghĩa nhất vào lúc này, khi bạn đã biết nó thay thế cái gì.

27.1 Khả năng quan sát

Bên ngoài `kubectl logs` là bộ ba tiêu chuẩn: **metric** (Prometheus cào các endpoint /actuator/prometheus mà mọi service đã mở sẵn; dashboard Grafana trên bộ ba RED — rate, errors, duration), **trace** (dây nối Zipkin đã có trong mọi cấu hình, hiện lấy mẫu ở mức không trong cụm — một biến môi trường bật nó lên khi có pod Zipkin), và **alert** (Alertmanager: một cú báo khi tỷ lệ lỗi của một service hay độ đầy của một PVC vượt ngưỡng). Việc này đến qua Helm chart `kube-prometheus-stack` — khoảnh khắc đã hứa khi bạn dùng Helm mà không tự viết chart (hộp quyết định của Chương 20, được giữ lời). Strimzi có thể xuất metric của broker vào cùng Prometheus đó với vài dòng `metricsConfig` trên resource Kafka — độ trễ của consumer group là alert Kafka đầu tiên đáng có.

TRÊN PHẦN CỨNG NÀY

Bộ đầy đủ tốn ~1,5 GB và không ít CPU — ngân sách nói là vừa, nhưng đây là thứ đầu tiên phải cắt khi tải cao. Khả năng quan sát có thể bỏ đi là món xa xỉ của homelab; các đội production tính kích cỡ cho nó trước tiên.

27.2 GitOps: Argo CD

Pipeline *đẩy* các lần triển khai (Jenkins giữ một `kubeconfig` và chạy `kubectl`). GitOps đảo chiều mũi tên: một agent trong cụm (Argo CD) *kéo* — liên tục so sánh manifest của một kho git với cụm và hội tụ. Điều đó mua được gì, bằng vốn từ bạn đã có: thông tin xác thực của cụm không bao giờ rời khỏi cụm (nỗi lo RBAC của Chương 22 tan biến — CI chỉ push image và commit một lần nâng tag); việc xóa cuối cùng cũng lan truyền (cạm bẫy `prune` của Chương 20, được giải quyết từ thiết kế); trôi (drift) cấu hình trở nên hữu hình (một Deployment bị sửa tay

hiện “OutOfSync”); và rollback là `git revert`. Jenkins giữ CI — build, test, push, cập nhật tag trong git — Argo CD nhận CD. Stage deploy của Jenkinsfile, tức Chương 23 của cuốn sách này, trở thành một commit.

27.3 Cho stack cổ điển về hưu

Sự dư thừa có chủ đích của Chương 2 và 3 nay có thể được thanh toán, từng service một:

- **Bỏ Eureka:** đổi các route của gateway từ `lb://course-service` sang `http://course-service:8082` — DNS của Service và các endpoint gác bằng readiness làm mọi thứ registry từng làm ở đây. Xóa một thứ triển khai, giải phóng 512 Mi.
- **Bỏ Config-server:** mount các file `configurations/` thành một ConfigMap và trở `spring.config.import` vào thư mục đã mount (`optional:file:/config/`). Cùng file, bớt một service, và thay đổi cấu hình trở thành thay đổi manifest chảy qua GitOps.

Cả hai cuộc di dời đều là điểm cộng cho CV chính vì chúng chạy *về phía* nền tảng: biết vì sao Eureka tồn tại *và* vì sao Kubernetes khiến nó lỗi thời là khác biệt giữa dùng một trong hai và hiểu cả hai.

27.4 Jenkins vào cụm

Phép đảo ngược kiểu homelab của Chương 22 — build trên controller, Jenkins nằm ngoài k3s — đảo lại gọn gàng một khi pipeline đã ổn định: plugin Kubernetes sinh một pod agent cho mỗi build, và bản thân Jenkins trở thành một StatefulSet với `JENKINS_HOME` trên một PVC. Cuộc di dời chạm tới mọi khái niệm của Phần V cùng lúc, đó là lý do nó được xếp cuối cùng.

27.5 OpenShift, nói ngắn

Các manifest được giữ khả chuyển có chủ đích: image không chạy bằng root của Jib (Chương 13) thỏa mãn mô hình bảo mật UID tùy ý của OpenShift, nên triển khai lên một sandbox OpenShift miễn phí hẳn chỉ cần bước dịch Ingress sang Route (Chương 18). Chạy thí nghiệm đó một lần sẽ kiểm chứng những tuyên bố về tính khả chuyển mà cuốn sách này vẫn đưa ra — và vốn từ của OpenShift (Route, SCC, BuildConfig) ánh xạ một-một lên các chương bạn đã đọc.

27.6 Bạn đang đứng ở đâu

Bỏ tên dự án đi, bản kiểm kê đọc như một bản mô tả công việc platform: bộ máy phân giải thuộc tính giúp một artifact chạy ở bất cứ đâu; log, consumer group và ngữ nghĩa giao nhận; image, layer và registry; điều hòa, probe, RBAC, storage class và operator; pipeline, danh tính deploy giới hạn bằng RBAC, và phép đảo chiều GitOps. Những ràng buộc của homelab không phải là giới hạn — chúng là giáo trình: mỗi hộp “trên phần cứng này” đều ép ra một quyết định mà tiền bạc thường che đi.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Bạn sẽ đặt alert trên metric nào trước tiên cho đường Kafka, và nó đo cái gì theo cách nói của Chương 10?
2. Triển khai kiểu đẩy so với kiểu kéo: chỉ ra thông tin xác thực của cụm nằm ở đâu trong mỗi kiểu, và giải thích việc dọn (prune) của GitOps giải quyết cạm bẫy của chương nào.
3. Phác thảo việc cho Eureka về hưu: cái gì đổi trong cấu hình của gateway, hai cơ chế Kubernetes nào thay cho đăng ký và liveness, và cái gì có thể xóa?
4. Vì sao image của Jib khiến thí nghiệm OpenShift rẻ, và phép dịch object nào vẫn còn lại?

Phần VIII

Đào sâu

Đào sâu: JWT ở quy mô lớn — xác minh ở đâu, token béo phì và khóa bất đối xứng

Chương 5 dạy thiết kế bảo mật của dự án đúng như nó đang có: token HS256, xác minh một lần tại gateway, danh tính chuyển tiếp dưới dạng header. Thiết kế đó đúng với một hệ thống có một gateway và năm service. Chương này là chương *đào sâu* đầu tiên của cuốn sách: lấy một thiết kế đã có trong dự án và đẩy nó tới khi gãy, theo cách một người phỏng vấn hay một đợt tăng lưu lượng gấp mười sẽ làm. Câu hỏi lấy từ một bài viết tiếng Việt được chia sẻ rộng rãi về JWT trong microservices;¹ câu trả lời lấy từ việc đọc code của dự án.

Ba câu hỏi sắp xếp chương này:

1. **Nên xác minh token ở đâu** — một lần tại gateway, hay trong từng service?
2. **Chuyện gì xảy ra khi token phình to** — và mẫu *phantom token* giữ nó gọn là gì?
3. **Chữ ký thực sự bảo đảm điều gì**, và vì sao lựa chọn giữa HS256 và RS256 quyết định ai được tin cậy với thứ gì?

28.1 Xác minh ở đâu: gateway hay từng service?

28.1.1 Mô hình 1: xác minh tập trung (điều dự án đang làm)

Đọc lại hai nửa của mẫu biên tin cậy. Ở biên, trong `JwtAuthenticationFilter` của gateway:

```
Claims claims = Jwts.parser()
    .verifyWith(signingKey)           ①
    .build()
    .parseSignedClaims(token)
    .getPayload();

String userId = claims.getSubject();
String role = claims.get(JwtConstants.CLAIM_ROLE, String.class);
```

¹“Lột trần sự thật về JWT — Phần 2: Trận chiến Microservices và nỗi ám ảnh Token béo phì”, viblo.asia, 2026.

CHƯƠNG 28. ĐÀO SÂU: JWT Ở QUY MÔ LỚN — XÁC MINH Ở ĐẦU, TOKEN BÉO PHÌ VÀ KHÓA BẮT ĐỐI XỨNG

```
ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
    .header(JwtConstants.HEADER_USER_ID, userId) ②
    .header(JwtConstants.HEADER_USER_ROLE, role)
    .build();
```

Và phía sau nó, trong `GatewayHeaderAuthFilter` của mỗi service (bản của course-service):

```
String userId = request.getHeader("X-User-Id");
String userRole = request.getHeader("X-User-Role");

if (userId != null && userRole != null) { ③
    var authorities = List.of(
        new SimpleGrantedAuthority("ROLE_" + userRole));
    var authentication = new UsernamePasswordAuthenticationToken(
        Long.valueOf(userId), null, authorities);
    SecurityContextHolder.getContext().setAuthentication(authentication);
}
```

- ① Mật mã chạy đúng một lần, ở đây. Khóa chính là `app.jwt.secret` mà `auth-service` dùng để ký.
- ② Danh tính rời gateway dưới dạng hai header HTTP thuần. Không gì ở phía sau phân biệt được gateway viết chúng hay ai khác viết.
- ③ Service *không* kiểm tra gì. Có hai header nghĩa là đã xác thực. Đó là toàn bộ hợp đồng.

Ưu điểm là thật: một nơi giữ secret, service không cần thư viện JWT lẫn xử lý khóa, và chi phí CPU kiểm tra chữ ký mỗi request chỉ trả một lần. Phần lớn tutorial Spring Cloud dừng ở đây.

28.1.2 Ngụy biện “mạng nội bộ đáng tin”

Mẫu này dựa trên một bất biến — *mọi* đường tới một service đều đi qua gateway. Trong file Compose của dự án, bất biến đó đã sai sẵn. `docker-compose.yml` publish thẳng `auth-service` ở 8081 và `course-service` ở 8082 ra host, để bạn debug được. Bất kỳ ai trên host vì thế có thể bỏ qua gateway:

THỰC HÀNH — GIẢ MẠO DANH TÍNH TRONG MỘT DÒNG

Với stack đang chạy, hỏi `course-service` danh sách khóa học của một giáo viên mà không hề đưa token:

```
$ curl -s -H "X-User-Id: 1" -H "X-User-Role: TEACHER" \
```



```
http://localhost:8082/courses | head -c 300
```

Nó trả lời. Giờ lặp lại qua gateway ở 8080 với cùng hai header và không có Authorization:

```
$ curl -s -H "X-User-Id: 1" -H "X-User-Role: TEACHER" \
  http://localhost:8080/api/courses
{"error": "Unauthorized", "message": "Missing or invalid Authorization header"}
```

Khác biệt giữa hai kết quả chính là toàn bộ mô hình bảo mật.

Trong Kubernetes các cổng publish biến mất (Chương 18), nhưng mạng pod phẳng vẫn còn: pod nào trong namespace cũng với được `course-service:8082`. Nguy hiểm nằm ở chỗ coi “bên trong” nghĩa là “đáng tin”. Chỉ cần một container bị chiếm — một image có lỗ hổng, một kubeconfig bị lộ, một dependency trong chuỗi cung ứng — kẻ tấn công giả mạo `X-User-Id: 1` tới mọi service cùng lúc.

CẠM BẤY — HEADER DANH TÍNH SỐNG SỐT QUA ĐƯỜNG DẪN CÔNG KHAI

Triệu chứng: không thấy gì, và đó chính là vấn đề. Nhìn đầu hàm `JwtAuthenticationFilter.filter`: request tới đường dẫn công khai (`/api/auth/refresh`, `/actuator/**`) được chuyển tiếp *nguyên vẹn* — gateway không xác minh token cũng không đụng tới header. Client vì thế có thể gửi `X-User-Id: 1` tới một đường dẫn công khai và header tới service còn nguyên. Hôm nay không endpoint công khai nào đọc nó, nên chưa khai thác được; nhưng bất biến “chỉ gateway mới viết các header này” không được cưỡng chế, nó chỉ đúng nhờ may mắn. Cách sửa là một default filter trong `api-gateway.yml` gỡ cả hai header khỏi mọi request *đi vào* trước khi filter JWT đặt chúng:

```
spring:
  cloud:
    gateway:
      default-filters:
        - RemoveRequestHeader=X-User-Id
        - RemoveRequestHeader=X-User-Role
```

Giờ các header chỉ có thể xuất phát từ gateway. Việc này đóng cánh cửa bên ngoài; nó không làm gì cho cánh cửa trong cụm.

28.1.3 Mô hình 2: zero trust — service nào cũng xác minh

Phương án thay thế gỡ bỏ sự tin cậy: mỗi service tự parse và xác minh JWT, và bỏ qua mọi header danh tính. Header giả giờ vô giá trị vì service không bao giờ đọc; token giả thì trượt kiểm tra chữ ký. Gateway vẫn có thể xác minh — loại token hỏng từ sớm rất rẻ — nhưng không gì ở phía sau *phụ thuộc* vào việc đó.

Chi phí xuất hiện ngay: mọi bên xác minh đều cần khóa. Với HS256 nghĩa là chép *secret key* tới mọi service, còn tệ hơn vấn đề nó giải quyết (Mục 28.3). Zero trust ở tầng ứng dụng chỉ hợp lý với khóa bất đối xứng, và đó là lý do hai nửa của chương này đi cùng nhau.

QUYẾT ĐỊNH — HEADER DO GATEWAY XÁC MINH, CHO TỚI LÚC NÀY

Dự án giữ Mô hình 1. Nó có đúng hai nơi giữ khóa, không có service bên thứ ba, và mọi service đều do cùng một pipeline triển khai — bán kính thiệt hại của giả định biên tin cậy chỉ là một namespace. Ba tín hiệu sẽ lật quyết định: một service do đội khác viết, một service mà bên gọi không chỉ là gateway (một consumer Kafka cũng mở HTTP, một batch job), hoặc một yêu cầu tuân thủ đòi danh tính phải xác minh được tại điểm sử dụng. Lúc đó bước đi là RS256 cộng xác minh tại từng service, không phải thêm header. Hệ thống production thường đi đường thứ ba: giữ header, và để *tầng vận chuyển* chứng minh người gửi bằng mutual TLS từ một service mesh (Istio, Linkerd). Gateway khi đó thành một đối tác được định danh bằng mật mã thay vì một hàng xóm không kiểm chứng được.

28.2 Chữ ký bảo đảm điều gì

Một JWT là `base64url(header).base64url(payload).signature`. Hai phần đầu được *encode*, không phải mã hóa: dán bất kỳ token nào của dự án vào một bộ giải mã và `sub`, `email`, `role` đọc được ngay. Chữ ký thêm đúng hai tính chất:

- **Toàn vẹn.** Đổi một byte của payload (`"role":"STUDENT"` thành `"TEACHER"`) là xác minh thất bại.
- **Xác thực.** Token do một bên giữ khóa ký tạo ra — và không ai khác.

Nó không thêm tính bí mật. Cả HS256 lẫn RS256 đều tính chữ ký trên cùng một chuỗi, `base64url(header) + "." + base64url(payload)`. Chúng chỉ khác ở *khóa nào tạo chữ ký và khóa nào kiểm tra nó*. Một khác biệt duy nhất đó quyết định toàn bộ cấu trúc tin cậy của một hệ thống.

28.3 HS256 so với RS256

28.3.1 HS256: một khóa, hai vai

HS256 là HMAC-SHA256: một hàm băm có trộn khóa. Bạn không cần nhớ cấu trúc (hai lượt SHA-256 lồng nhau với khóa XOR cùng hai pad cố định); bạn cần một tính chất. *Cùng một khóa vừa tạo vừa kiểm tra chữ ký*. Xác minh nghĩa là tính lại HMAC với bản khóa của mình rồi so byte. Không có khóa thì không tính được, nên không giả mạo được. Có khóa thì tính được, nên *có thể* giả mạo. Bên xác minh và bên ký không phân biệt được với nhau.

Hai bản sao của khóa đó trong dự án:

```
// auth-service/.../service/JwtService.java
this.signingKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));

// api-gateway/.../filter/JwtAuthenticationFilter.java
this.signingKey = Keys.hmacShaKeyFor(jwtSecret.getBytes(StandardCharsets.UTF_8));
```

Cả hai đọc `app.jwt.secret`; trong cụm cả hai mount cùng một Secret jwt (Chương 16). Gateway chỉ bao giờ cần *xác minh*, vậy mà nó giữ một khóa có thể *đúc* token cho bất kỳ user với bất kỳ vai trò nào. Chiếm được pod gateway là sở hữu mọi danh tính trong hệ thống. Mở rộng Mô hình 2 một cách ngây thơ — chép secret tới năm service — và bạn có sáu nơi như vậy.

28.3.2 RS256: hai khóa, mỗi khóa một vai

RS256 là chữ ký RSA trên một mã băm SHA-256. RSA dựa trên một bất đối xứng trong số học: nhân hai số nguyên tố lớn thì tức thì; tìm lại chúng từ tích thì bất khả thi.

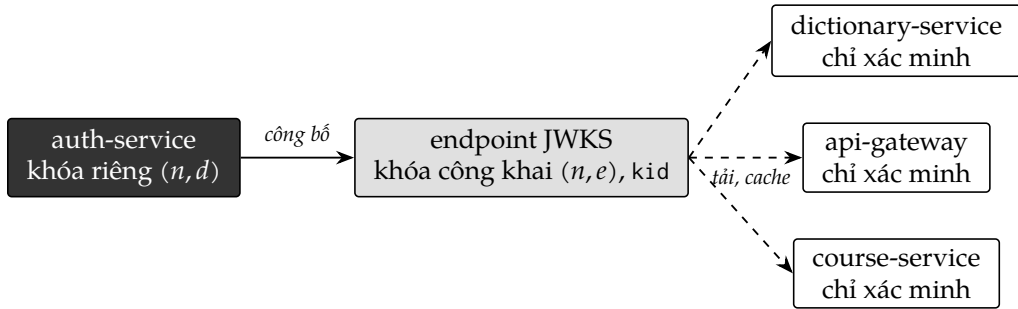
Sinh khóa, một lần:

1. Chọn hai số nguyên tố lớn p, q (mỗi số khoảng 1024 bit cho khóa 2048 bit). Tính $n = p \times q$.
2. Chọn số mũ công khai e ; thực tế luôn là 65537. Cặp (n, e) là **khóa công khai**.
3. Tính d sao cho $e \times d \equiv 1 \pmod{(p-1)(q-1)}$. Cặp (n, d) là **khóa riêng**. Tính d cần p và q ; chỉ biết n thì không suy ra được chúng.

Ký và xác minh:

```
h = SHA256(header "." payload), padded (PKCS#1 v1.5)
sign:  signature = h^d mod n      -- needs d: only auth-service
verify: signature^e mod n == h ?  -- needs e, n: anyone
```

Vì $e \times d \equiv 1$, nâng lũy thừa d rồi lũy thừa e trả về h . Ai có khóa công khai đều kiểm tra được chữ ký; không ai thiếu d tạo được chữ ký. Đó là bất đối xứng mà HS256 không có, và nó thay đổi ai được giữ thứ gì:



Hình 28.1: Cấu trúc khóa RS256: một bên ký, bao nhiêu bên xác minh cũng được. Lộ một bên xác minh không lộ gì cả.

- Auth-service giữ khóa riêng và là nơi *duy nhất* có thể phát hành token.
- Gateway và mọi service giữ khóa công khai. Lộ nó vô hại — nó công khai theo định nghĩa. Bạn có thể in nó trong cuốn sách này.
- Phân phối khóa không còn là bài toán quản lý bí mật: công bố khóa công khai tại một URL quy ước (một JWKS, JSON Web Key Set), và bên xác minh nào cũng tải lúc khởi động rồi cache lại.

Bạn đã tiêu thụ cấu trúc này rồi. Đăng nhập Google ở Chương 5 trả về một id_token ký RS256; OAuth2 client của Spring xác minh nó với JWKS của Google tại <https://www.googleapis.com/oauth2/v3/certs>. Google không bao giờ đưa bạn khóa riêng, và bạn không bao giờ cần nó.

28.3.3 Chi phí

	HS256	RS256 (2048-bit)
Vật liệu khóa	một secret, ≥ 32 byte	cặp riêng/công khai, $\approx 1,7$ KB PEM
Kích thước chữ ký	32 byte (43 ký tự)	256 byte (342 ký tự)
Ký	micro giây	khoảng 1 ms
Xác minh	micro giây	vài chục micro giây
Ai ký được	bất kỳ ai giữ khóa	chỉ người giữ khóa riêng
Ai xác minh được	bất kỳ ai giữ khóa	bất kỳ ai

Xác minh RSA rẻ hơn ký rất nhiều vì $e = 65537$ rất nhỏ trong khi d rộng 2048 bit. Bất đối xứng đó khớp với cấu trúc: một bên ký, nhiều bên xác minh. Nếu kích

thước chữ ký quan trọng, ES256 (ECDSA trên đường cong P-256) cho chữ ký 64 byte và khóa nhỏ hơn nhiều, và là mặc định hiện đại cho các identity provider mới.

CẠM BẦY — NHẦM LẪN THUẬT TOÁN (ALGORITHM CONFUSION)

Triệu chứng: một token đáng lẽ bị từ chối lại được chấp nhận — và log không thấy gì sai. Nguyên nhân: thư viện cũ đọc trường `alg` trong header để quyết định *cách* xác minh. Kẻ tấn công lấy một token RS256 hợp lệ, sửa header thành `"alg": "HS256"`, và ký bằng HMAC với secret là *chính các byte của khóa công khai* — những byte vốn dĩ công khai. Bên xác minh ngây thơ thấy HS256, dùng khóa công khai mình có làm secret HMAC, và phép kiểm tra qua. JWT chặn việc này bằng cách để *loại khóa* chọn thuật toán, không phải header: `verifyWith(SecretKey)` từ chối mọi token bất đối xứng và `verifyWith(PublicKey)` từ chối mọi token HMAC. Đó là lý do `JwtAuthenticationFilter` an toàn hôm nay, và là lý do bạn không bao giờ được xác minh bằng một phương thức chấp nhận “header nói gì thì làm nấy”.

28.3.4 Chuyển đổi dự án

Thay đổi đủ nhỏ để đọc trọn vẹn. Sinh cặp khóa một lần:

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 \
    -out jwt-private.pem
$ openssl pkey -in jwt-private.pem -pubout -out jwt-public.pem
```

PEM riêng chỉ đi vào Secret của `auth-service`; PEM công khai có thể đặt trong `ConfigMap`, vì nó không bí mật. `JwtService` ký bằng khóa riêng:

```
@Service
public class JwtService {

    private final PrivateKey privateKey; // sign
    private final PublicKey publicKey; // self-verify where needed

    public JwtService(@Value("${app.jwt.private-key-pem}") String privPem,
                     @Value("${app.jwt.public-key-pem}") String pubPem)
        throws GeneralSecurityException {
        KeyFactory kf = KeyFactory.getInstance("RSA");
        this.privateKey = kf.generatePrivate(
            new PKCS8EncodedKeySpec(decodePem(privPem)));
        this.publicKey = kf.generatePublic(
            new X509EncodedKeySpec(decodePem(pubPem)));
    }
}
```

CHƯƠNG 28. ĐÀO SÂU: JWT Ở QUY MÔ LỚN — XÁC MINH Ở ĐẦU, TOKEN BÉO PHÌ VÀ KHÓA BẮT ĐỐI XỨNG

```
public String generateAccessToken(User user) {
    return Jwts.builder()
        .header().keyId("2026-09-key1").and()           ②
        .subject(String.valueOf(user.getId()))
        .claim("email", user.getEmail())
        .claim("role", user.getRole().name())
        .issuedAt(new Date())
        .expiration(new Date(System.currentTimeMillis()
            + accessTokenExpiryMs))
        .signWith(privateKey, Jwts.SIG.RS256)           ③
        .compact();
}

private static byte[] decodePem(String pem) {
    String body = pem.replaceAll("-----BEGIN [A-Z ]+-----", "")
        .replaceAll("-----END [A-Z ]+-----", "")
        .replaceAll("\\s", "");
    return Base64.getDecoder().decode(body);
}
}
```

- ① PKCS#8 là định dạng chứa mà openssl genpkey ghi cho khóa riêng; X.509 SubjectPublicKeyInfo là thứ -pubout ghi. Bóc vỏ PEM, giải mã base64, xong.
- ② kid (key id) đặt tên cho khóa đã ký token này. Tồn vài byte bây giờ và cho phép xoay khóa về sau.
- ③ JJWT suy ra RS256 từ loại khóa; nêu tên tường minh để ghi lại ý định.

Gateway — và bất kỳ service nào muốn tự xác minh — chỉ giữ khóa công khai:

```
public JwtAuthenticationFilter(@Value("${app.jwt.public-key-pem}") String pubPem)
    throws GeneralSecurityException {
    this.publicKey = KeyFactory.getInstance("RSA")
        .generatePublic(new X509EncodedKeySpec(decodePem(pubPem)));
}

Claims claims = Jwts.parser()
    .verifyWith(publicKey)    // PublicKey: accepts RS/PS/ES only
    .build()
    .parseSignedClaims(token)
    .getPayload();
```

Không gì khác thay đổi: cùng claim, cùng header, cùng GatewayHeaderAuthFilter. Gateway không còn đúc được token, và api-gateway.yml không cần app.jwt.secret nữa.

28.3.5 Xoay khóa, và kid để làm gì

Khóa có tuổi: chính sách bảo xoay mỗi 90 ngày, hoặc một chiếc laptop chứa file PEM bị mất. Xoay khóa mà không đăng xuất toàn bộ người dùng là một nghi thức bốn bước:

1. Sinh cặp khóa mới; thêm khóa công khai của nó vào JWKS bên cạnh khóa cũ, dưới một kid mới.
2. Đợi tới khi mọi bên xác minh đã làm mới cache (hoặc khởi động lại chúng).
3. Chuyển auth-service sang ký bằng khóa riêng mới. Token mới mang kid mới; token cũ, còn đang lưu hành, mang kid cũ — và bên xác minh chọn đúng khóa công khai theo kid.
4. Sau khi thời gian sống dài nhất của token đã trôi qua (ở đây 15 phút cho access token), gỡ khóa cũ khỏi JWKS.

Không có kid, bên xác minh sẽ phải thử mọi khóa nó biết; có nó, tra cứu là chính xác và khóa cũ có thể nghỉ hưu một cách chắc chắn. Phiên bản PEM tĩnh ở trên chưa có endpoint JWKS; bước tự nhiên tiếp theo là phục vụ `/.well-known/jwks.json` từ auth-service và để các bên xác minh dùng `NimbusJwtDecoder.withJwkSetUri(...)` của Spring Security, thử tự động cache và tải lại khi gặp kid lạ.

28.4 Token béo phì (payload bloat)

Phi trạng thái rất quyến rũ: nếu token mang mọi thứ một service cần, không service nào phải gọi service khác để hỏi “user này được làm việc này không?” Các đội theo logic đó cho tới khi token mang vai trò, quyền, nhóm, feature flag và hồ sơ của user — và nặng 4 KB. Phép tính lúc đó quay lại chống họ. Một trang dashboard bắn 20 lời gọi API song song gửi đi 80 KB token trước khi có một byte dữ liệu nghiệp vụ nào. Nhân với vài chục nghìn user đồng thời và header xác thực chiếm một phần đo được của băng thông vào. Cookie, nếu token nằm ở đó, có trần 4 KB; một số proxy giới hạn kích thước header ở 8 KB; log ghi header phình theo.

Access token của dự án mỏng. `JwtService.generateAccessToken` ghi sub, email, role, iat, exp — khoảng 200 byte đã ký. Giữ nó như vậy bằng một quy tắc: **token mang danh tính; quyền hạn thì tra cứu**. Khoảnh khắc course-service cần biết “giáo viên này được sửa khóa học nào?”, cảm dỗ thêm claim `courseIds` xuất hiện. Hãy cưỡng lại. Danh sách đó thay đổi khi một khóa học được tạo, điều token không thể biết cho tới khi hết hạn; và nó lớn không giới hạn. Service vốn đã sở hữu dữ liệu ghi danh (Chương 7) — một truy vấn có index trả lời câu hỏi đúng và cập nhật.

28.5 Mẫu phantom token

Giả sử token béo là không tránh khỏi — một mô hình phân quyền cũ, một tích hợp với đối tác. Mẫu phantom token giữ token *bên ngoài* nhỏ xíu trong khi token *bên trong* vẫn tự chứa:

1. Auth-service tạo JWT đầy đủ nhưng *không* trả về. Nó cất JWT vào một kho nhanh (Redis) dưới một định danh mờ ngẫu nhiên, và chỉ trả về định danh đó — một UUID, 36 byte.
2. Gateway nhận UUID, tra cứu, và thay bằng JWT thật trước khi chuyển tiếp. Service nội bộ thấy một token ký bình thường.
3. Service nội bộ tự xác minh nó (RS256, JWKS). Bên trong vành đai, hệ thống phi trạng thái và zero trust; bên ngoài, client mang một token không có nội dung nào đọc được.

Đăng xuất trở nên tầm thường và *tức thì*: `DEL <uuid>` trong Redis, và request kế tiếp với định danh đó thất bại tại gateway với 401, bất kể JWT bên trong còn sống bao lâu. Đây là phần thưởng thật của mẫu này — nó sửa điểm yếu cấu trúc duy nhất của JWT, không thu hồi được.

Dự án đã chạy một nửa của nó. Nhìn `JwtService.generateRefreshTokenValue`:

```
public String generateRefreshTokenValue() {  
    return UUID.randomUUID().toString();  
}
```

Refresh token không phải JWT. Nó là một định danh mờ lưu trong bảng `refresh_tokens` (Flyway V2) và bị thu hồi bằng cách xóa dòng — `AuthService.logout` làm đúng thế. Access token vẫn là một JWT tự chứa. Vậy độ trễ thu hồi của dự án bị chặn trên bởi 15 phút của access token, và *cơ chế* thu hồi của nó là mẫu phantom áp dụng cho riêng thông tin xác thực sống dài.

CẠM BÃY — REFRESH KHÔNG XOAY VÒNG

Đọc `AuthService.refreshToken`: nó trả về *đúng* refresh token nó nhận được. Một refresh token bị đánh cắp một lần vì thế hợp lệ suốt bảy ngày, và vụ trộm không thể phát hiện. Thực hành trong ngành là *xoay vòng* (rotation): mỗi lần `/refresh` phát một refresh token mới và vô hiệu token cũ. Nếu token cũ có lúc nào bị đưa ra lại, hai bên đang giữ bản sao — thu hồi cả họ token. Bảng đã có đủ cột để làm việc này; thay đổi là một cặp xóa-và-chèn trong một transaction.

QUYẾT ĐỊNH — BA CÁCH THU HỒI MỘT ACCESS TOKEN

Khi admin vô hiệu một tài khoản, câu trả lời của dự án là “trong vòng 15 phút”. Các phương án, theo chi phí tăng dần:

1. **Đợi cho hết hạn.** Access token ngăn cộng refresh token có trạng thái. Không thêm hạ tầng; độ trễ thu hồi bằng thời gian sống của access token. Chấp nhận được với phần lớn ứng dụng, và là điều dự án đang làm.
2. **Danh sách cấm (deny-list).** Thêm claim `jti` (id token); khi đăng xuất hoặc vô hiệu, ghi `jti` vào Redis với TTL bằng phần đời còn lại của token; gateway tra Redis mỗi request. Một lượt tra nhanh mỗi request, và chỉ token bị thu hồi mới được lưu.
3. **Phantom token.** Định danh mờ bên ngoài, JWT bên trong, Redis làm bảng ánh xạ. Thu hồi tức thì và trình duyệt không có token đọc được — nhưng Redis giờ nằm trên đường găng của *mọi* request. Redis chết nghĩa là mọi user bị đăng xuất cùng lúc.

Phản biện của một kỹ sư senior với sự hào hứng của bài viết: phương án 3 đổi “không thu hồi được” lấy “một điểm lỗi đơn mới”. Phần lớn đội nên với tới phương án 2 trước và coi phương án 3 là phù hợp với yêu cầu cấp ngân hàng, nơi chính bài viết cũng thừa nhận session có thể là công cụ tốt hơn.

28.6 Chuyện gì gây lúc 3 giờ sáng

Phân tích sự cố là cách nhanh nhất để phân biệt hai mô hình khóa. Auth-service chết lúc 3 giờ sáng; không gì khác chết.

- **HS256, như đang triển khai.** Gateway giữ secret và xác minh cục bộ: mọi user đang đăng nhập tiếp tục dùng được cho tới khi access token hết hạn. Lời gọi refresh thất bại (auth-service là endpoint `/refresh`), nên phiên chết trong vòng 15 phút. Đăng nhập mới thất bại. Hồi phục tức thì khi auth-service trở lại.
- **RS256 với JWKS đã cache.** Giống hệt với các bên xác minh đang chạy: chúng đã cache khóa công khai lúc khởi động và không cần gì từ auth-service. Sự cố mới là một bên xác minh *lạnh* — một pod gateway khởi động lại trong lúc sự cố không tải được JWKS, và tùy cấu hình sẽ từ chối mọi token (fail closed) hoặc crash lúc boot. Giảm nhẹ: kèm khóa công khai hiện hành trong cấu hình làm phương án dự phòng, hoặc cho cache JWKS một thời gian ân hạn dài.
- **Phantom token.** Thêm Redis vào danh sách những thứ phải sống. Auth-service chết có cùng hệ quả như trên; Redis chết là sập toàn bộ lưu lượng

đã xác thực.

Bài học khái quát: phi trạng thái mua cho bạn sự độc lập khỏi bên phát hành tại thời điểm request. Mọi cơ chế bạn thêm để giành lại quyền kiểm soát — JWKS, deny-list, ánh xạ mờ — đều đưa trở lại một phụ thuộc, và bạn phải gọi tên được nó và nói được chuyện gì xảy ra khi nó không sẵn sàng.

28.7 Khi JWT là công cụ sai

Bài viết khép lại bằng một cảnh báo đáng nhắc lại, vì chính con đường của dự án (Chương 27) có một phương án monolith. JWT xứng đáng với độ phức tạp của nó khi *nhiều bên độc lập* phải trả lời “ai đang gọi?” mà không chia sẻ kho session: microservices, đăng nhập một lần xuyên ứng dụng, liên kết OAuth2/OIDC. Nó là công cụ sai khi hệ thống là một khối triển khai duy nhất có đủ chỗ cho session trong bộ nhớ, khi quyền thay đổi thường xuyên và phải có hiệu lực ngay, hoặc khi cơ quan quản lý đòi một phiên phải chấm dứt được trong mili giây. Những trường hợp đó, session phía server với cookie đơn giản hơn, thu hồi được, và nhàm chán — mà nhàm chán là một tính năng.

28.8 Ngoài dự án

Keycloak, Auth0, Cognito, Okta và Entra ID đều phát hành token RS256 hoặc ES256 và công bố JWKS; mọi resource server Spring trong ngành xác minh bằng cấu hình một dòng `jwk-set-uri` đã thấy ở Chương 5. Service mesh (Istio, Linkerd) hiện thực biến thể mTLS của zero trust để code ứng dụng vẫn đọc header trong khi tầng vận chuyển chứng minh ai viết chúng. Mẫu phantom token được sản phẩm hóa bởi Curity và các API gateway như Kong, Apigee dưới các tên “token introspection” và “opaque token exchange”. Biết phiên bản tự tay làm — một secret HS256, hai nơi giữ — là thứ khiến những sản phẩm đó đọc hiểu được.

28.9 Câu hỏi từ phòng vấn thật

Những câu này được hỏi như bài toán mở, không phải định nghĩa. Câu trả lời dưới đây là điều một kỹ sư senior nói trong hai phút đầu; người phỏng vấn sau đó đào vào đúng đánh đổi bạn vừa nêu.

Q: Làm sao vô hiệu một JWT trước khi nó hết hạn?

Bạn không làm được; bạn giới hạn thiệt hại. Access token sống vài phút, không phải vài giờ, nên “đợi hết hạn” là mức nền. Muốn thu hồi tức thì, thêm claim `jit` và một deny-list trong Redis với TTL bằng phần đời còn lại của token — một lượt tra mỗi request, chỉ lưu token bị thu hồi, và khi Redis không sẵn sàng có thể fail open hay fail closed theo chính sách. Mờ hoàn toàn (phantom token) cho thu hồi tức thì nhưng đặt Redis lên đường găng của mọi request. Trong dự án này refresh token đã là mờ và lưu trong cơ sở dữ liệu, nên “đăng xuất mọi nơi” là `DELETE FROM refresh_tokens WHERE user_id = ?` cộng thêm nhiều nhất 15 phút truy cập còn sót.

Q: Lưu lượng đăng nhập tăng từ 10 lên 10 000 mỗi giây. Cái gì gãy?

Ký trước tiên: ký RS256 tốn khoảng 1 ms CPU mỗi token, nên mười nghìn lần đăng nhập mỗi giây là mười nhân CPU chỉ để ký, trong khi HS256 gần như không đáng kể. Rồi tới băm mật khẩu, thứ cố ý chậm (bcrypt cost 10 khoảng 100 ms) — đó mới là bức tường thật, và câu trả lời là mở rộng ngang auth-service cộng rate limit theo IP và theo tài khoản tại gateway. Xác minh không phải vấn đề: verify RS256 mất vài chục micro giây và không cần mạng. Cũng kiểm tra rằng lệnh insert refresh token không dồn lên một index nóng và bảng refresh_tokens được dọn các dòng hết hạn.

Q: Hai region, một hệ thống auth. Cái gì phải chia sẻ?

Với HS256 là secret, một bài toán phân phối bạn không muốn có xuyên region. Với RS256 chỉ là bộ khóa công khai, thứ cache được, công khai, và có thể trữ an toàn trong khoảng chồng lẫn của chu kỳ xoay khóa. Đồng hồ quan trọng hơn khóa: exp và iat được so với đồng hồ của bên xác minh, nên cho phép một độ lệch nhỏ (clockSkewSeconds của JWT, thường 30–60 s) và chạy NTP ở mọi nơi. Refresh token có trạng thái, nên hoặc ghim refresh về region phát hành hoặc sao chép bảng; Postgres đơn lẻ của dự án sẽ là thứ đầu tiên phải đổi.

Q: Một token bị đánh cắp từ trình duyệt của người dùng. Bạn làm gì?

Khoanh vùng, rồi phát hiện. Khoanh vùng: thu hồi refresh token của user; access token chết trong vòng đời của nó. Phát hiện: xoay vòng refresh token biến một token cũ bị dùng lại thành tín hiệu — hai bên cùng đưa ra một họ token nghĩa là bị trộm, thu hồi cả họ. Phòng ngừa: giữ refresh token trong cookie `HttpOnly` để script bị tiêm không đọc được, và giữ access token trong bộ nhớ thay vì localStorage (Chương 38). Chỉ thêm ràng buộc thiết bị hoặc IP nếu bạn chấp nhận chi phí bảo động giả với người dùng di động.

Q: Sao không kiểm tra cơ sở dữ liệu ở mọi request cho xong?

Có thể, và monolith làm thế — đó là session. Lý do không làm trong kiến trúc service là mọi service sẽ cần kho user hoặc một lời gọi tới service có nó, đặt auth-service lên đường găng của mọi thứ và ghép mọi lần deploy vào nó. Token có chữ ký cho mỗi chặng tự trả lời “đây là ai” tại chỗ. Câu trả lời thật thà nêu cái giá — độ trễ thu hồi — và cái bạn đã trả để mua lại quyền kiểm soát: vòng đời ngắn, refresh token có trạng thái, và tùy chọn một deny-list.

Q: Trong token của bạn có gì, và cố ý không có gì?

Subject, vai trò, thời điểm phát hành, hạn dùng, và ở đây một e-mail — danh tính, không phải quyền hạn. Không có: danh sách permission, thành viên nhóm, bất cứ thứ gì thay đổi nhanh hơn hạn token, bất cứ thứ gì service khác sở hữu, và bất cứ thứ gì bí mật (payload đọc được). Nếu người phỏng vấn ép “cho permission vào để khỏi query”, câu trả lời senior là phép tính ở Mục 28.4: 4 KB nhân mọi request, và một quyết định phân quyền lỗi thời bị đóng băng suốt đời token.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Sau khi chuyển sang RS256, `api-gateway.yml` không còn chứa `app.jwt.secret`. Nếu toàn bộ config-server bị lộ, kẻ tấn công có giả mạo được token không? Giải thích file nào mới là thứ phải lộ.
2. Mô tả nghi thức xoay khóa 90 ngày và nêu chính xác kid giải quyết vấn đề gì ở bước 3.
3. Ba mươi service, mỗi cái xác minh token RS256 bằng JWKS tải lúc khởi động. Auth-service chết lúc 3 giờ sáng. User nào bị ảnh hưởng, khi nào, và sự kiện duy nhất nào trong lúc sự cố khiến một bên xác minh từ chối *toàn bộ* lưu lượng?
4. Một đồng nghiệp đề xuất thêm claim `courseIds` để course-service phân quyền mà không cần truy vấn. Nêu hai lý do từ chối không liên quan tới kích thước token.
5. Gateway chuyển tiếp request tới đường dẫn công khai mà không đựng header. Giải thích vì sao hôm nay đây là lỗ hổng tiềm ẩn chứ chưa phải đang hoạt động, và nêu cấu hình một dòng loại bỏ sự tiềm ẩn đó.
6. Admin vô hiệu một user đang đăng nhập. Cho biết độ trễ thu hồi với thiết kế hiện tại của dự án, và nêu phụ thuộc thêm mà mỗi thiết kế nhanh hơn trong hai thiết kế còn lại đưa vào.

Đào sâu: Cấu hình — Config Server, Secret và refresh ở quy mô lớn

Chương 2 giới thiệu config-server như “thêm một nguồn thuộc tính trong chồng”: một service HTTP nhỏ trao cho mỗi ứng dụng file YAML của nó lúc khởi động. Mô tả đó đầy đủ cho một chiếc laptop và thiếu sót nguy hiểm cho bất kỳ thứ gì khác. Chương này tháo rời đường đi cấu hình của dự án — server thực sự phục vụ gì, ai đọc được, chuyện gì xảy ra khi nó thay đổi, và chuyện gì xảy ra khi nó không có mặt — rồi xây lại theo hai cách khác nhau.

Ba câu hỏi sắp xếp chương này:

1. **Cấu hình tập trung giải quyết vấn đề gì**, và trong ba backend của nó (native, git, nền tảng) cái nào giải quyết tốt nhất cho hệ thống nào?
2. **Bán kính thiệt hại của một file cấu hình là gì** — ai đọc nó, quy tắc ưu tiên nào áp dụng, và một giá trị mặc định trong file được git theo dõi thực sự có nghĩa gì?
3. **Cái gì gây khi server chết**, khi một giá trị phải đổi lúc chạy, và khi một secret phải xoay?

29.1 Vì sao tập trung, và nó giải quyết gì

Quy tắc twelve-factor là *tách bạch nghiêm ngặt cấu hình khỏi code*: một image được build một lần và thẳng cấp nguyên vẹn qua các môi trường; chỉ cấu hình bao quanh nó khác đi. Mọi cơ chế cấu hình đều là một cách thỏa mãn quy tắc đó, và config-server là cơ chế đầu tiên hệ sinh thái Spring sinh ra. Nó giải quyết ba vấn đề cụ thể cùng lúc:

- **Trùng lặp.** Năm service dùng chung địa chỉ Kafka, endpoint tracing, mẫu log. Một tài liệu cho mỗi service ở một chỗ tốt hơn năm bản sao trôi dạt.
- **Nhận biết môi trường.** Cùng một service hỏi `/course-service/prod` hay `/course-service/default` và nhận đúng tài liệu, từ cùng một image.
- **Thay đổi không cần build lại.** Với backend git, một thay đổi cấu hình là một commit — được review, có audit, hoàn tác được — và có hiệu lực ở lần khởi động hay refresh kế tiếp, không build lại image nào.

Dự án nhận trọn lợi ích đầu, không nhận lợi ích thứ hai (chỉ có một profile,

default), và lợi ích thứ ba chỉ trên nguyên tắc: với backend native các tài liệu nằm trong chính jar của config-server, nên đổi một giá trị *đúng là* phải build lại.

QUYẾT ĐỊNH — BA BACKEND, BA HỆ THỐNG

Native classpath (dự án này): cấu hình và code trong một repo, một pipeline, một phiên bản. Đúng khi một đội sở hữu mọi thứ và mọi thay đổi cấu hình đăng nào cũng đi cùng thay đổi code. **Backend git**: cấu hình trong repository riêng với lịch sử riêng; đúng khi vận hành và phát triển là những người khác nhau, khi cùng một service chạy ở nhiều môi trường, hay khi kiểm toán viên hỏi “ai đổi timeout và khi nào”. **Nền tảng** (ConfigMap và Secret, không server): đúng khi mọi đích triển khai đều là Kubernetes, vì nền tảng đã cung cấp cấu hình có phiên bản, mount được, bảo vệ bằng RBAC và bớt một JVM phải chạy. Dự án đang ở cái đầu và hướng tới cái thứ ba (Mục 29.6); cái thứ hai là thứ phần lớn doanh nghiệp bạn phỏng vấn thực sự đang chạy.

29.2 Đọc đường đi thật

Theo dõi một thuộc tính, `spring.datasource.url` của `course-service`, từ file tới pod đang chạy. Nó bắt đầu trong jar của `config-server`:

```
# config-server/src/main/resources/configurations/course-service.yml
server:
  port: 8082
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_course ①
    username: postgres
    password: root ②
  app:
    s3:
      endpoint: ${APP_S3_ENDPOINT:http://localhost:9000} ③
      secret-key: ${APP_S3_SECRET_KEY:minioadmin}
```

File `application.yml` đóng gói của service nói lấy tài liệu đó ở đâu:

```
# course-service/src/main/resources/application.yml
spring:
  application:
    name: course-service ④
  config:
    import: optional:configserver:http://localhost:8888 ⑤
```

Và container ghi đè cả địa chỉ lẫn giá trị:

```
# deploy/k8s/base/course-service.yaml (env excerpt)
- { name: SPRING_CONFIG_IMPORT,
  value: "configserver:http://config-server:8888" } ⑥
- { name: SPRING_DATASOURCE_URL,
  value: "jdbc:postgresql://postgres-course:5432/ts_course" }
envFrom:
- secretRef: { name: postgres-course-app } ⑦
```

- ① Một giá trị cứng, không phải placeholder: tài liệu được phục vụ nói localhost. Nó chỉ đúng trên laptop.
- ② Mật khẩu cơ sở dữ liệu dạng cleartext, trong một file git theo dõi, phục vụ qua HTTP. Ý định là chỉ cho dev; không gì trong file nói thế.
- ③ Thành ngữ \$VAR:default: config-server phục vụ placeholder *chưa phân giải*; mỗi client tự phân giải với môi trường của mình lúc khởi động. Đó là lý do cùng một tài liệu chạy được trong Compose và Kubernetes — và là lý do một giá trị mặc định trong đó là giá trị mọi môi trường sẽ lặng lẽ dùng trừ khi được bảo khác.
- ④ Khóa tra cứu. Server ánh xạ course-service sang course-service.yaml.
- ⑤ optional: — nếu không gì trả lời ở 8888, cứ khởi động từ file đóng gói. Đúng cho một dev chạy một service trong IDE; sai cho một container.
- ⑥ Container thay cả dòng import: host mới, và *không* có optional:. Config-server không với tới được giờ là chết người lúc khởi động. Compose che việc này bằng depends_on: condition: service_healthy; Kubernetes thì không, và pod crash-loop cho tới khi server Ready (Chương 15).
- ⑦ Mật khẩu ở ② bị Secret ghi đè bằng SPRING_DATASOURCE_PASSWORD. Giá trị mặc định cleartext vì thế vô hại trong cụm — *chừng nào* Secret còn tồn tại. Xóa nó đi và service không thất bại; nó kết nối bằng root.

29.2.1 Ưu tiên, chính xác

Spring Boot dựng một Environment từ một danh sách có thứ tự các nguồn thuộc tính; nguồn đầu tiên định nghĩa một khóa sẽ thắng. Với dự án này thứ tự quan trọng, cao nhất trước:

1. Tham số dòng lệnh và SPRING_APPLICATION_JSON (không dùng ở đây).
2. Biến môi trường của OS — env và envFrom của Deployment, với relaxed binding biến SPRING_DATASOURCE_URL thành spring.datasource.url.
3. Tài liệu import từ config-server.
4. File application.yaml đóng gói đã thực hiện import.

Hai hệ quả dễ trả lời sai trong phỏng vấn. Thứ nhất, tài liệu được import

đứng trên file đã import nó: config-server ghi đè được mọi thứ đóng gói trong jar, nhưng không bao giờ ghi đè được thứ nền tảng đặt. Thứ hai, placeholder `$VAR:default` *không* phải quy tắc ưu tiên; nó được phân giải sau khi ưu tiên đã chọn xong tài liệu. Biến môi trường `APP_S3_SECRET_KEY` thắng giá trị mặc định không phải vì biến môi trường đứng trên config-server, mà vì chính văn bản của config-server đã yêu cầu nó.

THỰC HÀNH — ĐỌC THỨ SERVER PHỤC VỤ

Config-server không có xác thực và Compose publish cổng của nó. Hỏi nó tài liệu của course-service đúng như client làm:

```
$ curl -s http://localhost:8888/course-service/default | \
python -m json.tool | head -20
{
  "name": "course-service",
  "profiles": ["default"],
  "label": null,
  "version": null,
  "propertySources": [
    {
      "name": "classpath:/configurations/course-service.yml",
      "source": {
        "server.port": 8082,
        "spring.datasource.url": "jdbc:postgresql://localhost...",
        "spring.datasource.username": "postgres",
        "spring.datasource.password": "root",
```

Để ý `version: null` — backend native không có commit id để báo. Backend git điền vào đó mã SHA, và đó là cách bạn trả lời “pod này đang chạy cấu hình nào?” trong production.

29.3 Ưu và nhược, thành thật

Điểm mạnh	Trong dự án này
Một tài liệu cho mỗi service	Thật: năm file, không trùng lặp địa chỉ Kafka hay Zipkin.
Placeholder theo môi trường	Thật: cùng các file phục vụ Compose và Kubernetes không sửa gì.
Lấy lúc khởi động, không phụ thuộc lúc chạy	Thật: đã lên rồi, service không nói chuyện với server nữa.
Điểm yếu	
Không xác thực	Ai với tới 8888 đọc được mọi tài liệu, kể cả mật định và mật khẩu.
Native = build lại để đổi	Chỉnh một timeout là một lượt build image config-server và rollout.
Mặc định dùng được cho secret	JWT_SECRET và root có giá trị dự phòng dùng được trong git.
Ràng buộc thứ tự	Mọi pod phụ thuộc vào một pod không còn việc gì sau khi boot.
Chỉ một profile	default ở mọi nơi; môi trường chỉ khác nhau bằng biến môi trường.

Dòng cuối đáng một câu. Profile của Spring được thiết kế đúng cho việc tách dev/staging/prod, và config-server tự động phục vụ `course-service-prod.yml` chồng lên `course-service.yml` khi `SPRING_PROFILES_ACTIVE=prod`. Dự án không bao giờ dùng việc này; mọi khác biệt môi trường là một biến môi trường. Đó là một đơn giản hóa bảo vệ được — một cơ chế thay vì hai — nhưng nghĩa là các tài liệu được phục vụ không bao giờ nói được “giá trị này chỉ cho development”.

29.4 Cái gì gãy

CẠM BẦY — KHÓA KÝ NẪM TRONG REPOSITORY

Triệu chứng: không có, và đó chính là vấn đề. Mở `api-gateway.yml` và `auth-service.yml` được phục vụ:

```
app:
```

```
jwt:  
  secret: ${JWT_SECRET:mySecretKeyThatIsAtLeast256Bits...}
```

Nguyên nhân: giá trị mặc định là một khóa HS256 *hợp lệ*, commit vào một repository công khai, và create-secrets.sh mặc định JWT_SECRET thành đúng chuỗi đó. Bất kỳ triển khai nào quên đặt biến sẽ ký token bằng khóa cả internet đọc được — và với HS256 (Chương 28) đọc được khóa nghĩa là đúc được token cho bất kỳ user nào. Cách sửa có hai nửa: bỏ giá trị mặc định để biến thiếu thì thất bại lúc khởi động (\$JWT_SECRET không có dấu hai chấm — Spring từ chối phân giải), và coi mọi \$VAR:value trong tài liệu được phục vụ như một câu hỏi: “tôi có ổn không nếu production chạy với giá trị này?” Mật khẩu và khóa không bao giờ qua được bài kiểm tra đó; cổng và hostname thường qua.

CẠM BÃY — REFRESH MỘT SERVICE, HỎNG TẤT CẢ

Triệu chứng: sau khi “xoay JWT secret” bằng POST /actuator/refresh trên auth-service, mọi request qua gateway trả về 401 cho tới khi gateway khởi động lại. Nguyên nhân: refresh là *theo từng instance*. Auth-service đã dừng lại các bean @RefreshScope và giờ ký bằng khóa mới; gateway vẫn xác minh bằng khóa cũ. Kể cả khi cả hai được refresh, không có cửa sổ chồng lấn — mọi token phát trước lần refresh trở nên vô hiệu ngay khoảnh khắc gateway refresh. Một khóa đối xứng dùng chung giữa bên ký và bên xác minh không thể hot-swap được; nó cần nghi thức dựa trên kid của Chương 28. Quy tắc chung: refresh dành cho các giá trị mà bản cũ và bản mới cùng tồn tại được (một feature flag, một mức log, một timeout), không phải cho các giá trị hai bên phải thống nhất.

CẠM BÃY — KHỎE MẠNH NHƯNG SAI

Triệu chứng: config-server báo UP, service khởi động, giá trị sai đang có hiệu lực. Nguyên nhân: health indicator của config-server kiểm tra *backend* có trả lời — với native, là thư mục classpath có tồn tại. Nó không nói gì về nội dung. Một lỗi chính tả ở tên khóa YAML (access-key) được phục vụ vui vẻ và bị client bỏ qua vui vẻ, rồi client dùng giá trị mặc định của nó. Không bên nào log lỗi. Phòng thủ nằm ở client: bind cấu hình vào một record @ConfigurationProperties với @Validated và @NotBlank trên các trường bắt buộc, để khóa thiếu hay sai chính tả thất bại lúc khởi động với tên thuộc tính trong thông báo.

29.5 Chuyện gì gây lúc 3 giờ sáng

Pod config-server bị evict lúc 3 giờ sáng; không gì khác bị.

- **Service đang chạy: không sao.** Mỗi cái đã lấy tài liệu một lần lúc boot và giữ trong bộ nhớ. Không đường request nào chạm tới config-server. Đây là tính chất biện minh cho một replica duy nhất.
- **Service khởi động lại trong lúc sự cố: crash loop.** Import của nó không optional; lấy thất bại; JVM thoát; kubelet khởi động lại với back-off lũy thừa (Chương 24). Khi config-server trở lại, lần khởi động kế tiếp thành công — sau tối đa năm phút chờ back-off. Hệ thống hội tụ; nó không hội tụ nhanh.
- **Rolling update trong lúc sự cố: đứng.** Pod mới không bao giờ Ready, pod cũ tiếp tục phục vụ (maxUnavailable của Chương 15), và rollout chờ. Hành vi đúng, nhưng pipeline deploy sẽ báo timeout.
- **Khôi phục optional:: tệ hơn.** Pod khởi động lại boot từ giá trị mặc định đóng gói — localhost:5432, localhost:9092 — thành Ready (probe của nó là /v3/api-docs, không cần cơ sở dữ liệu), nhận lưu lượng, và thất bại mọi request. Sự cố trông như “config-server chết” giờ thành “course-service trả 500”, cách nguyên nhân hai bước nhảy.

Retry với back-off là điểm giữa: giữ import bắt buộc, nhưng cho client thử một lúc trước khi chết. Nó cần spring-retry và spring-boot-starter-aop trên classpath, rồi:

```
spring:
  cloud:
    config:
      fail-fast: true           # unreachable server is fatal ...
      retry:
        max-attempts: 20       # ... after 20 tries
        initial-interval: 2000 # 2 s, then x1.5 each time
        multiplier: 1.5
        max-interval: 15000    # capped at 15 s: ~4 minutes total
```

So với để kubelet làm việc retry: khởi động lại JVM tốn hai mươi giây nạp class mỗi lần và back-off tăng tới năm phút; retry trong tiến trình không tốn gì và chặn trên ở mười lăm giây. Cả hai đều hội tụ; một cái lãng phí một CPU mà chiếc máy hai nhân không có.

29.6 Nâng cấp giải pháp

Hai nâng cấp, theo thứ tự một doanh nghiệp sẽ làm.

29.6.1 Nâng cấp 1: backend git với giá trị mã hóa

Giữ config-server, chuyển tài liệu của nó vào một repository, và ngừng commit secret dạng cleartext. Phía server:

```
# config-server/src/main/resources/application.yml
server:
  port: 8888
spring:
  application:
    name: config-server
  profiles:
    active: git ①
  cloud:
    config:
      server:
        git:
          uri: https://git.example.com/ts/config-repo.git
          default-label: main ②
          search-paths: '{application}' ③
          clone-on-start: true ④
          username: ${CONFIG_GIT_USER}
          password: ${CONFIG_GIT_TOKEN}
      encrypt:
        key: ${CONFIG_ENCRYPT_KEY} ⑤
  management:
    endpoints:
      web:
        exposure:
          include: health
```

- ① Profile chọn backend. native và git là hai cái có sẵn; Vault và JDBC cũng tồn tại.
- ② Label là một branch hay tag. Ghim môi trường production vào release-2026.09 và một thay đổi cấu hình là một lượt merge, không phải một lượt sửa.
- ③ Một thư mục cho mỗi service trong repository: course-service/application.yml. Cùng tên file như hôm nay, sâu hơn một thư mục.
- ④ Clone lúc khởi động thay vì lúc có request đầu tiên, để health check chứng minh credential dùng được trước khi client nào hỏi.
- ⑤ Một khóa đối xứng cho các endpoint /encrypt và /decrypt. Đó là secret duy nhất config-server phải giữ, và nó phải đến từ nền tảng (một Secret của Kubernetes), không bao giờ từ một file trong bất kỳ repository nào.

Có khóa rồi, mã hóa một giá trị là một request, và repository chỉ lưu bản mã:

```
$ curl -s -X POST http://localhost:8888/encrypt \
```

```
-d 'a-real-256-bit-signing-key-generated-with-openssl'
AQBt3s6...b3f0e # ciphertext; safe to commit
```

```
# config-repo/api-gateway/application.yml
app:
  jwt:
    secret: '{cipher}AQBt3s6...b3f0e' # decrypted by the server on serve
```

Tiền tố {cipher} bảo server giải mã trước khi phục vụ. Repository giờ không chứa secret dùng được nào, bẫy giá trị mặc định biến mất (không có mặc định), và git log -- api-gateway/ trả lời ai đổi khóa và khi nào. Thứ nó *không* sửa: tài liệu được phục vụ vẫn đi qua mạng dạng cleartext tới một endpoint không xác thực. Thêm spring-boot-starter-security vào config-server với credential HTTP basic, và cấp cho mỗi client spring.cloud.config.username/password từ Secret riêng của nó.

29.6.2 Nâng cấp 2: cho server nghỉ hưu

Trên Kubernetes nền tảng có thể tự phục vụ các tài liệu. Kustomize sinh một ConfigMap cho mỗi service thẳng từ các file hiện có, nên lịch sử git không đổi:

```
# deploy/k8s/base/kustomization.yaml (excerpt)
configMapGenerator:
  - name: course-service-config
    files:
      # key in the ConfigMap = application.yml, value = the file
      - application.yml=../../config-server/src/main/resources/configurations/course-
        ↪ service.yml
generatorOptions:
  disableNameSuffixHash: false # hash suffix: a changed file = a
                              # new ConfigMap name = a rollout
```

Deployment mount nó như một thư mục và trỏ Spring vào thư mục thay vì server:

```
# deploy/k8s/base/course-service.yaml (excerpt)
spec:
  template:
    spec:
      containers:
        - name: course-service
          env:
            # replaces configserver:http://config-server:8888
            - { name: SPRING_CONFIG_IMPORT, value: "file:/config/" }
            - { name: SPRING_DATASOURCE_URL,
              value: "jdbc:postgres://postgres-course:5432/ts_course" }
```

```
envFrom:
  - secretRef: { name: postgres-course-app }
volumeMounts:
  - { name: config, mountPath: /config, readOnly: true }
volumes:
  - name: config
    configMap: { name: course-service-config }
```

Spring Boot coi dấu gạch chéo cuối là import thư mục và nạp `application.yml` từ đó, với đúng thứ tự ưu tiên mà tài liệu từ `config-server` từng có: trên file đóng gói, dưới biến môi trường. Không gì trong code hay jar của service thay đổi; Compose vẫn chạy vì import `optional:configserver`: đóng gói vẫn trỏ tới server (vẫn chạy, trong Compose). Rồi xóa `config-server.yaml` khỏi base, và cụm bớt một JVM, bớt một phụ thuộc khởi động, bớt một endpoint không xác thực.

Hậu tố hash trong tên là chi tiết khiến cách này tốt hơn server, chứ không chỉ tương đương. Một `ConfigMap` tên `course-service-config-7b9c4` đổi tên khi nội dung đổi; Deployment tham chiếu tên mới là một pod template mới; Kubernetes roll các pod. Thay đổi cấu hình, rollout và rollback là cùng một thao tác với thay đổi code — đúng thứ mục GitOps của Chương 27 muốn.

QUYẾT ĐỊNH — KHI NÀO BỎ HẸN CONFIG-SERVER

Bỏ khi ba điều đúng: mọi runtime đều là Kubernetes, không service nào cần giá trị đổi mà không khởi động lại pod, và secret đã sống trong nền tảng (đúng thế — Chương 16). Giữ khi bất kỳ điều nào sai: service trên VM hay trong Compose ở production, một yêu cầu refresh lúc chạy thật sự (hiếm; feature flag được phục vụ tốt hơn bởi hệ thống chuyên dụng), hay một tổ chức mà dấu vết audit cấu hình phải độc lập với repository triển khai. Người phỏng vấn hỏi “bạn có dùng Spring Cloud Config trên Kubernetes không?” đang chờ nghe danh sách này, và câu “nền tảng đã cung cấp sẵn”.

29.7 Ngoài dự án

Config-server backend git với giá trị `{cipher}` là chuẩn từ khoảng 2015 tới 2020 và vẫn là thứ bạn gặp ở phần lớn công ty Java. Hai thứ đã thay thế từng phần của nó. HashiCorp Vault (và các bản tương đương trên cloud, AWS Secrets Manager, Azure Key Vault) giữ secret với lease và xoay vòng; Spring Cloud Vault hay External Secrets Operator đưa chúng vào pod, và repository cấu hình chỉ giữ tham chiếu. Các dịch vụ feature flag (LaunchDarkly, Unleash, OpenFeature) tiếp quản trường hợp refresh lúc chạy với quy tắc nhắm mục tiêu và audit mà `@RefreshScope`

chưa bao giờ có. Thứ sống sót nguyên vẹn là kỷ luật: cấu hình ngoài image, thứ tự ưu tiên bạn thuộc lòng, và không secret dùng được trong bất kỳ file nào bạn commit.

29.8 Câu hỏi từ phỏng vấn thật

Q: Bạn phải xoay mật khẩu cơ sở dữ liệu cho năm mươi service mà không có downtime. Trình bày từng bước.

Xoay vòng không bao giờ là một lần ghi; nó là một sự chồng lấn. Thứ nhất, tạo credential mới trong cơ sở dữ liệu trong khi cái cũ vẫn dùng được — Postgres cho phép hai role hay hai mật khẩu cùng tồn tại, và các dịch vụ được quản lý (RDS, Cloud SQL) có “dual password” đúng cho việc này. Thứ hai, đưa giá trị mới vào Secret của nền tảng (hoặc vào repository cấu hình dưới dạng {cipher}) và kích hoạt các bên tiêu thụ: trên Kubernetes là một lượt rollout restart cho mỗi Deployment, việc mà mẹo hash tên ConfigMap làm hộ bạn; với Spring Cloud Bus là một POST /actuator/busrefresh mà mọi instance nhận qua Kafka. Thứ ba, theo dõi log kết nối của cơ sở dữ liệu cho tới khi không session nào dùng role cũ, rồi thu hồi. Cạm bẫy của dự án ở Chương 16 — “đổi Secret rồi, không có gì xảy ra” — là thứ bước hai ngăn chặn. Câu trả lời senior gọi tên sự chồng lấn một cách tường minh và nói bước nào được phép thất bại: thu hồi, thứ bạn chỉ cần hoãn lại.

Q: Một thay đổi cấu hình phải tới mọi instance trong vòng một phút. Khởi động lại thì quá chậm. Bạn làm gì?

Phân biệt loại giá trị trước. Một mức log, một rate limit, một feature flag: đánh dấu bean @RefreshScope, mở /actuator/busrefresh, thêm spring-cloud-starter-bus-kafka — dự án đã có Kafka — và một POST phát sóng một RefreshRemoteApplicationEvent; mỗi instance lấy lại tài liệu và dựng lại các bean refresh-scoped trong vài giây. Một URL cơ sở dữ liệu, một địa chỉ bootstrap Kafka, một khóa ký: *đừng* refresh những thứ đó, vì connection pool, producer hay bên xác minh giữ giá trị cũ trong các object không được dựng lại sạch sẽ; khởi động lại thay vào đó. Nếu yêu cầu thực ra là “flag cho thử nghiệm sản phẩm”, hãy nói rằng một dịch vụ feature flag (Unleash, LaunchDarkly) với SDK poll vài giây một lần là công cụ chuyên dụng, còn @RefreshScope là cách chữa cháy mười năm tuổi.

Q: Config server là điểm lỗi đơn. Bạn xử lý thế nào?

Bắt đầu bằng thu hẹp nó là điểm lỗi đơn *cho cái gì*: khởi động, không phải lưu lượng. Service đang chạy không bao giờ gọi lại nó, nên sự cố config-server chỉ ảnh hưởng khởi động lại và scale-out. Rồi giảm cả điều đó: hai replica sau Service (nó phi trạng thái với backend native hay git — mỗi replica một bản clone git là ổn), fail-fast cộng retry để pod đang boot chờ tới vài phút thay vì chết, và một readiness probe trên config-server để Service không bao giờ định tuyến tới replica còn đang clone. Câu trả lời mạnh nhất loại bỏ phụ thuộc: phục vụ tài liệu từ ConfigMap, và “điểm lỗi đơn” trở thành API server của Kubernetes, thứ bạn đã phụ thuộc cho mọi thứ khác.

Q: Làm sao giữ secret ngoài git mà cấu hình vẫn trong git?

Ba tầng, mỗi tầng chấp nhận được ở một quy mô khác nhau. Nhỏ nhất: `/encrypt` của config-server với khóa đối xứng chỉ server giữ; git lưu văn bản `{cipher}`, và repository bị lộ vô dụng nếu không có khóa. Trung bình: Secret của Kubernetes tạo ngoài luồng (`create-secrets.sh` của dự án) và được tham chiếu theo tên trong manifest, nên git giữ hình dạng nhưng không bao giờ giữ giá trị. Lớn nhất: một trình quản lý secret — Vault, AWS Secrets Manager — với External Secrets Operator đồng bộ vào Secret của Kubernetes, để xoay vòng và audit sống ở nơi secret sống. Cái bẫy cần gọi tên: một `$VAR:default` với giá trị mặc định dùng được là một secret trong git bất kể bạn chọn tầng nào.

Q: Hai pod của cùng một service đang chạy với cấu hình khác nhau. Sao lại thế, và ngăn bằng cách nào?

Nguyên nhân thường gặp: một rolling update kẹt nửa chừng (pod cũ với tài liệu cũ, pod mới với tài liệu mới — bình thường và tạm thời); một config-server có bản clone git cũ trên một replica vì clone-on-start tắt và replica đó không bao giờ làm mới; hay, tinh vi nhất, một `/actuator/refresh` gửi tới một instance qua load balancer tình cờ chọn nó. Phòng ngừa là biến cấu hình thành một phần của pod template: mount từ ConfigMap có tên mang hash nội dung, để “cấu hình nào” được trả lời bằng “ReplicaSet nào”. Phát hiện là phơi phiên bản cấu hình — mã SHA git mà config-server trả về dưới dạng `version`, hoặc tên ConfigMap — trong `/actuator/info`, và so sánh giữa các pod trước khi tin bất kỳ báo cáo “nó chạy trên một trong số chúng” nào.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. `course-service.yml` được phục vụ nói `password: root` và `Secret` của `Deployment` nói khác. Nêu cái nào thắng và vì sao, rồi mô tả chuyện gì xảy ra nếu `Secret` bị xóa và pod khởi động lại.
2. Giải thích vì sao `$APP_S3_SECRET_KEY:minioadmin` trong tài liệu được phục vụ không phải quy tắc ưu tiên, và `config-server` thực sự gửi gì cho client với khóa đó.
3. Một đồng nghiệp chạy `POST /actuator/refresh` trên cả `auth-service` và `api-gateway` sau khi đổi `JWT_SECRET`. Liệt kê hai cửa sổ lỗi riêng biệt và nêu cơ chế từ Chương 28 mà xoay khóa nóng thực sự cần.
4. `Config-server` chết; `course-service` được scale từ một lên hai replica trong lúc sự cố. Mô tả pod mới làm gì với `configserver:` (không `optional`), với `optional:configserver:`, và với `fail-fast` cộng `retry`.
5. Sau khi chuyển sang `ConfigMap`, cái gì khiến pod khởi động lại khi một giá trị đổi, và vì sao điều đó khiến việc chuyển đổi *có năng lực hơn* server nó thay thế chứ không chỉ tương đương?
6. Phòng vấn: “Bạn có triển khai `Spring Cloud Config` trên `Kubernetes` không?” Nêu ba điều kiện để bạn bỏ nó và một điều kiện để bạn giữ.

Đào sâu: Khám phá dịch vụ — bên trong Eureka, các kiểu lỗi và phương án Kubernetes

Chương 3 giải thích Eureka trong bốn gạch đầu dòng — đăng ký, heartbeat, cache, tự bảo toàn — và Chương 27 phác họa việc cho nó nghỉ hưu trong hai dòng. Giữa hai chỗ đó là mọi thứ người vận hành thực sự gặp: các bộ đếm thời gian quyết định một instance đã chết còn nhận lưu lượng bao lâu, ngưỡng chuyển registry sang chế độ nó ngừng nói thật, và sự thật khó xử rằng trên Kubernetes dự án chạy hai hệ thống khám phá cùng lúc mà gateway chỉ hỏi một trong hai. Chương này đo những thứ đó, làm gãy chúng, rồi chỉ ra cấu hình chính xác để gỡ Eureka mà không làm hỏng Compose.

Ba câu hỏi sắp xếp chương này:

1. **Registry giải quyết gì mà DNS không giải quyết**, và vì sao stack cổ điển đặt việc tra cứu ở *client*?
2. **Registry có thể cũ tới mức nào** — tính bằng giây, với phép tính — và sự cũ đó gây gì trong lúc deploy?
3. **Cái gì thay thế từng tính năng của Eureka** khi nền tảng là Kubernetes, và làm sao chuyển đổi mà không có ngày “cắt cầu dao”?

30.1 Vì sao cần registry, và vì sao ở client

Vấn đề là địa chỉ. Instance có IP đổi theo mỗi lần khởi động lại; bên gọi cần một cái tên ổn định. Có ba họ câu trả lời:

- **Load balancer phía server.** Bên gọi quay một địa chỉ cố định (một nginx, một AWS ELB); nó biết các instance. Đơn giản cho bên gọi, nhưng mỗi bước nhảy thêm độ trễ và bộ cân bằng tự nó phải được cập nhật.
- **Registry phía client** (Eureka, Consul). Mỗi instance đăng ký; mỗi bên gọi tải danh sách về và cân bằng cục bộ. Không thêm bước nhảy, không có bộ cân bằng phải scale, bên gọi sống sót qua sự cố registry nhờ cache — và mọi bên gọi phải nhúng một thư viện client.
- **DNS của nền tảng** (Service của Kubernetes). Nền tảng theo dõi readiness của pod và giữ danh sách endpoint của một IP ảo luôn cập nhật; bên gọi

dùng DNS thuần và không biết gì. Không thư viện, không tiến trình registry, nhưng chỉ bên trong nền tảng đó.

Netflix xây cái thứ hai năm 2012 vì họ chạy trên EC2 không có DNS nền tảng để dựa vào và hàng nghìn bên gọi JVM mang được thư viện. Spring Cloud thừa kế nó, và dự án này thừa kế từ Spring Cloud. Lịch sử đó là câu trả lời thành thật cho “vì sao Eureka”: không phải vì nó hợp nhất với một cụm Kubernetes năm 2026, mà vì stack cổ điển vẫn là thứ phần lớn công ty Java đang chạy và phần lớn phỏng vấn Java đang hỏi.

QUYẾT ĐỊNH — PHÍA CLIENT, RỒI PHÍA NỀN TẢNG

Dự án giữ Eureka qua Compose và các milestone Kubernetes đầu tiên vì hai lý do: Compose không có DNS gắn với readiness, nên registry là cách duy nhất để gateway bỏ qua một service đã lên nhưng chưa sẵn sàng; và chạy hai hệ thống song song là cách bạn học nền tảng thay thế cái gì. Khoảnh khắc Compose không còn là đích triển khai — nó vốn chỉ là công cụ dev — registry là chi phí thuần: thêm một JVM, thêm một phụ thuộc khởi động, và một ý kiến thứ hai về pod nào còn sống, bất đồng với kubelet tới bốn phút. Mục 30.6 thực hiện cú chuyển.

30.2 Đọc cấu hình thật

Server là một cổng và một annotation. Toàn bộ hành vi nằm ở các giá trị mặc định dự án không bao giờ đặt, nên đọc phía client trước:

```
# course-service/src/main/resources/application.yml
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/ ①
```

```
# deploy/k8s/base/course-service.yaml (env excerpt)
- { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE,
  value: "http://discovery-server:8761/eureka/" } ②
- { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" } ③
```

Và bên tiêu thụ, các route của gateway:

```
# configurations/api-gateway.yaml (excerpt)
- id: course-service-courses
  uri: lb://course-service ④
```

```
predicates:  
- Path=/api/courses/**  
filters:  
- StripPrefix=2
```

- ① Địa chỉ duy nhất client được cho biết. Mọi thứ khác — course-service ở đâu, auth-service ở đâu — trở về từ URL này dưới dạng một tài liệu registry.
- ② Trên Kubernetes discovery-server được *DNS nền tảng* phân giải thành một ClusterIP. Chính registry được tìm thấy bằng cơ chế nó sắp sao chép.
- ③ Đăng ký IP của pod, không phải tên pod (Chương 3). Địa chỉ đăng ký là thứ gateway sẽ quay, bỏ qua hoàn toàn Service course-service.
- ④ lb:// trao cái tên cho Spring Cloud LoadBalancer, nó hỏi cache cục bộ của Eureka client để lấy instance và chọn một cái theo vòng tròn. Service Kubernetes course-service:8082 không bao giờ được hỏi trên đường này.

Điểm ④ là câu cần nhớ từ chương này. Trên cụm, *readiness probe bảo vệ Service của Kubernetes, và gateway không dùng Service của Kubernetes*. Thứ duy nhất đứng giữa gateway và một pod chưa sẵn sàng là trạng thái Eureka — thứ pod tự báo về chính nó.

30.2.1 Các bộ đếm thời gian

Mọi thứ dưới đây là mặc định dự án thừa kế. Mỗi cái là một thuộc tính bạn đặt được; mỗi cái có lý do cho giá trị của nó.

Thuộc tính (eureka.*)	Mặc định	Ý nghĩa
instance.lease-renewal-interval-in-seconds	30 s	chu kỳ heartbeat
instance.lease-expiration-duration-in-seconds	90 s	im lặng lâu thế này = hết hạn
server.eviction-interval-timer-in-ms	60 s	chu kỳ quét lease hết hạn
server.response-cache-update-interval-ms	30 s	cache chỉ đọc của server
client.registry-fetch-interval-seconds	30 s	client tải lại registry
spring.cloud.loadbalancer.cache.ttl	35 s	cache của LoadBalancer
server.renewal-percent-threshold	0.85	ngưỡng tự bảo toàn

Giờ là phép tính người phỏng vấn muốn nghe.

Instance mới nhận lưu lượng. Spring Cloud đăng ký ngay khi web server khởi động (client Netflix thô sẽ đợi 40 s; EurekaAutoServiceRegistration của Spring ép một lần đăng ký theo yêu cầu). Cache chỉ đọc của server có thể cũ tới 30 s, lượt

tải registry của gateway chậm tới 30 s sau đó, và cache LoadBalancer của nó chậm tới 35 s sau nữa. Trường hợp xấu nhất: khoảng **95 giây** giữa “Tomcat đang lắng nghe” và “gateway biết”. Thông thường: một nửa.

Instance chết ngừng nhận lưu lượng. Pod bị giết mà không tắt êm (OOM, mất node). Lease hết hạn sau 90 s không heartbeat; lượt quét loại bỏ chạy mỗi 60 s, nên tới 150 s sau khi chết server vẫn liệt kê nó; cộng cùng 30 + 30 + 35 s cache: tới **245 giây** — bốn phút — trong đó gateway tiếp tục quay một IP không còn trả lời. Tắt êm gửi một DELETE lúc rời đi và bỏ qua 150 s đầu, còn lại tới 95 s cache cũ.

Tự bảo toàn. Mỗi phút server so heartbeat nhận được với heartbeat kỳ vọng: số instance $\times 2$ (một lần mỗi 30 s) $\times 0,85$. Dưới vạch, nó ngừng loại bỏ bất cứ thứ gì và dashboard hiện bằng đỏ. Dự án đăng ký năm client: kỳ vọng $5 \times 2 = 10$, ngưỡng $\lfloor 10 \times 0,85 \rfloor = 8$, và điều kiện là “nhiều hơn 8”. Giết *một* service và số đếm rơi xuống 8 — không nhiều hơn 8 — và registry vào chế độ tự bảo toàn. Trên stack này, một service crash nghĩa là *không* instance nào bị loại bỏ nữa cho tới khi nó quay lại. Netflix chỉnh ngưỡng đó cho hàng trăm instance, nơi mất 15 % trong một phút thật sự nghĩa là phân mảnh mạng; trên năm instance, nó nghĩa là thứ Ba.

THỰC HÀNH — XEM MỘT INSTANCE CHẾT VẪN SỐNG

Trong Compose, dừng course-service mà không cho nó hủy đăng ký, rồi hỏi registry về nó một phút sau:

```
$ docker compose kill course-service
$ sleep 60
$ curl -s -H 'Accept: application/json' \
  http://localhost:8761/eureka/apps/COURSE-SERVICE | \
  python -c "import sys,json; a=json.load(sys.stdin)['application']; \
    print([i['status'] for i in a['instance']])"
['UP']
```

Giờ mở <http://localhost:8761>. Sau lượt quét kế tiếp, instance hoặc biến mất — hoặc, vì bốn client còn lại gửi 8 lần gia hạn mỗi phút, trang mang bằng EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT, và course-service giữ UP lâu tùy bạn muốn đợi. Trong lúc đó `curl localhost:8080/api/courses` trả về 503 từ gateway ở mọi lần thử rơi vào địa chỉ đã chết.

30.3 Ưu và nhược, thành thật

- **Ứng hộ.** Không thêm bước nhảy mạng; bên gọi tiếp tục hoạt động qua sự cố registry nhờ cache; một cơ chế giống hệt trên laptop, trong Compose và trong cụm; một dashboard cho thấy ai đã đăng ký; và một instance có thể tự đánh dấu OUT_OF_SERVICE để rút lui có kiểm soát.
- **Phản đối, trong dự án này.** Một nguồn sự thật thứ hai về sự sống chậm hơn kubelet tới bốn phút; readiness probe bị bỏ qua trên đường của gateway (④ ở trên); tự bảo toàn chính cho một đội tàu lớn gấp trăm lần; một JVM 512 Mi không làm gì cho lưu lượng; prefer-ip-address là cách chữa cháy riêng cho nền tảng mà mọi manifest phải mang; và mọi service biên dịch với một thư viện client mà upstream đang ở chế độ bảo trì.
- **Trung lập.** Eureka là AP theo thiết kế: nó thích trả lời bằng dữ liệu cũ hơn là không trả lời. Đó là lựa chọn đúng cho một registry — bên gọi với danh sách hơi sai có thể thử lại; bên gọi không có danh sách thì không làm được gì — và cũng là lý do mọi con số ở trên đều là “tối”.

30.4 Cái gì gây

CẠM BẦY — ROLLING DEPLOY, BA MƯƠI GIÂY 503

Triệu chứng: `kubectl rollout restart deployment/course-service` thành công, readiness qua, và trong nửa phút gateway trả về 503 Service Unavailable trên `/api/courses`. Nguyên nhân: pod cũ hủy đăng ký khi SIGTERM và thoát; pod mới đăng ký với IP mới; nhưng LoadBalancer của gateway vẫn giữ IP cũ tới 95s cache. Service của Kubernetes chuyển tức thì và không ai hỏi nó. Sửa, theo chi phí tăng dần: một hook `preStop` ngủ 20–30s để pod cũ tiếp tục trả lời trong lúc cache cạn (`terminationGracePeriodSeconds` phải lớn hơn nó); cache ngắn hơn (`registry-fetch-interval-seconds: 5`, `spring.cloud.loadbalancer.cache.ttl: 5s`) với cái giá là tải lên registry; hoặc một filter Retry trên gateway cho lỗi kết nối để lần thử thứ hai rơi vào IP còn sống. Hoặc gỡ registry khỏi đường đi (Mục 30.6) và để Service làm việc nó sinh ra để làm.

CẠM BẦY — ĐÃ ĐĂNG KÝ, UP, VÀ TRẢ VỀ LỖI

Triệu chứng: dashboard hiện `course-service UP`; request thất bại với lỗi cơ sở dữ liệu. Nguyên nhân: trạng thái Eureka do instance tự báo, và mặc định nó là UP ngay khoảnh khắc đăng ký thành công — nó không hỏi actuator

health. Flyway có thể đã thất bại sau khi đăng ký, Postgres có thể đã biến mất, và instance vẫn nói UP mỗi 30 s. Sửa: `eureka.client.healthcheck.enabled: true` truyền actuator health sang trạng thái registry, để một health check DOWN thành DOWN trong Eureka ở heartbeat kế tiếp — 30 s sau, cộng cache. Readiness của Kubernetes sẽ rút pod trong 5 s; trên đường của gateway, nó không có cơ hội.

CẠM BÃY — HAI REPLICHA EUREKA, HAI CÂU TRẢ LỜI

Triệu chứng: sau khi scale discovery-server lên hai replica cho tính sẵn sàng, một số bên gọi thấy một instance mà những bên khác không thấy. Nguyên nhân: các peer sao chép đăng ký cho nhau bất đồng bộ; một client có defaultZone liệt kê cả hai địa chỉ nói chuyện với cái trả lời trước và có thể đọc một replica chưa nhận được bản sao. Mỗi server cũng đếm gia hạn độc lập, nên một cái có thể vào tự bảo toàn trong khi cái kia loại bỏ. Sửa: liệt kê cả hai peer trong defaultZone của mọi client và của mỗi server (chúng phải đăng ký với nhau), chấp nhận vài giây bất nhất là cái giá của AP, và đừng bao giờ scale một registry bạn sắp gỡ.

30.5 Chuyện gì gây lúc 3 giờ sáng

Pod discovery-server chết lúc 3 giờ sáng; không gì khác chết.

- **Lưu lượng gateway: không ảnh hưởng.** Eureka client của gateway giữ registry cuối cùng trong bộ nhớ và LoadBalancer tiếp tục định tuyến theo nó. Lướt tải registry thất bại mỗi 30 s và log cảnh báo; cache không hết hạn. Đây là lời hứa AP được giữ.
- **Heartbeat: thất bại lặng lẽ.** Mọi client log một lần gia hạn thất bại mỗi 30 s. Không gì khác đổi; lease là khái niệm của server và server đã đi.
- **Service khởi động lại trong lúc sự cố: vô hình.** Nó boot, đăng ký thất bại, thử lại ở nền — và nó không phục vụ gì qua gateway cho tới khi registry trở lại và 95 s cache cạn. Service Kubernetes của nó có nó làm endpoint trong vài giây; không ai dùng Service đó.
- **Service chết trong lúc sự cố: vẫn được định tuyến tới.** Không server, không loại bỏ. Gateway giữ IP chết cho tới khi registry trở lại và đã loại bỏ nó và cache đã quay vòng: trọn 245 s, tính từ lúc server trở lại.
- **Khi discovery-server trở lại: registry lạnh.** Nó bắt đầu rỗng. Trong `wait-time-in-ms-when-sync-empty` đầu tiên (mặc định 5 phút) nó từ chối phục

vụ một registry nó coi là chưa đầy đủ, trả về những gì nó có; client đăng ký lại ở heartbeat kế tiếp trong vòng 30 s. Hồi phục hoàn toàn: khoảng một phút sau khi pod Ready.

So với đường nền tảng: danh sách endpoint của một Service do control plane duy trì, thứ được sao chép và mọi thành phần khác đã phụ thuộc vào. Không có câu chuyện 3 giờ sáng riêng cho DNS Kubernetes vì nó không có tiến trình riêng nào để mất.

30.6 Nâng cấp giải pháp

Mục tiêu: trên Kubernetes, định tuyến bằng DNS của Service và để readiness probe gác lưu lượng; trong Compose, giữ Eureka để trải nghiệm dev không đổi. Một profile Spring tách hai bên, và config-server phục vụ tài liệu profile chồng lên tài liệu gốc.

30.6.1 Bước 1: tài liệu profile cho gateway

api-gateway.yml giữ các route lb://. Một api-gateway-k8s.yml mới bên cạnh thay thế chúng — cả danh sách, vì danh sách YAML ghi đè theo chỉ số và ghi đè một phần là cái bẫy:

```
# configurations/api-gateway-k8s.yml -- served when profile k8s is active
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: http://auth-service:8081
          predicates:
            - Path=/api/auth/**
          filters:
            - StripPrefix=2
        - id: auth-oauth2
          uri: http://auth-service:8081
          predicates:
            - Path=/oauth2/**
        - id: auth-oauth2-callback
          uri: http://auth-service:8081
          predicates:
            - Path=/login/oauth2/**
        - id: course-service
          uri: http://course-service:8082
          predicates:
```

①


```

- Path=/api/courses/**,/api/students/**,/api/assignments/**,/api/enrollments
↪ /**,/api/submissions/**
  filters:
    - StripPrefix=2
- id: dictionary-service
  uri: http://dictionary-service:8083
  predicates:
    - Path=/api/dictionary/**
  filters:
    - StripPrefix=2
httpClient:
  connect-timeout: 2000
  response-timeout: 10s
eureka:
  client:
    enabled: false

```

- ① Một tên Service và cổng. CoreDNS phân giải nó thành ClusterIP; kube-proxy cân bằng qua các endpoint Ready. Readiness probe giờ nằm trên đường của gateway.
- ② Không có registry để hỏi, phòng thủ duy nhất của gateway trước một endpoint chết là connect timeout nhanh; đằng nào kube-proxy cũng sẽ gỡ endpoint trong vài giây.
- ③ Eureka client vẫn trên classpath nhưng không làm gì: không đăng ký, không tải, không cảnh báo heartbeat. Đây là thứ cho phép một image phục vụ cả hai môi trường.

30.6.2 Bước 2: Feign không cần registry

AuthServiceClient của course-service phân giải auth-service qua LoadBalancer. Cho nó một URL cố định tùy chọn:

```

// course-service/.../client/AuthServiceClient.java
@FeignClient(
    name = "auth-service",
    url = "${app.clients.auth-service:}", // empty -> use lb://
    fallback = AuthServiceFallback.class)
public interface AuthServiceClient {
    @GetMapping("/users/{id}")
    UserDto getUserById(@PathVariable Long id);
}

```

Khi url phân giải thành chuỗi rỗng, Feign quay về phân giải cân bằng tải theo name; khi nó phân giải thành http://auth-service:8081, LoadBalancer bị bỏ qua. Tài liệu profile của course-service đặt nó:

```
# configurations/course-service-k8s.yml
app:
  clients:
    auth-service: http://auth-service:8081
eureka:
  client:
    enabled: false
```

30.6.3 Bước 3: các manifest

Diff so với `deploy/k8s/base/api-gateway.yaml`; mọi manifest service đổi cùng cách:

```
env:
+ - { name: SPRING_PROFILES_ACTIVE, value: "k8s" }
  - { name: SPRING_CONFIG_IMPORT,
    value: "configserver:http://config-server:8888" }
- - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE,
  value: "http://discovery-server:8761/eureka/" }
- - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }
```

Và trong `kustomization.yaml`:

```
resources:
- config-server.yaml
- discovery-server.yaml
- api-gateway.yaml
```

Compose không đưng tới: không profile, tài liệu gốc, `lb://`, Eureka. Cùng các image chạy ở cả hai. Khi Compose cuối cùng nghỉ hưu, xóa `spring-cloud-starter-netflix-eureka-client` khỏi mọi `pom.xml`, module `discovery-server`, và các route `lb://` trong tài liệu gốc — theo thứ tự đó, mỗi việc một commit riêng để pipeline kiểm chứng được.

THỰC HÀNH — CHỨNG MINH ĐƯỜNG NỀN TẢNG

Sau khi áp dụng profile, xác nhận gateway không còn quay thẳng IP pod — nó quay Service, mà endpoint theo readiness:

```
$ kubectl -n ts get endpoints course-service
NAME             ENDPOINTS          AGE
course-service   10.42.0.57:8082    3d
$ kubectl -n ts rollout restart deployment/course-service
$ while true; do curl -s -o /dev/null -w '%{http_code}\n' \
  -H "Authorization: Bearer $TOKEN" \
```

```

http://ts.local/api/courses; sleep 1; done
200
200
200 # no 503 window: the endpoint switched with readiness

```

Chạy cùng vòng lặp trước khi đổi và đếm số 503.

30.7 Ngoài dự án

Consul là registry bạn gặp ngoài JVM: một agent trên mỗi host tự chạy health check thay vì tin heartbeat tự báo, và phục vụ kết quả qua DNS lẫn HTTP, nên bên gọi không cần thư viện. Service mesh (Istio, Linkerd) đi thêm một bước và đặt một proxy sidecar cạnh mỗi pod làm khám phá, retry, timeout và mTLS, để ứng dụng chỉ còn `http://course-service`. Trong Spring, `spring-cloud-kubernetes` cho phép tên `lb://` phân giải qua API Kubernetes thay vì Eureka — hữu ích trong lúc chuyển đổi, không cần thiết khi chuyển xong. Và AWS Cloud Map, hậu duệ được quản lý của ý tưởng Eureka, tồn tại chủ yếu cho ECS, nơi không có kube-proxy để khiến nó thừa.

30.8 Câu hỏi từ phỏng vấn thật

Q: Gateway của bạn vẫn gửi lưu lượng tới một instance đã chết chín mươi giây trước. Vì sao, và cắt ngắn bằng cách nào?

Vì không gì trên đường đi nhận ra. Eureka biết một cái chết không êm chỉ qua heartbeat thiếu: 90 s hết hạn lease, rồi một lượt quét chạy mỗi 60 s, rồi 30 s cache server, 30 s tải của client, và 35 s cache LoadBalancer — tới bốn phút đầu-cuối, và nếu registry đã trượt vào tự bảo toàn, mãi mãi. Cắt phần đuôi trước: rút `registry-fetch-interval-seconds` và TTL LoadBalancer xuống 5 s, tồn lưu lượng registry mà bạn kham được dưới vài trăm instance. Rồi cắt phần đầu: làm cái chết trở nên êm (xử lý `SIGTERM`, `terminationGracePeriodSeconds` đủ dài) để DELETE được gửi đi và 150 s logic lease không bao giờ chạy. Rồi làm bên gọi bên bắt kể: một filter Retry cho lỗi kết nối, để một địa chỉ cũ tồn một lần thử lại, không phải một lỗi. Câu trả lời senior kết bằng “hoặc ngừng định tuyến bằng registry trên Kubernetes, vì kube-proxy gỡ endpoint trong vài giây sau khi readiness probe thất bại”.

Q: Làm sao một instance mới bắt đầu nhận lưu lượng trong rolling deploy với không lỗi nào?

Hai điều kiện và một chồng lẩn. Instance mới không được nhận lưu lượng trước khi phục vụ được: trên Kubernetes đó là readiness probe, trên Eureka đó là chỉ đăng ký sau khi context lên hẳn — Spring làm thế, nhưng một health check qua trước khi Flyway xong thì không, nên hãy probe thứ gì phụ thuộc cơ sở dữ liệu. Instance cũ phải tiếp tục phục vụ tới khi mọi bên gọi ngừng gửi: trên Kubernetes endpoint bị gỡ lúc SIGTERM nhưng request đang chạy hoàn tất trong grace period, và một preStop ngủ che cho bên gọi có cache cũ. Chồng lẩn là maxSurge: 1, maxUnavailable: 0: đưa cái mới lên, đợi Ready, rồi hạ cái cũ. Lỗi trong rollout hầu như luôn là một trong ba thứ — probe nói dối, cache sống lâu hơn pod, hay grace period ngắn hơn request chậm nhất — và bạn gọi tên được cái nào từ hình dạng của lỗi.

Q: Eureka so với DNS Kubernetes. Một phút.

Cả hai ánh xạ một tên tới một tập địa chỉ; chúng khác ở ai duy trì tập đó và ai tra cứu. Eureka: instance tự đăng ký, tự báo sức khỏe, và mọi bên gọi mang một client tải và cache danh sách — không cần nền tảng, chạy trên VM, AP với vài phút cũ, và mọi bên gọi là một JVM có thư viện. Kubernetes: control plane duy trì endpoint của Service từ label selector và readiness probe do chính nó chạy, kube-proxy cân bằng, bên gọi dùng DNS và không biết gì — vài giây cũ, không thư viện, không tiến trình registry, nhưng chỉ trong cụm. Trên Kubernetes, Eureka là ý kiến thứ hai, chậm hơn, về sự sống. Chỉ giữ nó nếu bạn buộc phải chạy cùng code ở nơi nền tảng không có mặt.

Q: Bạn chạy hai replica Eureka và chúng bất đồng về việc một instance còn sống không. Giờ sao?

Hãy lường trước; thiết kế cho nó. Sao chép giữa peer là bất đồng bộ và mỗi replica loại bỏ và tự bảo toàn theo bộ đếm riêng, nên một cửa sổ bất đồng là cố hữu — Eureka là AP và bất đồng là cái giá của việc trả lời trong lúc phân mảnh. Điều quan trọng là client chịu được: mọi client liệt kê cả hai replica, thử cái thứ hai khi thất bại, và coi registry là gợi ý mà một connect timeout có thể phủ quyết. Nếu bất đồng kéo dài quá vài phút, một replica đang tự bảo toàn còn cái kia không; so ngưỡng gia hạn với số instance thực, và trên đội tàu nhỏ hạ renewal-percent-threshold hoặc tắt hẳn tự bảo toàn. Câu trả lời sai là cố làm registry nhất quán; câu đúng là làm bên gọi thờ ơ với replica nào chúng đọc.

Q: Hai trăm service, mỗi cái tải registry mỗi ba mươi giây. Registry đang tan chảy. Bạn đối gì?

Bốn trăm lượt tải mỗi phút của một tài liệu liệt kê hai trăm ứng dụng là nút thắt đầu tiên, và câu trả lời của Eureka là tải delta: client hỏi /apps/delta sau lần tải đầy đủ đầu tiên, mặc định bật và chỉ hỏng khi hàng đợi delta của server bị tắt. Thứ hai, cache phản hồi chỉ đọc tồn tại chính xác để server tuần tự hóa registry một lần mỗi 30 s, không phải mỗi request — giữ nó. Thứ ba, chia theo zone: cấu hình availability-zone của Eureka cho client ưu tiên registry cục bộ và instance cục bộ, để mỗi server gánh một phần lượt tải. Quá một nghìn instance, câu trả lời của chính Netflix là thêm server Eureka sau DNS, và câu trả lời ngày nay là ở chỗ đó bạn đang ở trên một nền tảng làm khám phá họ bạn.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Tính độ trễ xấu nhất trước khi gateway định tuyến tới một pod course-service vừa khởi động, gọi tên từng cache trong chuỗi và mặc định của nó.
2. Một instance bị OOM killer giết. Liệt kê các bộ đếm phải trôi qua trước khi gateway ngừng quay nó, và nêu cái nào một lượt tắt êm sẽ bỏ qua.
3. Với năm client đăng ký, chỉ ra vì sao mất một cái đưa registry vào tự bảo toàn, và nêu cấu hình một dòng thay đổi điều đó trên đội tàu nhỏ.
4. Trên Kubernetes, cơ chế nào gác lưu lượng tới một pod trên đường của gateway hôm nay, và vì sao readiness probe không? Thay đổi cấu hình đơn nào đưa readiness trở lại đường đi?
5. Sau `eureka.client.enabled: false` dưới profile `k8s`, giải thích vì sao cùng image course-service vẫn đăng ký với Eureka trong Compose, và `url = "$app.clients.auth-service:"` làm gì ở mỗi môi trường.
6. Phòng vấn: “Vì sao registry tiếp tục phục vụ một instance đã chết trong khi Service Kubernetes thì không?” Trả lời theo ai quan sát sự sống và bằng cách nào.

Đào sâu: Gateway dưới tải — bên trong stack reactive, rate limit và việc của biên

Chương 4 dạy gateway như một bảng route: predicate, URI lb://, StripPrefix, một filter JWT. Đó là gateway vào một ngày yên ả. Chương này là gateway vào một ngày tệ — course-service chậm, một client kẹt trong vòng lặp retry, một pod khởi động lại đúng lúc không nên — và hỏi stack reactive thực sự mua được gì, biên của dự án đang thiếu gì, và ranh giới giữa việc của biên và việc của mọi người khác nằm ở đâu.

Ba câu hỏi sắp xếp chương này:

1. **Vì sao gateway không chặn (non-blocking)**, và chính xác chuyện gì hỏng nếu một filter chặn?
2. **Cái gì gây trước dưới tải** — và vì sao câu trả lời là “thứ không có timeout”?
3. **Cái gì thuộc về biên** — rate limit, circuit breaker, CORS — và cái gì không bao giờ nên?

31.1 Vì sao cần gateway, và nó giải quyết gì

Không có gateway, frontend phải giữ sáu địa chỉ, mỗi service phải tự xác minh JWT, và CORS, rate limit, header truy vết sẽ được làm sáu lần hoặc không lần nào. Gateway gom tất cả về một địa chỉ và một chỗ cho mọi mối quan tâm *xuyên suốt* — những thứ thuộc về *mọi* request chứ không thuộc về khóa học hay người dùng. Dạng phỏng vấn của câu hỏi là “API gateway làm được gì mà load balancer không làm được?” và câu trả lời senior là: load balancer chọn một backend; gateway *hiểu request* — nó có thể xác thực, viết lại, giới hạn tốc độ và chuyển hướng theo từng route, vì nó chạy code ứng dụng trên đường đi. Sức mạnh đó cũng là rủi ro: bất cứ thứ gì bạn đặt lên đường đó, mọi request đều phải trả giá.

QUYẾT ĐỊNH — GATEWAY ỨNG DỤNG PHÍA SAU ROUTER HẠ TẦNG

Cụm đã có Traefik (Chương 18). Vì sao vẫn giữ Spring Cloud Gateway phía sau? Vì hai thứ làm hai việc khác nhau: Traefik sở hữu host, TLS và việc chia nhánh theo path; gateway sở hữu những thứ chỉ ứng dụng mới biết — tên Eureka nào phục vụ /api/courses, JWT hợp lệ trông ra sao, header nào cần

chèn. Gộp chúng nghĩa là dạy Traefik secret JWT và hợp đồng header, việc khả thi (middleware ForwardAuth) nhưng đưa bảo mật ứng dụng vào cấu hình hạ tầng. Quyết định sẽ lật nếu nền tảng tự cung cấp danh tính (API Gateway cloud với Cognito hay IAP): lúc đó gateway Spring là một chặng thứ hai trả tiền mà không được gì.

31.2 Stack reactive, đọc từ filter

Gateway khai báo `web-application-type: reactive` trong `application.yml` của nó. Một dòng đó đổi mô hình mỗi request một thread của Tomcat lấy vòng lặp sự kiện (event loop) của Reactor Netty, và mọi dòng của filter JWT được viết cho mô hình đó:

```
@Override
public Mono<Void> filter(ServerWebExchange exchange,
                        GatewayFilterChain chain) { ①
    String path = exchange.getRequest().getURI().getPath();
    if (isPublicPath(path)) {
        return chain.filter(exchange); ②
    }
    String authHeader = exchange.getRequest().getHeaders()
        .getFirst(HttpHeaders.AUTHORIZATION);
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        return unauthorizedResponse(exchange, "Missing ..."); ③
    }
    String token = authHeader.substring(7);
    try {
        Claims claims = Jwts.parser().verifyWith(signingKey)
            .build().parseSignedClaims(token).getPayload(); ④
        ServerHttpRequest mutated = exchange.getRequest().mutate()
            .header(JwtConstants.HEADER_USER_ID, claims.getSubject())
            .header(JwtConstants.HEADER_USER_ROLE,
                claims.get(JwtConstants.CLAIM_ROLE, String.class))
            .build();
        return chain.filter(exchange.mutate().request(mutated).build());
    } catch (Exception e) {
        return unauthorizedResponse(exchange, "Invalid or expired token");
    }
}
```

- ① Kiểu trả về là toàn bộ câu chuyện. Một filter servlet trả về void *sau khi* request đã xong; cái này trả về một `Mono<Void>` — một *mô tả* công việc sẽ hoàn thành sau. Thread gọi filter được tự do ngay khi phương thức này trả về.

- ② `chain.filter(exchange)` không chuyển tiếp request; nó trả về Mono của chặng kế tiếp. Chưa có gì xảy ra. Chuỗi được lắp ráp trước và subscribe một lần.
- ③ Ngay cả nhánh lỗi cũng reactive: `writeWith` trả về một Mono hoàn thành khi Netty đã flush xong body.
- ④ Mẫu công việc thật duy nhất, và nó thuộc loại tốn CPU chỉ vài micro giây: một HMAC trên vài trăm byte. Đây là loại việc *được phép* chạy trên thread event loop. Một lời gọi cơ sở dữ liệu thì không.

31.2.1 Thread: bốn, không phải bốn trăm

Tomcat trả lời tính đồng thời bằng thread: mặc định 200, mỗi request đang bay một thread, mỗi thread đứng chờ trong lúc backend trả lời. Request của một proxy gần như toàn là chờ, nên một proxy Tomcat với 1.000 request đang bay cần 1.000 thread và khoảng một gigabyte cho stack. Reactor Netty chạy một nhóm nhỏ, cố định các thread *event loop*, mỗi thread ghép kênh hàng nghìn socket bằng I/O không chặn: một request đến, chuỗi filter được lắp ráp, lệnh ghi lên upstream được phát đi, và thread đi tiếp. Khi upstream trả lời, vòng lặp nào sở hữu socket đó sẽ tiếp tục chuỗi.

TRÊN PHẦN CỨNG NÀY

Reactor Netty định cỡ vòng lặp bằng `max(availableProcessors, 4)`. Trên hai lõi của ts-server, đó là **bốn** thread event loop cho toàn bộ gateway. Bốn thread phục vụ mọi request là lý do Deployment của gateway xin ít bộ nhớ hơn một service MVC (384 Mi so với 512 Mi, Chương 17) — và là lý do một thread bị chặn không phải vấn đề 0,5 % mà là vấn đề 25 %.

31.2.2 Chặn làm gì một event loop

Giả sử ai đó “cải tiến” filter bằng cách kiểm tra danh sách cấm trong Postgres:

```
// DO NOT: a JDBC call inside a WebFlux filter
boolean revoked = jdbcTemplate.queryForObject(
    "select exists(select 1 from revoked where jti = ?)",
    Boolean.class, claims.getId());
```

JDBC chặn theo đặc tả: thread gọi đứng chờ socket. Thread đó là một trong bốn event loop, và trong lúc chờ nó không phục vụ hàng trăm socket khác mà nó sở hữu. Request của chúng khựng lại — không thất bại, *khựng* — cho tới khi truy vấn trả về. Dưới một cơ sở dữ liệu chậm, cả bốn vòng lặp bị ghim trong vài giây và gateway nhận kết nối mà nó không xử lý nổi: mọi route, mọi user, một

triệu chứng. Điều tương tự đúng với `.block()` trên một `Mono`, `Thread.sleep`, một lời gọi `RestTemplate` đồng bộ, hay đọc file. Phương án reactive dời công việc ra khỏi vòng lặp và *ghép* nó vào chuỗi:

```
return Mono.fromCallable(() -> deny.isRevoked(claims.getId()))
    .subscribeOn(Schedulers.boundedElastic()) // off the loop
    .flatMap(revoked -> revoked
        ? unauthorizedResponse(exchange, "Token revoked")
        : chain.filter(exchange.mutate().request(mutated).build()));
```

Tốt hơn nữa là một client không chặn (Redis reactive, R2DBC) để không thread nào phải chờ cả. Quy tắc cho bất cứ thứ gì trên đường đi của gateway: *nếu phương thức của thư viện trả về một giá trị thay vì một Mono/Flux, nó không thuộc về filter*. Agent BlockHound của Reactor biến quy tắc đó thành một test thất bại.

31.2.3 Predicate, thứ tự, và phép tính `stripPrefix`

Route được thử theo thứ tự khai báo (tất cả có order mặc định là 0, và phép sắp xếp là ổn định); predicate khớp đầu tiên thắng. Bảng của dự án không có chồng lấn, nhưng một route bắt-tất-cả `Path=/api/**` trong tương lai đặt *phía trên* các route cụ thể sẽ nuốt chúng trong im lặng — đặt route bắt-tất-cả ở cuối hoặc cho nó order cao. Các filter gắn vào một route được đánh số tuần tự và chạy *sau* các global filter có order thấp hơn; `getOrder()` của filter JWT bằng `-1` đặt nó trước mọi filter của route, và đó là điều cho phép rate limiter của route (bên dưới) đọc header `X-User-Id` mà filter JWT vừa ghi.

CẠM BÃY — ĐOẠN PATH LẶP ĐÔI

Triệu chứng: frontend gọi `/api/courses/courses/5` và chạy được, nhưng gõ tay `/api/courses/5` thì `course-service` trả 404. Nguyên nhân: `StripPrefix=2` bỏ đúng hai đoạn, `/api` và `/courses`, nên `/api/courses/5` tới nơi thành `/5` — và không controller nào được ánh xạ ở đó. `courses` đầu tiên chọn *route*; `courses` thứ hai sống sót qua phép bỏ và khớp `@RequestMapping("/courses")`. Frontend cố ý lặp đoạn đó. Thiết kế sạch hơn là `StripPrefix=1` với controller giữ tiền tố tài nguyên của mình; dự án chọn giữ cho service không biết tới `/api` với cái giá là URL lặp đôi.

31.3 CORS ở biên

`CorsConfig` đăng ký một `CorsWebFilter` cho `/**`: `origin localhost:3000` và `5173`, sáu phương thức, header bất kỳ, cho phép credentials. Ba điều cần hiểu về chỗ nó

sống.

Vì sao ở đây mà không ở từng service. CORS là quy tắc của trình duyệt về *origin*, và origin mà trình duyệt nói chuyện là gateway. Một service trả lời Access-Control-Allow-Origin là trả lời một câu hỏi không ai hỏi nó; và nếu hai service bất đồng, trình duyệt sẽ thấy chính sách không nhất quán cho một API. Mỗi quan tâm của biên đặt ở biên.

Preflight tốn gì. Với bất kỳ request nào không “đơn giản” — một POST JSON, bất cứ thứ gì có header Authorization — trình duyệt gửi trước một request OPTIONS và đợi câu trả lời. Đó là thêm một vòng đi-về cho mỗi lời gọi, và mọi lời gọi API trong dự án này đều mang Authorization. Gateway tự trả lời preflight mà không đụng tới route nào, rất rẻ, nhưng vòng đi-về đó là người dùng trả. Access-Control-Max-Age cho trình duyệt cache câu trả lời; CorsConfig không đặt nó, nên Chrome rơi về năm giây. Một dòng, `config.setMaxAge(3600L)`, loại gần hết preflight khỏi một phiên.

Khi nào nó không bao giờ được hỏi tới. Trong cụm, frontend và /api chung một origin phía sau Traefik (Chương 26), nên không request nào là xuyên origin và filter đứng không. Dev là nơi duy nhất nó chạy — và proxy của Vite có thể khiến ngay cả dev cũng cùng origin. Vì thế cấu hình CORS ở đây không phải tính năng bảo mật; nó là tiện nghi phát triển không được mở rộng câu trả: `allowCredentials(true)` với origin ký tự đại diện bị trình duyệt từ chối chính vì nó sẽ cho bất kỳ site nào gọi API với cookie của người dùng.

31.4 Việc của biên, và việc không phải

Thuộc về biên	Không thuộc
Xác thực, header danh tính	Quy tắc phân quyền theo từng tài nguyên
Rate limit, hạn ngạch	Kiểm tra hợp lệ nghiệp vụ
Timeout, retry (lũy đẳng), circuit breaker	Gộp response từ nhiều service
CORS, kết thúc TLS, nén	Gọi cơ sở dữ liệu
Header truy vết, log truy cập	Biến đổi dữ liệu ngoài header/path

Cột bên phải là anti-pattern “smart pipes” mà thời ESB đã dạy: khi gateway biết quy tắc nghiệp vụ, mọi tính năng phải ship ở hai nơi và đội gateway thành nút thắt của tất cả. Ngoại lệ chính đáng duy nhất — ghép nhiều lời gọi service thành một response theo hình dạng màn hình — là một service riêng (*backend for frontend*), triển khai và mở rộng tách khỏi biên.

31.5 Ưu và nhược, thắng thần

- **Ưu:** một địa chỉ, một filter JWT, một chính sách CORS; route là cấu hình, đổi được mà không cần build lại (Chương 2).
- **Ưu:** mô hình reactive giữ hàng nghìn request đang bay trên bốn thread và khởi động trong vài giây.
- **Nhược:** gateway của dự án *không* đặt timeout nào cho HTTP client. Mặc định của Reactor Netty là 30 giây để kết nối và *không* có timeout phản hồi.
- **Nhược:** không circuit breaker, không rate limiter — client nào cũng gửi nhanh tùy thích, và một service chết được từng request phát hiện riêng lẻ.
- **Nhược:** replicas: 1 trong Kubernetes. Thành phần mọi request đều đi qua lại là thứ chỉ có một bản.
- **Nhược:** starter nó dùng (spring-cloud-starter-gateway) bị đánh dấu lỗi thời trong Spring Cloud 2025.0, thay bằng starter gateway-server-webflux; việc đổi tên hôm nay vô hại và sẽ là lỗi build ở một lần nâng cấp tương lai.

31.6 Cái gì gây

CẠM BẦY — MỘT DOWNSTREAM CHẬM KÉO CẢ ROUTE THEO

Triệu chứng: course-service chậm (một lock, một GC dài), và trong vòng một phút *mọi* request tới /api/courses/** thất bại với 503 hoặc treo — kể cả những request rẻ đáng lẽ thành công. Nguyên nhân: gateway giữ một connection pool theo từng host, cỡ $\max(\text{cores}, 8) \times 2 = 16$ trên máy này. Không có timeout phản hồi, mỗi request chậm giữ một kết nối trong pool lâu bằng thời gian course-service cần; request thứ mười bảy đợi trong hàng acquire 45 giây rồi thất bại. Mọi user xếp hàng sau người chậm nhất. Sửa: một timeout phản hồi (Mục 31.8) để kết nối kẹt được thu hồi, và một circuit breaker để route thất bại nhanh khi lỗi dồn cụm.

CẠM BẦY — 431 REQUEST HEADER FIELDS TOO LARGE

Triệu chứng: một user có nhiều cookie, hoặc một token với claim béo (Chương 28), nhận ngay 431 từ gateway mà log service không có gì. Nguyên nhân: Netty giới hạn *khối* header request ở 8 KB theo mặc định; gateway từ chối request trước khi bất kỳ filter nào chạy. Sửa: giữ token mỏng. Tăng server.max-http-request-header-size là câu trả lời hạng nhì, vì mỗi chặng phía sau gateway có giới hạn riêng.

CẠM BẦY — GATEWAY LÀ ĐIỂM LỖI ĐƠN

Triệu chứng: một lần rolling update gateway, một lần reboot node, hoặc một OOM kill — và toàn bộ ứng dụng sập, trong khi mọi service phía sau đều khỏe. Nguyên nhân: một replica. Gateway là phi trạng thái, nên cách sửa là replicas: 2 cộng một PodDisruptionBudget; thứ duy nhất làm việc mở rộng hơn là trạng thái theo từng instance như một rate limiter trong bộ nhớ, và đó chính là lý do phần nâng cấp bên dưới giữ trạng thái đó trong Redis.

31.7 Chuyện gì xảy ra lúc 3 giờ sáng

Postgres của course-service bắt đầu một đợt autovacuum dài; truy vấn từng mất 20 ms giờ mất 4 s. Đi theo đường đi.

1. Frontend bắn sáu request của một dashboard. Bốn cái chạm course-service và mất 4 s mỗi cái. Gateway giữ bốn kết nối trong pool suốt 4 s.
2. User bấm lại. Hai mươi user, mỗi người bốn request: 80 request tranh 16 kết nối. Hàng acquire dài ra; sau 45 s các request đang đợi thất bại với 503.
3. Request tới auth-service và dictionary-service *không bị ảnh hưởng* — pool là theo host — cho tới khi access token của user hết hạn và /refresh thành công nhưng dashboard vẫn lỗi, nên họ đăng xuất rồi đăng nhập lại, thêm tải.
4. Không ai bị gọi, vì gateway “khỏe”: /actuator/health trả 200 và event loop của nó không bị chặn. Sự cố vô hình ở biên và chỉ hiện ra dưới dạng độ trễ trong Zipkin (Chương 24).

Với timeout phản hồi 5 s và circuit breaker trên route, bước 2 trở thành: các lời gọi chậm bị cắt ở 5 s và tính là thất bại; sau mười lần, breaker mở; trong 10 s tiếp theo mọi request /api/courses thất bại dưới một mili giây với một body fall-back; pool không bao giờ cạn; auth và dictionary vẫn chạy; thay đổi trạng thái của breaker là một metric bạn có thể đặt cảnh báo. Cơ sở dữ liệu vẫn chậm — khả năng chịu lỗi không sửa nguyên nhân — nhưng bán kính thiệt hại là một route thay vì cả site.

31.8 Nâng cấp giải pháp

Ba bổ sung, toàn bộ là cấu hình cộng một bean. Dependency trước (pom.xml của gateway):

```
# spring-cloud-starter-circuitbreaker-reactor-resilience4j
```

```
# -> the reactive CircuitBreaker gateway filter
# spring-boot-starter-data-redis-reactive
# -> backing store for RequestRateLimiter (non-blocking client)
```

Rồi api-gateway.yml trong config-server:

```
spring:
  data:
    redis:
      host: ${REDIS_HOST:localhost}          # redis:6379 in compose
  cloud:
    gateway:
      httpclient:
        connect-timeout: 2000                ①
        response-timeout: 5s
        pool:
          max-connections: 64                ②
          acquire-timeout: 2000
      default-filters:
        - RemoveRequestHeader=X-User-Id      ③
        - RemoveRequestHeader=X-User-Role
      routes:
        - id: course-service-courses
          uri: lb://course-service
          predicates:
            - Path=/api/courses/**
          filters:
            - StripPrefix=2
            - name: CircuitBreaker            ④
              args:
                name: courseCb
                fallbackUri: forward:/fallback/course
                statusCodes: "500,502,503,504"
            - name: RequestRateLimiter        ⑤
              args:
                redis-rate-limiter.replenishRate: 20
                redis-rate-limiter.burstCapacity: 40
                redis-rate-limiter.requestedTokens: 1
                key-resolver: "#{@userKeyResolver}"
  resilience4j:
    circuitbreaker:
      instances:
        courseCb:
          sliding-window-size: 10
          minimum-number-of-calls: 10
          failure-rate-threshold: 50
          slow-call-duration-threshold: 3s    ⑥
          slow-call-rate-threshold: 80
          wait-duration-in-open-state: 10s
```

```
permitted-number-of-calls-in-half-open-state: 3
timelimiter:
  instances:
    courseCb:
      timeout-duration: 5s
```

⑦

- ① Kết nối trong 2 s hoặc bỏ cuộc; phản hồi đầy đủ trong 5 s hoặc kết nối bị đóng và trả về pool. Có thể ghi đè theo route qua metadata của route cho một endpoint chậm đã biết như tải file lên.
- ② Mười sáu kết nối mỗi host là chặt một khi timeout làm request ngăn lại; 64 là thoải mái trên máy này mà vẫn có giới hạn.
- ③ Cách sửa của Chương 28: header danh tính chỉ có thể do gateway ghi.
- ④ Filter Resilience4j reactive. Thất bại (và các mã trạng thái liệt kê) tính vào breaker; khi mở, request được chuyển tiếp tới /fallback/course *bên trong gateway*.
- ⑤ Token bucket trong Redis: nạp lại 20 token mỗi giây, tích tối đa 40, mỗi request tiêu một. Vượt quá thì trả 429 với X-RateLimit-Remaining: 0. Redis chứ không phải bộ nhớ, để hai replica gateway dùng chung một bucket cho mỗi user.
- ⑥ Không có dòng này, lời gọi chậm không phải thất bại — ngưỡng chậm mặc định là 60 s. Ba giây nghĩa là “chậm hơn thiết kế”.
- ⑦ Breaker của gateway bực mỗi lời gọi trong một TimeLimiter có mặc định là **một giây**. Bỏ trống, một lần tải lên khỏe mạnh 1,2 s sẽ làm breaker nhảy. Đặt tường minh và giữ nó $\geq$ timeout phản hồi.

Key resolver và endpoint fallback:

```
@Configuration
public class RateLimitConfig {

    /** Bucket per authenticated user; anonymous callers share
     * a bucket per client IP (public paths: login, refresh). */
    @Bean
    public KeyResolver userKeyResolver() {
        return exchange -> {
            String user = exchange.getRequest().getHeaders()
                .getFirst(JwtConstants.HEADER_USER_ID);
            if (user != null) {
                return Mono.just("u:" + user);
            }
            var addr = exchange.getRequest().getRemoteAddress();
            return Mono.just("ip:" + (addr == null ? "unknown"
                : addr.getAddress().getHostAddress()));
        };
    }
}
```

```
@RestController
public class FallbackController {

    /** Served by the gateway itself when a breaker is open.
     * 503 + Retry-After tells clients (and browsers) to back off. */
    @GetMapping("/fallback/course")
    public ResponseEntity<Map<String, String>> course() {
        return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE)
            .header(HttpHeaders.RETRY_AFTER, "10")
            .body(Map.of("error", "course-service unavailable",
                "retryAfterSeconds", "10"));
    }
}
```

QUYẾT ĐỊNH — GIỚI HẠN TỐC ĐỘ THEO KHÓA NÀO

Theo *user id* là mặc định đúng cho một API đã xác thực: nó khớp với thứ bạn đang bảo vệ (phần công bằng của một tenant) và không né được bằng cách đổi IP. Theo *IP* là lựa chọn duy nhất trước khi xác thực — login và refresh — và là lựa chọn đúng ở đó, vì mỗi đe dọa là nhờ credential từ một nguồn; nó sai phía sau NAT công ty, nơi một IP là một nghìn người dùng. Theo *API key* dành cho client máy và đối tác, nơi key là hợp đồng. Biên production thường xếp chồng hai lớp: giới hạn thô theo IP trên đường dẫn công khai và giới hạn theo user trên mọi thứ còn lại.

THỰC HÀNH — ĐẬP VÀO BUCKET

Với Redis đã thêm vào `docker-compose.yml` và gateway khởi động lại, bắn 50 request với tư cách một user nhanh hơn bucket nạp lại:

```
$ for i in $(seq 1 50); do
    curl -s -o /dev/null -w "%{http_code} " \
        -H "Authorization: Bearer $TOKEN" \
        http://localhost:8080/api/courses/courses
done; echo
200 200 200 ... 200 429 429 429 429 429 429 429 429 429
$ curl -si -H "Authorization: Bearer $TOKEN" \
    http://localhost:8080/api/courses/courses | grep -i ratelimit
X-RateLimit-Remaining: 0
X-RateLimit-Burst-Capacity: 40
X-RateLimit-Replenish-Rate: 20
```

Bốn mươi cái thành công (phần burst), rồi 429 cho tới khi một giây trôi qua và hai mươi token quay lại. Dừng course-service và lặp lại: sau

mười thất bại, cùng lệnh đó trả về JSON fallback dưới một mili giây, và /actuator/circuitbreakers cho thấy courseCb ở trạng thái OPEN.

31.9 Ngoài dự án

Kong, Envoy (và ingress của Istio xây trên nó), nginx, và các gateway cloud (AWS API Gateway, Apigee) hiện thực cùng bốn ý tưởng — timeout, breaker, rate limit, viết lại header — dưới dạng cấu hình gốc thay vì filter Spring, với token bucket thường nằm trong bộ nhớ riêng của proxy trên từng node cộng một kho chung cho giới hạn toàn cục. “Outlier detection” của Envoy là circuit breaker; filter “local rate limit” của nó là bucket. Điều bạn học ở đây chuyển thẳng sang: những câu hỏi người phỏng vấn hỏi về bất kỳ thứ nào trong số đó — trạng thái nằm ở đâu, chuyện gì xảy ra khi kho chết, thứ bậc timeout là gì — có cùng câu trả lời.

31.10 Câu hỏi từ phỏng vấn thật

Q: Lưu lượng tăng 10× sáng thứ Hai. Cái gì gây trước trong gateway, và bạn tinh chỉnh gì?

Không phải event loop. Bốn vòng lặp làm I/O không chặn xử lý được nhiều hơn mười lần lưu lượng của dự án này rất xa; CPU trên gateway là xác minh HMAC và chép header, chỉ vài micro giây. Cái gây trước là bất cứ thứ gì *không có giới hạn*: connection pool theo host (mặc định 16 ở đây) đầy ngay khi bất kỳ downstream nào chậm, và không có timeout phản hồi thì nó không bao giờ rút. Nên thứ tự tinh chỉnh là: timeout phản hồi và kết nối để kết nối được thu hồi; nâng trần pool lên mức downstream thực sự phục vụ được (max-connections, và một acquire-timeout ngắn để request xếp hàng thất bại nhanh thay vì chết đông); circuit breaker theo route để service đang vật lộn bị cắt bớt thay vì xếp hàng; rồi mở rộng ngang — hai hoặc ba replica phía sau Service với HPA theo CPU, việc tầm thường với một gateway phi trạng thái một khi trạng thái rate limit nằm trong Redis. Giới hạn bộ nhớ cũng quan trọng: Netty giữ buffer trực tiếp ngoài heap, nên giới hạn 512 Mi với MaxRAMPercentage=75 chưa 128 Mi cho chúng; dưới lượng đệm 10×, đó là nơi một OOM kill sẽ đến từ.

Q: Một downstream chậm và cả site treo. Vì sao, và bạn làm gì?

Vì site dùng chung thứ đang cạn kiệt. Nếu gateway có một connection pool theo host, chỉ route của host đó phải suy giảm — nên “cả site” nổi lên có thứ khác

đang dùng chung: một event loop bị chặn bởi lời gọi đồng bộ trong filter, một rate limiter toàn cục khóa sai thứ, hoặc frontend bắn mọi lời gọi trong một `Promise.all` nên một phản hồi chậm giữ cả trang. Sửa theo thứ tự đó: rà filter tìm lời gọi chặn (BlockHound trong test), cho route chậm một timeout và breaker để nó thất bại trong mili giây với body fallback, và cho frontend render trang từng phần. Điểm senior là khả năng chịu lỗi nói về *cô lập*: mục tiêu không phải course-service không bao giờ chậm mà là sự chậm của nó không với tới được trang đăng nhập.

Q: Làm sao rate limit theo user xuyên ba replica gateway?

Trạng thái phải dùng chung, nếu không giới hạn là theo từng replica và lệch ba lần. `RequestRateLimiter` của Spring Cloud Gateway chạy một token bucket dưới dạng script Lua trong Redis, nguyên tử, khóa theo bất cứ thứ gì `KeyResolver` trả về — ở đây `u:<userId>` từ header mà filter JWT đã đặt, rơi về IP của client trên đường dẫn công khai. Các con số là chính sách: `replenishRate` là số request mỗi giây duy trì được, `burstCapacity` là thứ một lần tải trang được tiêu cùng lúc; đặt burst cỡ số lời gọi của một màn hình. Redis thành một phụ thuộc, nên quyết định chế độ lỗi của nó một cách có chủ ý: mặc định của filter là *cho qua* khi Redis không với tới được, đúng với ứng dụng nội bộ và sai với ứng dụng công khai đối mặt lạm dụng. Với lưu lượng rất cao, một bucket cục bộ theo replica đặt trước bucket dùng chung hấp thụ phần lớn vòng đi-về tới Redis.

Q: Bạn đặt xác thực ở đâu, và không đặt logic ở đâu?

Xác thực — “token này hợp lệ không và của ai” — ở biên, một lần, vì đó là cùng một câu hỏi cho mọi request và vì đó là nơi credential đi vào hệ thống. Phân quyền — “user này được chấm bài nộp này không” — trong service sở hữu bài nộp, vì chỉ nó biết dữ liệu. Gateway có thể cưỡng chế các quy tắc thô vẫn còn hình dạng request (“/api/admin/** cần vai trò ADMIN”) nhưng khoanh khắc một quy tắc cần một dòng trong cơ sở dữ liệu, nó thuộc về phía sau. Không gì có hình dạng nghiệp vụ nằm trong gateway: không gộp, không biến đổi body, không gọi kho dữ liệu. Phép thử tôi dùng khi review: nếu một thay đổi quy tắc miền đòi deploy gateway, ranh giới đã bị vượt.

Q: Làm sao deploy gateway mà không rơi request nào?

Tối thiểu hai replica, một `PodDisruptionBudget` với `minAvailable: 1`, và rolling update với `maxUnavailable: 0` để pod mới Ready trước khi pod cũ bị gỡ. Readiness phải thật: probe nên thất bại khi registry Eureka còn rỗng, nếu không pod mới nhận request trước khi nó route được đi đâu. Lúc tắt, pod cũ

phải ngừng nhận và hoàn tất request đang bay: `server.shutdown: graceful` với `spring.lifecycle.timeout-per-shutdown-phase` dài hơn timeout phản hồi, và một `preStop sleep` vài giây để việc gỡ endpoint của kube-proxy lan truyền trước khi tiến trình nhận `SIGTERM`. Khoảng hở cuối đó — danh sách endpoint và vòng đời pod do hai controller khác nhau cập nhật — là thứ làm rơi các request mà người ta đổ cho “rolling update”.

Q: Một tích hợp đối tác cần một response ghép từ ba service. Gateway?

Không. Đó là một backend-for-frontend: một service nhỏ gọi ba service song song, chịu được một cái lỗi, và định hình response cho client đó. Nó mở rộng và deploy theo lịch riêng, sở hữu timeout riêng, và giữ gateway là một proxy. Đặt nó vào gateway thì mọi thay đổi của đối tác là một lần deploy biên, và event loop của gateway bắt đầu giữ trạng thái fan-out cho lời gọi chậm nhất trong ba.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Filter JWT trả về `chain.filter(exchange)` mà không đợi gì. Giải thích cái gì đã và chưa xảy ra vào lúc dòng đó trả về, và vì sao một filter servlet không thể viết theo cách này.
2. Một đồng nghiệp thêm một lượt tra Redis đồng bộ vào filter JWT để kiểm tra danh sách cấm. Dưới một lần Redis khựng 200 ms, chuyện gì xảy ra với request tới các route *khác*, và vì sao tác động trên máy này là 25 % cho mỗi lời gọi bị khựng?
3. course-service treo ở 10 % request. Với cấu hình gateway hiện tại của dự án, bao nhiêu request treo đồng thời sẽ chặn *toàn bộ* lưu lượng tới service đó, và thiết lập duy nhất nào thay đổi con số ấy?
4. Filter `CircuitBreaker` của gateway được thêm vào một route mà không có khối `timelimiter`. Một lần tải lên khỏe mạnh mất 1,5 s. Chuyện gì xảy ra, và vì sao?
5. Vì sao trạng thái của rate limiter nằm trong Redis thay vì bộ nhớ của gateway, và đánh đổi là gì khi Redis chết?
6. Mọi request từ frontend đều mang `Authorization`. Điều đó hàm ý gì cho preflight CORS khi dev, thiết lập duy nhất nào giảm chúng, và vì sao tất cả không áp dụng trong cụm?

Đào sâu: Gọi đồng bộ — OpenFeign, Resilience4j và lỗi dây chuyền

Chương 6 vạch ranh giới giữa hai cách service nói chuyện: HTTP khi bên gọi cần câu trả lời ngay, sự kiện khi nó cần một sự thật được ghi nhận. Chương này ở lại phía HTTP và hỏi câu hỏi mà mọi hệ thống phân tán rồi cũng hỏi lúc 3 giờ sáng: khi service tôi đang gọi chậm, chuyện gì xảy ra với *tôi*? Dự án có đúng thư viện trên classpath — OpenFeign cho lời gọi, Resilience4j cho sự bảo vệ — và, như bạn sẽ thấy, đã nổi chúng theo cách không bảo vệ được gì. Điều đó khiến nó thành ca dạy học tốt hơn một ca hoàn chỉnh.

Ba câu hỏi sắp xếp chương này:

1. **Một lời gọi đồng bộ tồn gì** ngoài độ trễ của nó — và vì sao một phụ thuộc chậm tệ hơn một phụ thuộc chết?
2. **Circuit breaker quyết định thế nào**, với các con số thật của dự án, và time-out, retry và breaker tương tác ra sao?
3. **Fallback nên trả về gì**, và khi nào lời gọi không nên tồn tại?

32.1 Vì sao Feign, và nó giải quyết gì

course-service thảnh thỏang cần biết một user là ai — một cái tên để in lên bài nộp, một vai trò để kiểm tra. auth-service sở hữu dữ liệu đó. Các lựa chọn là: gọi auth-service qua HTTP, chép dữ liệu sang course-service qua sự kiện, hoặc đặt cả hai vào một cơ sở dữ liệu. Dự án giữ các cơ sở dữ liệu tách biệt (Chương 7) và dành sự kiện cho các sự thật đã xảy ra; tra cứu theo id là một câu hỏi, nên nó là một lời gọi.

OpenFeign biến lời gọi đó thành một interface:

```
@FeignClient(name = "auth-service",
              fallback = AuthServiceFallback.class) ①
public interface AuthServiceClient {

    @GetMapping("/users/{id}") ②
    UserDto getUserById(@PathVariable("id") Long id); ③

}
```

- ① name là tên ứng dụng trên Eureka. Feign hỏi discovery client các instance của auth-service và cân bằng tải giữa chúng ở phía client — không gateway, không host cứng. fallback nêu tên lớp sẽ được gọi thay khi lời gọi thất bại. Giữ ý đó lại.
- ② Cùng các annotation Spring MVC như một controller, đọc ngược lại: đây là thứ cần *gửi*. Path là của chính service (/users/{id}), không phải của gateway (/api/auth/users), vì lời gọi service-tới-service bỏ qua gateway.
- ③ Kiểu trả về là một record từ common. Record đó là *hợp đồng*: auth-service phải sinh JSON giải tuần tự được thành nó.

Thứ Feign giải quyết là mã lặp và sự ràng buộc vào địa chỉ. Thứ nó không giải quyết là tính chất duy nhất quan trọng dưới tải: mỗi lời gọi giữ một thread Tomcat trong course-service lâu bằng thời gian auth-service cần để trả lời.

QUYẾT ĐỊNH — MỘT LỜI GỌI, KHÔNG PHẢI MỘT BẢN SAO — TẠM THỜI

Với một tra cứu theo id hiếm khi xảy ra và câu trả lời hiếm khi đổi, một lời gọi là thứ đúng đơn giản nhất, và Feign làm nó trong hai dòng. Quyết định sẽ lật khi bất kỳ điều nào trong ba điều thành sự thật: tra cứu nằm trên đường nóng (mọi danh sách khóa học cần tên giáo viên), dữ liệu cần có trong lúc auth-service chết (chăm bài không được dừng vì đăng nhập hỏng), hoặc hình dạng của UserDto bắt đầu đổi theo các bản phát hành của auth-service. Khi đó course-service nên giữ bản sao riêng của những trường nó cần, nuôi bằng các sự kiện auth-service đã phát (ts.user.registered, ts.user.activated) — mẫu mà Chương 7 đã áp dụng cho học sinh.

32.2 Hợp đồng trong common, và rủi ro phiên bản của nó

UserDto là một record với năm trường: id, email, firstName, lastName, role. UserController của auth-service không trả về nó — nó trả về UserResponse của riêng mình. Hai lớp hôm nay khớp nhau nhờ trùng hợp tên trường, và mặc định bỏ qua thuộc tính lạ của Jackson che đi khoảng hở: auth-service có thể thêm trường thoải mái, và course-service sẽ không nhận ra. Nó sẽ nhận ra một lần đổi tên. Đổi firstName thành givenName trong UserResponse biên dịch được ở cả hai phía, deploy được ở cả hai phía, và để course-service đọc tên là null — không ngoại lệ, không dòng log. Phiên bản phỏng vấn: “bạn đánh phiên bản một API nội bộ thế nào?” và câu trả lời là: chỉ thay đổi có tính bổ sung, không bao giờ đổi tên hay xóa; DTO của bên tiêu thụ bỏ qua thuộc tính lạ; một consumer-driven contract test (Spring Cloud Contract, Pact) trong build của auth-service thất bại khi UserResponse không còn thỏa UserDto. Phiên bản trong URL (/v2/users) dành

cho thay đổi phá vỡ không tránh được, và nó nghĩa là chạy cả hai chừng nào còn bên tiêu thụ nào chưa theo kịp.

32.3 Circuit breaker, với các con số của dự án

Breaker của Resilience4j là một máy trạng thái trên một *cửa sổ trượt* các kết quả gọi gần đây. Cấu hình của dự án trong `course-service.yml`:

```
resilience4j:
  circuitbreaker:
    instances:
      authService:
        sliding-window-size: 10           ①
        failure-rate-threshold: 50       ②
        wait-duration-in-open-state: 10s ③
        permitted-number-of-calls-in-half-open-state: 3 ④
```



Hình 32.1: Ba trạng thái của breaker với các ngưỡng của dự án này. Lời gọi bị từ chối ở OPEN đi tới fallback mà không chạm mạng.

- ① Cửa sổ giữ 10 kết quả gần nhất (theo số lượng; phương án kia là theo thời gian, “60 giây gần nhất”). Breaker không đánh giá cho tới khi cửa sổ có đủ `minimum-number-of-calls` kết quả — không đặt ở đây, nên mặc định bằng cỡ cửa sổ. Dịch ra: 9 lời gọi đầu sau khởi động có thể *toàn bộ* thất bại mà breaker vẫn CLOSED.
- ② Ở 10 kết quả, nếu 5 trở lên thất bại, breaker mở. Một “thất bại” là một ngoại lệ breaker không bỏ qua. Để ý thứ *không* được cấu hình: `slow-call-duration-threshold`. Mặc định của nó là 60 s, nên một lời gọi mất 30 s là một *thành công* với breaker này. Phụ thuộc chậm — trường hợp phổ biến — không bao giờ làm nó nhảy.
- ③ OPEN trong 10 s: mọi lời gọi bị từ chối bằng `CallNotPermittedException` trong micro giây và chuyển tới fallback. `auth-service` được nghỉ 10 giây.
- ④ Rồi HALF_OPEN: 3 lời gọi được cho qua làm thăm dò. Nếu tỷ lệ thất bại của

chúng $\geq 50\%$ (2 trong 3), quay lại OPEN thêm 10s; nếu không, CLOSED với cửa sổ mới.

Giờ tới phần khiến chương này thăng thấn. Instance `authService` đó không gắn với gì cả. Không phương thức nào mang `@CircuitBreaker(name = "authService")`, và tích hợp riêng của Feign — thứ sẽ bọc `getUserById` trong một breaker tên `AuthServiceClient#getUserById(Long)` — đang tắt, vì công tắc `openfeign.circuitbreaker.enabled` mặc định là `false`, và starter cung cấp `CircuitBreakerFactory` nó cần (`starter circuitbreaker-resilience4j`) không có trên classpath. Với công tắc đó tắt, thuộc tính `fallback` ở ❶ bị lặng lẽ bỏ qua. Và `AuthServiceClient` hôm nay không được gọi từ lớp service nào. Dự án có một breaker cấu hình cho một lời gọi không xảy ra, bảo vệ không gì cả, với một `fallback` sẽ không chạy. Đó là mẫu vật hoàn hảo của cách khả năng chịu lỗi thường bị cấu hình sai trong production: mọi mảnh có mặt, không mảnh nào được nối.

32.4 Timeout xếp chồng

Khi lời gọi được nối đúng, ba bộ đếm giờ bao quanh nó, và chúng phải được sắp thứ tự nếu không các bộ bên ngoài chỉ là trang trí.

- **Timeout socket của Feign.** `connectTimeout` và `readTimeout` trên HTTP client. Không đặt trong dự án, nên mặc định áp dụng: 10s để kết nối, 60s để đọc. Một `auth-service` treo ghim một thread `course-service` suốt một phút.
- **TimeLimiter của Resilience4j.** Một hạn chót cho toàn bộ lời gọi được bọc, độc lập với socket. Nó chỉ áp dụng cho kiểu trả về bất đồng bộ (`CompletableFuture`), đó là lý do lời gọi Feign đồng bộ phải dựa vào timeout socket.
- **Hạn chót của bên gọi.** `response-timeout` của gateway (Chương 31), và phía sau nó là sự kiên nhẫn của trình duyệt. Nếu gateway bỏ cuộc ở 5s trong khi Feign đợi 60s, người dùng thấy lỗi ở 5s và `course-service` giữ một thread bận thêm 55 giây để phục vụ không ai.

Quy tắc: timeout của mỗi chặng *ngắn hơn* chặng trước nó. Trình duyệt 10s > gateway 5s > Feign read của `course-service` 2s. Một timeout nổ bên trong một request đã bị bỏ rơi là công sức lãng phí; một timeout nổ muộn hơn của bên gọi là một chỗ rò.

32.5 Retry: chỉ cho lời gọi lặp lại được

@Retry của Resilience4j phát lại một lời gọi thất bại. Trên GET /users/{id} thì vô hại: đọc hai lần cũng như đọc một lần. Trên POST /enrollments thì là một lần ghi danh trùng, vì timeout không cho bạn biết request đầu đã được xử lý hay chưa. Vì thế retry chỉ dành cho thao tác *lũy đẳng* (idempotent), hoặc thao tác được làm cho lũy đẳng bằng một khóa client cung cấp mà server khử trùng lặp theo đó. Thêm hai quy tắc từ production: retry với *lùi theo hàm mũ có jitter*, nếu không mọi client retry cùng một khoảng khắc và đợt sóng thứ hai lớn hơn đợt đầu (“bão retry”); và retry *bên trong* breaker, để một breaker đang mở chặn retry ngay từ đầu. Thứ tự aspect mặc định của Resilience4j — Retry ngoài cùng, rồi CircuitBreaker, rồi TimeLimiter, rồi Bulkhead — làm đúng điều này: mỗi lần retry đi qua breaker, và lời gọi bị từ chối thì không được retry.

32.6 Bulkhead: giới hạn vùng nổ

Bulkhead giới hạn số lời gọi đồng thời được phép đang bay tới một phụ thuộc. Bulkhead semaphore: một bộ đếm; lời gọi đồng thời thứ 26 bị từ chối ngay. Bulkhead thread pool: lời gọi chạy trên một pool riêng để thread của bên gọi không bao giờ là kẻ đứng chờ — cần thiết với một ứng dụng servlet có pool Tomcat hữu hạn. course-service chạy Tomcat với 200 thread. Không có bulkhead, 200 lời gọi auth-service chậm tiêu sạch từng thread, và course-service không trả lời nổi một probe sức khỏe, nói gì tới danh sách khóa học. Với bulkhead semaphore 25, tối đa 25 thread từng chờ auth-service và 175 còn lại cho mọi thứ khác. Đó là sự cô lập mà tiêu đề chương hứa hẹn.

32.7 Fallback trả về gì

```
@Component
public class AuthServiceFallback implements AuthServiceClient {
    @Override
    public UserDto getUserById(Long id) {
        return null;
    }
}
```

- ① Một null mà bên gọi phải nhớ kiểm tra. Mọi nơi tiêu thụ getUserById giờ có một nhánh ẩn, và một lần quên kiểm tra là một NullPointerException chỉ nổ khi auth-service chết — đúng lúc bạn ít muốn một lỗi mới nhất.

Một fallback có bốn lựa chọn trung thực, theo thứ tự ưu tiên: một giá trị *đã cache* từ lời gọi thành công gần nhất (cũ nhưng thật); một giá trị *suy giảm* mà UI hiển thị được (“Giáo viên không rõ”) và được định kiểu rõ là như vậy; một kết quả *rỗng* nơi miền cho phép; hoặc một *lỗi có kiểu* mà bên gọi ánh xạ được thành 503 với `Retry-After`. `null` không phải cái nào trong số đó. Nó là sự vắng mặt của một quyết định, đẩy sang mọi bên gọi. Fallback cũng cần *nguyên nhân*: một phương thức fallback hai tham số (`Throwable last`) phân biệt được một lần breaker từ chối với một 404, và 404 — user không tồn tại — *không* được biến thành “thử lại sau”.

32.8 Ưu và nhược, thăng thấn

Dự án có gì	Thiếu gì
Client khai báo, phân giải qua Eureka	Client không bao giờ được gọi
Một instance breaker với ngưỡng hợp lý	Không gì gắn vào nó; công tắc breaker của Feign đang tắt
Một lớp fallback đã đăng ký Resilience4j trên classpath	Bị bỏ qua; và nó trả về <code>null</code> Không timeout, không retry, không bulk-head, không ngưỡng chậm
Actuator đã mở	Endpoint breaker không nằm trong danh sách mở

Và thêm một khoảng hở không liên quan tới khả năng chịu lỗi: `SecurityConfig` của `auth-service` chỉ cho phép các đường dẫn công khai; `/users/{id}` cần một ngưỡng đã xác thực, thứ `GatewayHeaderAuthFilter` của nó dựng từ `X-User-Id`. Feign không gửi header nào như vậy. Lời gọi, một khi được nối, sẽ trả 401 ở mọi lần thử — và 401 là một ngoại lệ, nên mười lần sẽ mở breaker chống lại một service hoàn toàn khỏe mạnh.

32.9 Cái gì gãy

CẠM BÃY — FALLBACK KHÔNG BAO GIỜ CHẠY

Triệu chứng: `auth-service` bị dừng, và thay vì `null` của fallback, `course-service` ném `FeignException` với stack trace xuyên qua load balancer, và request trả về 500. Nguyên nhân: `@FeignClient(fallback = ...)` chỉ được tôn trọng khi `spring.cloud.openfeign.circuitbreaker.enabled=true` và có một bean `CircuitBreakerFactory`, thứ cần dependency `spring-cloud-starter-circuitbreaker-resilience4j`. Thiếu một trong hai,

Spring không log gì và thuộc tính trở nên trơ. Sửa ở Mục 32.11.

CẠM BẦY — SERVICE KHỎE, BREAKER MỞ

Triệu chứng: sau một lần deploy auth-service, breaker của course-service mở và cứ mở, quay vòng OPEN → HALF_OPEN → OPEN, dù auth-service trả lời curl bình thường. Nguyên nhân: mọi lời gọi nhận 401 (thiếu header danh tính), hoặc 404 cho một user đã xóa, và cả hai đều tính là thất bại. Sửa: lan truyền danh tính trên lời gọi service-tới-service (một RequestInterceptor, bên dưới), và bảo breaker ngoại lệ nào *không* phải lỗi phụ thuộc bằng ignore-exceptions — lỗi phía client nghĩa là request sai, không phải service chết.

CẠM BẦY — THREAD 60 GIÂY

Triệu chứng: auth-service treo (deadlock, GC kẹt); CPU của course-service rảnh, vậy mà nó ngừng trả lời *mọi* request, và readiness probe của nó thất bại nên Kubernetes ngừng route tới nó. Nguyên nhân: read timeout mặc định 60 s của Feign nhân với 200 thread của Tomcat. Mỗi tra cứu treo giữ một thread một phút; với tốc độ request vừa phải, mọi thread đều đứng chờ trong vài giây. Breaker không giúp được — với ngưỡng chậm mặc định, lời gọi treo không phải thất bại cho tới khi hết timeout. Sửa: read timeout 2 s, một ngưỡng chậm, và bulkhead 25.

32.10 Chuyện gì gây lúc 3 giờ sáng

Postgres của auth-service bắt đầu một đợt vacuum dài; /users/{id} đi từ 20 ms lên 2 s. Giả sử lời gọi Feign đã được nối vào endpoint danh sách khóa học, như thiết kế của dự án mời gọi, và không gì khác thay đổi.

1. Mỗi danh sách khóa học giờ mất 2 s thay vì 20 ms. Thread Tomcat trong course-service bận lâu hơn 100× cho mỗi request.
2. Ở 50 danh sách mỗi giây, 100 thread đang chờ auth-service tại bất kỳ thời điểm nào. Ở 100 mỗi giây, cả 200 đều chờ, và request xếp hàng trong backlog accept của Tomcat.
3. Readiness probe của course-service (/actuator/health thậm chí không được mở ở đây — Chương 17 dùng probe TCP) vẫn qua, vì socket vẫn nhận kết nối. Gateway tiếp tục gửi lưu lượng.

4. Connection pool của gateway tới course-service đầy; route /api/courses của nó suy giảm cho tất cả mọi người, kể cả các request chưa từng cần auth-service. Chuỗi lỗi của Chương 31 bắt đầu cao hơn một chặng.
5. auth-service, trong khi đó, nhận 100 request mỗi giây từ một bên gọi sẽ vớt câu trả lời sau timeout của gateway — tải khiến sự tranh chấp của đợt vacuum tệ hơn.
6. Breaker không bao giờ mở: 2 s dưới mặc định chậm 60 s, và không lời gọi nào thất bại. Cơ chế duy nhất có thể cắt tải đang nhìn vào sai tín hiệu.

Với cấu hình bên dưới: lời gọi bị cắt ở 2 s (Feign read timeout) và tính là chậm trên 1 s; sau mười kết quả với $\geq 80\%$ chậm, breaker mở; danh sách trả về với tên giáo viên đã cache hoặc suy giảm dưới một mili giây; tối đa 25 thread từng đứng chờ; tải của auth-service giảm còn ba lời gọi thăm dò mỗi mười giây; và thay đổi trạng thái của breaker nằm trên một dashboard.

32.11 Nâng cấp giải pháp

Dependency trước: thêm spring-cloud-starter-circuitbreaker-resilience4j vào POM của course-service (nó mang tới CircuitBreakerFactory mà Feign cần; resilience4j-spring-boot3 giữ lại cho annotation và metric). Rồi course-service.yml:

```
spring:
  cloud:
    openfeign:
      circuitbreaker:
        enabled: true
      client:
        config:
          auth-service:
            connect-timeout: 1000
            read-timeout: 2000
            logger-level: basic
resilience4j:
  circuitbreaker:
    instances:
      authService:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 10
        minimum-number-of-calls: 10
        failure-rate-threshold: 50
        slow-call-duration-threshold: 1s
        slow-call-rate-threshold: 80
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
```

```

automatic-transition-from-open-to-half-open-enabled: true
register-health-indicator: true ④
ignore-exceptions: ⑤
  - feign.FeignException.NotFound
  - feign.FeignException.BadRequest
retry:
  instances:
    authService:
      max-attempts: 3 ⑥
      wait-duration: 200ms
      enable-exponential-backoff: true
      exponential-backoff-multiplier: 2
      enable-randomized-wait: true
      randomized-wait-factor: 0.5
      retry-exceptions:
        - java.net.SocketTimeoutException
        - feign.RetryableException
      ignore-exceptions:
        - feign.FeignException.FeignClientException
bulkhead:
  instances:
    authService:
      max-concurrent-calls: 25 ⑦
      max-wait-duration: 0
timelimiter:
  instances:
    authService:
      timeout-duration: 3s ⑧
      cancel-running-future: true
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,circuitbreakers,circuitbreakerevents
health:
  circuitbreakers:
    enabled: true

```

- ① Công tắc khiến `fallback = ...` thành thật. Feign giờ bọc mỗi phương thức trong một breaker tên `AuthServiceClient#getUserById(Long)`; các annotation tường minh bên dưới dùng instance `authService` cho chính sách phía bên gọi.
- ② Timeout theo từng client, khóa theo name của Feign. Hai giây là rộng rãi cho một tra cứu theo id bình thường mất 20 ms, và ngắn hơn hẳn 5 s của gateway.
- ③ Chậm giờ được tính. Trên 1 s là “chậm hơn thiết kế”; khi 80 % cửa sổ là chậm, breaker mở dù mọi lời gọi rồi cũng thành công.

- ④ Trạng thái breaker xuất hiện trong `/actuator/health` và dưới dạng một gauge `Micrometer (circuitbreaker.state)` — thứ bạn đặt cảnh báo.
- ⑤ Một user không tồn tại là một câu trả lời, không phải sự cố. Phản hồi 4xx không tính là thất bại cũng không kích hoạt fallback.
- ⑥ Ba lần thử với 200 ms, rồi 400 ms, mỗi lần jitter $\pm 50\%$. Chỉ lỗi vận chuyển được retry; bất kỳ HTTP 4xx nào là chung cuộc. Tổng trường hợp xấu nhất: $3 \times 2\text{s} + 0,6\text{s}$, vẫn có giới hạn và vẫn dưới một phút — nhưng dài hơn 5 s của gateway, đó là lý lẽ cho tối đa 2 lần thử trên đường request và 3 chỉ trong công việc nền.
- ⑦ Hai mươi lăm thread được phép chờ auth-service; lời gọi thứ 26 thất bại ngay với `BulkheadFullException` và đi fallback. `max-wait-duration: 0` nghĩa là không xếp hàng — xếp hàng chính là thứ ta đang tránh.
- ⑧ Chỉ dùng bởi biến thể `CompletableFuture` bên dưới; vô hại nếu không.

Lớp bọc phía service, với lan truyền danh tính và một fallback nói rõ nó là gì:

```
/** Forwards the caller's identity on service-to-service calls so
 * auth-service's GatewayHeaderAuthFilter accepts them. */
@Configuration
public class FeignIdentityConfig {
    @Bean
    public RequestInterceptor identityPropagation() {
        return template -> {
            var auth = SecurityContextHolder.getContext()
                .getAuthentication();
            if (auth != null && auth.getPrincipal() instanceof Long id) {
                template.header(JwtConstants.HEADER_USER_ID, id.toString());
                auth.getAuthorities().stream().findFirst().ifPresent(a ->
                    template.header(JwtConstants.HEADER_USER_ROLE,
                        a.getAuthority().replace("ROLE_", "")));
            }
        };
    }
}

/** The one place course-service asks who a user is. */
@Service
@RequiredArgsConstructor
public class UserLookupService {

    private final AuthServiceClient authServiceClient;
    private final Map<Long, UserDto> lastKnown =
        new ConcurrentHashMap<>();

    @Retry(name = "authService")
    @CircuitBreaker(name = "authService", fallbackMethod = "degraded")
```

①

```

@Bulkhead(name = "authService")
public UserDto findUser(Long id) {
    UserDto user = authServiceClient.getUserById(id);
    lastKnown.put(id, user);
    return user;
}

/** Same signature plus the cause; called for breaker rejections,
 * bulkhead rejections, timeouts and 5xx -- never for 4xx. */
private UserDto degraded(Long id, Throwable cause) { ②
    UserDto cached = lastKnown.get(id);
    if (cached != null) {
        return cached; // stale, real
    }
    return new UserDto(id, null, "Unknown", "user", null);
}
}

```

- ① Một cache giá-trị-tốt-gần-nhất. Với một replica, một map là đủ; với nhiều replica, cùng ý tưởng đó thuộc về Redis với TTL, hoặc — cách sửa thật — một bảng cục bộ nuôi bằng sự kiện user.
- ② Fallback có kiểu và nhìn thấy được: bên gọi nhận một UserDto có tên nói “Unknown”, không bao giờ null. Throwable cho bạn log *vì sao* (breaker mở hay timeout) ở mức WARN một lần mỗi lần đổi trạng thái thay vì mỗi lời gọi.

Các metric cần theo dõi, tất cả đã xuất sang Prometheus bởi thiết lập Actuator sẵn có: `resilience4j_circuitbreaker_state` (một gauge theo từng trạng thái từng instance; cảnh báo khi `state="open"` bằng 1 quá một phút), `resilience4j_circuitbreaker_calls_seconds` theo kind (successful, failed, ignored) cho tốc độ, `resilience4j_bulkhead_available_concurrent_calls` tiến về không như chỉ báo sớm, và `http_client_requests_seconds` của chính Feign cho độ trễ thô đã khởi đầu tất cả.

QUYẾT ĐỊNH — KHI NÀO LỜI GỌI KHÔNG NÊN TỒN TẠI

Giữ lời gọi đồng bộ chừng nào nó nằm ngoài đường nóng, hiếm, và chịu được câu trả lời cũ. Thay nó bằng bản sao cục bộ nuôi bằng sự kiện khi một trang danh sách cần nó (độ trễ và ràng buộc), khi việc chấm bài phải chạy trong lúc auth-service chết (tính sẵn sàng), hoặc khi hai đội phát hành theo nhịp khác nhau (trôi hợp đồng). Thay nó bằng một cache TTL ngắn đặt trước lời gọi khi dữ liệu nóng nhưng không được cũ quá một phút và nhất quán cuối cùng qua sự kiện không đáng một consumer mới. Bản năng senior là đếm số phụ thuộc đồng bộ trên mỗi request và coi mỗi cái là một phép nhân xác suất lỗi: ba phụ thuộc 99,9% nối tiếp là một endpoint 99,7%.

THỰC HÀNH — XEM BREAKER MỞ

Với phần nâng cấp đã áp dụng và stack đang chạy, dừng auth-service và gọi một endpoint dùng UserLookupService mười hai lần:

```
$ docker compose stop auth-service
$ for i in $(seq 1 12); do
    curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" \
      -H "Authorization: Bearer $TOKEN" \
      http://localhost:8080/api/courses/courses/1
done
200 1.04s ... (10 slow lines: retries then fallback)
200 0.01s <- breaker open: fallback without a network call
200 0.01s
$ curl -s http://localhost:8082/actuator/circuitbreakers | jq
{ "circuitBreakers": { "authService": {
  "failureRate": "100.0%", "slowCallRate": "0.0%",
  "bufferedCalls": 10, "failedCalls": 10, "state": "OPEN" } } }
```

Mọi phản hồi đều là 200 với “Unknown user” làm giáo viên — trang suy giảm thay vì thất bại. Khởi động lại auth-service, đợi 10 s, và /actuator/circuitbreakerevents cho thấy ba lời gọi thăm dò HALF_OPEN thành công và trạng thái trở về CLOSED.

32.12 Ngoài dự án

Cùng bộ máy dưới những cái tên khác: Hystrix là bản gốc của Netflix và đã nghỉ hưu; Resilience4j là kẻ kế nhiệm trong thế giới Java; Envoy và Istio hiện thực timeout, retry có ngân sách, và loại trừ outlier trong sidecar để code ứng dụng không chứa gì trong số đó, hướng đi mà các đội tàu lớn đã chọn. gRPC mang hạn chót (deadline) trong giao thức — hạn chót của bên gọi lan truyền tới mọi chặng, giải quyết bài toán “timeout xếp chồng” ngay từ cấu trúc. SDK cloud (AWS, GCP) đi kèm lùi theo hàm mũ có jitter sẵn vì cùng lý do chương này khuyến nghị. Từ vựng là phổ quát; một người phỏng vấn nói “bulkhead” hay “retry budget” có ý đúng như chương này.

32.13 Câu hỏi từ phỏng vấn thật

Q: Độ trễ auth-service đi từ 20 ms lên 2 s. Đi qua với tôi chuyện gì xảy ra trong course-service, và bạn chặn nó thế nào.

Mỗi request chạm auth-service giữ một thread Tomcat lâu gấp trăm lần. Ở tốc độ vừa phải, pool 200 bão hòa trong vài giây, hàng đợi accept đầy, và course-service ngừng trả lời mọi thứ — kể cả các endpoint chưa từng gọi auth-service và readiness probe. Ở thượng nguồn, pool theo host của gateway tới course-service đầy và route của nó suy giảm cho mọi user. Không gì trong cấu hình mặc định chặn được điều này: Feign đợi 60 s, và một lời gọi 2 s không phải chậm với một breaker có ngưỡng chậm 60 s. Để chặn, theo thứ tự đòn bẩy: read timeout 2 s để không thread nào đợi lâu hơn hạn chót của gateway; bulkhead khoảng 25 để 175 thread còn lại tiếp tục phục vụ; ngưỡng chậm 1 s ở 80 % để breaker mở theo độ trễ, không chỉ theo lỗi; và một fallback trả về tên đã biết gần nhất thay vì null. Rồi, nếu tra cứu nằm trên đường nóng, bỏ hẳn lời gọi bằng bản sao cục bộ nuôi bằng sự kiên user. Cách đặt vấn đề senior: mục tiêu là độ trễ của auth-service hiện ra dưới dạng một cái tên cũ trên trang, không phải một sự cố của khóa học.

Q: Bạn định cỡ timeout qua ba chặng thế nào?

Từ ngoài vào trong, mỗi cái ngắn hơn cái trước. Trình duyệt hay client di động là ngân sách ngoài cùng — chẳng hạn 10 s trước khi người dùng bỏ cuộc. Timeout phản hồi của gateway phải nằm trong đó, 5 s, để client nhận 503 sạch sẽ thay vì treo. Feign read timeout của course-service phải nằm trong *cái đó* và chừa chỗ cho retry: 2 s với tối đa một lần retry giữ trường hợp xấu nhất quanh 4 s. Bất kỳ timeout nào dài hơn của bên gọi là một chỗ rò — công sức bỏ ra cho một request đã bị bỏ rơi — và bất kỳ chính sách retry nào cũng phải tính vào phép cộng, vì ba lần thử ở 2 s là 6 s cộng backoff. Trong production bạn cũng đặt timeout *kết nối* riêng và ngắn hơn nhiều (1 s), vì không kết nối được thì chẩn đoán trong một vòng đi-về. Các hệ thống tốt nhất lan truyền một header hạn chót từ biên để mỗi chặng tự tính ngân sách còn lại; gRPC làm điều này ngay trong giao thức.

Q: Bao retry — bạn ngăn thế nào?

Ba biện pháp, cần tất cả. Thứ nhất, jitter: lùi theo hàm mũ đơn thuần để mọi client thất bại cùng lúc retry cùng lúc, nên đợt sóng thứ hai đồng bộ và lớn hơn; ngẫu nhiên hóa thời gian chờ $\pm 50\%$ dãn nó ra. Thứ hai, ngân sách retry: retry theo phần trăm lưu lượng (mặc định của Envoy là 20 %), để khi một phụ thuộc thực sự chết, retry không thể nhân ba tải của nó. Resilience4j xấp xỉ điều này bằng

cách đặt Retry ngoài breaker — một breaker đang mở từ chối các lần thử trước khi chúng thành retry. Thứ ba, tính lũy đẳng: chỉ retry GET và các thao tác mang khóa lũy đẳng mà server xử trùng lặp; nếu không một timeout trên POST thành một lần ghi danh trùng. Và chỉ retry lỗi vận chuyển và 503 — một 400 hay 404 là tắt định và sẽ lỗi lại. Dấu hiệu của bão trong metric là một phụ thuộc có tốc độ request tăng khi tỷ lệ thành công của nó giảm.

Q: Breaker đang mở, nhưng user cần dữ liệu. Fallback của bạn trả về gì?

Thứ trung thực nhất có sẵn, định kiểu để bên gọi không nhầm nó với giá trị thật. Thứ tự ưu tiên: giá trị thành công gần nhất từ một cache sống ngắn (cũ nhưng thật — tên giáo viên của mười phút trước là ổn); một giá trị thay thế suy giảm rõ ràng mà UI render được (“Giáo viên không rõ”) không bao giờ giả dạng dữ liệu; một tập rỗng nơi miền cho phép; hoặc một ngoại lệ có kiểu trở thành 503 với Retry-After. Không bao giờ null, thứ biến một sự kiện chịu lỗi thành NullPointerException ở đâu đó khác. Fallback cũng phải thấy nguyên nhân: 404 nghĩa là user không tồn tại và nên hiện ra như thế, không bị nuốt vào “thử lại sau”. Và fallback phải rẻ và cục bộ — một fallback gọi service khác là một phụ thuộc thứ hai trên đường lỗi.

Q: Làm sao bạn biết breaker đang mở lúc 3 giờ sáng?

Vì nó là một metric có cảnh báo, không phải một dòng log. Resilience4j xuất `resilience4j_circuitbreaker_state` như một gauge theo từng trạng thái; một rule như “state=open” bằng 1 quá 60 giây” gọi ai đó với tên phụ thuộc trong cảnh báo. Ghép với các chỉ báo sớm: `slow_call_rate` tăng trước khi breaker nhảy, số chỗ trống bulkhead tiến về không, và `p99` của phụ thuộc từ `http_client_requests_seconds`. Breaker cũng nuôi `/actuator/health` khi bật `register-health-indicator`, nên `readiness probe` có thể phản ánh nó — dù thường bạn *không* muốn một breaker mở kéo service ra khỏi vòng xoay, vì nó đang phục vụ fallback chính là để ở lại. Và log *lần chuyển trạng thái* ở WARN một lần, không phải mỗi lời gọi bị từ chối, nếu không log thành sự cố kế tiếp.

Q: Bạn đặt breaker ở gateway hay ở service?

Cả hai, cho hai loại lỗi khác nhau. Breaker của gateway (Chương 31) bảo vệ user khỏi cả một service chậm; nó mở theo route và phục vụ một body fallback. Breaker của service bảo vệ thread pool của chính service khỏi một trong các phụ thuộc của nó và là nơi fallback hiểu miền sống — chỉ course-service biết một tên giáo viên thiếu là sống được. Gateway không thể biết điều đó. Sai lầm là chỉ có cái ở

biên và cho rằng các service phía sau vì thế an toàn; chuỗi lỗi mà chương này đi qua diễn ra hoàn toàn phía sau gateway.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Dự án cấu hình một breaker tên `authService`. Liệt kê ba lý do riêng biệt khiến nó hiện không bảo vệ lời gọi nào, và thay đổi một dòng sửa từng lý do.
2. Với `sliding-window-size: 10` và các mặc định còn lại, bao nhiêu lần thất bại liên tiếp có thể xảy ra sau khởi động trước khi breaker mở, và vì sao một lời gọi mất 30 s không được tính?
3. Sắp thứ tự và biện minh: timeout phản hồi của gateway, timeout của trình duyệt, Feign read timeout, số lần retry.
4. `AuthServiceFallback.getUserById` trả về `null`. Nếu hai khiếm khuyết cụ thể kéo theo và chữ ký hai tham số sửa được cái tinh vi hơn.
5. `auth-service` trả 401 cho mọi lời gọi Feign. Giải thích nguyên nhân trong cấu hình bảo mật của dự án, tác động lên breaker, và hai thay đổi giải quyết nó.
6. Ba phụ thuộc đồng bộ mỗi cái 99,9% sẵn sàng nối tiếp trên một endpoint. Tính sẵn sàng của nó là bao nhiêu, và hai thay đổi thiết kế nào nâng nó lên mà không đụng tới các phụ thuộc?

Đào sâu: Dữ liệu xuyên service — mỗi service một cơ sở dữ liệu, dữ liệu trùng lặp và outbox

Chương 7 đặt ra quy tắc: mỗi service sở hữu cơ sở dữ liệu của mình, và con đường duy nhất tới dữ liệu của service khác là API hoặc sự kiện của nó. Quy tắc dễ phát biểu và tồn kém để sống cùng. Chương đào sâu này đi theo mẫu dữ liệu duy nhất mà dự án sao chép giữa các service — tên và e-mail của một học viên — từ lúc nó được ghi vào Postgres của auth-service tới lúc nó xuất hiện trong Postgres của course-service, và hỏi chuyện gì có thể hỏng trên đường đi. Câu trả lời là bài toán ghi kép (dual write), và cách sửa là transactional outbox.

Ba câu hỏi sắp xếp chương này:

1. **Mỗi service một cơ sở dữ liệu mua được gì**, và cấm điều gì — join, khóa ngoại, transaction — mà một monolith coi là hiển nhiên?
2. **Dữ liệu trùng lặp giữ đúng bằng cách nào**, và cửa sổ chính xác mà dự án đang sai hôm nay là gì?
3. **“Lưu dòng và phát sự kiện” có thể nguyên tử được không** — và nếu không, thứ gần nhất với nó là gì?

33.1 Vì sao chọn nó, và nó giải quyết gì

Mở `docker-compose.yml`: hai container Postgres 17, `postgres-auth` giữ `ts_auth` và `postgres-course` giữ `ts_course`, trên volume và cổng riêng. Không gì trong `course-service` biết địa chỉ của cái đầu. Sự cô lập mua bốn thứ:

- **Quyền sở hữu schema.** Auth-service có thể thêm cột `totp_secret`, đổi tên `provider_id`, hay chạy một migration Flyway phá hủy mà không cần phối hợp với ai. Các migration v3 và v4 ở Chương 7 đã diễn ra đúng như thế.
- **Triển khai độc lập.** Một migration khóa bảng `users` một phút làm nghẽn đăng nhập, không phải danh sách khóa học.
- **Mở rộng và tinh chỉnh độc lập.** Cơ sở dữ liệu khóa học sẽ lớn theo bài nộp; cơ sở dữ liệu auth thì không. Chúng có thể nằm trên đĩa khác nhau, có connection pool khác nhau, lịch sao lưu khác nhau.

- **Chặn cứng sự ghép nối vô tình.** Trong một cơ sở dữ liệu dùng chung, câu `JOIN users ON student.user_id = users.id` đầu tiên viết dưới áp lực deadline hàn schema của hai đội lại với nhau mãi mãi. Ở đây join là bất khả, nên nó không bao giờ được viết.

Nó cấm đúng bằng ấy:

- **Không join xuyên service.** “Liệt kê học viên khóa 7 kèm e-mail” không thể là một truy vấn nếu e-mail nằm ở nơi khác.
- **Không khóa ngoại xuyên service.** `student.user_id` trong `V1__create_course_tables.sql` là `BIGINT NOT NULL UNIQUE` và không gì hơn — Postgres không thể kiểm tra user 42 tồn tại trong một cơ sở dữ liệu nó không nhìn thấy.
- **Không transaction xuyên service.** Một thao tác nghiệp vụ phải đối cả hai cơ sở dữ liệu không thể rollback như một khối.

QUYẾT ĐỊNH — SAO CHÉP HỌC VIÊN, KHÔNG GỌI LẤY USER

Course-service cần tên và e-mail học viên trên mọi danh sách lớp và trong mọi thông báo bài tập. Có hai thiết kế: gọi auth-service mỗi lần (thậm chí có sẵn một Feign client cho việc đó, `AuthServiceClient.getUserById`, với fallback trả về null), hoặc giữ một bản sao cục bộ trong bảng `student`. Dự án chọn bản sao, và Feign client là code chết — không class nào trong course-service gọi nó. Lý do: danh sách lớp được đọc nhiều hơn rất nhiều so với số lần tên thay đổi; một lời gọi đồng bộ cho mỗi dòng sẽ khiến tính sẵn sàng của course-service là tích của tính sẵn sàng auth-service (Chương 6); và bản sao cục bộ có thể index, join, phân trang bằng SQL thuần. Cái giá là bản sao có thể cũ, và phần còn lại của chương nói về cũ đến đâu, và làm sao chặn trên nó. Quyết định lật lại khi dữ liệu thay đổi thường xuyên hoặc phải là nguồn tin cậy lúc đọc — số dư tài khoản, một quyền hạn — nơi một lời gọi, hoặc cache với vô hiệu hóa tường minh, mới là thiết kế trung thực.

33.2 Một học viên tới course-service bằng cách nào

Lần theo đường đi thật. Nó bắt đầu ở `AuthService` của auth-service, được đánh dấu `@Transactional` ở mức class:

```
public void activate(String code) {
    User user = userRepository.findByActivationCode(code)
        .orElseThrow(() ->
            new IllegalArgumentException("Invalid activation code"));
}
```

CHƯƠNG 33. ĐÀO SÂU: DỮ LIỆU XUYỀN SERVICE — MỖI SERVICE MỘT CƠ SỞ DỮ LIỆU, DỮ LIỆU TRÙNG LẬP VÀ OUTBOX

```
user.setActivated(true);
user.setActivationCode(null);
userRepository.save(user);                                ①

userActivatedPublisher.publish(user);                    ②
}
```

UserActivatedPublisher dựng một record UserActivatedEvent — id user, e-mail, họ tên, vai trò — và gọi kafkaTemplate.send("ts.user.activated", event.userId(), event). Ở phía kia, UserActivatedListener của course-service:

```
@KafkaListener(topics = "ts.user.activated", groupId = "course-group")
public void onUserActivated(UserActivatedEvent event) {
    if (!ROLE_STUDENT.equalsIgnoreCase(event.role())) {
        return;
    }
    Long userId = Long.parseLong(event.userId());
    if (studentRepository.findById(userId).isPresent()) { ③
        return;
    }
    Student student = Student.builder()
        .userId(userId)
        .firstName(event.firstName())
        .lastName(event.lastName())
        .email(event.email())
        .build();
    studentRepository.save(student);                        ④
}
```

- ① Dòng đang bản bên trong một transaction chưa commit. save() trên một entity đã được quản lý là no-op cho tới khi flush.
- ② Sự kiện được trao cho Kafka producer *bên trong* cùng transaction, trước commit ở lúc thoát method. Ghi nhớ dòng này; nó là toàn bộ vấn đề.
- ③ Listener lũy đẳng (idempotent) theo khóa tự nhiên: giao lại lần hai cùng sự kiện thấy dòng đã có và trả về. Đây là lý do dự án sống sót qua giao nhận ít-nhất-một-lần (Chương 34) mà không cần bảng khử trùng — ở đây.
- ④ Bản sao. Từ giờ, course-service trả lời mọi truy vấn danh sách lớp từ dòng này và không bao giờ hỏi auth-service nữa.

Cửa sổ nhất quán vì thế là: từ commit ở auth-service tới commit ở course-service. Trên một stack yên tĩnh đó là mili giây. Với notification-service và course-service cùng nằm trên đường consumer, và broker trên cùng cái máy hai nhân (Chương 25), nó có thể là vài giây. Một giáo viên ghi danh học viên “ngay lập tức” sau khi học viên kích hoạt có thể nhận 404 Student not found từ EnrollmentService.enroll — hành vi đúng, trải nghiệm bất ngờ.

33.2.1 Thứ hoàn toàn không được lan truyền

Giờ tìm sự kiện thứ hai. Không có. `ts.user.activated` bắn một lần, lúc kích hoạt. Một user sau đó đổi tên hay e-mail trong auth-service không bao giờ được phản ánh vào student; ghi chú “future: Kafka event propagates e-mail changes” trong DATABASE-DESIGN.md là lời thừa nhận. Thông báo bài tập sẽ tiếp tục đi tới địa chỉ cũ. Đây là trạng thái trung thực của nhất quán cuối cùng trong dự án: *cuối cùng* cho việc tạo, *không bao giờ* cho việc cập nhật. Mục 33.5 thêm sự kiện còn thiếu.

33.3 Ưu và nhược, thắng thần

	Cơ sở dữ liệu dùng chung	Mỗi service một CSDL (ở đây)
Đọc xuyên entity	một JOIN	bản sao cục bộ, hoặc hai lời gọi
Toàn vẹn	khóa ngoại	code ứng dụng + sự kiện
Ghi nhiều entity	một transaction	saga hoặc outbox
Đổi schema	phối hợp mọi chủ sở hữu	chủ sở hữu tự quyết
Bán kính sự cố	một CSDL, mọi service	một service
Độ cũ	không	vài giây (tạo), không giới hạn (cập nhật) hôm nay

Điểm yếu thật của dự án, gọi tên:

- Sự kiện cập nhật không tồn tại; bản sao trôi vĩnh viễn.
- Lưu-và-phát không nguyên tử (mục kế tiếp).
- Fallback của Feign trả null, nên nếu ai đó nối AuthServiceClient vào, trạng thái circuit mở sẽ hiện ra dưới dạng NullPointerException thay vì một phản hồi suy giảm rõ ràng.
- Không có đường báo cáo: “bao nhiêu học viên đã kích hoạt mỗi khóa” cần dữ liệu từ cả hai cơ sở dữ liệu và không có read model nào cho nó.

33.4 Bài toán ghi kép

Dòng ② ở trên ghi vào hai hệ thống — Postgres và Kafka — không chia sẻ transaction manager. Dù bạn đặt thứ tự thế nào, một trong hai sự cố là khả dĩ:

- **Commit, rồi mất sự kiện.** `send()` là bất đồng bộ: nó nối vào một bộ đệm phía client và trả về một future. Method thoát, transaction commit, và JVM bị giết (OOMKilled, một rolling update, `kubectl delete pod`) trước khi bộ đệm được flush. User đã kích hoạt; course-service không bao giờ nghe về nó; học viên không bao giờ ghi danh được, và không gì log lỗi.
- **Phát, rồi rollback.** Bộ đệm flush trong vài mili giây; broker ack; rồi commit thất bại — một ràng buộc unique, kết nối rớt, deadlock. Course-service tạo học viên cho một user không tồn tại. Một dòng *ma*.

Cả hai hiểm theo từng request và chắc chắn theo từng năm. Phiên bản phỏng vấn: “bạn lưu một đơn hàng và phát `OrderCreated`; làm sao bảo đảm cả hai xảy ra?” Câu trả lời junior là “bọc trong try/catch”. Câu trả lời senior bắt đầu bằng “không thể, với hai tài nguyên”, rồi gọi tên outbox.

CẠM BÃY — ĐĂNG KÝ TREO SÁU MƯƠI GIÂY, RỒI THẤT BẠI

Triệu chứng: khi Kafka chết, `POST /api/auth/register` không thất bại nhanh — nó treo một phút rồi trả 500. Nguyên nhân: producer phải lấy metadata topic trước lần gửi đầu; không broker nào với được, nó chặn trong `max.block.ms` (mặc định 60 000 ms) rồi ném ngoại lệ. Ngoại lệ lan ra khỏi `register()`, transaction rollback, và user *không* được đăng ký — vì broker gửi mail chết. Một phụ thuộc lẽ ra bất đồng bộ đã thành điểm lỗi đồng bộ với timeout một phút. Outbox bên dưới biến chuyện này thành “đã đăng ký; e-mail đến trễ”.

CẠM BÃY — BẢN SAO SỐNG LÂU HƠN BẢN GỐC

Triệu chứng: e-mail bài tập bị trả lại từ một địa chỉ user đã đổi nhiều tháng trước. Nguyên nhân: `student.email` được `UserActivatedListener` ghi một lần và không có sự kiện `ts.user.updated`. Cách sửa: phát ra một sự kiện như thế, và để listener upsert. Mục 33.5 cho thấy bên phát; thay đổi ở listener là một `findById().map(update).orElseGet(create)`.

33.5 Nâng cấp giải pháp: transactional outbox

Ý tưởng là ghi sự kiện vào *cùng cơ sở dữ liệu*, trong *cùng transaction*, với dòng nghiệp vụ — và để một tiến trình riêng chép nó sang Kafka sau đó. Postgres bảo đảm dòng và sự kiện commit cùng nhau hoặc không gì cả; relay bảo đảm sự kiện cuối cùng tới Kafka; tính lũy đẳng của consumer bảo đảm relay thử lại không gây hại. Đây là cài đặt đầy đủ cho auth-service.

Bảng, dưới dạng migration Flyway `v5__create_outbox.sql`:

```

CREATE TABLE outbox (
  id          UUID PRIMARY KEY,
  aggregate_type VARCHAR(64) NOT NULL, -- 'user'
  aggregate_id VARCHAR(64) NOT NULL, -- also the Kafka key
  event_type  VARCHAR(64) NOT NULL, -- 'UserActivated'
  topic       VARCHAR(128) NOT NULL,
  payload     JSONB       NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  published_at TIMESTAMPTZ,
  attempts    INT         NOT NULL DEFAULT 0
);
-- the relay's only query: oldest unpublished first
CREATE INDEX idx_outbox_unpublished
  ON outbox (created_at) WHERE published_at IS NULL;

```

Entity cố tình ngu ngốc — một dòng, không phải domain object:

```

@Entity @Table(name = "outbox")
@Getter @Setter @NoArgsConstructor
public class OutboxEvent {
  @Id private UUID id;
  private String aggregateType;
  private String aggregateId;
  private String eventType;
  private String topic;
  @JdbcTypeCode(SqlTypes.JSON)
  private String payload; // serialized event
  private Instant createdAt;
  private Instant publishedAt;
  private int attempts;
}

public interface OutboxRepository extends JpaRepository<OutboxEvent, UUID> {
  // SKIP LOCKED: two relay instances never pick the same row
  @Query(value = "
    SELECT * FROM outbox WHERE published_at IS NULL
    ORDER BY created_at LIMIT :n FOR UPDATE SKIP LOCKED",
    nativeQuery = true)
  List<OutboxEvent> lockUnpublished(@Param("n") int n);
}

```

Publisher không còn đụng tới Kafka. Nó ghi một dòng, và vì được gọi từ bên trong transaction của AuthService, dòng đó commit cùng với user:

```

@Component
@RequiredArgsConstructor
public class UserActivatedPublisher {

```

```
private final OutboxRepository outbox;
private final ObjectMapper mapper;

public void publish(User user) {
    UserActivatedEvent event = new UserActivatedEvent(
        String.valueOf(user.getId()), user.getEmail(),
        user.getFirstName(), user.getLastName(),
        user.getRole().name());
    OutboxEvent row = new OutboxEvent();
    row.setId(UUID.randomUUID());
    row.setAggregateType("user");
    row.setAggregateId(event.userId());
    row.setEventType("UserActivated");
    row.setTopic("ts.user.activated");
    row.setPayload(toJson(event));
    row.setCreatedAt(Instant.now());
    outbox.save(row);

}

private String toJson(Object o) {
    try { return mapper.writeValueAsString(o); }
    catch (JsonProcessingException e) {
        throw new IllegalStateException(e);
    }
}
```

Relay chạy mỗi giây, lấy một lô đã khóa, gửi từng dòng *đồng bộ*, và đánh dấu:

```
@Component
@RequiredArgsConstructor
@Slf4j
public class OutboxRelay {

    private final OutboxRepository outbox;
    private final KafkaTemplate<String, String> kafka;

    @Scheduled(fixedDelay = 1000)
    @Transactional
    public void relay() {
        for (OutboxEvent row : outbox.lockUnpublished(100)) {
            ProducerRecord<String, String> rec = new ProducerRecord<>(
                row.getTopic(), row.getAggregateId(), row.getPayload());
            rec.headers().add("eventId",
                row.getId().toString().getBytes(UTF_8));
            rec.headers().add("eventType",
                row.getEventType().getBytes(UTF_8));
            try {
```



```

        kafka.send(rec).get(10, TimeUnit.SECONDS); ⑥
        row.setPublishedAt(Instant.now());
    } catch (Exception e) {
        row.setAttempts(row.getAttempts() + 1);
        log.warn("outbox {} attempt {} failed: {}",
                row.getId(), row.getAttempts(), e.toString());
        break; ⑦
    }
}
}
}

```

- ① Id sự kiện được đúc ở đây, một lần, và sống qua mọi lần thử lại. Nó là khóa khử trùng của consumer (Chương 34).
- ② Id aggregate trở thành key Kafka, nên mọi sự kiện về một user rơi vào một partition, theo thứ tự.
- ③ Một INSERT thuần, trong transaction của bên gọi. Nếu activate() rollback, dòng này cũng vậy: không sự kiện ma. Nếu commit, dòng đã bền vững trước khi Kafka dính lú: không mất sự kiện.
- ④ Transaction của relay giữ các khóa FOR UPDATE SKIP LOCKED, nên replica auth-service thứ hai chạy cùng scheduler lấy những dòng khác.
- ⑤ Header mang danh tính và loại mà không làm bản payload JSON mà các class DTO giải tuần tự.
- ⑥ Nơi duy nhất trong auth-service chờ Kafka, và nó là một luồng nền. Request HTTP không bao giờ chặn trên broker nữa.
- ⑦ Khi thất bại thì dừng lô, giữ thứ tự (dòng 2 không được phát trước dòng 1 cho cùng key), và để nhíp kế tiếp thử lại. attempts là thứ bạn đặt cảnh báo.

Thêm `@EnableScheduling` vào `AuthServiceApplication` và đổi `KafkaTemplate` thành `<String, String>`, vì payload đã là JSON. Consumer thấy đúng thứ nó từng thấy.

Hai dòng sổ sách hoàn tất nó: một lệnh hàng đêm `DELETE FROM outbox WHERE published_at < now() - interval '7 days'` giữ bảng nhỏ, và cùng hình dạng `UserActivatedPublisher` cho bạn `ts.user.updated` miễn phí — gọi nó từ bất cứ đâu lưu thay đổi hồ sơ, với `eventType = "UserUpdated"`, và để `UserActivatedListener` upsert thay vì insert.

THỰC HÀNH — XEM OUTBOX HẤP THỤ MỘT SỰ CỐ BROKER

Với outbox đã có, dừng Kafka, kích hoạt một user, và nhìn:

```

$ docker compose stop kafka
$ curl -s -X POST localhost:8080/api/auth/activate?code=$CODE
{"message":"Account activated"} # instant, no 60s hang

```

```
$ docker compose exec postgres-auth psql -U postgres ts_auth -c \  
"select event_type, attempts, published_at from outbox"  
event_type | attempts | published_at  
UserActivated | 3 |  
$ docker compose start kafka  
# ...a few seconds later  
UserActivated | 3 | 2026-09-07 03:04:11.20+00
```

Rồi xem log của course-service tìm Provisioned student row. Việc kích hoạt chưa bao giờ gặp rủi ro, và học viên xuất hiện ngay khi broker xuất hiện.

33.5.1 Change data capture: relay cấp công nghiệp

Relay kiểu polling đơn giản và thêm tối đa một giây độ trễ cộng một truy vấn mỗi giây mỗi replica. Ở quy mô lớn relay được thay bằng *change data capture*: Debezium đọc đuôi write-ahead log của Postgres, thấy INSERT INTO outbox, và tự phát nó lên Kafka, không cần scheduler ứng dụng và độ trễ dưới 100 ms. Code ứng dụng y hệt — ghi dòng — đó là lý do nên áp dụng outbox sớm: relay là phần duy nhất thay đổi.

33.6 Saga: khi hai service đều phải thay đổi

Ghi danh hôm nay là cục bộ — course, student và enrollment đều nằm trong ts_course, nên EnrollmentService.enroll là một transaction bình thường. Lần ghi xuyên service thực sự đầu tiên mà dự án sẽ gặp là *xóa tài khoản*: auth-service phải xóa user; course-service phải xóa học viên, các ghi danh, bài nộp, và file bài nộp trong MinIO (Chương 9). Không có transaction nào trải qua từng ấy.

Một *saga* là chuỗi các transaction cục bộ trong đó mỗi bước phát một sự kiện kích hoạt bước kế, và mỗi bước có một hành động *bù* (compensating) nếu bước sau thất bại:

1. auth-service đánh dấu user DELETING (chưa xóa) và ghi UserDeletionRequested vào outbox.
2. course-service xóa dữ liệu và file của học viên, rồi phát StudentDataDeleted — hoặc, khi thất bại, StudentDataDeletionFailed.
3. auth-service, khi nhận sự kiện thành công, xóa hẳn user; khi nhận sự kiện thất bại, bù bằng cách khôi phục ACTIVE và báo cho vận hành.

Đây là *choreography*: không có điều phối viên, mỗi service phản ứng với sự kiện. Nó hợp với hai hay ba bước. Quá số đó — xóa xuyên năm service, với ràng

buộc thứ tự — đồ thị sự kiện trở nên không đọc nổi, và các đội chuyển sang *orchestration*: một thành phần (saga orchestrator, hoặc một workflow engine như Temporal) giữ máy trạng thái và gửi *lệnh* tới từng service. Đánh đổi là thêm một service để vận hành lấy một luồng bạn thực sự đọc được.

Hai tính chất mọi saga phải có, và người phỏng vấn kiểm tra: *khóa ngữ nghĩa* — trạng thái DELETING ngăn user đăng nhập trong vài giây saga chạy — và *bước lũy đẳng*, để giao nhận ít-nhất-một-lần của Chương 34 không thể xóa hai lần hay bù hai lần.

33.7 Đọc xuyên service

Khoảng trống còn lại là báo cáo. “Học viên đã kích hoạt mỗi khóa, kèm lần đăng nhập cuối” cần dữ liệu auth và dữ liệu khóa học, và không service nào có cả hai. Ba câu trả lời, theo chi phí tăng dần:

- **Ghép API.** Gateway hoặc một backend-for-frontend nhỏ gọi cả hai service và join trong bộ nhớ. Ổn cho một trang, sai cho một báo cáo hằng đêm trên mọi khóa học.
- **Một read model.** Một cơ sở dữ liệu reporting riêng đăng ký *mọi* sự kiện miền và duy trì các bảng phi chuẩn hóa theo hình dạng truy vấn. Đây là CQRS: ghi đi tới các service sở hữu, đọc đi tới một mô hình dựng từ sự kiện của chúng. Outbox ở trên là điều kiện tiên quyết — không có sự kiện đáng tin, read model trôi âm thầm.
- **Một kho dữ liệu.** CDC từ mọi cơ sở dữ liệu service vào một kho phân tích. Read model cho những người không phải bạn.

33.8 Chuyện gì xảy lúc 3 giờ sáng

- **postgres-course chết; user vẫn kích hoạt.** Không có outbox, không gì mất ở phía auth, nhưng UserActivatedListener ném ngoại lệ ở save(). Error handler mặc định của Spring thử lại mười lần liên tiếp, rồi *log và bỏ qua* record (Chương 34). Sự kiện đã tiêu thụ, học viên không bao giờ được tạo, và dòng log là dấu vết duy nhất. Có outbox thì phía auth an toàn, nhưng phía consumer vẫn cần topic thử lại — outbox sửa producer, không sửa consumer.
- **Một sự kiện bị phát lại.** Ai đó reset offset của course-group để phục hồi sau sự cố trên. Mọi ts.user.activated kể từ đầu retention được giao lại. findByIdById().isPresent() biến từng cái thành no-op. Lũy đẳng theo khóa tự nhiên là thứ biến phát lại thành thao tác thường ngày thay vì sự cố.

- **Thứ tự.** UserActivated và một UserUpdated tương lai cho cùng user được key theo userId nên chung partition; cập nhật không thể vượt mặt kích hoạt. Sự kiện cho các user *khác nhau* có thể xen kẽ tự do, ổn thôi — không có bất biến xuyên user. Đổi key, hoặc gửi từ hai producer khác nhau, và bảo đảm này biến mất.
- **Relay kẹt.** Một dòng độc — payload broker từ chối vì kích thước — khiến ⑦ break ở mọi nhịp, và các dòng phía sau xếp hàng mãi mãi. Cảnh báo trên max(attempts) và trên count(*) where published_at is null; sau N lần thử, chuyển dòng sang bảng outbox_dead và để hàng đợi chảy tiếp.

QUYẾT ĐỊNH — KHI MỘT CƠ SỞ DỮ LIỆU LÀ LỰA CHỌN TỐT HƠN

docs/PLAN-A mô tả cùng sản phẩm dưới dạng modular monolith: một ứng dụng Spring Boot, một cơ sở dữ liệu ts, và student.user_id là khóa ngoại thật. Mọi vấn đề trong chương này — ghi kép, trôi dữ liệu, saga, read model — biến mất ở đó, thay bằng một JOIN và một transaction. Đó không phải khoản tiết kiệm nhỏ. Bố cục microservice (PLAN-B) chỉ xứng đáng với chi phí khi những điều bị cấm chính là mục đích: các đội phải triển khai không cần nhau, dữ liệu phải mở rộng riêng, hoặc tổ chức đủ lớn để một schema dùng chung sẽ bị join chéo trong cơn nóng giận. Với một đội ba người dự án này cố ý xây quá tay, và chặng đường phía trước ở Chương 27 thẳng thắn về điều đó. Câu trả lời senior trong phòng vẫn không phải “microservices” hay “monolith”; nó là “modular monolith cho tới khi một ranh giới có lý do để trở thành ranh giới mạng — và khi đó, outbox ngay từ ngày đầu”.

33.9 Ngoài dự án

Outbox là mẫu đứng sau “outbox event router” của Debezium, transactional messaging của Axon và Eventuate, và các dạng TransactionalEventPublisher trong mọi codebase hướng sự kiện nghiêm túc. Saga kiểu orchestration được sản phẩm hóa bởi Temporal, Camunda và AWS Step Functions; kiểu choreography đơn giản là “sự kiện kèm hành động bù” và một sơ đồ ai đó phải giữ cho cập nhật. Netflix, Uber, và mọi ngân hàng bạn kể tên được đều chạy một tổ hợp nào đó của ba thứ này — outbox cho độ tin cậy, saga cho ghi nhiều bước, read model CQRS cho truy vấn vượt ranh giới sở hữu. Điều duy nhất không ai trong số họ làm là điều dự án đang làm hôm nay: gọi send() bên trong một transaction và hy vọng.

33.10 Câu hỏi từ phỏng vấn thật

Q: Bạn cần một báo cáo join user và khóa học, mà chúng nằm ở hai cơ sở dữ liệu. Làm thế nào?

Trước hết tôi hỏi cần tươi đến đâu và lớn đến đâu. Cho một màn hình, ghép API: một backend-for-frontend gọi `/users?ids=` và `/courses`, join trong bộ nhớ, ổn tới vài trăm dòng. Cho một báo cáo thật tôi dựng read model: một cơ sở dữ liệu reporting đăng ký `ts.user.activated`, `ts.user.updated` và các sự kiện khóa học, duy trì một bảng `course_roster` phi chuẩn hóa theo hình dạng truy vấn. Đó là CQRS — các service sở hữu giữ phần ghi, read model trả lời phần đọc xuyên service. Điều kiện tiên quyết là sự kiện đáng tin, nên tôi đặt outbox vào trước; một read model nuôi bằng những lời gọi `send()` kiểu cố-gắng-hết-sức sẽ trôi và không ai nhận ra cho tới khi bộ phận tài chính nhận ra. Cho phân tích ở quy mô công ty, cùng ý tưởng là CDC vào kho dữ liệu. Điều tôi từ chối làm là cấp cho job báo cáo một read replica của cả hai cơ sở dữ liệu và một join xuyên cơ sở dữ liệu — đó là sự ghép nối qua cơ sở dữ liệu dùng chung mà kiến trúc này tồn tại để ngăn, quay lại bằng cửa hông.

Q: Hai service phải cùng thành công hoặc không service nào. Làm thế nào, không dùng two-phase commit?

Một saga. Mỗi service làm một transaction cục bộ và phát sự kiện; bước kế phản ứng; mỗi bước có hành động bù. Trong dự án, xóa tài khoản: auth-service đặt `DELETING` và phát `UserDeletionRequested`; course-service xóa dữ liệu và phát `StudentDataDeleted` hoặc `...Failed`; auth-service hoặc xóa hẳn hoặc khôi phục `ACTIVE`. Hai thứ khiến nó đúng thay vì hy vọng: khóa ngữ nghĩa — trạng thái `DELETING` chặn đăng nhập trong vài giây saga chạy — và bước lũy đẳng, vì Kafka sẽ giao ít nhất một lần. Tôi chọn choreography cho hai hay ba bước và orchestration (Temporal, Camunda, hoặc một máy trạng thái tự viết) khi đồ thị sự kiện cần một sơ đồ mới theo dõi được. 2PC bị loại không phải vì nó sai mà vì Kafka và Postgres không chia sẻ transaction coordinator, và giữ lock xuyên một lời gọi mạng là cách bạn nhận cuộc gọi lúc 3 giờ sáng.

Q: Một sự kiện bị mất giữa commit cơ sở dữ liệu và lần gửi Kafka. Làm sao để điều đó bất khả?

Bằng cách không bao giờ gửi từ luồng request. Transactional outbox: sự kiện là một dòng trong bảng outbox ghi trong cùng transaction với dòng nghiệp vụ, nên Postgres bảo đảm cả hai hoặc không gì. Một relay — một poller `@Scheduled` với `FOR`

UPDATE SKIP LOCKED, hoặc Debezium đọc WAL — phát nó sau đó và đánh dấu. Relay là ít-nhất-một-lần, nên dòng mang một id sự kiện để consumer khử trùng. Trong dự án này nó còn gỡ một sự cố tiềm ẩn: hôm nay `kafkaTemplate.send()` bên trong `AuthService.register` chặn trong `max.block.ms` khi broker chết, nên một sự cố broker mail khiến đăng ký thất bại sau sáu mươi giây. Với outbox, đăng ký commit trong mili giây và e-mail đến trễ thay vì lượt đăng ký bị mất.

Q: Làm sao chuyển một cơ sở dữ liệu dùng chung sang mỗi service một cơ sở dữ liệu một cách an toàn?

Strangler, không phải big bang. Bước một: cầm máu — tìm mọi join và khóa ngoại xuyên schema và định tuyến chúng qua API của service sở hữu hoặc một bản sao cục bộ, trong khi các bảng vẫn ở một cơ sở dữ liệu. Đó là phần khó và có thể mất nhiều tháng; tách vật lý sau đó là một cuối tuần. Bước hai: cấp cho mỗi service credential và schema riêng, để một join vô tình thất bại ngay lúc biên dịch. Bước ba: ghi kép hoặc CDC các bảng sang cơ sở dữ liệu mới, kiểm đếm và checksum, lật phần đọc, rồi phân ghi, giữ bảng cũ chỉ-đọc thêm một bản phát hành, rồi drop. Xuyên suốt, outbox đi vào *trước* khi tách, vì khoảnh khắc join biến mất là bạn dựa vào sự kiện để giữ bản sao đúng. “Hai server Postgres, không phải hai schema” của Chương 7 là trạng thái cuối; bạn tới đó bằng schema-mỗi-service trước.

Q: Bạn đã nhân bản dữ liệu Student vào course-service. Làm sao giữ nó đúng?

Tôi chấp nhận nó sẽ cũ và tôi chặn trên độ cũ. Việc tạo tới trên `ts.user.activated`; cập nhật cần một `ts.user.updated` dự án chưa phát — đó là sửa đầu tiên. Cả hai key theo `userId`, nên chung partition và cập nhật không thể vượt mặt việc tạo. Listener upsert theo khóa tự nhiên, khiến phát lại an toàn. Sự kiện tới từ outbox, nên không cái nào mất; consumer ghi lại id sự kiện đã xử lý, nên không cái nào áp dụng hai lần. Rồi tôi đo: một job hàng đêm so một mẫu `student.email` với `auth-service` và cảnh báo khi trôi, vì mỗi cơ chế ở trên đều có một lỗi tôi chưa tìm ra. Điều tôi không làm là đặt e-mail vào JWT — việc đó đóng băng nó suốt đời token và ghép schema qua hợp đồng token (Chương 28).

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Đọc `AuthService.activate`. Mô tả hai thứ tự sự cố khác nhau khiến `auth-service` và `course-service` bất đồng, và nói thứ tự nào tạo ra một dòng lẽ ra không tồn tại.
2. Kafka không với được. Giải thích vì sao `POST /register` hôm nay mất

khoảng một phút để thất bại và không tạo user, nêu tên thiết lập producer chịu trách nhiệm. Cùng request đó làm gì sau khi có outbox?

3. Trong relay, vì sao FOR UPDATE SKIP LOCKED là cần thiết, và chuyện gì hỏng với hai replica auth-service nếu thay nó bằng một SELECT thuần?
4. Vì sao break (không phải continue) theo sau một lần gửi thất bại trong vòng lặp relay? Nêu kịch bản mà phương án kia làm hỏng.
5. E-mail của một học viên đổi trong auth-service. Nêu course-service biết gì về việc đó hôm nay, và liệt kê ba thay đổi code khép lại khoảng trống.
6. Thiết kế xóa tài khoản xuyên auth-service, course-service và MinIO như một saga choreography: gọi tên các trạng thái, các sự kiện, hành động bù, và giải thích trạng thái DELETING bảo vệ khỏi điều gì.

Đào sâu: Bảo đảm giao nhận Kafka — ít nhất một lần, consumer lũy đẳng và dead letter

Chương 10 đưa ra quy tắc ngón tay cái — Kafka là ít-nhất-một-lần, nên hãy làm consumer lũy đẳng — và Chương 11 cho thấy phòng thủ poison pill. Chương đào sâu này đọc các thiết lập producer và consumer mà dự án thực sự chạy, phần lớn nó chưa từng viết ra, và đi theo một thông điệp qua mọi nơi nó có thể bị nhân đôi hoặc mất. Rồi bổ sung hai thứ mà một consumer production cần và dự án còn thiếu: một bản ghi khứ trùng và một đường thử lại kết thúc ở dead-letter topic thay vì một dòng log.

Ba câu hỏi sắp xếp chương này:

1. **Dự án đứng ở đâu** trên thang nhiều-nhất-một-lần / ít-nhất-một-lần / đúng-một-lần — không phải theo ý định, mà theo các mặc định nó thừa hưởng?
2. **Chính xác khi nào một thông điệp được giao hai lần**, và consumer phải làm gì để điều đó vô hại?
3. **Chuyện gì xảy ra với một thông điệp không xử lý được**, và nó nên đi đâu thay vì đi vào hư không?

34.1 Vì sao chọn nó, và nó giải quyết gì

Dự án dùng Kafka cho một việc: tách một lần ghi nghiệp vụ khỏi các hiệu ứng phụ của nó. Một user đăng ký; một mail phải đi; một dòng học viên phải xuất hiện trong cơ sở dữ liệu khác. Làm những việc đó inline sẽ khiến đăng ký chậm và mong manh như SMTP và một Postgres thứ hai. Làm qua một log khiến chúng *cuối cùng* xảy ra, sống sót qua khởi động lại consumer, và phát lại được. Chương 6 đã nói về tách đồng bộ/bất đồng bộ; điều nó chưa nói là “cuối cùng” có nghĩa chính xác trong Kafka, gồm năm hay sáu thiết lập, và mặc định của các thiết lập đó là thứ dự án đang chạy hôm nay.

QUYẾT ĐỊNH — SỰ KIỆN, KHÔNG PHẢI LỆNH — VÀ KHI NÀO MỘT LỜI GỌI MỚI ĐÚNG

ts.user.registered nói “việc này đã xảy ra”; nó không nói “hãy gửi e-mail”. Notification-service tự quyết đăng ký nghĩa là gì với nó, và course-service bỏ qua topic này hoàn toàn. Đó là phong cách sự kiện, và là lý do một consumer thứ tư có thể thêm vào mà không đụng auth-service. Phong cách lệnh — một thông điệp SendEmail gửi đích danh cho một handler — ghép bên gửi vào sự tồn tại của bên nhận và thường là dấu hiệu rằng người ta thực ra muốn một lời gọi đồng bộ. Và đôi khi đúng là thế: khi bên gọi cần *kết quả* ngay (user này có tồn tại không? file này đã quét chưa?), một lời gọi Feign với circuit breaker là thiết kế trung thực, và khoắc cho nó cái áo request-reply qua Kafka chỉ thêm hai chặng và một correlation id. Quy tắc của dự án: sự kiện cho các sự thật mà service khác phản ứng; lời gọi cho các câu hỏi bên gọi phải có đáp án trước khi tiếp tục.

34.2 Những thiết lập dự án chưa từng viết

34.2.1 Producer

Course-service dựng producer bằng tay trong KafkaProducerConfig:

```
Map<String, Object> configProps = new HashMap<>();
configProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
configProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
    StringSerializer.class);
configProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
    JsonSerializer.class);
return new DefaultKafkaProducerFactory<>(configProps);
```

Ba dòng — và mọi thứ khác là mặc định của client Kafka 3.9. Từ Kafka 3.0 các mặc định đó là những giá trị an toàn:

```
acks=all
enable.idempotence=true
retries=2147483647
max.in.flight.requests.per.connection=5
delivery.timeout.ms=120000
max.block.ms=60000
linger.ms=0
```

Auth-service và notification-service dùng factory tự cấu hình của Spring Boot từ YAML, thứ đặt cùng hai serializer và không gì khác, nên cả ba producer hành

- ① Key serializer quan trọng hơn vẻ ngoài. Mọi publisher trong dự án đều truyền key: `event.userId()` trong `UserRegisteredPublisher` và `UserActivatedPublisher`, `assignmentId` trong `AssignmentCreatedPublisher`. Kafka bấm key để chọn partition; key bằng nhau, partition bằng nhau, và broker giữ thứ tự trong một partition. Đó là toàn bộ bảo đảm thứ tự theo user, và nó là một chuỗi ký tự.

Các mặc định nghĩa là gì, từng cái một:

- **acks=all**: leader chờ mọi replica đồng bộ trước khi xác nhận. Với `replicas: 1` trong `kafka-topics.yaml` và một broker duy nhất trong Compose, “all” là một — độ bền bằng một cái đĩa. Trong production nó đi cùng `replication.factor=3` và `min.insync.replicas=2`: một lần ghi sống sót khi một broker chết, và nếu hai broker chết producer nhận `NotEnoughReplicas` thay vì một lần ghi bản sao-đơn âm thầm.
- **Producer lũy đăng**: mỗi producer nhận một id và đánh số thông điệp theo partition; một lần thử lại sau khi mất ack được broker nhận ra và không nối thêm lần hai. Điều này khép lại bản sao *phía producer* của Chương 10. Nó không làm gì cho phía consumer.
- **retries không giới hạn, bị chặn bởi `delivery.timeout.ms`**: producer thử lại lỗi tạm thời tới hai phút, rồi làm future thất bại. `send()` đã trả về từ lâu; không ai trong dự án đọc future đó.

CẠM BẦY — YAML KHÔNG LÀM GÌ CẢ

Triệu chứng: bạn thêm `spring.kafka.producer.acks: 1` vào `course-service.yml` để tăng tốc một bài load test, và không gì thay đổi. Nguyên nhân: `KafkaProducerConfig` khai báo bean `ProducerFactory` riêng, việc đó tắt auto-configuration của Boot cho factory ấy — khối producer trong YAML không được ai đọc trong `course-service`. `Auth-service`, không có bean như vậy, sẽ tuân theo nó. Hai service, một quy ước YAML, hai hành vi. Cách sửa: xóa bean viết tay (factory của Boot làm y hệt) hoặc chuyển mọi thuộc tính vào đó; không bao giờ cả hai.

34.2.2 Consumer

YAML consumer trong `notification-service.yml` và `course-service.yml` đặt tên group, `auto-offset-reset: earliest`, và các `deserializer` xử lý lỗi. Mọi thứ về *commit* lại là mặc định, lần này của Spring Kafka:

- `enable.auto.commit=false`. Spring tắt commit theo hẹn giờ của client và tự commit.
- `AckMode.BATCH`. Sau khi listener trả về cho *mọi* record của một `poll()`, Spring commit offset cao nhất của lô. Xử lý và commit cách nhau bằng thời gian xử lý của lô.
- `DefaultErrorHandler` với `FixedBackOff(0, 9)`: một listener ném ngoại lệ được thử lại thêm chín lần không chờ, rồi record được *log và bỏ qua*. Không gửi đi đâu. Không đếm ở đâu mà dự án đọc.
- `max.poll.records=500`, `max.poll.interval.ms=300000`: một lô tới 500 record phải xử lý xong trong năm phút, nếu không consumer bị coi là đã chết.

Ghép bốn thứ đó lại và vị trí của dự án là: ít-nhất-một-lần, với bản sao khả dĩ ở mọi lần crash hay rebalance, và với mất mát âm thầm sau mười lần thử thất bại. Cả thứ tự lẫn mất mát đều không được viết trong file nào bạn có thể grep.

34.3 Chính xác khi nào xảy ra bản sao

Dòng thời gian tạo ra một bản sao, với notification-group chạy hai instance A và B trên một topic có hai partition:

```
t0  A polls partition 0, gets offsets 40..49  (committed: 40)
t1  A processes 40..47: eight e-mails sent
t2  A is killed (rolling update, OOMKilled, SIGKILL)
    -- offset 40 is still the committed position --
t3  group rebalances; partition 0 is assigned to B
t4  B polls from the committed offset: 40
t5  B processes 40..49: ten e-mails sent
    -- users 40..47 receive their activation mail twice --
```

Không gì ở đây là lỗi. Nó là hợp đồng: Kafka hứa offset sẽ không tiến qua một record bạn chưa xử lý xong; cái giá là một record bạn *đã* xử lý xong nhưng chưa commit sẽ quay lại. Mọi consumer làm bất cứ điều gì không đảo ngược được — gửi mail, chèn dòng, trừ tiền thẻ — phải được viết cho t5.

Hai biến thể quan trọng với dự án này. `AckMode.RECORD` commit sau mỗi record thay vì mỗi lô: ít bản sao hơn khi crash (một thay vì tới 500), nhiều lượt commit hơn. Và biến thể *consumer chậm* không cần crash nào: nếu xử lý một lô lâu hơn `max.poll.interval.ms`, broker loại consumer khỏi group, instance khác nhận partition từ commit cuối, và *cả hai* cuối cùng xử lý cùng những record — cái đầu chưa nhận ra mình đã bị đuổi.

CẠM BẦY — MỘT BÀI TẬP, MỘT CƠN BÃO E-MAIL TRÙNG

Triệu chứng: giáo viên tạo bài tập trong khóa có 300 học viên; một số học viên nhận thông báo ba lần; log consumer hiện `Member ... has left the group` và một `rebalance`. Nguyên nhân: `AssignmentCreatedListener` lặp qua `enrolledStudentEmails` và gọi `javaMailSender.send()` cho từng địa chỉ, đồng bộ, trên luồng poll. Một giây mỗi vòng SMTP, 300 e-mail là năm phút — đúng bằng `max.poll.interval.ms`. Consumer bị đuổi giữa vòng lặp, partition được gán lại, và chủ mới bắt đầu vòng lặp lại từ học viên đầu tiên. Cách sửa, theo chi phí: tăng `max.poll.interval.ms` (vá tạm), phát một record `ts.notification.email` cho mỗi người nhận để mỗi mail là một đơn vị công việc riêng (hình dạng đúng), và làm việc gửi lũy đăng theo người nhận (Mục 34.5).

34.4 Ưu và nhược, thăng thấn

	Dự án hôm nay	Nền production
Độ bền producer	<code>acks=all</code> , 1 replica	3 replica, <code>min.insync=2</code>
Bản sao producer	không (lũy đăng)	không (lũy đăng)
Bản sao consumer	có thể, chưa xử lý	có thể, đã khử trùng
Record thất bại	log và bỏ sau 10 lần	topic thử lại, rồi DLT
Thứ tự	theo key, 1 partition	theo key, N partition
Việc chậm	trên luồng poll	tách ra, hoặc bất đồng bộ
Quan sát	không có lag	cảnh báo lag, cảnh báo DLT

Điểm yếu trung thực của dự án: mất mát âm thầm sau mười lần thử; fan-out e-mail trên luồng poll; không khử trùng nơi hiệu ứng phụ không tự nhiên lũy đăng (mail); không có metric lag nào ai nhìn; và topic một partition, khiến thứ tự tầm thường và song song bất khả.

34.5 Nâng cấp giải pháp

34.5.1 Một consumer lũy đăng có sổ ghi những gì đã thấy

`UserActivatedListener` đã lũy đăng theo khóa tự nhiên — giao lại lần hai tìm thấy học viên và trả về. Nơi không có khóa tự nhiên, ghi lại id sự kiện. Outbox của

Chương 33 đặt một header eventId duy nhất lên mọi thông điệp; consumer giữ một bảng các id nó đã xử lý xong, và ghi dòng đó *trong cùng transaction* với thay đổi nghiệp vụ. Flyway V3__create_processed_events.sql trong course-service:

```
CREATE TABLE processed_events (
    event_id    UUID PRIMARY KEY,
    consumer    VARCHAR(64) NOT NULL,    -- listener name
    processed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- retention: rows older than the topic's retention are useless
CREATE INDEX idx_processed_events_at ON processed_events (processed_at);
```

Listener, viết lại để đọc header và giành id trước:

```
@KafkaListener(topics = "ts.user.activated", groupId = "course-group")
@Transactional
public void onUserActivated(
    UserActivatedEvent event,
    @Header("eventId") String eventId) {

    int claimed = jdbc.update("""
        INSERT INTO processed_events (event_id, consumer)
        VALUES (?, :uuid, 'UserActivatedListener')
        ON CONFLICT (event_id) DO NOTHING""", eventId);

    if (claimed == 0) {
        log.info("duplicate event {} ignored", eventId);
        return;

    if (!ROLE_STUDENT.equalsIgnoreCase(event.role())) {
        return;
    }
    Long userId = Long.parseLong(event.userId());
    Student student = studentRepository.findById(userId)
        .orElseGet(() -> Student.builder().userId(userId).build());
    student.setFirstName(event.firstName());
    student.setLastName(event.lastName());
    student.setEmail(event.email());
    studentRepository.save(student);
}
```

- ① Một transaction cơ sở dữ liệu bao quanh việc giành và việc thay đổi. Nếu save() thất bại, lượt giành cũng rollback, và lần thử lại có thể giành lại. Nếu JVM chết sau commit nhưng trước khi commit offset Kafka, lần giao lại rơi vào ③.
- ② ON CONFLICT DO NOTHING là toàn bộ thuật toán khử trùng: một insert-nếu-chưa-có-nguyên-tử. Không có race đọc-rời-ghi giữa hai instance consumer cùng

nhận một record sau rebalance.

- ③ Trả về bình thường, không ném ngoại lệ: offset phải tiến qua một bản sao.
- ④ Nhân tiện, `upsert` thay vì `insert-nếu-thiếu`, để cùng listener này phục vụ luôn sự kiện `ts.user.updated` mà Chương 33 thêm vào.

Notification-service không có cơ sở dữ liệu, và mail là hiệu ứng phụ duy nhất bạn không thể thu hồi. Các lựa chọn, chẳng hạn về chi phí: chấp nhận thỉnh thoảng một mail trùng (điều dự án ngầm làm); hoặc cho service một kho nhỏ — Redis SET `eventId 1 NX EX 604800` trước khi gửi, cùng lượt giành đó trong một lệnh; hoặc làm chính mail lũy đăng bằng cách đặt id sự kiện vào header `Message-ID`, để hộp thư khử trùng thứ consumer không làm được.

34.5.2 Topic thử lại và dead-letter topic

Thay “thử lại mười lần trong một mili giây, rồi quên” bằng thử lại có chờ, và một đích đến cho thứ vẫn thất bại. `@RetryableTopic` của Spring Kafka làm điều này bằng topic thay vì thread, nên một record kẹt không chặn partition phía sau nó:

```
@RetryableTopic(
    attempts = "4",                                ①
    backoff = @Backoff(delay = 2_000, multiplier = 3), ②
    include = { MailSendException.class,
                MailAuthenticationException.class }, ③
    dltStrategy = DltStrategy.FAIL_ON_ERROR)
@KafkaListener(topics = "ts.user.registered",
    groupId = "notification-group")
public void onUserRegistered(UserRegisteredEvent event) {
    emailService.sendActivationEmail(
        event.email(), event.firstName(), event.activationCode());
}

@DltHandler                                         ④
public void onDeadLetter(UserRegisteredEvent event,
    @Header(KafkaHeaders.EXCEPTION_MESSAGE) String reason,
    @Header(KafkaHeaders.ORIGINAL_OFFSET) long offset) {
    log.error("DLT ts.user.registered offset={} user={} reason={}",
        offset, event.userId(), reason);
    meter.counter("kafka.dlt", "topic", "ts.user.registered")
        .increment();                                ⑤
}
```

- ① Một lần thử trên topic chính, ba lần trên các topic thử lại.
- ② Độ trễ 2, 6 và 18 giây — khoảng nửa phút để MailDev hay một SMTP relay quay lại. Spring tạo các topic `ts.user.registered-retry-2000`, `-retry-6000`,

- retry-18000 và ts.user.registered-dlt; dưới Strimzi (Chương 25) hãy khai báo chúng là object `KafkaTopic` hoặc để auto-creation bật.
- ③ Chỉ thử lại thứ mà chờ đợi sửa được. Lỗi giải tuần tự hay `NullPointerException` đi thẳng tới DLT; thử lại nó ba lần là diễn kịch.
 - ④ Handler dead-letter là một listener bình thường trên topic -dlt. Nó *không* được ném ngoại lệ.
 - ⑤ Một counter, để “DLT không rỗng” là một cảnh báo (khả năng quan sát của Chương 27) thay vì một phát hiện tình cờ.

Làm gì với một dead letter là quyết định vận hành, không phải code: một cảnh báo bần; ai đó đọc header `EXCEPTION_MESSAGE`; bản sửa được đưa lên; và các record được *phát lại* — chép từ DLT về topic chính bằng một công cụ admin nhỏ hoặc Kafka UI — nơi consumer lữ đấng bảo đảm những cái đã thành công bị bỏ qua.

THỰC HÀNH — XEM BẬC THANG THỬ LẠI VÀ DEAD LETTER

Làm hỏng SMTP, đăng ký một user, và đọc các topic:

```
$ docker compose stop maildev
$ curl -s -X POST localhost:8080/api/auth/register -H \
  'Content-Type: application/json' -d '{"email":"dlt@test.io", \
  "password":"Passw0rd!", "firstName":"D", "lastName":"L", \
  "activationMethod":"EMAIL"}'
$ docker compose exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:19092 --list | grep registered
ts.user.registered
ts.user.registered-dlt
ts.user.registered-retry-18000
ts.user.registered-retry-2000
ts.user.registered-retry-6000
```

Khoảng 26 giây sau request, notification-service log DLT ts.user.registered offset=0 user=... reason=Mail server connection failed. Khởi động lại MailDev, chép record từ DLT sang ts.user.registered trong Kafka UI (localhost:9090), và mail kích hoạt tới nơi.

34.5.3 Đúng một lần, và những gì nó không bao phủ

Transaction của Kafka cho đúng-một-lần với một hình dạng cụ thể: tiêu thụ từ topic A, sản xuất sang topic B, commit cả offset lần đầu ra một cách nguyên tử (isolation.level=read_committed ở bên đọc). Các bộ xử lý luồng cần nó; dự án không có giai đoạn nào như vậy. Transaction *không* trải qua Kafka và Postgres, hay Kafka và SMTP — khoảnh khắc hiệu ứng phụ rời khỏi broker, bạn quay về ít-nhất-một-lần cộng lũy đẳng. Người phỏng vấn hỏi “có thể đạt đúng-một-lần không?” muốn

đúng sự phân biệt đó: có bên trong Kafka, không xuyên hệ thống, và outbox cộng một bảng khử trùng là cách bạn đạt hiệu quả đó xuyên hệ thống dù sao đi nữa.

34.6 Chuyện gì gây lúc 3 giờ sáng

- **notification-service chết bốn giờ.** Không gì mất: record tích lại trên topic, retention là 168 giờ trong Compose và mặc định bảy ngày của Strimzi. Consumer lag — khoảng cách giữa offset cuối của partition và offset đã commit của group — lớn lên vài nghìn. Khi khởi động lại service tiêu thụ chúng ở tốc độ SMTP; mọi mail kích hoạt đến trễ, không cái nào thiếu. Điều sẽ làm mất chúng: một sự cố dài hơn retention, hoặc một lần triển khai lại dưới group id mới, bắt đầu từ earliest và gửi lại mọi mail trong bảy ngày lịch sử.
- **Broker chết.** Producer chặn trong `max.block.ms` ở lần lấy metadata đầu, rồi ném ngoại lệ — sự cố đăng ký sáu mươi giây của Chương 33. Producer đã kết nối sẵn đệm tới `buffer.memory` (32MB) và thử lại trong `delivery.timeout.ms`; sau hai phút future thất bại và, vì không ai đọc nó, sự kiện biến mất. Consumer đơn giản là poll không được gì. Outbox biến tất cả chuyện này thành một cột `attempts` tăng dần.
- **Phát lại từ hôm qua.** Vận hành `reset course-group` về một mốc thời gian bằng `kafka-consumer-groups.sh --reset-offsets --to-datetime`. Mọi sự kiện từ đó được giao lại; với lượt giành `processed_events` mỗi cái tốn một insert thất bại. Không có nó, mail đi hai lần và dòng học viên được thử tạo hai lần (chỉ được cứu bởi ràng buộc `UNIQUE(user_id)`).
- **Một key nóng.** Mọi sự kiện cho một khóa học rất lớn rơi vào một partition; thêm partition không giúp gì vì key chọn partition. Đổi key — theo người nhận thay vì theo bài tập — hoặc chấp nhận thông lượng của một key là thông lượng của một consumer.

34.7 Ngoài dự án

Mọi Kafka được quản lý — Confluent Cloud, AWS MSK, Aiven — đều giao `acks=all`, ba replica và `min.insync.replicas=2` làm điểm khởi đầu, và bán dashboard lag như thứ đầu tiên bạn nhìn. Topic thử lại và DLT chuẩn tới mức Parallel Consumer của Confluent, consumer proxy của Uber và `@RetryableTopic` của Spring đều cài cùng bậc thang; khác biệt ở cách chúng giữ thứ tự theo key qua các lần thử lại (Spring mặc định không giữ — một record thử lại có thể được xử lý sau một record muộn hơn cùng key, ổn cho mail và không ổn cho bút toán sổ cái). RabbitMQ và SQS cho bạn ack theo từng thông điệp và dead-letter queue như

tính năng của broker thay vì mẫu thư viện; thứ chúng không thể cho là phát lại, chính là lý do một log được chọn ngay từ đầu.

34.8 Câu hỏi từ phỏng vấn thật

Q: Làm sao tăng số thông điệp Kafka xử lý được trong một đơn vị thời gian?

Tôi bắt đầu từ consumer, vì đó là nơi giới hạn của dự án nằm. Partition là đơn vị song song: một group không thể có nhiều consumer hoạt động hơn số partition của topic, và mọi topic trong `kafka-topics.yaml` đều có một. Nên thay đổi đầu tiên là số partition — 6 hoặc 12, chọn dư vì partition thêm được nhưng không bao giờ bớt được, và thêm sau này làm đổi phép băm key-sang-partition, phá thứ tự theo key với dữ liệu đang bay. Rồi khớp consumer với partition: `spring.kafka.listener.concurrency=3` mỗi pod nhân hai replica cho sáu thread trên sáu partition. Kế đến, đưa việc chậm ra khỏi luồng poll: một `AssignmentCreated` gửi 300 e-mail đồng bộ là một lô năm phút làm sập `max.poll.interval.ms`; thay vào đó phát một record cho mỗi người nhận, để thông lượng tăng theo partition. Phía producer, gom lô: `linger.ms=5--20` và `batch.size=64--256 KB` cho client gửi hàng trăm record mỗi request, và `compression.type=lz4` hoặc `zstd` cắt byte trên đường truyền ba đến năm lần với JSON. `acks=1` tăng thông lượng producer chừng gấp đôi với cái giá mất một lần ghi khi leader chết trước khi nhân bản — tôi giữ `acks=all` cho bất cứ thứ gì không tái tạo được. Phía consumer, `max.poll.records` và `fetch.min.bytes` đổi độ trễ lấy ít vòng đi hơn, bị chặn bởi thứ vừa trong `max.poll.interval.ms`. Và tôi để ý key nóng: nếu một partition gánh 80% tải vì chọn key dở, không gì ở trên giúp được và cách sửa là một key khác.

Q: Consumer lag đang tăng. Bạn chẩn đoán thế nào?

Trước hết tôi xác định nó là một trong ba thứ: consumer chậm, consumer chết, hay producer nhanh lên. `kafka-consumer-groups.sh --describe --group notification-group` cho lag theo partition và member nào sở hữu mỗi cái; lag trên *mọi* partition mà member vẫn có mặt nghĩa là xử lý chậm; lag trên *một số* với member biến mất nghĩa là một instance chết hoặc kẹt trong rebalance; lag xuất hiện ở một thời điểm rõ ràng với tiêu thụ đều nghĩa là một đợt bùng nổ producer. Với xử lý chậm tôi nhìn thời gian mỗi record của listener — trong dự án này là độ trễ SMTP — và sự khuấy đảo rebalance trong log (`Member ... has left`), nghĩa là `max.poll.interval.ms` đang bị chạm và group đang quay cuồng. Cách sửa đi theo chẩn đoán: thêm partition và concurrency, đưa lời gọi chậm ra khỏi luồng, hoặc, nếu đợt bùng nổ là hợp lệ, để nó chảy hết và chỉ cảnh báo khi lag già hơn

SLO tính bằng giây, không phải lớn hơn một số record. Metric để vẽ là lag theo thời gian, không theo số lượng.

Q: Một thông điệp bị xử lý hai lần. Vì sao, và làm sao cho an toàn?

Vì Kafka commit offset sau khi xử lý, và tiến trình chết, hoặc bị đuổi vì chậm, giữa hai việc đó. Đó là ít-nhất-một-lần hoạt động đúng thiết kế. Làm cho an toàn nghĩa là mọi consumer hoặc có khóa lũy đăng tự nhiên — `findById` của `UserActivatedListener` — hoặc giành một id sự kiện duy nhất trong cùng transaction với hiệu ứng phụ: `INSERT ... ON CONFLICT DO NOTHING` vào `processed_events`, trả về nếu không dòng nào. Id phải tới từ producer, đúc một lần và mang qua các lần thử lại, chính là thứ dòng outbox cho bạn. Với hiệu ứng phụ không có transaction — e-mail — tôi đặt id vào Message-ID của thư hoặc giành nó trong Redis bằng `SET NX`. Điều tôi không làm là chuyển sang `AckMode.RECORD` và gọi là đã sửa; việc đó thu hẹp cửa sổ từ một lô xuống một record và để nguyên lỗi tại chỗ.

Q: Làm sao bảo đảm thứ tự cho một user?

Key mọi sự kiện về user đó theo id user, để phép băm đặt chúng vào một partition, và giữ một thread consumer cho mỗi partition. Đó là toàn bộ bảo đảm, và nó có ba cách vỡ: producer lũy đăng bị tắt với `max.in.flight` trên một, cho phép một lô thử lại rơi sau lô muộn hơn; đổi số partition, làm băm lại key; và topic thử lại, chèn lại một record thất bại phía sau những record kế tiếp nó. Với dự án, key `userId` trên `ts.user.*` đã làm việc này. Nếu nghiệp vụ cần phát biểu mạnh hơn — “cập nhật được áp dụng sau tạo, luôn luôn” — tôi thêm số phiên bản vào sự kiện và để consumer từ chối bất cứ thứ gì cũ hơn cái nó có, điều biến thứ tự từ thuộc tính của broker thành bất biến của consumer.

Q: Một broker chết. Producer làm gì?

Với ba broker và `replication.factor=3`, controller bầu leader mới từ các replica đồng bộ cho mỗi partition bị ảnh hưởng trong vài giây; producer nhận `NotLeaderOrFollower`, làm mới metadata, và thử lại — vô hình với ứng dụng vì retries không giới hạn và `delivery.timeout.ms` là hai phút. Nếu broker chết kéo ISR xuống dưới `min.insync.replicas`, producer nhận `NotEnoughReplicas` và tiếp tục thử lại; với `acks=all` đó là sự từ chối đúng dẫn việc ghi một bản sao duy nhất. Trên stack Compose một broker của dự án không có failover: các lần gửi được đệm, future thất bại sau hai phút, và dòng outbox là thứ cứu sự kiện. Điều duy nhất tôi kiểm tra khi review là có ai đọc future không — trong dự án này không ai, nên lỗi phía producer là âm thầm.

Q: Làm sao phát lại một topic từ hôm qua?

Dừng consumer group, reset offset của nó, khởi động lại: `kafka-consumer-groups.sh --reset-offsets --group course-group --topic ts.user.activated --to-datetime 2026-09-06T00:00:00.000 --execute`. Group phải không hoạt động, nếu không lệnh từ chối. Trước khi chạy tôi trả lời hai câu hỏi: consumer có lũy đẳng với mọi thứ kể từ mốc đó không — vì mọi record sẽ quay lại — và dữ liệu còn đó không, phụ thuộc vào retention (168h ở đây) và topic có compact không. Với một record đơn lẻ tôi phát lại từ DLT thay vì reset cả group. Và tôi không bao giờ phát lại vào một group gửi e-mail mà chưa xác nhận đường khứ trùng, vì “chúng ta gửi lại 40 000 mail kích hoạt” là sự cố theo sau một lần reset tự tin.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Nêu tên các thiết lập producer khiến producer của dự án lũy đẳng và bền vững, và giải thích vì sao `acks=all` trên stack Compose yếu hơn nghe có vẻ.
2. Vẽ (bằng lời) dòng thời gian trong đó `notification-group` gửi cùng một mail kích hoạt hai lần mà không có crash nào, và nêu tên thiết lập mà giá trị của nó định nghĩa điểm kích hoạt.
3. Vì sao insert vào `processed_events` và lần ghi nghiệp vụ phải chung một transaction? Mô tả sự xen kẽ đi sai nếu insert commit trước một mình.
4. Trong cấu hình `@RetryableTopic`, giải thích vì sao `include` chỉ liệt kê ngoại lệ mail và chuyện gì xảy ra với một `NullPointerException`.
5. `Course-service` khai báo bean `ProducerFactory` riêng. Nêu hệ quả cho khối `spring.kafka.producer` trong YAML của nó, và cách sửa một dòng.
6. `Notification-service` đã chết chín ngày. Nêu chuyện gì xảy ra khi khởi động lại, thiết lập nào quyết định, và điều gì khác đi nếu nó khởi động lại với `group-id: notification-v2`.

Đào sâu: File ở quy mô lớn — presigned URL, endpoint, và vì sao byte không thuộc về Postgres

Chương 9 giới thiệu MinIO như “S3 tại nhà” và giải thích lỗi hai endpoint. Chừng đó đủ để bài nộp hoạt động. Chương này hỏi thiết kế đó tốn gì khi file trở nên nhiều, lớn và quan trọng — khoảnh khắc bài luận đã chấm của một học sinh là bản sao duy nhất ở bất kỳ đâu. Code của dự án là mẫu vật: một `S3Config`, một `S3StorageService`, một cột tên `file_object_key`.

Ba câu hỏi sắp xếp chương này:

1. **Byte nên sống ở đâu** — trong cơ sở dữ liệu, trên đĩa, hay trong kho đối tượng — và mỗi lựa chọn tốn chính xác những gì?
2. **Ai nên chuyển byte** — trình duyệt đi qua service, hay trình duyệt đi thẳng tới kho với một giấy phép có chữ ký?
3. **Chuyện gì xảy ra khi hai kho bất đồng** — một dòng gọi tên một object không tồn tại, hay một object không dòng nào gọi tên?

35.1 Vì sao kho đối tượng, và nó giải quyết gì

Một bài nộp có hai nửa: các dữ kiện về nó (ai, khi nào, lần thứ mấy, điểm bao nhiêu) và chính file. Dữ kiện có tính quan hệ và bé xíu; file thì mờ đục và có thể nặng 20 MB. File có ba nơi để ở.

Trong Postgres, dưới dạng bytea. Một kho, một transaction, một bản backup. Cũng là nơi rắc rối bắt đầu. Mỗi lần đọc dòng kéo cả blob qua shared buffers; pg_dump lớn thêm bằng kích thước mọi bài luận từng nộp; mỗi lần cập nhật ghi lại toàn bộ giá trị và để lại một dead tuple cho autovacuum; write-ahead log chở mỗi byte hai lần (WAL, rồi page); replication đẩy nó tới mọi standby; và một lượt tải xuống phải hiện thân trọn vẹn trong JVM trước khi tới socket. Postgres làm được, và tới vài gigabyte không ai để ý. Quá mức đó, mọi tác vụ vận hành — backup, restore, độ trễ replication, vacuum — bị chi phối bởi dữ liệu mà cơ sở dữ liệu chẳng thêm giá trị nào.

Trên một volume mount vào service. Nhanh và đơn giản cho tới khi service có hai replica trên hai node, thứ đầu tiên Kubernetes muốn làm. Một claim

ReadWriteOnce (Chương 19) gắn với một node; ReadWriteMany cần NFS hoặc một storage driver, và giờ service tự sở hữu bố cục file, quyền, dọn dẹp và backup của riêng nó. Địa cũng nằm trong vùng lỗi của pod.

Trong kho đối tượng. Không gian tên phẳng, API HTTP, độ bền theo nội dung, và — quyết định — khả năng đưa cho client một URL rồi bước ra khỏi đường dữ liệu. Backup thành một quy tắc replication của bucket. Phục vụ file thành việc của CDN. Cơ sở dữ liệu chỉ giữ một key 500 ký tự.

QUYẾT ĐỊNH — MỘT KEY TRONG POSTGRES, BYTE TRONG MINIO

Dự án lưu metadata trong bảng submission và file dưới một key trong bucket ts-submissions. Dòng tham chiếu object bằng tên; không gì tham chiếu ngược từ object tới dòng. Các phương án khác thua ở trực quan trọng với một trường học: bài nộp được ghi một lần, đọc vài lần, và giữ nhiều năm — hồ sơ mà kho đối tượng sinh ra để phục vụ và cơ sở dữ liệu làm tệ nhất. Quyết định chỉ đảo ngược nếu file rất nhỏ (vài KB), cần nguyên tử với dòng trong cùng transaction, và được đọc mỗi lần truy cập dòng — một ảnh đại diện thu nhỏ, chẳng hạn. Ngay cả khi đó câu trả lời thật thà thường vẫn là “vẫn S3, cộng một cache”.

35.2 Đọc code của dự án

35.2.1 Hai client, hai endpoint

Cửa ngõ của mọi thứ là S3Config. Đọc nó với môi trường compose trong đầu (APP_S3_ENDPOINT=http://minio:9000, APP_S3_PUBLIC_ENDPOINT=http://localhost:9000):

```
@Bean
public S3Client s3Client() {
    return S3Client.builder()
        .endpointOverride(URI.create(endpoint))           ①
        .region(Region.of(region))
        .credentialsProvider(credentialsProvider())
        .serviceConfiguration(S3Configuration.builder()
            .pathStyleAccessEnabled(true).build())         ②
        .build();
}

@Bean
public S3Presigner s3Presigner() {
    return S3Presigner.builder()
        .endpointOverride(URI.create(publicEndpoint))     ③
        .region(Region.of(region))
```

```
.credentialsProvider(credentialsProvider())
.serviceConfiguration(S3Configuration.builder()
    .pathStyleAccessEnabled(true).build())
.build();
}
```

- ① Client chuyển byte nói chuyện với tên *nội bộ*. Trong Compose hay trong cụm, minio là một tên DNS trên mạng riêng (Chương 18); từ trình duyệt nó chẳng là gì.
- ② Địa chỉ kiểu path-style: bucket nằm trong đường dẫn URL, nên request là /ts-submissions/key trên host minio:9000. Mặc định của AWS là kiểu virtual-host, bucket là một subdomain DNS — host ts-submissions.s3.amazonaws.com, đường dẫn /key. MinIO cũng làm được, nhưng chỉ khi nó biết domain của chính mình (MINIO_DOMAIN) và có wildcard DNS cho domain đó; trên mạng gia đình cả hai đều không có, nên dạng path là dạng duy nhất phân giải được.
- ③ Presigner không bao giờ gửi request. Nó chỉ tính chữ ký — và phải tính cho host mà *trình duyệt* sẽ liên hệ. Nó là một bean riêng chỉ vì lý do đó. Mục 35.3 giải thích vì sao không thể đổi host sau khi ký.

35.2.2 Key, upload, download

S3StorageService đủ ngắn để đọc trọn; các dòng quan trọng là đây:

```
@PostConstruct
void ensureBucketExists() {
    try {
        s3Client.headBucket(HeadBucketRequest.builder()
            .bucket(bucket).build());
    } catch (NoSuchBucketException e) { ①
        s3Client.createBucket(CreateBucketRequest.builder()
            .bucket(bucket).build());
    }
}

public String upload(MultipartFile file, Long assignmentId,
    Long studentId) {
    String key = "submissions/%d/%d/%s-%s".formatted(
        assignmentId, studentId, UUID.randomUUID(), original); ②
    s3Client.putObject(PutObjectRequest.builder()
        .bucket(bucket).key(key)
        .contentType(file.getContentType())
        .contentLength(file.getSize()).build(),
        RequestBody.fromInputStream(file.getInputStream(),
            file.getSize())); ③
}
```

```

    return key;
}

public String presignedDownloadUrl(String key) {
    GetObjectPresignRequest presignRequest = GetObjectPresignRequest
        .builder()
        .signatureDuration(Duration.ofMinutes(presignedExpiryMinutes))
        .getObjectRequest(GetObjectRequest.builder()
            .bucket(bucket).key(key).build())
        .build();
    return s3Presigner.presignGetObject(presignRequest).url().toString();
}

```

- ① Chỉ `NoSuchBucketException` được bắt. Một `connection refused` — MinIO chưa lên — là `SdkClientException`, thoát ra ngoài, làm hỏng việc tạo bean, và kéo cả service xuống. Manifest Kubernetes của MinIO nói vậy ngay ở comment đầu: `course-service` sẽ `CrashLoopBackOff` cho tới khi MinIO trả lời. Compose che điều này bằng `depends_on: service_healthy`.
- ② Key là toàn bộ mô hình dữ liệu của một kho đối tượng. S3 không có thư mục; dấu gạch chéo chỉ là ký tự trong chuỗi. Nhưng *tiền tố* là thứ bạn liệt kê theo, nên `submissions/{assignmentId}/{studentId}/` nghĩa là “mọi lần nộp của học sinh này cho bài này” là một lời gọi `ListObjectsV2` với một tiền tố. UUID bảo đảm key mới cho mỗi lần nộp, nên không gì bị ghi đè và versioning không cần thiết cho an toàn. Tên file gốc được nối vào để dễ đọc trong console; cơ sở dữ liệu giữ riêng nó trong `file_name` vì key không phải chỗ tốt cho dấu cách, Unicode, hay ...
- ③ Byte chảy *xuyên qua* JVM này: hỗ trợ multipart của Spring đã parse chúng (đổ ra file tạm khi vượt một ngưỡng nhỏ), và giờ chúng được stream tới MinIO. Thread request bị chiếm suốt lượt truyền, hai lần.

Và đường nối với nửa quan hệ, trong `SubmissionService`:

```

@Transactional
public SubmissionResponse submit(...) {
    ...
    if (hasFile) {
        String key = s3StorageService.upload(file, assignmentId,
            student.getId());
        submission.setFileObjectKey(key);
        submission.setFileName(file.getOriginalFilename());
    }
    submission = submissionRepository.save(submission);
    return toResponse(submission);
}

```

- ① Một lời gọi HTTP tới hệ thống khác, bên trong một transaction cơ sở dữ liệu. Nó không, và không thể, là một phần của transaction đó.
- ② Nếu insert này thất bại — ràng buộc unique trên (assignment_id, student_id, attempt_number) thua một cuộc đua giữa hai tab, hay Postgres thoảng không với tới được — transaction rollback và object đã upload ở ① nằm lại trong bucket, mang một key không dòng nào sẽ ghi lại. Chương 11 gọi đây là bài toán dual-write khi hệ thống thứ hai là Kafka; vẫn bài toán đó với một hệ thống thứ hai khác.

35.3 Signature Version 4, và vì sao host được hàn chết vào

Một presigned URL là một request S3 bình thường với phần xác thực chuyển từ header sang chuỗi truy vấn. Link tải xuống của dự án trông thế này (đã ngắt dòng):

```
http://localhost:9000/ts-submissions/submissions/7/3/2f9c...-essay.pdf
?X-Amz-Algorithm=AWS4-HMAC-SHA256
&X-Amz-Credential=minioadmin/20260907/us-east-1/s3/aws4_request
&X-Amz-Date=20260907T091500Z
&X-Amz-Expires=900
&X-Amz-SignedHeaders=host
&X-Amz-Signature=3c1f...e9a0
```

Chữ ký được tính qua hai giai đoạn. Trước hết một *khóa ký* được dẫn xuất từ secret qua một chuỗi HMAC trên ngày, region, tên dịch vụ và chuỗi cố định aws4_request, nên bản thân secret không bao giờ chạm vào request. Rồi khóa đó ký một *string to sign* chứa SHA-256 của một *canonical request*: phương thức HTTP, URI chuẩn hóa (bucket và key, kiểu path), chuỗi truy vấn đã sắp xếp, các header được ký, và mã băm của payload. Nhìn X-Amz-SignedHeaders=host: header Host luôn được ký. Đổi host — minio thành localhost, hay localhost thành 127.0.0.1 — là canonical request đổi, mã băm đổi, chữ ký không khớp nữa, và MinIO trả lời 403 SignatureDoesNotMatch. Không có cách nào ký với tên nội bộ rồi viết lại link cho thế giới bên ngoài. Ai ký phải biết chính xác origin mà trình duyệt sẽ dùng.

THỰC HÀNH — CỐ TÌNH LÀM HỎNG MỘT CHỮ KÝ

Đăng nhập, nộp một bài có file, và chép downloadUrl mà API trả về. Nó hoạt động đúng như được phát:

```
$ curl -s -o /dev/null -w "%{http_code}\n" "$URL"
200
```


Giờ chỉ đổi host, sang một địa chỉ tới đúng container MinIO đó:

```
$ curl -s "${URL}/localhost/127.0.0.1" | head -c 160
<?xml version="1.0" encoding="UTF-8"?><Error>
<Code>SignatureDoesNotMatch</Code><Message>The request signature we
calculated does not match the signature you provided.
```

Cùng server, cùng object, cùng thông tin xác thực — và một header Host khác là đủ. Đó là toàn bộ lý do public-endpoint tồn tại.

CẠM BẦY — LINK TẢI XUỐNG CHẠY TRONG COMPOSE VÀ HỎNG TRONG CỤM

Triệu chứng: trên cụm, bấm vào file của một bài nộp mở ra `http://localhost:9000/...` và trình duyệt báo connection refused. Nguyên nhân: Deployment của course-service đặt `APP_S3_ENDPOINT` là `http://minio:9000` nhưng không bao giờ đặt `APP_S3_PUBLIC_ENDPOINT`, nên giá trị mặc định `http://localhost:9000` của config-server được ký vào mọi link — một địa chỉ từng nghĩa là “laptop của lập trình viên” và giờ nghĩa là “laptop của người dùng”. Sửa hai phần: mở cổng S3 của MinIO qua một quy tắc Ingress trên host riêng của nó (chữ ký path-style bao gồm cả đường dẫn, nên Ingress không được thêm tiền tố), và đặt public endpoint thành origin đó:

```
- { name: APP_S3_PUBLIC_ENDPOINT,
  value: "https://files.ts-server.tailbfe002.ts.net" }
```

Chừng nào chưa có hostname thứ hai, giải pháp tạm thật thà là stream lượt tải qua course-service (GetObject vào body của response) để trình duyệt không bao giờ cần với tới MinIO — chậm hơn, nhưng đúng.

35.4 Ai chuyển byte

Đường upload hôm nay là: trình duyệt → Traefik → api-gateway → course-service → MinIO. Mỗi chặng thấy mỗi byte. Service cường chế `max-file-size: 20MB` và `max-request-size: 25MB` trong cấu hình multipart, nên trường hợp xấu nhất có giới hạn; nhưng mỗi lượt upload giữ một thread Tomcat và một file tạm suốt thời gian của nó, gateway phản ứng (Chương 4) chuyển tiếp body từng khối nhưng vẫn tiêu event loop cho việc đó, và mọi giới hạn body hay idle timeout ở tầng ingress áp lên trước tất cả. Dưới hạn nộp — 200 học sinh upload lúc 23:59 — thread pool của service là nút thắt cho công việc nó không cần làm.

Phương án thay thế là mẫu Chương 9 nhắc tới cho tải xuống, áp dụng cho

tải lên: service ký một PUT cho một key nó chọn, trình duyệt upload thẳng tới MinIO, và chỉ khi đó service mới ghi dòng. Service tốn vài micro giây mỗi file và không bao giờ thấy byte. Cái nó từ bỏ: không kiểm tra được nội dung trước khi nó hạ cánh (quét virus, nhận diện loại) và phải biết, từ đâu đó, rằng upload thực sự hoàn tất. Mục 35.8 xây dựng điều này cho từ tế.

Quá khoảng 100 MB, một PUT đơn là công cụ sai trên mọi mạng: *multipart upload* chia object thành các phần ít nhất 5 MB, upload song song, tiếp tục sau sự cố bằng cách gửi lại một phần, và ráp chúng phía server trong một lời gọi cuối. S3TransferManager của SDK tự làm việc này trên một ngưỡng. Cái đuôi vận hành: upload bị bỏ dở để lại các phần vô hình vẫn bị tính tiền và đếm — mọi bucket nên mang một quy tắc lifecycle hủy multipart upload đang dở sau vài ngày.

35.5 Ưu và nhược, thẳng thắn

Điểm mạnh	Dự án trả giá ở đâu
Cơ sở dữ liệu giữ nhỏ; backup chỉ là metadata	Hai kho phải backup, và chỉ một trong số đó có transaction
API chuẩn: MinIO hôm nay, S3 ngày mai với hai biến môi trường	Hai endpoint phải giữ cho đúng, và cụm đặt sai một
Presigned download đưa service ra khỏi đường dữ liệu	Upload vẫn đi qua service, mỗi file một thread
Key cho “thư mục” miễn phí theo tiền tố	Thiết kế key là cánh cửa một chiều; đổi tên nghĩa là copy
Độ bền là việc của kho	Trên phần cứng này nó là một container và một claim 5 GB
Thông tin xác thực là hai chuỗi	Đó là thông tin <i>root</i> của MinIO, trong mọi pod course-service

Dòng cuối đáng nhấn mạnh. minioadmin tạo được bucket, xóa được bucket, và đọc được mọi object trong mọi bucket. Service cần PutObject, GetObject và HeadBucket trên một bucket. Quyền tối thiểu ở đây là một user MinIO với policy giới hạn vào `arn:aws:s3:::ts-submissions/*`; trên AWS là một IAM role gắn với ServiceAccount của pod (IRSA) để không tồn tại khóa tĩnh nào. Người phỏng vấn hỏi “service của bạn xác thực với S3 thế nào?” đang nghe xem bạn nói “một IAM role” hay “một key trong biến môi trường” — và, nếu là về sau, bạn có biết đó là câu trả lời yếu hơn không.

35.6 Chuyện gì gây

CẠM BẦY — SERVICE KHÔNG CHỊU KHỞI ĐỘNG KHI THIẾU BUCKET

Triệu chứng: course-service CrashLoopBackOff trên cụm mới; log kết thúc bằng Error creating bean with name 's3StorageService' với một Connection refused nằm sâu vài tầng nguyên nhân. Nguyên nhân: kiểm tra bucket trong @PostConstruct coi mọi thất bại ngoài “thiếu bucket” là chí mạng, mà pod MinIO vẫn đang kéo image. Back-off khởi động lại của kubelet cuối cùng rơi vào lúc MinIO sẵn sàng, nên nó tự lành — chậm. Sửa: bắt cả SdkClientException, log cảnh báo, và để readiness probe (Chương 17) giữ lưu lượng tránh xa cho tới lần headBucket thành công đầu tiên; một phụ thuộc lưu trữ nên làm suy giảm tính năng file, không phải cả service.

CẠM BẦY — OBJECT MỒ CÔI

Triệu chứng: bucket chứa nhiều object hơn số key khác null trong bảng submission; không ai để ý suốt một năm; claim 5 GB đầy. Nguyên nhân: dual write trong submit. Upload trước, insert sau: insert hỏng là object mồ côi. Insert trước, upload sau: upload hỏng để lại dòng trống vào khoảng không, và học sinh thấy “đã đính kèm file” với một link chết. Thử tự nào cũng không tự an toàn. Cách giải chuẩn là làm một phía lũy đẳng và phía kia đối chiếu được: insert dòng với trạng thái PENDING_FILE và key đã chọn, upload, rồi lật trạng thái sang SUBMITTED trong transaction thứ hai; một job hằng đêm xóa object dưới các key mà dòng vẫn PENDING_FILE sau một giờ, và liệt kê tiền tố để tìm key không dòng nào gọi tên. Mục 35.8 gấp điều này vào luồng mới, nơi nó gần như miễn phí.

35.7 Chuyện gì gây lúc 3 giờ sáng

Pod MinIO biến mất — node khởi động lại, claim chưa gắn lại. Đi qua từng endpoint:

- **Bài nộp dạng text và link** vẫn chạy: upload chỉ được gọi khi có file, và save chỉ chạm Postgres.
- **Bài nộp có file** thất bại với 500 sau ngân sách retry của SDK — vài giây giữ thread mỗi lượt. Dưới tải, các thread này tích tụ và bỏ đói cả các bài nộp text lẽ ra đã thành công. Đây là lập luận của Chương 17 cho một bulkhead: một executor riêng, có giới hạn, cho các lời gọi lưu trữ.

- **Liệt kê bài nộp** chạy và thậm chí trả về link tải xuống. Ký là thuần tính toán; không request nào được gửi. Link chỉ hỏng khi bấm, và đó là cách hỏng đúng — metadata là thật, byte tạm thời không sẵn có.
- **Khởi động lại course-service** trong lúc sự cố thì không lên được, vì lý do trong cạm bẫy ở trên. Một sự cố của kho file đã thành sự cố của chấm điểm, ghi danh, và mọi thứ khác service làm.

Giờ chiều ngược lại: Postgres chết và MinIO bình thường. Mọi thứ thất bại, kể cả các upload lẽ ra đã thành công ở tầng lưu trữ — và mỗi upload chạm tới ① trong submit rồi hỏng ở ② để lại một object mồ côi. Một cú chớp cơ sở dữ liệu lúc 3 giờ sáng sinh ra một đồng nhỏ object vô chủ, và đó là lý do cụ thể job đối chiếu tồn tại.

35.8 Nâng cấp giải pháp: presigned PUT

Bản nâng cấp đưa byte ra khỏi service và đóng cửa sổ mồ côi cùng lúc. Ba lời gọi thay cho một:

1. POST /assignments/{id}/submissions/upload-url — service kiểm tra học sinh được nộp, chọn key, ký một PUT, và insert một dòng PENDING_FILE.
2. Trình duyệt PUT file lên MinIO với URL đó.
3. POST /assignments/{id}/submissions kèm key — service xác nhận object tồn tại bằng một HEAD, rồi hoàn tất dòng.

Storage service có thêm hai phương thức. Cả hai đều trọn vẹn:

```
/** Sign a PUT for a key we chose; the browser must send exactly this
 * content type, because it is part of the signature. */
public String presignedUploadUrl(String key, String contentType,
                                long contentLength) {
    PutObjectRequest put = PutObjectRequest.builder()
        .bucket(bucket)
        .key(key)
        .contentType(contentType)
        .contentLength(contentLength)           ①
        .build();
    PutObjectPresignRequest req = PutObjectPresignRequest.builder()
        .signatureDuration(Duration.ofMinutes(10))
        .putObjectRequest(put)
        .build();
    return s3Presigner.presignPutObject(req).url().toString();
}

/** Confirm an object exists and report its size; empty if missing. */
public Optional<Long> objectSize(String key) {
```

```

try {
    HeadObjectResponse head = s3Client.headObject(
        HeadObjectRequest.builder().bucket(bucket).key(key).build());
    return Optional.of(head.contentLength());
} catch (NoSuchKeyException e) {
    return Optional.empty();
}
}

```

- ① Ký Content-Length và Content-Type biến chúng thành hợp đồng: trình duyệt gửi kích thước hay loại khác sẽ nhận 403. Đó là cách giới hạn 20 MB sống sót khi rời khỏi service.
- ② HEAD tốn một vòng đi-về và không byte nào. Đó là bằng chứng rõ nhất có thể rằng bước 2 đã xảy ra.

Submission service tách phương thức cũ làm hai. Tiền tố key được dựng từ dữ kiện phía server, nên client không thể nhận vợ upload của người khác:

```

public UploadTicket prepareUpload(Long assignmentId, Long studentUserId,
    String fileName, String contentType,
    long size) {
    Assignment assignment = requireOpenAssignment(assignmentId);
    Student student = requireEnrolled(assignment, studentUserId);
    if (size > MAX_FILE_BYTES) {
        throw new ApiException(413, "File exceeds 20 MB");
    }
    String key = "submissions/%d/%d/%s-%s".formatted(
        assignmentId, student.getId(), UUID.randomUUID(),
        safeName(fileName));
    Submission pending = Submission.builder()
        .assignment(assignment).student(student)
        .attemptNumber(nextAttempt(assignmentId, student.getId()))
        .fileObjectKey(key).fileName(fileName)
        .status("PENDING_FILE")
        .build();
    submissionRepository.save(pending);
    String url = s3StorageService.presignedUploadUrl(key, contentType,
        size);
    return new UploadTicket(pending.getId(), key, url);
}

public SubmissionResponse completeUpload(Long submissionId,
    Long studentUserId) {
    Submission s = findOrThrow(submissionId);
    if (!s.getStudent().getUserId().equals(studentUserId)
        || !"PENDING_FILE".equals(s.getStatus())) {
        throw new ApiException(409, "Nothing to complete");
    }
}

```

CHƯƠNG 35. ĐÀO SÂU: FILE Ở QUY MÔ LỚN — PRESIGNED URL, ENDPOINT, VÀ VÌ SAO BYTE KHÔNG THUỘC VỀ POSTGRES

```
}
long size = s3StorageService.objectSize(s.getFileObjectKey())
    .orElseThrow(() -> new ApiException(400,
        "Upload not found; request a new upload URL")); ③
if (size > MAX_FILE_BYTES) {
    throw new ApiException(413, "File exceeds 20 MB");
}
s.setStatus("SUBMITTED");
return toResponse(submissionRepository.save(s));
}
```

- ① `safeName` bỏ dấu phân cách đường dẫn và ký tự ngoài ASCII; tên hiển thị ở lại trong `file_name`.
- ② Dòng tồn tại *trước* byte. Crash ở bất kỳ đâu sau điểm này để lại một dòng `PENDING_FILE` mà job dọn dẹp tìm được, thay vì một object không ai tìm được.
- ③ Con đường duy nhất tới `SUBMITTED` là qua một `HEAD` thành công. Dòng không bao giờ nhận có một file không ở đó.

Job dọn dẹp giờ là một truy vấn, không phải một lượt quét bucket:

```
@Scheduled(cron = "0 15 * * *") // hourly, at :15
public void reapAbandonedUploads() {
    List<Submission> stale = submissionRepository
        .findByStatusAndSubmittedAtBefore("PENDING_FILE",
            LocalDateTime.now().minusHours(1));
    for (Submission s : stale) {
        s3StorageService.deleteQuietly(s.getFileObjectKey()); // idempotent
        submissionRepository.delete(s);
    }
}
```

MinIO phải cho phép `PUT` cross-origin từ trình duyệt. Quy tắc CORS của bucket được đặt một lần bằng MinIO client:

```
$ mc cors set local/ts-submissions cors.json
# cors.json: allow PUT from the frontend origin, expose ETag
{ "CORSRules": [ { "AllowedOrigins": ["http://localhost:3000"],
    "AllowedMethods": ["PUT", "GET"],
    "AllowedHeaders": ["Content-Type", "Content-Length"],
    "ExposeHeaders": ["ETag", "MaxAgeSeconds": 3600 } ] }
```

Và phía frontend là một fetch trơn; không instance axios, không gateway, không JWT — URL *chính là* thông tin xác thực:

```
const ticket = await api.post(
    `/assignments/${id}/submissions/upload-url`,
```

```

{ fileName: file.name, contentType: file.type, size: file.size });

const put = await fetch(ticket.url, {
  method: 'PUT',
  headers: { 'Content-Type': file.type }, // must match the signature
  body: file,
});
if (!put.ok) throw new Error(`upload failed: ${put.status}`);

await api.post(`/submissions/${ticket.submissionId}/complete`);

```

QUYẾT ĐỊNH — PRESIGNED PUT SO VỚI UPLOAD QUA PROXY

Đi qua proxy khi service phải thấy byte trước khi lưu — quét virus, mã hóa lại ảnh, nhận diện content-type — hoặc khi client không thể với tới endpoint lưu trữ. Presign khi file lớn hoặc nhiều và service chẳng thêm gì cho lượt truyền: thread pool, giới hạn body ở ingress, và chẳng mạng kép đều biến mất. Trường hợp của dự án là về thứ hai, với một lưu ý quyết định thứ tự công việc: presigned upload đòi một origin MinIO trình duyệt với tới được, thứ cụm chưa có. Cạm bẫy ở trên và bản nâng cấp này chung một điều kiện tiên quyết — một hostname công khai cho file — và nên được lên lịch cùng nhau.

35.9 Ngoài dự án

Trên AWS, cùng đoạn code chạy với S3 khi bỏ endpoint override và thay khóa tĩnh bằng IAM role; CloudFront signed URL đóng vai presigner với một CDN đứng trước, và S3 *POST policy* là người anh em cũ hơn, thân thiện với form trình duyệt, của presigned PUT, có sẵn điều kiện content-length-range. Google Cloud Storage và Azure Blob Storage có cùng ba ý tưởng dưới tên khác — signed URL, SAS token, resumable upload. MinIO production chạy dạng phân tán với erasure coding qua nhiều ổ đĩa và node; một container với một claim là hình hài để phát triển, và SDK không phân biệt được, đó chính là ý nghĩa của chuẩn. Phiên bản phỏng vấn của chương này là một câu hỏi — “người dùng upload một video 2 GB; dẫn tôi đi qua” — và câu trả lời senior nêu presigned hoặc multipart upload, hợp đồng kích thước và loại trong chữ ký, một bước xác nhận, một job dọn mớ côi, và một quy tắc lifecycle cho các phần dang dở, theo gantt đúng thứ tự đó.

35.10 Câu hỏi từ phỏng vấn thật

Q: Người dùng bắt đầu upload video 1 GB và service hết bộ nhớ. Sửa đi.

Trước hết xác định byte là vấn đề, không phải metadata: một heap dump giữa lúc upload cho thấy các buffer multipart hay một `ByteArrayOutputStream`, và giới hạn bộ nhớ của pod (Chương 17) là thứ giết nó. Sửa ngắn hạn là bảo đảm Spring đổ ra đĩa (`file-size-threshold` nhỏ, một volume tạm đủ chỗ) và stream tới kho với `Content-Length` biết trước, như `S3StorageService.upload` đã làm — đưa bạn tới hàng trăm MB mà heap không phình, nhưng vẫn một thread và một file tạm mỗi upload. Sửa thật là đưa service ra khỏi đường đi: `presigned PUT` cho mọi thứ dưới khoảng 100 MB, và multipart upload với `S3TransferManager` hoặc chia phần phía trình duyệt cho phần trên, kèm quy tắc lifecycle hủy upload dang dở sau bảy ngày. Khi đó bộ nhớ của service độc lập với kích thước file theo cấu trúc, và `max-file-size` trở thành một `Content-Length` đã ký thay vì một cấu hình servlet.

Q: Chúng ta cần phục vụ mười nghìn lượt tải xuống mỗi giây. Làm sao, mà không chạm vào service?

`Presigned GET` đã nghĩa là service không làm I/O nào mỗi lượt tải, chỉ một chữ ký — vài micro giây, không mạng. Mười nghìn mỗi giây vì thế là câu hỏi về kho và biên. Kho đối tượng mở rộng đọc theo chiều ngang bằng cách phân bổ tiền tố key; một CDN phía trước (CloudFront với `signed URL`, hay CDN nào kéo từ origin là bucket) phục vụ object lặp lại từ cache và hấp thụ fan-out. Giữ việc ký rẻ bằng cách không cache gì — chữ ký là phi trạng thái — và đặt hạn dùng khớp cửa sổ cache của CDN để một object đã cache không sống lâu hơn quyền của nó bao nhiêu. Cái bấy là ký trong mỗi lời gọi liệt kê: `toResponse` trong dự án ký cho mỗi dòng của mỗi danh sách, ổn với hạn 15 phút nhưng nghĩa là một danh sách nghìn bài nộp tính nghìn chuỗi HMAC. Ký lười, ở một endpoint “lấy link” riêng, khi lưu lượng biến mình cho nó.

Q: Object có trong bucket nhưng dòng thì thiếu, hoặc dòng có mà object không. Đối chiếu thế nào?

Nói trước rằng cả hai là kết quả tất yếu của dual write và cách sửa là thiết kế, không phải một script. Thiết kế: ghi dòng ở trạng thái `PENDING` với key trước khi upload, xác nhận bằng `HEAD` trước khi đánh dấu hoàn tất, và không bao giờ để dòng nhận một file nó chưa xác minh. Đối chiếu khi đó có hai nửa rẻ. Dòng kẹt ở `PENDING` lâu hơn hạn `presign` bị xóa cùng object của chúng — một truy vấn, không

phải quét. Object không dòng được tìm bằng cách liệt kê tiền tố và anti-join với bảng, việc bạn kham nổi hằng đêm vì tiền tố là theo bài tập; trên AWS một báo cáo S3 Inventory thay cho lượt liệt kê. Làm gì thì làm, hãy để lệnh xóa object lũy đẳng để job chạy lại được.

Q: Bảo vệ một presigned URL thế nào?

Bốn núm vặn, và câu trả lời senior gọi tên cả bốn. *Phạm vi*: ký một phương thức trên một key; URL GET không dùng để PUT được, và URL PUT không dùng cho key khác được vì URI chuẩn hóa được ký. *Hạn dùng*: phút, không phải ngày; dự án dùng 15 cho tải xuống và 10 là đủ cho tải lên — đủ dài cho kết nối chậm, đủ ngắn để một link rò rỉ trong log chat đã chết khi ai đó để ý. *Hợp đồng nội dung*: ký Content-Type và Content-Length khi upload để client không đổi được điều nó đã khai. *Danh tính người ký*: thông tin xác thực đằng sau chữ ký nên là một user quyền tối thiểu trên đúng bucket đó, để ngay cả người ký bị chiếm cũng không liệt kê hay xóa được. Và nói điều người ta hay quên: presigned URL là một bearer credential — không log ở đâu, chỉ trả về qua HTTPS, và không bao giờ đặt vào một redirect mà proxy có thể cache.

Q: Chuyển từ MinIO sang AWS S3 không gián đoạn.

Vì code nói API S3, cuộc di cư là một lần chuyển dữ liệu cộng một thay đổi cấu hình, và mẹo nằm ở thứ tự. Thiết lập bucket replication hoặc chạy `mc mirror --watch` từ MinIO sang bucket S3 cho tới khi độ trễ chỉ còn vài giây. Chuyển *ghi* trước bằng cách đổi `APP_S3_ENDPOINT` và thông tin xác thực (giờ là một IAM role) trên một rolling restart — object mới hạ cánh ở S3, mirroring tiếp tục chép bất kỳ thứ gì còn hạ cánh ở MinIO từ các pod chưa khởi động lại. Chuyển *public endpoint* sau cùng, sau một lượt mirror cuối, để link tải xuống chỉ bao giờ được ký cho host có object. Key không đổi, nên cơ sở dữ liệu không cần migration. Giữ MinIO chỉ-đọc một tuần làm phương án lùi, rồi ngừng. Thứ gãy là cách địa chỉ: kiểu virtual-host trở nên khả dụng, kiểu path vẫn chạy, và đoạn code nào ghép URL bằng tay thay vì hỏi SDK chính là nơi bạn sẽ tìm thấy lỗi.

Q: Chi phí lưu trữ tăng mỗi tháng. Làm gì?

Do theo tiền tố trước — bài tập nào, năm nào. Rồi dùng công cụ của chính kho thay vì logic ứng dụng: một quy tắc lifecycle chuyển object cũ hơn một học kỳ sang tầng ít truy cập, và cũ hơn chính sách lưu giữ sang lưu trữ lạnh hoặc xóa; một quy tắc hủy multipart upload dang dở, thứ vô hình mà vẫn tính tiền; và tắt versioning, hoặc có hạn, trừ khi nhu cầu tuân thủ nói khác. Trong ứng dụng,

ngừng lưu thứ tái tạo được (thumbnail, bản chuyển mã) và khử trùng lặp theo mã băm nội dung nếu học sinh upload cùng một mẫu. Ở dự án này con số nhỏ — 250 lượt upload 20 MB là đầy claim 5 GB — nhưng cùng bộ quy tắc đó là thứ giữ hóa đơn S3 phẳng ở quy mô gấp nghìn lần.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Hai bean, `S3Client` và `S3Presigner`, chung thông tin xác thực và region, khác nhau một dòng. Nêu dòng đó, và giải thích chuyện gì sai nếu presigner dùng endpoint nội bộ.
2. Một đồng nghiệp đề xuất giữ một endpoint và để gateway viết lại `minio:9000` thành host công khai trong JSON trả về. Dùng cấu trúc chữ ký Version 4, giải thích vì sao link đã viết lại trả về 403.
3. Trong `SubmissionService.submit`, upload diễn ra bên trong một phương thức `@Transactional`. Nêu transaction bao phủ và không bao phủ điều gì, và mô tả thứ để lại khi save thất bại.
4. Bố cục key là `submissions/{assignment}/{student}/{uuid}-{name}`. Nêu một thao tác bố cục này làm rẻ và một thao tác nó làm đắt, và nói vì sao UUID khiến versioning của bucket không cần thiết cho an toàn.
5. MinIO chết lúc 3 giờ sáng. Trong các việc này, việc nào vẫn thành công: nộp câu trả lời text, liệt kê bài nộp, mở link tải xuống, khởi động lại `course-service`? Giải thích mỗi việc trong một mệnh đề.
6. Trong luồng presigned PUT, kẻ tấn công lấy được một URL upload hợp lệ từ phiên của chính mình và cố gắn object đó vào bài nộp của học sinh khác. Chỉ ra hai kiểm tra chặn họ.

Đào sâu: Từ điển trên MongoDB — mô hình document, tìm kiếm regex và những đánh đổi thẳng thắn

Chương 8 lập luận cho từ điển là service document của dự án: một mục từ được đọc như một khối, ghi như một khối, và lồng nhau tự nhiên. Lập luận đó vững. Chương này thử nó theo cách một kỹ sư senior hoài nghi sẽ làm — đọc các truy vấn service thực sự chạy, hỏi mỗi truy vấn tốn gì ở mười nghìn từ mỗi người dùng, và thừa nhận nơi một cơ sở dữ liệu quan hệ cũng làm tốt không kém. Mục tiêu không phải đảo ngược quyết định mà để bạn bảo vệ được cả hai phía của nó trong phỏng vấn, với code ngay trước mặt.

Ba câu hỏi sắp xếp chương này:

1. **Mô hình document mua được gì ở đây**, và Postgres với jsonb sẽ tốn gì thay vào đó?
2. **Truy vấn tìm kiếm thực sự thực thi thế nào** — phần nào chạm index, phần nào quét, và trang được dựng ở đâu?
3. **Chuyện gì gây khi từ điển lớn lên** — và trong số đó, cái nào là lỗi của Mongo, cái nào là lỗi của code?

36.1 Vì sao MongoDB ở đây, và nó giải quyết gì

Mở WordDefinition. Nó là toàn bộ schema của service:

```
@Document(collection = "word_definitions")
public class WordDefinition {
    @Id private String id;
    private String word;
    private String createdByUserId;
    private String meaning;
    private String usage;
    private String notes;
    private List<String> examples;
    private List<String> tags;
    /** Image attachments stored inline as base64 data URLs. */
    private List<String> images;
    private Map<String, Object> extra;
```

CHƯƠNG 36. ĐÀO SÂU: TỪ ĐIỂN TRÊN MONGODB — MÔ HÌNH DOCUMENT, TÌM KIẾM REGEX VÀ NHỮNG ĐÁNH ĐỔI THẮNG THẮN

```
@CreatedDate private Instant createdAt;  
@LastModifiedDate private Instant updatedAt;  
}
```

- ① Mọi truy vấn trong service đều giới hạn theo trường này. Nó là hàng rào tenant và, như sẽ thấy, là thứ duy nhất index giúp được.
- ② Hai mảng nhúng trong document. Trong Postgres đây là hai bảng con hoặc hai cột text[]; ở đây chúng là một phần của mục từ và không tốn gì để lấy cùng.
- ③ Ảnh dưới dạng chuỗi base64 inline. Đây là “byte trong cơ sở dữ liệu” của Chương 35 khoác áo document, kèm một trần cứng: một document BSON không được vượt 16 MB. Một tá ảnh điện thoại và từ đó không lưu được nữa.
- ④ Một map tự do. Lối thoát hiểm của mô hình document: các trường code chưa biết tới. Cũng là lý do “không schema” cần kỷ luật (Chương 8).

Mô hình mua được gì: một lượt đọc trả về mọi thứ trang cần, không join, không N+1 qua các bảng; thêm trường là thay đổi code, không phải migration; và hình dạng trên đường truyền là hình dạng trên đĩa, khiến service gần như toàn là keo dán. Nó tốn gì: quan hệ giữa các mục từ (WordLink) không nằm trong document và phải duyệt bằng tay; mọi trường mảng không index được theo tiền tố cho tìm kiếm chuỗi con; và không gì ngăn một document lớn không giới hạn.

QUYẾT ĐỊNH — MONGODB, VÀ LẬP LUẬN CHO VIỆC LẼ RA CHỌN POSTGRES

Từ điển là một sổ tay theo người dùng với các trường lồng nhau, tùy chọn, tiến hóa — sân nhà của mô hình document — và không chia sẻ gì với các service quan hệ, nên cách ly không tốn gì. Đó là lý do tốt. Phản biện thật thà: Postgres có jsonb cho phần lồng nhau, text[] với index GIN cho tag, pg_trgm cho đúng kiểu tìm chuỗi con service này chạy, và CTE đệ quy cho đồ thị từ, tất cả trong một cơ sở dữ liệu đội đã vận hành hai lần. Câu trả lời senior cho “vì sao Mongo?” vì thế không phải “vì document”; mà là “vì dữ liệu có hình dạng document và chúng tôi chấp nhận một cơ sở dữ liệu thứ hai để service triển khai độc lập và để học polyglot persistence — và đây là cái giá”. Điều gì sẽ lật: yêu cầu join từ với khóa học hay người dùng, hoặc một đội vận hành không muốn chạy engine thứ hai.

36.2 Đọc đường tìm kiếm

Endpoint ai cũng dùng là tìm kiếm. Nó chạy qua ba mẫu code. Truy vấn trong repository:

```
@Query("{ 'createdByUserId': ?0, '$or': [ "
    + "{ 'word': { '$regex': ?1, '$options': 'i' } }, "
    + "{ 'tags': { '$regex': ?1, '$options': 'i' } } ] }")
List<WordDefinition> searchByWordOrTag(String userId, String regex);
```

Index đang tồn tại, từ MongoIndexConfig:

```
mongoTemplate.indexOps("word_definitions")
    .ensureIndex(new Index().on("createdByUserId", Sort.Direction.ASC));
```

Và phương thức service biến danh sách thành trang:

```
List<WordDefinitionResponse> responses = wordDefinitionRepository
    .searchByWordOrTag(userId, escapeRegex(query.trim())) ①
    .stream()
    .map(w -> toResponse(w, userId)) ②
    .sorted(Comparator.comparing(WordDefinitionResponse::word,
        String.CASE_INSENSITIVE_ORDER)) ③
    .toList();
int start = (int) pageable.getOffset();
int end = Math.min(start + pageable.getPageSize(), responses.size());
List<WordDefinitionResponse> pageContent = start < responses.size()
    ? responses.subList(start, end) : List.of(); ④
return new PageImpl<>(pageContent, pageable, responses.size());
```

- ① Chữ người dùng gõ được ghép vào một biểu thức chính quy. `escapeRegex` thêm gạch chéo ngược trước mọi ký tự đặc biệt trước, nên `a.*` khớp đúng ba ký tự đó và `(` không làm server sập vì lỗi parse. Bỏ qua bước này là có regex injection: nhẹ thì kết quả sai, nặng thì một pattern thảm họa ghim chết một lõi CPU.
- ② `toResponse` chạy thêm hai truy vấn mỗi từ — cha và con từ `word_links` — nên một lượt tìm khớp n từ phát ra $2n + 1$ vòng đi-về. Đây là bài toán $N+1$, tái sinh trong một cơ sở dữ liệu vốn được cho là tránh nó.
- ③ Sắp xếp diễn ra trong JVM, trên *mọi* kết quả khớp, không phải trên trang.
- ④ Phân trang cũng vậy. Mongo trả về mọi kết quả; service giữ hai mươi và vớt phần còn lại. Trang 50 tốn đúng bằng trang 1, cộng thêm việc sao chép.

36.2.1 Server thực thi nó thế nào

Hỏi chính Mongo. Với stack đang chạy:

THỰC HÀNH — ĐỌC KẾ HOẠCH THỰC THI

```
$ docker compose exec mongodb mongosh ts_dictionary --quiet --eval '
db.word_definitions.find({ createdByUserId: "1", $or: [
  { word: { $regex: "clou", $options: "i" } },
  { tags: { $regex: "clou", $options: "i" } } ] })
.explain("executionStats").executionStats'
```

Lược bớt, câu trả lời trông thế này:

```
{ nReturned: 3,
  totalKeysExamined: 412,
  totalDocsExamined: 412,
  executionStages: {
    stage: "FETCH",
    filter: { $or: [ { word: { $regex: "clou", $options: "i" } },
                    { tags: { $regex: "clou", $options: "i" } } ] },
    inputStage: {
      stage: "IXSCAN",
      indexName: "createdByUserId_1",
      indexBounds: { createdByUserId: [ "[\"1\"", \"1\"]" ] } } } }
```

Ba dòng kể trọn câu chuyện: IXSCAN trên createdByUserId thu hẹp về 412 document của người dùng này; FETCH rồi nạp từng document từ đĩa; filter chạy cả hai regex trên mỗi document. Tỷ số totalDocsExamined / nReturned là 137 — mỗi từ trả về, 137 từ được đọc rồi loại.

Vì sao regex không dùng được index: B-tree sắp xếp chuỗi theo các ký tự đầu. ^clou (có neo, phân biệt hoa thường) ánh xạ thành một khoảng key và Mongo sẽ dùng index trên word cho nó. clou không neo có thể xuất hiện bất kỳ đâu trong chuỗi, và i nghĩa là thứ tự byte của index thậm chí không khớp phép so sánh; không có khoảng nào để nhảy tới. Bộ lập kế hoạch của Mongo nói rõ điều này: một \$regex không phân biệt hoa thường hoặc không neo là quét toàn bộ phần mà các điều kiện khác để lại. Ở đây đó là lát của người dùng, và vì thế index createdByUserId đang làm toàn bộ việc hữu ích, còn index kép đề xuất về sau không thay đổi phần regex — nó thay đổi điều xảy ra *sau* đó.

36.2.2 Điều kiện trên mảng

tags là một mảng. Một điều kiện vô hướng trên trường mảng khớp nếu *bất kỳ*

phần tử nào khớp — không cần `$elemMatch` cho một điều kiện đơn. Đó là tiện lợi thật so với SQL, nơi cùng điều kiện là một `EXISTS` vào bảng con. Cũng vì thế regex chạy một lần cho mỗi tag mỗi document: một document có tám tag tốn tám lần đánh giá pattern bên cạnh một lần cho `word`.

36.3 Đồ thị

`WordLink` là một danh sách kề: một document mỗi cạnh với `parentWordId`, `childWordId`, `position`. Index ở cả hai đầu là đúng — “con của X” và “cha của X” mỗi cái là một lượt tra index. Chi phí nằm ở việc duyệt:

```
private WordGraphResponse buildGraphNode(String wordId, String userId,
                                         Set<String> visited) {
    if (visited.contains(wordId)) return null;
    visited.add(wordId);
    WordDefinition word = wordDefinitionRepository
        .findByIdAndCreatedByUserId(wordId, userId).orElse(null); ①
    if (word == null) return null;
    List<WordLink> childLinks = wordLinkRepository
        .findByParentWordIdOrderByPosition(wordId);                ②
    List<WordGraphResponse> children = childLinks.stream()
        .map(link -> buildGraphNode(link.getChildWordId(), userId,
                                     visited))
        .filter(Objects::nonNull).toList();
    return new WordGraphResponse(word.getId(), word.getWord(),
                                  word.getMeaning(), children);
}
```

- ① Một truy vấn mỗi node.
- ② Thêm một nữa mỗi node. Một đồ thị 2,000 từ tốn 4,000 vòng đi-về, tuần tự, trên thread request. Mỗi lượt dưới một mili giây trên cùng host; qua một chằng mạng là một trang nhiều giây. Nhúng `childIds` vào document cha sẽ biến nó thành hai truy vấn tổng cộng (find mọi từ của người dùng, dựng trong bộ nhớ) — với cái giá cập nhật hai document mỗi liên kết, mà không có transaction giữ chúng cùng nhau.

Mongo có một toán tử đồ thị, `$graphLookup`, duyệt `word_links` đệ quy phía server trong một aggregation. Nó trả về tập phẳng các node với tới được kèm trường độ sâu, và cây được dựng lại trong bộ nhớ. Tập `visited` của dự án làm bằng tay điều `$graphLookup` làm với `maxDepth`; khác biệt là $2n$ vòng đi-về so với một.

36.4 Transaction và nhất quán, thật thà

Mongo hỗ trợ transaction ACID nhiều document từ bản 4.0 — trên một replica set. Dự án chạy một mongod đơn (Compose lần StatefulSet), và một node standalone không thể bắt đầu transaction nào. Nên deleteWord là hai lượt ghi độc lập: xóa định nghĩa, rồi deleteByParentWordIdOrChildWordId. Crash giữa hai lượt để lại các cạnh trở vào một từ đã mất; việc duyệt chịu được (orElse(null), rồi lọc), là tư thế phòng thủ đúng, nhưng là một hợp đồng toàn vẹn dữ liệu ẩn. Cách sửa không tốn dòng code nào và một dòng vận hành là chạy replica set một node (--replSet rs0 cộng một lời gọi initiate): transaction trở nên khả dụng, và cả change stream bạn sẽ muốn cho mẫu outbox của Chương 11.

Write concern và read preference quan trọng ngay khi có node thứ hai. Mặc định w: majority nghĩa là một lượt ghi được xác nhận đã nằm trên đa số node; đọc từ secondary có thể trả về trạng thái cũ hơn. Với một sổ tay một người dùng, đọc-được-điều-mình-vừa-ghi trên primary là điều người dùng mong đợi, và là mặc định — nói vậy trong phỏng vấn, và rằng bạn sẽ xem lại nó trước khi thêm read replica.

36.5 Ưu và nhược, thắng thắn

Điểm mạnh	Dự án trả giá ở đâu
Một document, một lượt đọc, không join	Liên kết nằm riêng; đồ thị là $2n$ truy vấn
Mảng là hạng nhất (tags, examples)	Tìm chuỗi con trên chúng là quét từng phần tử
Không migration cho trường mới	extra và images lớn không kiểm soát tới 16 MB
Tìm kiếm regex chạy ngay không cần gì	Nó không dùng được index trừ khi có neo và phân biệt hoa thường
Phân trang chỉ cách một skip/limit	Đường tìm kiếm phân trang trong Java sau khi nạp tất cả
Cơ sở dữ liệu độc lập, triển khai độc lập	Không auth trên mongod trong Compose lần cụm

Dòng cuối không phải đặc tính của Mongo mà là một sự thật của dự án đáng biết trước khi ai đó hỏi: image chính thức chỉ bật xác thực khi đặt thông tin root, và cả hai manifest đều không đặt. Trong một mạng chỉ ClusterIP đó là rủi ro có cân nhắc, được ghi trong mongodb.yaml; nó là thứ đầu tiên phải đổi nếu service có

ngày chia cụm với bất kỳ thứ gì không đáng tin.

36.6 Chuyện gì gây

CẠM BẦY — TÌM KIẾM CHẬM DẦN THEO MỖI TỪ, RỒI HẾT GIỜ

Triệu chứng: độ trễ tìm kiếm tăng tuyến tính theo từ điển của người dùng; tới vài nghìn từ, timeout của frontend nổ và trang không hiện gì. Nguyên nhân: ba hệ số nhân chồng lên cùng một request — regex quét mọi document trong lát của người dùng; toResponse rồi phát hai truy vấn mỗi kết quả; và sắp xếp cùng phân trang diễn ra sau cả hai. Cơ sở dữ liệu đang làm đúng điều được yêu cầu. Sửa theo thứ tự hiệu quả: phân trang trong cơ sở dữ liệu (bên dưới), gộp các lượt tra liên kết thành một truy vấn \$in mỗi trang, và neo hoặc thay thế regex.

CẠM BẦY — DOCUMENT QUÁ LỚN

Triệu chứng: lưu một từ thất bại với BsonMaximumSizeExceededException: ... exceeds 16793600 bytes. Nguyên nhân: images giữ các data URL base64, và base64 phình thêm một phần ba; năm ảnh 3 MB là document vượt giới hạn. Sửa: lưu ảnh trong MinIO dưới tiền tố dictionary/{userId}/{wordId}/ và giữ key trong document — quy tắc của Chương 35, áp dụng ở đây. Trong lúc chờ, giới hạn mảng bằng validation ở WordDefinitionRequest.

CẠM BẦY — REGEX KHÔNG BAO GIỜ TRẢ VỀ

Triệu chứng: một request ghim một lỗi ở 100 % và các truy vấn còn lại của service xếp hàng sau nó. Nguyên nhân: một pattern như (a+)+\$ trên chuỗi dài — backtracking thảm họa. escapeRegex ngăn người dùng gửi một cái như vậy; rủi ro quay lại vào ngày ai đó thêm “tìm kiếm nâng cao” truyền pattern thô, hoặc quên escape ở một endpoint mới. Phòng thủ nhiều lớp: một maxTimeMS trên truy vấn (@Meta(maxExecutionTimeMs) của Spring Data) để pattern chạy hoang bị giết phía server sau một giây.

36.7 Chuyện gì gây lúc 3 giờ sáng

Node của StatefulSet MongoDB khởi động lại; pod ở trạng thái pending bốn phút trong khi claim gắn lại.

- **dictionary-service** trả về 500 trên mọi endpoint. Readiness probe của nó (Chương 17) không có kiểm tra cơ sở dữ liệu, nên gateway vẫn định tuyến tới nó. Người dùng thấy lỗi thay vì một trang “dịch vụ không sẵn sàng”.
- **Mọi thứ khác** không bị chậm. Không service nào khác giữ kết nối Mongo, không service nào gọi dictionary-service, và không topic Kafka nào phụ thuộc vào nó. Đây là mỗi-service-một-cơ-sở-dữ-liệu hoạt động đúng thiết kế: bán kính thiệt hại của sự cố là đúng một tính năng.
- **Khi pod trở lại**, connection pool của driver tự kết nối lại; không cần khởi động lại và không có cache phải làm ấm. Create-on-write của Mongo nghĩa là một claim *sai* sẽ lên với dữ liệu rỗng và im lặng (cạm bẫy của Chương 8) — kiểm tra `show dbs` trước khi tin một báo cáo “mất dữ liệu”.

Cái 3 giờ sáng còn lại thì chậm hơn: claim 5 GB đầy. Mongo không từ chối gọn một lượt ghi ở thời điểm đó; nó khựng lại, journal thất bại, và process có thể thoát. Vì document có thể lớn lên (ảnh, extra), không có trần nào trong schema ngăn được. Cảnh báo thuộc về mức dùng của PVC, không phải ứng dụng.

36.8 Nâng cấp giải pháp

Bốn thay đổi, mỗi cái trọn vẹn, theo thứ tự đem lại hiệu quả.

36.8.1 Phân trang trong cơ sở dữ liệu

Thay truy vấn trả về danh sách bằng truy vấn trả về trang. Spring Data Mongo cần một truy vấn đếm riêng cho trang từ `@Query`:

```
@Query(value = "{ 'createdByUserId': ?0, '$or': [ "
    + "{ 'word': { '$regex': ?1, '$options': 'i' } }, "
    + "{ 'tags': { '$regex': ?1, '$options': 'i' } } ] }",
    count = true)
long countByWordOrTag(String userId, String regex);

@Query("{ 'createdByUserId': ?0, '$or': [ "
    + "{ 'word': { '$regex': ?1, '$options': 'i' } }, "
    + "{ 'tags': { '$regex': ?1, '$options': 'i' } } ] }")
Page<WordDefinition> searchByWordOrTag(String userId, String regex,
    Pageable pageable); ①
```

```
public Page<WordDefinitionResponse> searchWords(String userId,
    String query, Pageable pageable) {
    Pageable sorted = PageRequest.of(pageable.getPageNumber(),
        pageable.getPageSize(),
```

```

        Sort.by("word").ascending());
    Page<WordDefinition> page = (query == null || query.isBlank()
        ? wordDefinitionRepository.findByCreatedByUserId(userId, sorted)
        : wordDefinitionRepository.searchByWordOrTag(userId,
            escapeRegex(query.trim()), sorted);
    return toResponses(page, userId);
}

```

- ① Spring Data nổi skip, limit và sort vào truy vấn. Regex vẫn quét lát của người dùng, nhưng chỉ hai mươi document được nạp, tuần tự hóa và ánh xạ.
- ② Sắp xếp chuyển sang server. Thứ tự không phân biệt hoa thường cần một collation trên truy vấn hoặc collection (`{locale: "en", strength: 2}`); thêm nó vào index bên dưới để sort dùng được.
- ③ Các lượt tra liên kết thành một lô mỗi trang, tiếp theo.

36.8.2 Gộp lô các liên kết

```

private Page<WordDefinitionResponse> toResponses(Page<WordDefinition> page,
    String userId) {
    List<String> ids = page.map(WordDefinition::getId).toList();
    Map<String, List<String>> parents = new HashMap<>();
    Map<String, List<String>> children = new HashMap<>();
    for (WordLink l : wordLinkRepository.findByChildWordIdIn(ids)) { ①
        parents.computeIfAbsent(l.getChildWordId(), k -> new ArrayList<>())
            .add(l.getParentWordId());
    }
    for (WordLink l : wordLinkRepository
        .findByParentWordIdInOrderByPosition(ids)) {
        children.computeIfAbsent(l.getParentWordId(), k -> new ArrayList<>())
            .add(l.getChildWordId());
    }
    return page.map(w -> toResponse(w,
        parents.getDefault(w.getId(), List.of()),
        children.getDefault(w.getId(), List.of())));
}

```

- ① Hai truy vấn \$in mỗi trang thay vì hai mỗi từ: $2n + 1$ thành 3. Cả hai dùng index sẵn có trên `childWordId` và `parentWordId`.

36.8.3 Index cho sort, và tìm kiếm có neo

```

// MongoIndexConfig
mongoTemplate.indexOps("word_definitions").ensureIndex(

```

```
new CompoundIndexDefinition(  
    new Document("createdByUserId", 1).append("word", 1))  
    .collation(Collation.of("en").strength(2))); ①
```

- ① Với index này, “từ của người dùng này theo thứ tự chữ cái” là một lượt đi dọc index, nên danh sách không lọc và mọi trang đã sort tránh được sort trong bộ nhớ. Và tìm theo *tiền tố* — {word: {\$regex: "^clou"}} với cùng collation — thành một lượt seek khoảng trên cùng index. Với từ điển, tiền tố thường là điều người dùng muốn khi gõ.

36.8.4 Text index cho tìm-từ-ở-bất-kỳ-đâu

Khi thực sự cần chuỗi con ở bất kỳ đâu, thôi dùng regex:

```
// MongoIndexConfig: one text index per collection, weighted  
mongoTemplate.indexOps("word_definitions").ensureIndex(  
    new TextIndexDefinition.TextIndexDefinitionBuilder()  
        .onField("word", 10F)  
        .onField("tags", 5F)  
        .onField("meaning", 1F)  
        .build()); ①
```

```
// DictionaryService  
public Page<WordDefinitionResponse> textSearch(String userId,  
    String terms, Pageable pageable) {  
    TextCriteria text = TextCriteria.forDefaultLanguage()  
        .matchingAny(terms.split("\\s+")); ②  
    Query q = TextQuery.queryText(text).sortByScore()  
        .addCriteria(Criteria.where("createdByUserId").is(userId))  
        .with(pageable);  
    List<WordDefinition> hits = mongoTemplate.find(q, WordDefinition.class);  
    long total = mongoTemplate.count(Query.of(q).skip(0).limit(0),  
        WordDefinition.class);  
    return toResponses(new PageImpl<>(hits, pageable, total), userId);  
}
```

- ① Text index tách token và rút gốc từ; cloud khớp clouds và clouded, và một kết quả trong word xếp trên cùng kết quả đó trong meaning. Nó không khớp *bên trong* một token (lou không tìm ra cloud) — đó là đánh đổi.
- ② Không regex, không escape, không backtracking. Index được hỏi trước; createdByUserId lọc các kết quả.

QUYẾT ĐỊNH — TIỀN TỔ CÓ NEO SO VỚI FULL TEXT

Regex neo tiền tổ trên index kép trả lời “từ bắt đầu bằng...” trong vài micro giây và không cần loại index mới; nó là mặc định đúng cho ô gõ-là-gợi-ý. Text index trả lời “mục từ về...” với xếp hạng và rút gốc, với cái giá là index lớn hơn và không khớp giữa từ. Regex không neo không phân biệt hoa thường — thứ dự án có — trả lời “bất kỳ thứ gì chứa...” và trả một lượt quét cho nó. Khuyến nghị là tiền tổ cho ô tìm kiếm, text search sau một nút “tìm mọi nơi” riêng, và regex về hưu. Atlas Search (Lucene dưới API của Mongo) cho cả hai cùng lúc, nhưng chỉ trên Atlas; trên một mongod tự host, hai công cụ này là tất cả.

36.9 Ngoài dự án

Mọi cơ sở dữ liệu document đều có hình dạng của chương này: DynamoDB với mẫu truy cập khóa-phân-vùng-trước và hoàn toàn không có tìm chuỗi con; Couchbase với N1QL và một index full-text; Elasticsearch là sidecar mà các kho document mọc ra khi tìm kiếm trở thành sản phẩm. jsonb cộng pg_trgm của Postgres là câu trả lời phần lớn đội nên cân nhắc trước bất kỳ thứ nào trong số đó — nó biến đúng `ILIKE '%clou%'` của service này thành một lượt seek index GIN. Cách đặt vấn đề senior cho “SQL hay NoSQL?” không bao giờ là engine; mà là các mẫu truy cập viết ra trước, rồi engine nào có index khớp với chúng.

36.10 Câu hỏi từ phòng vấn thật

Q: Tìm kiếm từ điển chậm. Bạn kiểm tra gì trước?

Kế hoạch thực thi, trước mọi thứ: `explain("executionStats")` trên đúng truy vấn service gửi, và hai con số từ nó — `totalDocsExamined` so với `nReturned`, và stage nào mang filter. Ở service này nó cho thấy một `IXSCAN` trên `createdByUserId` rồi một `FETCH` xem xét hàng trăm document mỗi kết quả, vì regex không neo không phân biệt hoa thường không dùng được index nào. Rồi nhìn điều xảy ra sau truy vấn: ở đây, hai lượt tra liên kết mỗi kết quả và sort cùng phân trang trong bộ nhớ. Tôi sẽ sửa theo thứ tự đó — phân trang trong cơ sở dữ liệu, gộp lô liên kết bằng `$in`, neo regex thành tiền tổ trên index kép `{createdByUserId, word}` với collation không phân biệt hoa thường — và đọc lại kế hoạch sau mỗi bước. Nếu “bất kỳ đâu trong từ” là yêu cầu thật, đó là một text index, không phải regex nhanh hơn.

Q: Khi nào bạn không chọn MongoDB?

Khi mẫu truy cập có tính quan hệ: dữ liệu đọc theo cách nhóm khác với cách ghi, các bất biến trải qua nhiều thực thể (một bài nộp cần bài tập và học sinh của nó tồn tại), hay báo cáo join qua cả miền. Khi đội đã vận hành Postgres tốt và “document” chỉ cách một cột jsonb. Khi transaction nhiều document là chuyện thường ngày chứ không phải ngoại lệ — Mongo có, nhưng trên replica set, với ngữ nghĩa retry bạn phải tự code. Và khi dữ liệu sẽ được tìm theo chuỗi con ở quy mô lớn: pg_trgm là một dòng index; Mongo cần Atlas Search hoặc một engine ngoài. Từ điển của dự án qua được các bài kiểm tra đó vì nó theo người dùng, lồng nhau, và không join gì; miền khóa học trượt cả ba, và đó là lý do nó ở Postgres.

Q: Phân trang một trăm nghìn kết quả mà không nạp chúng.

Không bao giờ bằng skip ở độ sâu lớn — skip(90000) vẫn đi qua chín mươi nghìn mục index. Dùng phân trang *keyset*: sort trên một khóa có index và gần như duy nhất (word, phân định bằng _id) và hỏi {word: {\$gt: lastWord}} với limit(20); mỗi trang là một lượt seek index bất kể độ sâu. Trả về khóa cuối làm cursor mở thay vì số trang. Chỉ giữ truy vấn đếm nếu UI thực sự cần tổng, và cache nó, vì đếm một regex có lọc là lại chính lượt quét đó. Phân trang subList của dự án là ngược lại tất cả những điều này: nó nạp mọi thứ và trả về hai mươi.

Q: Một service dùng Mongo, một service dùng Postgres. Giữ dữ liệu của chúng nhất quán thế nào?

Bạn không làm được, theo nghĩa transaction — không có two-phase commit qua chúng và bạn cũng không nên muốn. Bạn giữ chúng nhất quán *cuối cùng* qua sự kiện (Chương 11): service sở hữu commit thay đổi của mình và công bố một sự thật, service kia tiêu thụ và cập nhật bản sao của riêng nó, và một bảng outbox ở phía sở hữu khiến “đã commit” và “đã công bố” là một. Nơi dự án cần điều này hôm nay là id người dùng: dictionary-service lưu createdByUserId là một chuỗi do auth-service phát, và không bao giờ cần join trên nó, nên không cần đồng bộ gì. Nếu có ngày cần tên hiển thị của người dùng, câu trả lời là một consumer UserRegistered ghi vào một collection users nhỏ — không phải một lời gọi Feign mỗi từ, và không phải một cơ sở dữ liệu dùng chung.

Q: Hình dạng document đã đổi. Migrate không gián đoạn.

Mở rộng, di chuyển, thu hẹp. Triển khai code đọc được cả hai hình dạng và ghi hình dạng mới (một alias @Field hay một converter xử lý việc đổi tên); chạy một job nền viết lại document cũ theo lô với updateMany trên một khoảng _id có giới

hạn, có tiết lưu, lũy đẳng, tiếp tục được; rồi, khi đếm hình dạng cũ về không, triển khai code chỉ đọc hình dạng mới. Mongo làm bước đầu dễ vì không gì cưỡng chế schema, và nguy hiểm cũng vì lý do đó — thêm một JSON Schema validator trên collection ở cuối để hình dạng cũ không quay lại được. Với trường images trong dự án này, migration còn là một lần chuyển dữ liệu: viết lại mỗi data URL thành một object MinIO và thay phần tử mảng bằng key của nó.

Q: Mô hình hóa đồ thị từ — nhúng hay tham chiếu, và duyệt tốn gì?

Tham chiếu, như dự án làm, khi cạnh có thuộc tính riêng (position) và cả hai chiều đều được truy vấn; nhúng một mảng childIds khi đồ thị nông, chủ yếu đọc, và cha-tới-con là chiều duy nhất quan trọng. Chi phí duyệt là phần thật thà: với tham chiếu và đệ quy ngây thơ đó là hai vòng đi-về mỗi node — 4,000 cho đồ thị 2,000 từ — chính là điều buildGraphNode làm. Cách sửa, theo thứ tự, là nạp mọi cạnh của người dùng trong một truy vấn và duyệt trong bộ nhớ; hoặc một \$graphLookup với maxDepth; hoặc nhúng. Chu trình cần một tập visited dù đường nào; dự án có sẵn. Và nói to đánh đổi: nhúng biến một lần tạo liên kết thành hai lượt ghi document mà không có transaction trên node standalone.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Từ kết quả explain, totalDocsExamined là 412 và nReturned là 3. Nếu stage đang loại bỏ, nói vì sao index createdByUserId không giúp được nó, và nêu một thay đổi duy nhất trên pattern cho phép index trên word phục vụ nó.
2. searchWords với page=50&size=20 trên người dùng có 3,000 từ: Mongo trả về bao nhiêu document, bao nhiêu truy vấn word_links chạy, và trang được dựng ở đâu?
3. deleteWord phát hai lượt ghi không có transaction. Giải thích vì sao transaction không khả dụng trên triển khai này, crash giữa hai lượt để lại gì, và dòng nào của buildGraphNode chịu được điều đó.
4. Một người dùng đánh sáu ảnh vào một từ và lưu thất bại. Nêu exception, phép tính tạo ra nó, và quy tắc lưu trữ từ Chương 35 sửa được nó.
5. Với text index trên word, tags và meaning, truy vấn “lou” không trả về gì dù “cloud” tồn tại. Giải thích, và nói loại index nào trả lời được.
6. MongoDB chết bốn phút lúc 3 giờ sáng. Liệt kê cái gì hỏng, cái gì không, và một thay đổi probe duy nhất biến 500 thành một “chưa sẵn sàng” sạch sẽ.

Đào sâu: Khả năng quan sát — trace xuyên Feign và Kafka, lấy mẫu, và những cảnh báo đáng giá

Chương 24 dạy bạn debug stack từ ngoài vào trong — describe trước logs — và Chương 27 gọi tên bộ ba tiếp theo: metric, trace, cảnh báo. Chương đào sâu này mở toang phần dây nối đã có sẵn trong mọi service và hỏi nó thực sự làm gì. Câu trả lời ít hơn cấu hình gợi ý: dependency tracing nằm trên mọi classpath, mẫu log in ra trace id, cụm lấy mẫu ở mức không, và endpoint Prometheus mà Chương 27 trông cậy thì chưa tồn tại. Đọc lý do vì sao là con đường nhanh nhất để hiểu truy vết phân tán hoạt động thế nào.

Ba câu hỏi sắp xếp chương này:

1. **Một trace bắt đầu ở đâu, và sống sót thế nào** qua các chặng trình duyệt → gateway → Feign → Kafka → consumer — và ở đâu trong dự án này nó lạng lẽ chết?
2. **Lấy mẫu tốn gì** ở 1.0, ở 0.0, và ở những con số hệ thống production thực sự dùng?
3. **Tín hiệu nào xứng đáng một cảnh báo**, và phải thêm gì trước khi bắt kỳ cảnh báo nào nổ được?

37.1 Vì sao chọn nó, và nó giải quyết gì

Log trả lời “tiền trình này đã nói gì?” Metric trả lời “hệ thống đang ra sao, tính gộp, theo thời gian?” Trace trả lời câu hỏi hai thứ kia không thể: “chuyện gì đã xảy ra với *đúng một request này* khi nó đi qua năm service và một message broker?” Trong monolith, một stack trace chính là một trace. Trong dự án này, một cú click “tạo bài tập” chạm tới gateway, course-service, auth-service (qua Feign), Kafka, và notification-service; không file log đơn lẻ nào thấy hết.

Dự án chọn Micrometer Tracing với cầu nối Brave, báo cáo về Zipkin. pom.xml của mỗi service mang cùng một cặp:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-brave</artifactId> ①
```



```

</dependency>
<dependency>
  <groupId>io.zipkin.reporter2</groupId>
  <artifactId>zipkin-reporter-brave</artifactId> ②
</dependency>

```

- ① Micrometer Tracing là *facade* mà Spring Boot 3 instrument lên; Brave (tracer của dự án Zipkin) là một trong hai hiện thực, cái còn lại là OpenTelemetry. Cầu nối là thứ biến các *observation* của Spring — Micrometer Observation API mà mọi starter Boot 3 phát ra — thành span.
- ② Reporter gửi các span đã hoàn tất tới Zipkin qua HTTP. Đổi đúng một artifact này lấy `opentelemetry-exporter-otlp` là cùng các span đó trở sang Tempo, Jaeger, hay một vendor.

QUYẾT ĐỊNH — BRAVE VÀ ZIPKIN, KHÔNG PHẢI OPENTELEMETRY

Brave cộng Zipkin là con đường ngắn nhất từ số không tới một màn hình trace hoạt động: một container, không collector, không thương lượng giao thức, và auto-configuration của Spring Boot biết cả hai rất rõ. Đó là lựa chọn đúng để học cơ chế và cho một homelab. Điều sẽ lật nó: một ngôn ngữ thứ hai trong stack (SDK OpenTelemetry phủ tất cả với một giao thức đường truyền), một backend vendor (Datadog, Honeycomb, Grafana Cloud đều nói OTLP bản địa), hoặc nhu cầu lấy mẫu theo đuôi (tail-based), thứ đòi một collector đứng giữa ứng dụng và kho. Việc chuyển đổi là cơ học — đổi hai dependency, đổi thuộc tính endpoint — vì code chưa bao giờ chạm trực tiếp vào Brave. Người phỏng vấn hỏi “Zipkin hay Jaeger hay Tempo?” và mong đợi đúng câu trả lời này: backend thay được, chuẩn instrumentation mới là quyết định tồn tại lâu.

37.2 Một trace đi qua dự án này như thế nào

37.2.1 Cấu hình, trong mọi service

Năm tài liệu service trong config-server đều kết thúc giống nhau:

```

management:
  tracing:
    sampling:
      probability: 1.0 ①
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans ②

```

```
endpoints:
  web:
    exposure:
      include: health,info,metrics,prometheus ③
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]" ④
```

- ① Mọi request đều được trace. Ổn trên laptop; xem Mục 37.3 để biết nó tồn tại ở nơi khác. Trong cụm, mọi Deployment ghi đè thành 0.0 với chú thích “Zipkin chưa có trong cụm”.
- ② Compose ghi đè host thành zipkin:9411; reporter POST các lô JSON tới đây.
- ③ Exposure là một *danh sách tên*. Nêu tên prometheus không tạo ra endpoint; endpoint chỉ tồn tại khi micrometer-registry-prometheus nằm trên classpath, và không pom.xml nào trong repository có nó. Hôm nay /actuator/prometheus trả 404 trên mọi service — kế hoạch ở Chương 27 giả định một endpoint vẫn còn phải thêm vào.
- ④ Mẫu log đọc hai khóa MDC mà câu hỏi tracing điền vào. Khi không có span đang hoạt động, giá trị dự phòng - in ra rỗng, và chính điều đó là dấu hiệu chẩn đoán: trace id rỗng trong một dòng log nghĩa là code chạy ngoài mọi ngữ cảnh được trace.

37.2.2 Từng chặng một

Theo dõi một POST /api/courses/7/assignments từ trình duyệt.

Trình duyệt → gateway. Trình duyệt không gửi header trace nào; axios không biết gì về tracing. Gateway là ứng dụng WebFlux và Spring Cloud Gateway được instrument qua Observation API, nên observation phía server của request đi vào *khởi tạo* một trace mới: một trace id ngẫu nhiên 128 bit và một span gốc. Khi gateway chuyển tiếp tới course-service, WebClient reactive nó dùng cũng được instrument, và propagator ghi ngữ cảnh vào request đi ra. Mặc định của Spring Boot 3 quan trọng ở đây: Brave được cấu hình để *phát* W3C Trace Context (traceparent: 00-<traceId>-<spanId>-01) và *đọc* cả W3C *lần* hai định dạng B3. Vậy dự án này nói W3C trên đường truyền và vẫn hiểu một bên gọi B3 cũ hơn.

Gateway → course-service. Course-service là servlet MVC. Observation phía server của nó đọc traceparent, tiếp tục cùng trace, và mở một span con. Mọi thứ log bên trong controller giờ mang trace id đó — mẫu ở ④ phía trên in nó ra.

course-service → auth-service qua Feign. AuthServiceClient là một client Spring Cloud OpenFeign. OpenFeign kèm sẵn một capability Micrometer, bật mặc định, bọc mỗi lời gọi trong một observation; propagator của Brave thêm

traceparent vào request Feign. Auth-service tiếp tục trace. Trong Zipkin, lời gọi Feign hiện ra như một span client trong course-service lồng dưới span controller, cùng một span server khớp trong auth-service. Nếu circuit breaker của Resilience4j ngắt lời gọi (Chương 6), không span client nào được tạo — fallback đã chạy, đường truyền không hề bị chạm, và trace hiện ra khoảng trống.

course-service → Kafka. Ở đây trace chết. Đọc producer của dự án:

```
@Bean
public ProducerFactory<String, Object> producerFactory() {
    Map<String, Object> configProps = new HashMap<>();
    configProps.put(BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
    configProps.put(KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
    configProps.put(VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
    return new DefaultKafkaProducerFactory<>(configProps);
}

@Bean
public KafkaTemplate<String, Object> kafkaTemplate() {
    return new KafkaTemplate<>(producerFactory()); ①
}
```

- ① Một `KafkaTemplate` dựng tay. Spring Kafka *có thể* lan truyền ngữ cảnh trace — nó ghi traceparent vào header của record và mở một span producer — nhưng chỉ khi `setObservationEnabled(true)` được gọi trên template, hoặc `spring.kafka.template.observation-enabled=true` được đặt cho bean tự cấu hình. Bean này không làm cả hai, và thuộc tính vắng mặt trong mọi tài liệu cấu hình. Record rời đi không có header. Phía tiêu thụ, `@KafkaListener` trong `notification-service` sẽ cần `spring.kafka.listener.observation-enabled=true` để đọc một header. Cả hai nửa đều tắt. E-mail sinh ra từ cú click, với Zipkin, là một sự kiện không liên quan.

`UserRegisteredPublisher` của `auth-service` dùng template tự cấu hình và có cùng khoảng trống vì cùng lý do: thứ thiếu là thuộc tính, không phải bean.

Ranh giới scheduler. `CourseLifecycleScheduler` chạy `@Scheduled` lúc 01:00. Không gì instrument một trigger cron; dòng log “Auto-archived 3 course(s)” của nó in ra với trace id và span id rỗng. Điều tương tự đúng với mọi phương thức `@Async` trừ khi executor được bọc bằng một decorator lan truyền ngữ cảnh. Trace là thuộc tính của một *request*; công việc không do request nào khởi đầu thì không có trace trừ khi bạn trao cho nó một cái.

37.2.3 Đối chiếu log

Với mẫu ở ④, một lệnh grep xuyên log của cả năm service theo một trace id ghép lại câu chuyện Zipkin về — hữu ích đúng lúc Zipkin chết hoặc bị lấy mẫu bỏ qua:

```
$ docker compose logs --no-color api-gateway course-service \
  auth-service | grep 6f2c1a9e4b3d8e10
api-gateway      | INFO [api-gateway,6f2c1a9e4b3d8e10,9a1b...] ...
course-service   | INFO [course-service,6f2c1a9e4b3d8e10,c04d...] ...
auth-service     | INFO [auth-service,6f2c1a9e4b3d8e10,e77f...] ...
```

Để ý thứ vắng mặt: không có dòng notification-service, vì không trace id nào vượt qua Kafka. Lệnh grep cho thấy lỗ hổng đúng như Zipkin sẽ cho thấy.

37.3 Lấy mẫu

probability: 1.0 nghĩa là mọi trace được ghi và mọi span được gửi. Chi phí tuyến tính theo lưu lượng: mỗi span là vài trăm byte JSON, được gom lô và POST bởi một thread nền, rồi được Zipkin đánh index và lưu. Ở lưu lượng của dự án, chẳng đáng gì. Ở 1.000 request mỗi giây với sáu span mỗi request, đó là 6.000 span mỗi giây, cỡ 2 MB/s dữ liệu vào và hàng chục gigabyte lưu trữ mỗi ngày — phần lớn ghi lại những request thành công trong 20 ms mà không ai sẽ nhìn tới.

Lấy mẫu *theo đầu* (head-based) — thứ probability cấu hình — quyết định tại span gốc có giữ trace hay không, và lan truyền quyết định trong cờ của traceparent để mọi service phía sau đồng ý. Nó rẻ và nhất quán, và nó mù: ở 0.01 nó giữ 1% trace chọn ngẫu nhiên, nên request duy nhất thất bại lúc 3 giờ sáng được giữ với xác suất 0.01. Lấy mẫu *theo đuôi* (tail-based) hoãn quyết định tới khi biết toàn bộ trace và giữ mọi trace có lỗi hoặc vượt ngân sách độ trễ, cộng một mẫu ngẫu nhiên phần còn lại. Nó cần một thành phần đếm tròn trace trước kho — một OpenTelemetry Collector — và đó là lý do nó ngoài tầm với của dây nối trực tiếp Brave-tới-Zipkin.

QUYẾT ĐỊNH — TỶ LỆ LẤY MẪU

Laptop và Compose: 1.0, vì bạn đang debug và lưu lượng chính là bạn. Cụm hôm nay: 0.0, thành thật về việc chưa có pod Zipkin — lấy mẫu ở mức không cũng tắt cả chi phí tạo span, không chỉ chi phí gửi. Cụm có Zipkin: bắt đầu ở 0.1; hạ xuống 0.01 khi kho lớn nhanh hơn tốc độ bạn đọc nó. Câu trả lời senior cho “tỷ lệ lấy mẫu bao nhiêu?” là “head-based 1–10% làm nền, tail-based cho lỗi và request chậm khi đã có collector” — và câu tiếp theo luôn là “mà trace lỗi mới là thứ bạn thực sự muốn, nên hãy tính ngân sách cho collector”.

37.4 Metric: cái gì có và cái gì chưa

/actuator/metrics của Actuator đã tồn tại trên mọi service và liệt kê vài trăm meter: bộ nhớ JVM và GC, request HTTP server gắn tag `uri`, `status` và `method`, metric client Kafka, trạng thái pool HikariCP, và trạng thái circuit breaker của Resilience4j trong course-service. Nó trả lời từng meter một, theo tên, qua HTTP — một công cụ debug, không phải công cụ giám sát. Giám sát nghĩa là một hệ thống kéo tất cả chúng mỗi 15 giây, lưu thành chuỗi thời gian, và đánh giá quy tắc. Hệ thống đó là Prometheus, và endpoint của nó đang thiếu (xem ③ ở trên).

Hai bộ từ vựng mang vào phòng vấn. **RED** dành cho service theo request: Rate, Errors, Duration — ba thứ mà timer `http.server.requests` đã cung cấp. **USE** dành cho tài nguyên: Utilization, Saturation, Errors — CPU, heap, pool Hikari, consumer lag của Kafka. Một dashboard mỗi service nên hiện RED ở trên và USE bên dưới; cảnh báo nên nổ trên một triệu chứng RED mà người dùng cảm nhận được, và các panel USE giải thích vì sao.

37.5 Ưu và nhược

Điểm mạnh	Điểm yếu, hôm nay
Không đổi code: mọi chặng được các starter instrument, và mẫu log chỉ một dòng.	Lan truyền qua Kafka tất ở cả producer lẫn consumer; con đường thú vị nhất của hệ thống là con đường vô hình.
Facade trung lập vendor: Brave và Zipkin đổi được sang OTLP mà không chạm mã nguồn.	Không registry Prometheus trên class-path nào; prometheus trong danh sách exposure chỉ là ước vọng.
transparent W3C trên đường truyền, chấp nhận B3 — tương tác được với mọi thứ hiện đại.	Chỉ lấy mẫu head-based; một lỗi ở mức 1% được giữ 1% số lần.
Đối chiếu log hoạt động cả khi không có Zipkin.	Zipkin chạy :latest không volume: trace bốc hơi mỗi lần compose down.

37.6 Cái gì gây

CẠM BẦY — TRACE KẾT THÚC Ở KAFKA

Triệu chứng: trong Zipkin, trace POST /assignments kết thúc ở span controller; thông báo theo sau là một trace một-span riêng biệt, hoặc không có. Nguyên nhân: cả `spring.kafka.template.observation-enabled` lẫn `spring.kafka.listener.observation-enabled` đều không được đặt, nên không header `traceparent` nào được ghi vào hay đọc từ record. Cách sửa: cả hai thuộc tính, trong tài liệu cấu hình của cả hai service (course-service và auth-service phát; notification-service tiêu thụ) — và template dựng tay trong `KafkaProducerConfig` phải gọi `setObservationEnabled(true)` vì thuộc tính chỉ cấu hình bean tự cấu hình. Listing ở Mục 37.8 làm cả ba việc.

CẠM BẦY — TAG CÓ SỐ LƯỢNG GIÁ TRỊ KHÔNG GIỚI HẠN

Triệu chứng: heap của một service leo dần suốt nhiều ngày và /actuator/metrics mất vài giây để trả lời; cuối cùng OOMKilled. Nguyên nhân: một tag có tập giá trị không bị chặn. Kinh điển là một handler ánh xạ /courses/{id} bị ai đó viết lại để dựng đường dẫn bằng tay — tag uri khi đó mang /courses/42, /courses/43, ..., mỗi cái một chuỗi thời gian, mãi mãi. Chuyện tương tự xảy ra khi span được gắn tag bằng user id hay e-mail. Cách sửa: gắn tag bằng *mẫu* (/courses/{id}), không bao giờ bằng giá trị; đặt định danh vào log annotation của span hay dòng log, những thứ không được đánh index theo giá trị. Tag uri mặc định của Spring là mẫu đường dẫn chính để tránh điều này.

CẠM BẦY — MỘT 401 KHÔNG CÓ TRACE

Triệu chứng: người dùng báo “tôi bị đăng xuất” và không có trace nào cho request của họ. Nguyên nhân: `JwtAuthenticationFilter` của gateway từ chối ở thứ tự -1, trước khi định tuyến; observation phía server vẫn ghi request, nhưng với lấy mẫu ở 0.0 trong cụm thì không gì được xuất đi đâu cả. Cách sửa không phải tracing: đây là câu hỏi *metric*. Đếm `http.server.requests` với `status=401` tại gateway và cảnh báo khi tăng vọt — một lần deploy sai JWT_SECRET hiện ra như một vách đá trên bộ đếm đó trong vòng 15 giây.

37.7 Chuyện gì gây lúc 3 giờ sáng

Zipkin chết lúc 3 giờ sáng; các service vẫn chạy.

- **Độ trễ request: không đổi.** Reporter là bất đồng bộ. Span hoàn tất đi vào một hàng đợi trong bộ nhớ; một thread nền rút nó ra theo lô qua HTTP. Thread request không bao giờ chờ Zipkin. Đây là tính chất cần nói đầu tiên trong phỏng vấn, vì nỗi sợ “tracing làm chậm service của tôi” là phản đối phổ biến.
- **Bộ nhớ: bị chặn.** Hàng đợi có dung lượng cố định (`queuedMaxSpans`, mặc định 10.000). Khi đầy, span mới bị *bỏ*, được đếm, và một cảnh báo được log — reporter không phình heap để giữ chúng. Bạn mất tầm nhìn, không mất service.
- **Log: vẫn đối chiếu được.** Trace id được sinh cục bộ; mẫu MDC vẫn in chúng. Lệnh grep ở mục đối chiếu hoạt động đúng như trước. Đây là lý lẽ cho việc luôn có mẫu log, bất kể backend trace.
- **Hồi phục: tự động.** Reporter thử lại mỗi lô ở lần rút kế tiếp; khi Zipkin trở lại, span mới chảy tiếp. Các span bị bỏ trong lúc sự cố mất luôn — không có replay.

Giờ theo chiều ngược lại: *lưu lượng* tăng vọt lúc 3 giờ sáng với lấy mẫu ở 1.0. Hàng đợi đầy, span bị bỏ, và chính dòng log cảnh báo trở thành tiếng ồn ở hàng trăm dòng mỗi giây. Đó là hình hài cụ thể của đánh đổi lấy mẫu — và lý do đặt tỷ lệ trước cơn tăng vọt, không phải trong lúc đó.

37.8 Nâng cấp giải pháp

Bốn bổ sung biến đây nối thành một hệ thống giám sát. Mỗi cái đều trọn vẹn; thêm theo thứ tự.

1. Endpoint Prometheus. Một dependency mỗi service (BOM Boot của parent quản lý phiên bản):

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Có nó trên classpath, danh sách exposure hiện có làm `/actuator/prometheus` thành thật. Thêm các công tắc tracing đang thiếu, và một tag chung để mọi chuỗi nói rõ nó đến từ service nào:

```
management:
```

CHƯƠNG 37. ĐÀO SÂU: KHẢ NĂNG QUAN SÁT — TRACE XUYÊN FEIGN VÀ KAFKA, LẤY MẪU, VÀ NHỮNG CẢNH BÁO ĐÁNG GIÁ

```
prometheus:
  metrics:
    export:
      enabled: true
  metrics:
    tags:
      application: ${spring.application.name} # every series, one tag
  tracing:
    sampling:
      probability: ${TRACING_SAMPLE:0.1} # env-overridable
spring:
  kafka:
    template:
      observation-enabled: true # producer span + traceparent header
    listener:
      observation-enabled: true # consumer span, continues the trace
```

Và template dựng tay trong course-service, thứ bỏ qua thuộc tính:

```
@Bean
public KafkaTemplate<String, Object> kafkaTemplate() {
    KafkaTemplate<String, Object> template =
        new KafkaTemplate<>(producerFactory());
    // Properties configure only the auto-configured template; a
    // hand-built one must opt in explicitly or traces end here.
    template.setObservationEnabled(true);
    return template;
}
```

2. Scrape trong cụm. Với kube-prometheus-stack đã cài (Chương 27), một ServiceMonitor bảo operator của nó scrape Service nào. Một object phủ cả năm, theo label:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: ts-services
  namespace: ts
  labels:
    release: kube-prometheus-stack # the selector the chart installs
spec:
  selector:
    matchLabels:
      monitoring: actuator # add this label to each Service
  endpoints:
    - port: http # the Service port *name*
      path: /actuator/prometheus
      interval: 15s
```


Service của auth-service và course-service phải cho phép /actuator/prometheus trong SecurityConfig của chúng để việc này chạy — cùng phép cho phép mà Chương 17 cần cho probe. Hạn chế nó trong mạng pod bằng NetworkPolicy thay vì để mở ra Ingress.

3. Log có cấu trúc. Grep xuyên container hoạt động trên laptop; trong cụm, Loki hay Elasticsearch parse JSON đáng tin hơn nhiều so với một mẫu ngoặc vuông. Bộ mã hóa JSON của Logback là một dependency cộng một logback-spring.xml:

```
<dependency>
  <groupId>net.logstash.logback</groupId>
  <artifactId>logstash-logback-encoder</artifactId>
  <version>8.0</version>
</dependency>
```

```
<configuration>
  <springProperty name="app" source="spring.application.name"/>
  <appender name="JSON"
    class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <customFields>{"application":"${app}"}</customFields>
      <!-- traceId and spanId come from the MDC automatically -->
      <includeMdcKeyName>traceId</includeMdcKeyName>
      <includeMdcKeyName>spanId</includeMdcKeyName>
    </encoder>
  </appender>
  <springProfile name="!local">
    <root level="INFO"><appender-ref ref="JSON"/></root>
  </springProfile>
  <springProfile name="local">
    <include resource="org/springframework/boot/logging/logback/base.xml"/>
  </springProfile>
</configuration>
```

Việc tách theo profile giữ mẫu đọc-được-bằng-mắt trên laptop và phát một object JSON mỗi dòng ở mọi nơi khác. Mỗi dòng giờ mang traceId như một trường; truy vấn log theo trace id là một lượt tra index, không phải quét.

4. Hai cảnh báo gắn với điều người dùng cảm nhận. Quy tắc Prometheus, trong kind PrometheusRule của operator:

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: ts-slo
  namespace: ts
```

CHƯƠNG 37. ĐÀO SÂU: KHẢ NĂNG QUAN SÁT — TRACE XUYỀN FEIGN VÀ KAFKA, LẤY MẪU, VÀ NHỮNG CẢNH BÁO ĐÁNG GIÁ

```
labels: { release: kube-prometheus-stack }
spec:
  groups:
    - name: ts.slo
      rules:
        - alert: GatewayErrorBudgetBurn
          expr: |
            sum(rate(http_server_requests_seconds_count{
              application="api-gateway", status=~"5.."}[5m]))
              /
            sum(rate(http_server_requests_seconds_count{
              application="api-gateway"}[5m])) > 0.02
          for: 10m
          labels: { severity: page }
          annotations:
            summary: "Gateway 5xx ratio above 2% for 10 minutes"
        - alert: NotificationConsumerLag
          expr: |
            sum by (topic) (
              kafka_consumer_fetch_manager_records_lag_max{
                application="notification-service"}) > 500
          for: 15m
          labels: { severity: ticket }
          annotations:
            summary: "notification-group is {{ $value }} records behind"
```

Cái đầu là cảnh báo *triệu chứng* trên tỷ lệ “errors” của RED tại nơi duy nhất mọi request đi qua; 2% trong mười phút là ngân sách khởi điểm, không phải luật. Cái thứ hai là câu trả lời của Chương 27 cho “metric Kafka nào trước”: lag tăng suốt 15 phút nghĩa là mail không đi ra, bất kể sức khỏe của pod nói gì. Để ý tên meter: Spring Kafka đăng ký records-lag-max của chính client theo cách đặt tên của Micrometer, và registry Prometheus hiển thị nó bằng gạch dưới.

THỰC HÀNH — CHỨNG MINH TRACE VƯỢT QUA KAFKA

Sau khi thêm hai thuộc tính observation-enabled và dòng template, khởi động lại course-service và notification-service dưới Compose, tạo một bài tập qua UI, rồi hỏi Zipkin về trace theo id in trong log của course-service:

```
$ curl -s "http://localhost:9411/api/v2/trace/6f2c1a9e4b3d8e10" \
  | jq -r '.[]' | "\(.localEndpoint.serviceName) \(.name)"
api-gateway          http post
course-service       http post /courses/{id}/assignments
course-service       http get                               # Feign to auth
auth-service         http get /users/{id}
course-service       ts.assignment.created send
notification-service ts.assignment.created process
```

Trước thay đổi, hai dòng cuối không tồn tại. Span consumer có cha là span producer; e-mail giờ quy được về cú click đã gây ra nó.

37.9 Ngoài dự án

Các stack production hội tụ về cùng một hình dạng với những cái tên khác nhau. Instrumentation: SDK OpenTelemetry hoặc Java agent, thứ tự instrument mà không cần cả starter. Thu thập: một OpenTelemetry Collector mỗi node hay mỗi cụm, nơi lấy mẫu tail-based, che dữ liệu, và phân phối tới nhiều backend cư trú. Lưu trữ: Tempo cho trace, Loki cho log, Prometheus hay Mimir cho metric — bộ ba Grafana — hoặc Jaeger, Elasticsearch và Thanos; hoặc một vendor nhận OTLP và tính tiền theo dung lượng. Mạch phỏng vấn chạy xuyên tất cả: “làm sao tìm một request chậm xuyên các service?” được trả lời bằng trace id trong log và một kho trace; “làm sao biết trước khi người dùng báo?” bằng cảnh báo RED ở biên; “tracing tốn gì?” bằng thảo luận lấy mẫu ở trên — và phiên bản senior của mọi câu trả lời gọi tên thành phần ở lại (instrumentation và trace id trong mọi dòng log) và những thành phần thay được (mọi thứ phía sau exporter).

37.10 Câu hỏi từ phỏng vấn thật

Q: Độ trễ p99 tăng gấp đôi đêm qua. Hãy dẫn tôi qua cách tìm nguyên nhân.

Bắt đầu từ metric, không phải log: histogram `http_server_requests` tại gateway, tách theo `uri`, cho bạn biết trong một truy vấn liệu *mọi* route đều chậm (hạ tầng — node, mạng, GC) hay chỉ một route (một nhánh code hoặc một downstream). Rồi lấy một trace chậm từ khung giờ đó — với lấy mẫu head-based ở 1% sẽ có vài trăm — và đọc thác span: span chậm hoặc là một span con (một lời gọi downstream, chẳng hạn chẳng Feign tới auth-service mất 800 ms thay vì 8) hoặc là khoảng trống giữa các span con (chính service đó, thường là một lời gọi cơ sở dữ liệu hay một lock, hiện ra như thời gian trong span cha không được span con nào phủ). Nếu là khoảng trống, trace id đưa bạn tới log của service đó đúng phút ấy; layout JSON làm việc đó thành một lượt tra index. Chi tiết senior là thứ tự — tổng hợp, rồi một mẫu đại diện, rồi log của nó — và gọi tên thứ bạn ước có: một liên kết exemplar từ bucket histogram thẳng tới một trace, thứ các stack gốc OpenTelemetry cung cấp còn Zipkin thì không.

Q: Bạn không thể trace 100%. Bạn lấy mẫu gì, và không bao giờ bỏ gì?

Lấy mẫu head-based 1–10% cho trạng thái ổn định, quyết định tại gateway và lan truyền trong cờ của traceparent để các service phía sau không bao giờ bất đồng. Không bao giờ bỏ lỗi và không bao giờ bỏ phần đuôi chậm — điều lấy mẫu head-based không thể hứa, vì span gốc được lấy mẫu trước khi ai biết request sẽ thất bại. Nên câu trả lời thật có hai tầng: một collector lấy mẫu tail-based giữ mọi trace có 5xx hoặc vượt ngân sách độ trễ, cộng một nền ngẫu nhiên 1% cho hình dáng lưu lượng bình thường. Nói thẳng về chi phí: lấy mẫu tail-based đếm trọn trace trong bộ nhớ collector suốt cửa sổ quyết định, thường 10–30 s, nên collector được định cỡ theo lưu lượng, không theo thứ bạn giữ. Trong dự án này đó là bước từ zipkin-reporter-brave sang một exporter OTLP cộng một collector; code ứng dụng không đổi.

Q: Một request biến mất khỏi trace ở Kafka. Vì sao, và sửa thế nào?

Vì ngưỡng cảnh trace chỉ vượt qua Kafka dưới dạng header của record, và không ghi chúng trừ khi template được bảo. Trong Spring Kafka hai thuộc tính làm việc đó, cả hai mặc định false trong Boot 3: phía producer, `template.observation-enabled`; phía consumer, `listener.observation-enabled` (cả hai dưới `spring.kafka`). Điểm tinh tế đáng nói to: thuộc tính chỉ cấu hình template tự cấu hình, nên một bean `KafkaTemplate` dựng tay — đúng thứ `KafkaProducerConfig` của `course-service` làm — cần `setObservationEnabled(true)` trong code. Sau khi sửa, span consumer có cha là span producer và toàn bộ đường e-mail treo dưới request HTTP gốc. Câu tiếp theo người phỏng vấn muốn: span consumer là *con*, nhưng nó có thể chạy vài giây hay vài giờ sau khi request kết thúc, nên màn hình trace sắp theo thời gian sẽ hiện một khoảng trống dài — khoảng trống đó là độ trễ hàng đợi, và bản thân nó là metric cần theo dõi.

Q: Bạn sẽ đánh thức người trực vì cái gì với stack này?

Triệu chứng người dùng cảm nhận, diễn đạt thành SLO, không bao giờ là tài nguyên thô. Hai cảnh báo gọi người: tỷ lệ 5xx của gateway vượt ngân sách lỗi suốt mười phút — điểm duy nhất mọi request đi qua — và độ trễ p99 tại gateway vượt mục tiêu trong cùng cửa sổ. Một cảnh báo mức ticket: consumer lag của notification tăng suốt mười lăm phút, vì đó là “mail không đi ra” ở dạng duy nhất hệ thống diễn đạt được. Mọi thứ khác — CPU, heap, pod khởi động lại, pool Hikari cạn — là panel dashboard giải thích một cảnh báo, không phải bản thân cảnh báo; cảnh báo CPU 80% đánh thức người vì một hệ thống đang hoạt động. Khung senior là ngân sách lỗi: ở mục tiêu sẵn sàng 99,9% bạn được đốt 43

phút lỗi mỗi tháng, và cảnh báo nổ trên *tốc độ đốt*, đó là lý do tỷ lệ được đo trong một cửa sổ chứ không trên một request thất bại đơn lẻ.

Q: Request của một người dùng chạm năm service. Làm sao tìm năm dòng log đó?

Bằng trace id, thứ mọi service đều in vì mẫu log có `%X{traceId}` và cầu nối tracing điền MDC ở mọi chặng. Trên laptop đó là grep xuyên log container; trong cụm là truy vấn trên một kho log đánh index trường đó, và đó là lý do log được gửi dưới dạng JSON thay vì parse bằng regex. Hai chi tiết tách một câu trả lời senior: thứ nhất, trace id phải được *trả về cho người dùng* — một header phản hồi `X-Trace-Id` từ gateway — để một ticket hỗ trợ trích dẫn được nó thay vì “khoảng 3 giờ chiều”; thứ hai, bất kỳ thứ gì chạy ngoài request (`@Scheduled`, `@Async`, một consumer Kafka không lan truyền) log ra id rỗng, và bạn nên biết những nhánh code nào của mình thuộc loại đó trước sự cố.

Q: Zipkin chết. Production có chậm đi không?

Không, và bạn nên nói được vì sao từ thiết kế của reporter: span được trao cho một hàng đợi trong bộ nhớ và một thread nền POST chúng theo lô, nên không thread request nào chờ Zipkin; hàng đợi bị chặn ở 10.000 span và bỏ khi tràn, nên bộ nhớ có tràn; các lô thất bại được log và thử lại ở lần rút kế tiếp, nên hồi phục là tự động. Thứ bạn mất là span trong cửa sổ sự cố — không có replay. Cái bẫy trong câu hỏi là trường hợp ngược lại: một exporter *đồng bộ*, hay một hàng đợi không giới hạn, sẽ biến sự cố giám sát thành sự cố ứng dụng, và đó là tính chất cần kiểm tra trước khi đưa vào bất kỳ agent hay exporter nào. Và cách giảm nhẹ cho các span bị mất cũng giống mọi thứ khác trong chương này: trace id vẫn nằm trong mọi dòng log, nên log mang sự đối chiếu ngay cả khi kho trace không thể.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Nêu chính xác thuộc tính hay lời gọi phương thức đang thiếu ở mỗi trong hai nửa phía Kafka của dự án, và giải thích vì sao chỉ đặt thuộc tính sẽ sửa được auth-service nhưng không sửa được course-service.
2. Một request tới gateway mang `X-B3-TraceId` từ một proxy upstream cũ. Dự án tiếp tục trace đó hay khởi tạo trace mới, và mặc định nào của Boot 3 quyết định điều đó? Header nào rời gateway hướng về course-service?
3. `/actuator/prometheus` nằm trong mọi danh sách exposure mà vẫn trả 404. Giải thích, và nêu tên đúng một artifact thay đổi câu trả lời.

4. Zipkin chết một giờ dưới lưu lượng bình thường. Cho biết chuyện gì xảy ra với độ trễ request, với heap, và với các span sinh ra trong giờ đó — và cái nào trong ba thứ sẽ đổi nếu reporter là đồng bộ.
5. Ở probability: 0.01, tỷ lệ request *thất bại* có trace là bao nhiêu, và thành phần kiến trúc nào cần có để đưa tỷ lệ đó lên 1.0 mà không trace mọi request thành công?
6. Một đồng nghiệp thêm `span.tag("email", user.getEmail())` vào span đăng nhập. Dự đoán sự cố trong những tuần tiếp theo và nêu quy tắc ngăn nó.

Đào sâu: Trình duyệt gặp gateway — nơi cất token, luồng refresh và CORS

Chương 26 đặt ứng dụng React lên cùng origin với API và ghi chú, gần như thoáng qua, rằng CORS “đơn giản là không bao giờ được hỏi tới”. Chương 28 đẩy thiết kế token từ phía server. Chương đào sâu này lấy đầu kia của đường dây: trình duyệt, runtime duy nhất trong hệ thống mà bạn không kiểm soát, nơi chạy code do bất kỳ ai chèn được thể script vào trang của bạn, và sẵn lòng gửi cookie tới form của kẻ tấn công. Mọi quyết định về nơi một token sống là quyết định về mối đe dọa nào trong số đó bạn chấp nhận.

Ba câu hỏi sắp xếp chương này:

1. **Token sống ở đâu** trong trình duyệt, và mỗi chỗ ở phơi nó ra trước đòn tấn công nào?
2. **Chuyện gì xảy ra khi hết hạn** — luồng refresh chạy thế nào, và vì sao mười tab phải sinh ra đúng một lời gọi refresh?
3. **CORS bảo vệ cái gì**, nó tồn tại mỗi request, và vì sao một origin làm nó biến mất?

38.1 Vì sao chọn nó, và nó giải quyết gì

Server phát hai thông tin xác thực (Chương 5): một JWT access token 15 phút và một refresh token mớ 7 ngày. Trình duyệt phải giữ cả hai qua các lần tải lại trang, đính cái thứ nhất vào mọi lời gọi API, và đổi cái thứ hai lấy cái thứ nhất mới khi hết hạn — mà người dùng không nhận ra. Câu trả lời của dự án là một store Zustand với middleware persist và một interceptor của axios. Đọc store trước:

```
export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      accessToken: null,
      refreshToken: null,
      user: null,
      isAuthenticated: false,
      setTokens: (accessToken, refreshToken) =>
        set({ accessToken, refreshToken, isAuthenticated: true }),
      setUser: (user) => set({ user }),
    })
  )
) ①
```

```
logout: () =>
  set({ accessToken: null, refreshToken: null,
    user: null, isAuthenticated: false }),
  {}),
  { name: 'auth-storage' }
)
);
```

- ① persist ghi toàn bộ state vào storage ở mỗi thay đổi và nạp lại khi tải trang. Đó là thứ sống sót qua một lần tải lại; không có nó, mỗi lần làm mới trang là một lần đăng xuất.
- ② Khóa lưu trữ. Backend mặc định là `localStorage`, nên mở DevTools, Application, Local Storage, và chúng ở đó: cả hai token, dạng chữ thường, dưới `auth-storage`. Bất kỳ script nào chạy trên trang đọc được chúng bằng một dòng.

QUYẾT ĐỊNH — LOCALSTORAGE, CHO PHIÊN BẢN ĐẦU

Dự án cắt cả hai token trong `localStorage` vì đó là thứ đơn giản nhất sống sót qua tải lại trang, hoạt động giống hệt trên `localhost:3000` và `ts.local`, không cần cấu hình cookie ở server, và né hắc CSRF — một token do JavaScript tự tay đánh vào không bao giờ được trình duyệt tự gửi. Cái giá là token chỉ an toàn đúng bằng mọi script trên trang. Điều đó chấp nhận được khi trang chỉ tải bundle của chính nó. Nó hết chấp nhận được vào ngày một script bên thứ ba (một thẻ analytics, một widget chat, một dependency npm bị chiếm) chạy trên cùng origin: bất kỳ cái nào cũng đọc được `auth-storage` và chuyển một refresh token 7 ngày đi nơi khác. Nâng cấp ở Mục 38.7 chuyển refresh token vào cookie `HttpOnly` và giữ access token trong bộ nhớ — nhiều bộ phận chuyển động hơn, nhưng script không còn trộm được thông tin xác thực sống dài.

38.2 Luồng refresh, như đã viết

38.2.1 Interceptor

Mọi lời gọi API đi qua một instance axios:

```
const apiClient = axios.create({ baseURL: '/api' }); ①

apiClient.interceptors.request.use((config) => {
  const token = useAuthStore.getState().accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
```



```

    return config;
  });

  apiClient.interceptors.response.use(
    (response) => response,
    async (error) => {
      const originalRequest = error.config;
      if (error.response?.status === 401
        && !originalRequest._retry) {
        originalRequest._retry = true;
        const refreshToken = useAuthStore.getState().refreshToken;
        if (refreshToken) {
          try {
            const { data } = await axios.post(
              '/api/auth/refresh', { refreshToken });
            useAuthStore.getState()
              .setTokens(data.accessToken, data.refreshToken);
            originalRequest.headers.Authorization =
              `Bearer ${data.accessToken}`;
            return apiClient(originalRequest);
          } catch {
            useAuthStore.getState().logout();
            window.location.href = '/login';
          }
        }
      }
      return Promise.reject(error);
    }
  );

```

- ① Base URL tương đối. Khi phát triển, proxy của Vite chuyển `/api` tới `localhost:8080`; trong cụm Traefik làm việc đó (Chương 26). Trình duyệt không bao giờ thấy origin thứ hai.
- ② Header bearer do code dính vào mọi request. Đây là sự miễn nhiễm CSRF nhắc ở trên: một form trên `evil.example` không thể khiến trình duyệt thêm header này.
- ③ Một 401 từ gateway kích hoạt một lần thử lại cho mỗi request; body JSON “Invalid or expired token” đến từ filter `JwtAuthenticationFilter` của gateway. Cờ được đánh dấu trên object config để 401 thứ hai rơi xuống bên gọi.
- ④ Lời gọi refresh cố ý dùng `axios` *trần*, không phải `apiClient`: nếu không, một 401 từ chính `/refresh` sẽ vào lại interceptor này và lặp.
- ⑤ Request gốc được phát lại với token mới. `await` của bên gọi hoàn tất như thể không có gì xảy ra.

Điều này đúng cho một request. Nó sai cho một trang.

38.2.2 Bài toán single-flight

DashboardPage mount và TanStack Query bắn các truy vấn song song: khóa học, bài tập, hồ sơ người dùng, số đếm từ điển. Nếu access token hết hạn khi laptop đóng nắp, tất cả trả 401 trong cùng vài mili giây, và mỗi cái chạy interceptor độc lập. Bốn request, bốn lời gọi tới /api/auth/refresh, bốn access token mới, và — vì setTokens chạy bốn lần — phản hồi nào tới sau cùng thì thắng. Hôm nay điều đó vô hại vì auth-service trả cùng một refresh token mỗi lần (Chương 28 đã chỉ ra đây là việc xoay vòng còn thiếu). Thêm xoay vòng, điều bạn nên làm, và cuộc đua thành chí mạng: lần refresh đầu vô hiệu token mà ba cái kia sắp đưa ra, chúng thất bại, và người dùng bị đăng xuất bởi chính cơ chế lẽ ra giữ họ lại.

Cách sửa là refresh *single-flight*: 401 đầu tiên khởi động một lần refresh và lưu promise của nó; mọi 401 đồng thời await cùng promise đó thay vì tự khởi động; khi nó hoàn tất, tất cả phát lại với cùng token mới. Mục 38.7 có code đầy đủ. Người phỏng vấn hỏi câu này dưới dạng “mười tab gặp 401 cùng lúc — bao nhiêu lời gọi refresh?” và câu trả lời senior là “mỗi tab một nếu không phối hợp, tổng cộng một nếu có khóa xuyên tab”, theo sau là gọi tên công cụ cho trường hợp xuyên tab: Web Locks API hoặc một BroadcastChannel, vì mỗi tab có heap JavaScript riêng và không thấy promise đang chờ của tab khác.

38.2.3 Đăng xuất và trang callback OAuth

Header.handleLogout POST refresh token tới /api/auth/logout, nơi xóa dòng của nó, rồi dọn store và điều hướng tới /login. Việc đó thu hồi thông tin xác thực 7 ngày ngay lập tức. Access token 15 phút không bị thu hồi — không thể, theo Chương 28 — nên một bản sao lấy trước khi đăng xuất vẫn hoạt động cho tới khi hết hạn. “Làm sao đăng xuất một người dùng ở mọi nơi?” có cùng câu trả lời với thêm một bước: xóa *tất cả* dòng refresh token của người dùng, không chỉ dòng trình duyệt này giữ, và chấp nhận cửa sổ 15 phút hoặc thêm deny-list theo jti.

Đăng nhập Google hạ cánh ở OAuthCallbackPage với cả hai token là tham số truy vấn — OAuth2LoginSuccessHandler dựng redirect theo cách đó. Trang đọc chúng, cất chúng, và điều hướng với replace: true để URL chứa token không nằm lại trong lịch sử. Chi tiết cuối quan trọng hơn về ngoài: một URL được chép vào lịch sử trình duyệt, vào header Referer của bất kỳ tài nguyên nào trang tải tiếp theo, và vào access log của server. replace rút ngắn thời gian phơi nhiễm; nó không xóa được khỏi access log của gateway và auth-service, nơi giờ chứa một refresh token hợp lệ dạng chữ thường.

38.3 CORS: chính sách bạn không đang dùng

Chính sách same-origin là quy tắc của trình duyệt rằng một script tải từ origin A không được *đọc* phản hồi từ origin B. CORS là giao thức để B nới lỏng quy tắc đó theo từng origin. Gateway của dự án có một CorsConfig liệt kê localhost:3000 và localhost:5173, cho phép mọi method và header, và đặt allowCredentials(true). Đọc nó đối chiếu với hai cách ứng dụng thực sự được phục vụ:

- **Phát triển.** Vite phục vụ trang ở :3000 và proxy /api tới :8080. Trình duyệt gửi request tới :3000; Vite chuyển tiếp phía server. Không tồn tại request cross-origin, nên không preflight, nên CorsConfig không bao giờ được dùng.
- **Cụm.** Traefik định tuyến /api tới gateway và mọi thứ khác tới nginx trên cùng host (Chương 26). Lại cùng origin.

CorsConfig vì thế phục vụ đúng một kịch bản: ai đó chạy bundle đã build từ một origin khác — chẳng hạn mở dist/index.html qua một static server ở :5173 không có proxy. Nó là lưới an toàn, và là lưới rộng rãi: addAllowedHeader("*") cộng credentials là chính sách rộng nhất Spring chấp nhận mà không ném lỗi.

CORS sẽ tồn tại nếu nó hoạt động là điều đáng biết cho phỏng vấn. Một request với body JSON và header Authorization không phải “request đơn giản”, nên trình duyệt gửi trước một preflight OPTIONS hỏi method và header có được phép không. Đó là thêm một vòng đi-về mỗi request khác nhau, có thể cache bằng Access-Control-Max-Age, thứ cấu hình của dự án không đặt — nên mọi lời gọi sẽ preflight. Trên đường truyền 40 ms, mọi lời gọi API thành 80 ms trước khi có việc gì được làm. Một origin loại bỏ hoàn toàn chi phí đó, và đó là lý lẽ hiệu năng cho thiết kế của Chương 26 bên cạnh lý lẽ bảo mật.

38.4 Ưu và nhược

	localStorage + bearer (hôm nay)	Cookie HttpOnly
XSS trộm token	có, cả hai token, một dòng JS	không — script không đọc được cookie
CSRF	miễn nhiệm: header do code đính	phơi nhiễm trừ khi có SameSite + token CSRF hoặc kiểm tra header tùy biến
Sống qua tải lại Nhiều tab	có, qua persist mỗi tab đọc storage; re- fresh đưa nhau	có, trình duyệt giữ nó cùng cookie ở mọi tab; re- fresh vẫn đưa
Chạy cross-origin	có	cần Allow-Credentials và danh sách origin chính xác
Thay đổi server	không	set-cookie khi đăng nhập, đọc cookie khi refresh

Tóm tắt thành thật mà một kỹ sư senior đưa ra: bearer-trong-storage đối một bài toán CSRF bạn giải được lấy một bài toán XSS bạn không giải được. Bất kỳ script bị chèn nào đánh bại nó hoàn toàn. Cookie đối theo chiều ngược lại, và bài toán của nó có cách sửa cơ học đã biết.

38.5 Cái gì gãy

CẠM BẦY — BỐN LẦN REFRESH, MỘT NGƯỜI THẮNG

Triệu chứng: sau khi để ứng dụng mở qua đêm, cú click đầu tiên chạy, rồi người dùng bị đưa tới /login dù tài khoản không có gì sai. Nguyên nhân: các 401 song song mỗi cái chạy interceptor; với xoay vòng refresh token ở server, lần refresh thứ hai đưa ra token mà lần đầu vừa vô hiệu, và nhánh catch đăng xuất người dùng. Cách sửa: interceptor single-flight ở Mục 38.7. Không có xoay vòng, triệu chứng chỉ là lời gọi lãng phí — đó là lý do cuộc đua vô hình hôm nay.

CẠM BẦY — TOKEN TRONG URL

Triệu chứng: một refresh token xuất hiện trong access log của gateway, trong log của Traefik, và trong lịch sử trình duyệt của một máy dùng chung.

Nguyên nhân: `OAuth2LoginSuccessHandler` truyền cả hai token làm tham số truy vấn của `redirect` tới `/oauth/callback`. Cách sửa: đặt token vào *fragment* của URL (`#accessToken=...`) — fragment không bao giờ được gửi tới server nào — hoặc tốt hơn, chỉ trả về một mã dùng một lần mà trang đổi lấy token bằng POST. Thiết kế cookie bên dưới loại refresh token khỏi URL hoàn toàn.

CẠM BẦY — MỘT 401 KHÔNG LIÊN QUAN TỚI TOKEN

Triệu chứng: mật khẩu sai ở trang đăng nhập làm khung ứng dụng lóe lên chốc lát, rồi quay về `/login`. Nguyên nhân: `auth-service` trả lời đăng nhập sai bằng 401; `interceptor` coi *mọi* 401 là “token hết hạn” và, nếu một refresh token cũ còn trong storage từ phiên trước, thử refresh, thất bại, rồi chạy `logout()` cộng chuyển hướng cứng. Cách sửa: miễn các endpoint `auth` khỏi đường refresh (`originalRequest.url` bắt đầu bằng `/auth/`), và dọn store khi trang đăng nhập mount.

38.6 Chuyện gì gây lúc 3 giờ sáng

Ba sự cố chỉ client thấy.

- **Laptop đóng nắp tám ngày.** Khi quay lại, access token đã chết từ lâu và refresh token hết hạn hôm qua. Request đầu nhận 401, refresh nhận 401 (`auth-service` xóa dòng hết hạn và ném lỗi), `interceptor` đăng xuất người dùng. Hành vi đúng — nhưng mọi truy vấn đang bay trên trang thất bại trước, và `TanStack Query` có thể thử lại mỗi cái ba lần trước khi chuyển hướng tới nơi. Đặt `retry: false` cho 401 trong tùy chọn mặc định của query client, nếu không người dùng nhìn vòng xoay mười giây trước khi được đưa tới đăng nhập.
- **Đồng hồ client lệch.** Dự án không bao giờ kiểm tra `exp` ở client, và thế là đúng: đồng hồ client không đáng tin và gateway quyết định. Một client “thông minh” chủ động refresh khi `exp` sắp tới sẽ, trên máy có đồng hồ nhanh 20 phút, refresh ở mọi request — hoặc, chậm 20 phút, không bao giờ refresh và vẫn nhận 401. Hãy để 401 của server là tín hiệu.
- **CORS cấu hình sai chỉ ở production.** Triệu chứng là lỗi console “No Access-Control-Allow-Origin header” trên một request trả 200 trong tab network. Trình duyệt nhận được phản hồi và từ chối trao nó cho script. Nó xảy ra khi trang và API ở hai origin khác nhau trong production (CDN cho bundle, gateway ở host khác) và danh sách cho phép của gateway vẫn ghi `localhost`.

Cách kiểm tra luôn giống nhau: so header request `Origin` với danh sách `allowedOrigins`, từng ký tự, cả scheme và cổng. Và để ý hình dạng của lỗi: nó không bao giờ hiện với server, nơi log ghi một request thành công.

38.7 Nâng cấp giải pháp

Thiết kế đích: access token chỉ sống trong bộ nhớ JavaScript (mất khi tải lại, khôi phục bằng một lần refresh âm thầm); refresh token sống trong cookie `HttpOnly` mà JavaScript không đọc được và trình duyệt chỉ gửi tới endpoint refresh.

1. Auth-service đặt cookie. Khi đăng nhập, xác minh 2FA, và refresh, controller trả access token trong body và refresh token trong cookie:

```
private ResponseCookie refreshCookie(String value, long maxAgeSec) {
    return ResponseCookie.from("refresh_token", value)
        .httpOnly(true)           // no document.cookie access
        .secure(true)             // HTTPS only; false on plain localhost
        .sameSite("Strict")       // never sent from another site
        .path("/api/auth/refresh") // only this endpoint sees it
        .maxAge(maxAgeSec)
        .build();
}

@PostMapping("/login")
public ResponseEntity<LoginResponse> login(
    @Valid @RequestBody LoginRequest request) {
    LoginResponse r = authService.login(request);
    if (r.refreshToken() == null) {           // 2FA or password change
        return ResponseEntity.ok(r);
    }
    return ResponseEntity.ok()
        .header(HttpHeaders.SET_COOKIE, refreshCookie(
            r.refreshToken(),
            jwtService.getRefreshTokenExpiryMs() / 1000).toString())
        .body(new LoginResponse(r.accessToken(), null,
            false, false, null)); // token not in the body
}

@PostMapping("/refresh")
public ResponseEntity<LoginResponse> refresh(
    @CookieValue(name = "refresh_token", required = false)
    String cookie) {
    if (cookie == null) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
    }
    LoginResponse r = authService.refreshToken(
        new TokenRefreshRequest(cookie));
}
```

```

    return ResponseEntity.ok()
        .header(HttpHeaders.SET_COOKIE, refreshCookie(
            r.refreshToken(),
            jwtService.getRefreshTokenExpiryMs() / 1000).toString())
        .body(new LoginResponse(r.accessToken(), null,
            false, false, null));
}

@PostMapping("/logout")
public ResponseEntity<Void> logout(
    @CookieValue(name = "refresh_token", required = false)
    String cookie) {
    if (cookie != null) authService.logout(cookie);
    return ResponseEntity.noContent()
        .header(HttpHeaders.SET_COOKIE,
            refreshCookie("", 0).toString()) // delete it
        .build();
}

```

path của cookie là thuộc tính mạnh nhất trong bốn: ngay cả khi một endpoint nào khác có lỗi hỏng, trình duyệt không bao giờ đính cookie này vào nó. Để ý path là góc nhìn của *gateway* (/api/auth/refresh), vì trình duyệt nói chuyện với gateway; gateway bỏ tiền tố sau đó. SameSite=Strict là phòng thủ CSRF: một form POST xuyên site không mang cookie chút nào. Yêu cầu body JSON trên /refresh là lớp thứ hai — một form HTML thuần không gửi được application/json mà không có preflight, thứ thiết kế same-origin sẽ không cấp cho site khác.

2. Client giữ access token trong bộ nhớ. Store bỏ persist và refresh token hoàn toàn:

```

export const useAuthStore = create<AuthState>()((set) => ({
    accessToken: null,           // memory only: gone on reload, by design
    user: null,
    isAuthenticated: false,
    setAccessToken: (accessToken) =>
        set({ accessToken, isAuthenticated: true }),
    setUser: (user) => set({ user }),
    logout: () =>
        set({ accessToken: null, user: null, isAuthenticated: false }),
}));

```

Khi tải trang, bộ nhớ trống; ứng dụng thực hiện một lần refresh âm thầm trước khi render các route được bảo vệ. Một lần tải lại vì thế tốn một vòng đi-về tới /api/auth/refresh, và nếu cookie không còn, người dùng ở /login — cùng hành vi như phiên hết hạn.

3. Refresh single-flight với phát lại. Interceptor giữ một promise đang chờ

và một hàng đợi request chờ nó:

```
let refreshing: Promise<string> | null = null; // module-level lock

async function refreshAccessToken(): Promise<string> {
  if (!refreshing) { // first 401 starts it
    refreshing = axios
      .post('/api/auth/refresh', null, { withCredentials: true })
      .then(({ data }) => {
        useAuthStore.getState().setAccessToken(data.accessToken);
        return data.accessToken as string;
      })
      .finally(() => { refreshing = null; }); // release the lock
  }
  return refreshing; // others await it
}

apiClient.interceptors.response.use(
  (response) => response,
  async (error) => {
    const original = error.config;
    const isAuthCall = original.url?.startsWith('/auth/');
    if (error.response?.status !== 401 || original._retry
        || isAuthCall) {
      return Promise.reject(error);
    }
    original._retry = true;
    try {
      const token = await refreshAccessToken(); // shared promise
      original.headers.Authorization = `Bearer ${token}`;
      return apiClient(original); // replay
    } catch {
      useAuthStore.getState().logout();
      window.location.href = '/login';
      return Promise.reject(error);
    }
  }
);
```

Mười 401 đồng thời giờ sinh ra một POST; chín cái kia await cùng promise và phát lại với token nó trả về. `withCredentials: true` là thứ khiến axios gửi cookie — và là thứ khiến request thành *credentialed* với CORS. Để phối hợp xuyên tab, bọc thân của `refreshAccessToken` trong `navigator.locks.request('refresh', ...)`: Web Locks API là toàn tiến trình, nên các tab thay phiên, và lần refresh của tab thứ hai thấy cookie đã được xoay bởi tab đầu và đơn giản là thành công.

4. Gateway, nếu có ngày xuất hiện origin thứ hai. Request *credentialed* cấm wildcard; origin phải được liệt kê chính xác và preflight được cache:


```

spring:
  cloud:
    gateway:
      globalcors:
        cors-configurations:
          '[/**]':
            allowed-origins:
              - https://app.teachersupporter.example # exact, with scheme
            allowed-methods: [GET, POST, PUT, PATCH, DELETE, OPTIONS]
            allowed-headers: [Authorization, Content-Type]
            allow-credentials: true # cookies cross-origin
            max-age: 3600 # cache the preflight

```

Với SameSite=Strict cookie vẫn không đi tới một site khác, nên một triển khai thực sự xuyên site cần SameSite=None; Secure và chấp nhận phơi nhiễm CSRF đi kèm — lý do thiết kế same-origin của Chương 26 là thứ nên giữ.

THỰC HÀNH — XEM COOKIE LÀM VIỆC CỦA NÓ

Sau khi triển khai phiên bản cookie, đăng nhập qua UI, rồi kiểm tra trình duyệt giữ gì và gửi gì:

```

$ curl -si -c jar.txt -X POST http://localhost:8080/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"teacher@ts.local","password":"secret"}' | grep -i set-cookie
Set-Cookie: refresh_token=3f9c...; Path=/api/auth/refresh; Max-Age=604800;
HttpOnly; SameSite=Strict
$ curl -s -b jar.txt -X POST http://localhost:8080/api/auth/refresh \
  | jq .accessToken
"eyJhbGciOiJIUzI1NiJ9..."
$ curl -s -b jar.txt -o /dev/null -w '%{http_code}\n' \
  http://localhost:8080/api/courses
401

```

Dòng cuối là thuộc tính path đang làm việc: cookie tồn tại trong jar nhưng curl, giống trình duyệt, không gửi nó tới /api/courses. Trong DevTools cùng cookie đó hiện ổ khóa ở cột HttpOnly và document.cookie trả về chuỗi rỗng.

38.8 Ngoài dự án

Mẫu ở trên — access token trong bộ nhớ, refresh token trong cookie HttpOnly, refresh âm thầm khi tải — là thứ SPA SDK của Auth0, của Okta, và phần lớn ứng dụng web ngân hàng hiện thực, thường kèm xoay vòng refresh và phát hiện tái

sử dụng ở server. Phương án thay thế loại trình duyệt khỏi việc xử lý token hoàn toàn là *backend-for-frontend*: một server nhỏ (hoặc chính gateway) giữ token trong session phía server, và trình duyệt chỉ mang một cookie session bình thường; Spring Cloud Gateway hỗ trợ điều này với filter `TokenRelay` và đăng nhập OIDC. Người phỏng vấn hỏi “bạn cất JWT ở đâu trong trình duyệt?” mong đợi “không ở đâu mà script đọc được, và refresh token không bao giờ trong URL” theo sau là bảng đánh đổi ở trên. “Còn XSS thì sao?” có một câu trả lời không hề về nơi cất: một Content Security Policy ngăn script lạ chạy ngay từ đầu, thứ phòng thủ mà mọi lựa chọn nơi cất vẫn phụ thuộc vào.

38.9 Câu hỏi từ phỏng vấn thật

Q: Bạn cất JWT ở đâu trong trình duyệt, và vì sao?

Bắt đầu bằng tách hai token, vì chúng có thời gian sống và bán kính thiệt hại khác nhau. Access token 15 phút có thể sống trong bộ nhớ JavaScript: mất nó khi tải lại tốn một lần refresh âm thầm, và script trộm được nó có mười lăm phút. Refresh token là thông tin xác thực quan trọng — bày ngày ở đây — và nó thuộc về một cookie `HttpOnly`, `Secure`, `SameSite=Strict` giới hạn bằng Path tới endpoint refresh, để không script nào đọc được và trình duyệt chỉ gửi nó tới một URL. `localStorage`, thứ dự án dùng hôm nay, bảo chứa được cho phiên bản đầu và không bảo chứa được khi bất kỳ script bên thứ ba nào chạy trên trang: cách một `JSON.parse` là một tuần truy cập. Câu kết senior là nơi cất là tuyến thứ hai; tuyến đầu là Content Security Policy ngăn script bị chèn thực thi ngay từ đầu.

Q: Mười tab nhận 401 cùng một khoảnh khắc. Bao nhiêu lời gọi refresh xảy ra?

Với interceptor như đã viết, mười — mỗi tab có bộ nhớ riêng và request đang chờ riêng, và trong một tab mỗi truy vấn song song chạy interceptor riêng rẽ, nên có thể hơn mười. Với xoay vòng refresh token ở server đó không phải lãng phí, đó là sự cố: lần refresh đầu xoay token, chín cái kia đưa ra token cũ và bị từ chối, và cơ chế phát hiện tái sử dụng mà server tốt hiện thực thu hồi cả họ token. Trong một tab cách sửa là promise dùng chung — 401 đầu khởi động refresh, các 401 sau await nó, tất cả phát lại với kết quả. Xuyên tab là `navigator.locks`: các tab lấy khóa lần lượt, và tab thứ hai thấy cookie đã xoay và thành công với một lần refresh bình thường. Điều đáng chú ý thêm là server nên khoan dung một cửa sổ ân hạn ngắn trên token trước đó, vài giây, vì việc mạng đảo thứ tự nghĩa là token “cũ” có thể tới hợp lệ sau lần xoay.

Q: XSS so với CSRF — mỗi lựa chọn nơi cất phơi bạn ra trước cái nào?

Bearer token trong storage: miễn nhiễm CSRF, vì trình duyệt không bao giờ tự đính header `Authorization`, và phơi nhiễm hoàn toàn trước XSS, vì bất kỳ script nào trên origin đọc được storage. Cookie: miễn nhiễm việc XSS đọc khi `HttpOnly`, và phơi nhiễm trước CSRF vì trình duyệt đính cookie vào bất kỳ request nào tới host đó, kể cả từ trang của kẻ tấn công — trừ khi `SameSite=Strict` hay `Lax` chặn nó, và một kiểm tra header tùy biến hay token hỗ trợ cho client cũ. Vậy lựa chọn thật là giữa một bài toán có cách sửa cơ học (CSRF) và một bài toán không có (XSS). Kỹ sư senior thêm rằng XSS trên trang có cookie `HttpOnly` vẫn có thể dùng phiên trong lúc trang mở — chỉ là không rút được thông tin xác thực ra — và đó là lý do access token ngăn và thu hồi phía server vẫn quan trọng.

Q: Người dùng đóng laptop tám ngày rồi quay lại. Chuyện gì xảy ra?

Cả hai thông tin xác thực đều chết: access token từ mười lăm phút, refresh token từ một ngày. Lời gọi API đầu nhận 401, interceptor gọi refresh, auth-service thấy dòng hết hạn, xóa nó và trả 401, client đăng xuất và chuyển hướng tới `/login`. Đó là kết quả đúng; câu hỏi chất lượng là người dùng thấy gì trên đường đi. Mọi truy vấn đang bay thất bại trước, và nếu query client thử lại request thất bại ba lần với backoff — mặc định của TanStack Query — chuyển hướng đến sau vài giây vòng xoay và các 401 lặp lại trong log. Cấu hình query client không thử lại với 401, chạy một lần refresh khi ứng dụng mount trước khi render route được bảo vệ, và cú quay lại sau tám ngày là một lần bật sạch về đăng nhập với route dự định được giữ cho sau đó.

Q: Làm sao đăng xuất một người dùng ở mọi nơi?

Phía refresh là một thao tác cơ sở dữ liệu: xóa mọi dòng trong `refresh_tokens` của người dùng đó, không chỉ dòng trình duyệt hiện tại đưa ra. Mọi thiết bị khác thất bại ở lần refresh kế tiếp và về trang đăng nhập trong vòng mười lăm phút. Phía access là bài toán của Chương 28 — token đã ký không thể thu hồi — nên hãy nói thật về cửa sổ và nêu hai cách đóng nó: một deny-list theo `jti` trong Redis được gateway kiểm tra, hoặc access token ngắn hơn. Trong dự án này thay đổi cụ thể là một phương thức repository, `deleteAllByUser(user)`, gọi từ một endpoint `/auth/logout-all` mới và từ hành động vô hiệu người dùng của admin. Nhắc cái bấy phòng vấn: một “đăng xuất” phía client chỉ dọn storage không thu hồi gì cả — token vẫn hợp lệ ở bất kỳ ai đã sao chép nó.

Q: Lỗi CORS xuất hiện ở production nhưng không bao giờ ở development. Bạn kiểm tra gì?

Trước hết, request có cross-origin ở production mà không ở development hay không — trong dự án này proxy của Vite và Traefik đều giữ trang và API trên một origin, nên lỗi CORS ở production nghĩa là việc triển khai đã tách chúng: CDN cho bundle, host hay cổng khác cho gateway. Rồi so header `Origin` trình duyệt gửi, từng byte kể cả scheme và cổng, với danh sách cho phép của gateway; `https://app.example` và `https://app.example:443` là hai chuỗi khác nhau với Spring. Rồi quy tắc credentials: với cookie hay `withCredentials`, origin wildcard bị trình duyệt từ chối ngay cả khi server gửi nó, và `Access-Control-Allow-Credentials: true` phải có mặt. Rồi preflight: một `OPTIONS` rơi vào route bỏ tiền tố và chuyển tiếp tới service sẽ trả 401 của service đó thay vì các header CORS, nên CORS phải được trả lời tại gateway trước khi định tuyến — điều `CorsWebFilter` làm. Cuối cùng, nhìn log server và mong đợi không thấy gì: request ở đó thành công; trình duyệt mới là bên từ chối, và đó là hình dạng của mọi bài toán CORS.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Mở DevTools trên ứng dụng đang chạy và nêu tên khóa storage giữ cả hai token. Viết biểu thức JavaScript một dòng mà một script bị chèn sẽ dùng để đọc refresh token, và nói nó làm được gì với token đó trong bảy ngày tới.
2. Interceptor gọi `axios` trần cho lần refresh thay vì `apiClient`. Giải thích vòng lặp sẽ xảy ra nếu không, từng bước, bắt đầu từ một 401 trên `/refresh`.
3. Với xoay vòng refresh token bật ở server, lần theo bốn 401 song song qua interceptor như đã viết và cho biết ở bước nào người dùng bị đăng xuất. Rồi giải thích vì sao phiên bản single-flight tránh được.
4. Thiết kế cookie đặt `Path=/api/auth/refresh`. Nêu hai đòn tấn công khác nhau mà thuộc tính này làm cùn, và giải thích vì sao path được viết theo góc nhìn của gateway chứ không phải auth-service.
5. `CorsConfig` liệt kê hai origin và cho phép credentials. Trong mode triển khai nào trong hai mode của dự án nó được hỏi tới, và một thay đổi triển khai duy nhất nào sẽ khiến nó quan trọng?
6. Người dùng báo lỗi CORS; log gateway cho thấy request trả 200. Giải thích vì sao cả hai quan sát đúng cùng lúc, và liệt kê ba giá trị header bạn sẽ so sánh trước tiên.

Đáp án tự kiểm tra

Chương 1 — Spring Boot

1. Biến môi trường thắng: biến môi trường đứng trên tài liệu của config-server, và tài liệu đó đứng trên YAML đóng gói trong jar. Đúng với container vì image phải bất biến giữa các môi trường; những gì khác nhau do runtime bơm vào.
2. @Configuration, @ComponentScan, @EnableAutoConfiguration. Cái cuối cùng: auto-configuration phụ thuộc vào classpath, nên gỡ một dependency sẽ âm thầm gỡ luôn các bean mà nó kích hoạt.
3. Nó được giải thành chuỗi ký tự placeholder theo đúng nghĩa đen. Vì thế service vẫn *khởi động* được dù không có thông tin xác thực Google — OAuth chỉ hỏng lúc người dùng bấm, chứ không hỏng lúc boot.
4. Việc mở (exposure) khiến endpoint tồn tại; Spring Security vẫn phân quyền từng request. Nếu /actuator/** không được cho phép, probe nhận 401, và kubelet tính đó là thất bại.

Chương 2 — Config Server

1. Biến môi trường > config-server > YAML đóng gói.
2. Native phục vụ file từ thư mục classpath bên trong jar; git thêm khả năng kiểm toán, review, hoàn tác, và thay đổi cấu hình mà không cần build lại config-server.
3. optional: cho phép service boot khi config-server sập — nhưng khi đó nó boot với các giá trị mặc định localhost và hỏng về sau, ở rất xa nguyên nhân.
4. Trong application.yml của chính nó, đóng gói trong jar của nó. Nguyên tắc bootstrap: nguồn cấu hình không thể tự cấu hình cho mình; mọi chuỗi như vậy đều cần một gốc tự chứa.

Chương 3 — Eureka

1. course-service đăng ký (POST) lúc khởi động; discovery client của gateway làm mới cache registry của nó (mặc định tối đa 30 s); rồi việc định tuyến cân

- bằng tải từ cache. Trường hợp xấu nhất để một phiên bản mới nhận được lưu lượng: khoảng đăng ký + làm mới cache, tức hàng chục giây.
2. Compose khiến *tên* service phân giải được, và hostname trùng với tên đó; trong Kubernetes hostname của pod là tên pod, thứ không ai phân giải được — nên phải đăng ký IP thay vào đó.
 3. Ưu tiên tính sẵn sàng của dữ liệu registry hơn tính chính xác. Có lợi: một switch khởi động lại làm im bật mọi heartbeat nhưng các phiên bản vẫn đang chạy. Có hại: sập thật hàng loạt để lại các phiên bản đã chết vẫn nằm trong registry.
 4. Danh sách endpoint của Service thay cho registry; readiness probe thay cho heartbeat; kube-proxy/DNS thay cho tra cứu phía client; ReplicaSet thay cho việc phiên bản mới tự đăng ký.

Chương 4 — API Gateway

1. Trình duyệt → ts.local (file hosts) → Traefik (Ingress: /api → Service api-gateway) → pod gateway (predicate /api/auth/**, StripPrefix=2, không đòi JWT với /api/auth) → pod auth-service do Eureka phân giải, đến nơi dưới dạng /login → controller.
2. Google kiểm tra redirect URI từng byte một; nó đã được đăng ký với đường dẫn đầy đủ /login/oauth2/..., nên phải đến nơi nguyên vẹn. Các route API thì cất tiền tố vì controller được viết không có tiền tố.
3. Mọi request mang token do auth-service ký đều trượt bước kiểm tra của gateway — 401 đồng loạt trên các route được bảo vệ trong khi bản thân việc đăng nhập có vẻ vẫn chạy.
4. Proxy là bài toán I/O với độ đồng thời cao — event loop rất hợp. course-service làm việc JDBC kiểu blocking; reactive sẽ làm nó phức tạp mà không có lợi ích gì cho đến khi chứng minh được điều ngược lại.

Chương 5 — Bảo mật

1. CSRF bám vào một thông tin xác thực có sẵn trong môi trường (cookie phiên được gửi tự động). JWT trong header Authorization được code gắn vào một cách tường minh; một request giả mạo xuyên site không thể thêm nó vào.
2. Mười lăm phút (thời hạn của nó). Không gì cả — chữ ký không thể thu hồi. /refresh: refresh token được đối chiếu với cơ sở dữ liệu, nơi việc thu hồi được lưu.

-
3. Bất biến: mọi đường mạng đến một service đều đi qua filter JWT của gateway. Bị phá khi: một Ingress định tuyến thẳng đến service (route /courses đã xóa), hoặc bất kỳ bên gọi nào trong cụm giả mạo header trên mạng pod phẳng.
 4. Client-secret không bao giờ rời khỏi server; authorization code thì đi qua trình duyệt. Code chỉ dùng một lần, sống ngắn, và vô giá trị nếu không có client-secret lúc đổi lấy token.
 5. HS256: một secret dùng chung; ai kiểm tra được thì cũng ký được. RS256: khóa riêng ký, khóa công khai kiểm tra — nên một IdP có thể cho hàng nghìn service kiểm tra token mà không service nào trong số đó đúc được token.

Chương 6 — Các service và module common

1. (a) HTTP — bên gọi cần dữ liệu ngay. (b) Sự kiện — một sự thật mà bên khác phản ứng theo; việc chấm điểm không được chờ mail. (c) Không cái nào: kiểm tra token là phép mật mã cục bộ với secret dùng chung; không cần gọi đi đâu.
2. auth-service chỉ *sản xuất* — từng lần gửi hóng riêng lẻ, boot vẫn thành công. notification-service chỉ *tiêu thụ*; không có broker thì nó chỉ là chi phí đứng không, nên nó đi kèm Kafka.
3. Thuộc về common: các record sự kiện — hai bên phải thống nhất hình dạng. Không bao giờ: logic nghiệp vụ/entity — hành vi dùng chung trói chu kỳ phát hành của các service vào nhau và che mất ranh giới.
4. Consumer triển khai trước producer, và thay đổi chỉ được thêm vào (trường mới, tùy chọn). Khi đó consumer cũ bỏ qua trường mới và consumer mới chịu được sự kiện cũ.

Chương 7 — Postgres, JPA, Flyway

1. Schema của hai lập trình viên âm thầm lệch nhau (update áp dụng theo từng cơ sở dữ liệu, không có lịch sử); và thay đổi phá hủy là bất khả thi — update không bao giờ drop/đổi tên, nên refactor entity lạng lẽ lệch khỏi schema.
2. Mỗi dòng lịch sử lưu checksum của script tương ứng; script bị sửa sẽ lệch checksum và Flyway từ chối khởi động — lịch sử là dấu vết kiểm toán, và việc sửa sẽ làm giả nó.
3. Phát hành 1: thêm cột mới, ghi cả hai (hoặc dùng trigger). Migration backfill.

Phát hành 2: đọc cột mới, ngừng ghi cột cũ. Phát hành 3: drop cột cũ. Mỗi bước tương thích với phiên bản ngay trước nó.

4. Ứng hộ: cô lập lỗi và phạm vi ảnh hưởng — một server sập, nâng cấp, hay cấu hình sai không thể chạm tới dữ liệu của server kia. Phản đối: gấp đôi bộ nhớ và bề mặt vận hành với một homelab. Cloud có quản lý thường chọn cô lập bằng thông tin xác thực trên một instance vì server là đơn vị tính tiền.

Chương 8 — MongoDB

1. Mục từ điển: tự chứa, đọc/ghi như một khối, lồng nhau tự nhiên, không cần nhất quán giữa các mục. Enrollment: tham chiếu hai aggregate khác và đòi hỏi nhất quán giữa chúng — join và transaction chính là điểm mấu chốt.
2. Vào code ứng dụng (và validator của collection nếu muốn). Kỷ luật: lớp document đọc ngược được (trường mới là tùy chọn), và script sửa dữ liệu tương minh khi đổi hình dạng.
3. Postgres từ chối kết nối đến cơ sở dữ liệu không tồn tại; Mongo tạo cơ sở dữ liệu ở lần ghi đầu tiên, nên tên sai cho ra kết quả rỗng. Ôn ào tốt hơn — lỗi bạn thấy lúc triển khai hơn hẳn lỗi bạn thấy lúc kiểm toán.
4. Chỉ ClusterIP + không có Ingress nghĩa là chỉ bên gọi trong cụm mới chạm tới được; sự tương đồng với compose cũng giữ ma sát khi dev bằng không. Mất hiệu lực ngay khi nó bị lộ ra ngoài cụm hoặc dùng chung cụm với workload không đáng tin.

Chương 9 — MinIO / S3

1. Service ký (micro giây) thay vì stream (giây); storage phục vụ byte trực tiếp. Kiểu lỗi mới: URL nhúng sẵn endpoint và thời hạn — endpoint sai hay lệch đồng hồ làm hỏng tải xuống chỉ ở trình duyệt, service không nhìn thấy.
2. Service ký chạm tới MinIO qua minio:9000 (DNS của cụm); trình duyệt không phân giải được tên đó. Chữ ký bao trùm cả host, nên viết lại sau khi ký sẽ làm chữ ký vô hiệu — vì thế phải ký với một endpoint thứ hai, công khai.
3. Thời hạn (15 phút) giới hạn cửa sổ thiệt hại; URL chỉ cấp một object, không phải cả bucket; và không thể ký lại nếu không có khóa.
4. Thay đổi: endpoint, thông tin xác thực (lý tưởng là chuyển sang IAM role), bảo đảm về độ bền. Không đổi: mọi lời gọi SDK, logic bucket/key/presign — API chính là chuẩn.

Chương 10 — Kafka cốt lõi

1. Key theo `userId` — thứ tự theo key chính là thứ tự theo partition. Số partition $\geq$ số consumer tối đa bạn từng muốn có trong group; độ song song bị chặn trên bởi số partition.
2. Các group độc lập nhau: mỗi group giữ offset riêng cho từng partition, nên cả hai group đều duyệt hết toàn bộ log; tiêu thụ là đọc không phá hủy cộng với một con trỏ theo từng group.
3. `LISTENERS` = socket được bind; `ADVERTISED_LISTENERS` = địa chỉ *trả về cho client* trong metadata. Bootstrap dùng địa chỉ bạn cung cấp; mọi kết nối sau đó dùng địa chỉ được quảng bá — nên lỗi chỉ bắt đầu sau một cái bắt tay thành công.
4. Retry và sập sau khi xử lý đều gây gửi lại; broker không thể biết bạn đã hành động chưa. Notification: khử trùng lặp theo id sự kiện (hoặc chấp nhận một mail trùng hiểm hoi — lũy đẳng bằng cách chịu đựng).

Chương 11 — Spring Kafka

1. `send()` đệm ở phía client và trả về một future; không cần broker cho đến lúc giao. Sự cố lộ ra theo từng lần gửi: future thất bại sau timeout, và sự kiện đang đệm mất trắng nếu tiến trình chết.
2. Khi chưa có offset đã commit, `earliest` bắt đầu từ record cũ nhất còn được giữ — lịch sử được xử lý lại. Một khi group đã commit offset, thiết lập này không bao giờ được hỏi đến nữa.
3. Không có: giải tuần tự ném lỗi ngay trong poll, offset không bao giờ tiến, partition kẹt mãi ở một record. Có: lỗi được vật chất hóa thành một thất bại có kiểu, error handler bỏ qua (hoặc retry rồi chuyển tiếp) — ở production đầy đủ, chuyển đến `<topic>.DLT`.
4. Giải tuần tự JSON khởi tạo các lớp được nêu tên trong header; danh sách tin cậy không giới hạn cho phép một message bắt consumer dựng lên kiểu bất kỳ trên classpath — một vector tiêm gadget.
5. Khi “ghi DB + phát sự kiện” không được phép lệch nhau. Nó làm cho thay đổi trạng thái và *việc ghi lại ý định phát* trở thành nguyên tử — một transaction cục bộ; một relay đảm bảo việc phát cuối cùng sẽ diễn ra.

Chương 12 — Docker

1. Namespace (góc nhìn cô lập) và cgroup (giới hạn tài nguyên). Không có kernel khách: khởi động tính bằng mili giây và không tốn chi phí hệ điều hành cho từng instance — nhưng đồng thời: kernel dùng chung (cô lập yếu hơn VM) và JVM nhìn thấy tài nguyên của host trừ khi được bảo khác đi.
2. Bên trong container, localhost là chính container đó. Compose sửa bằng DNS theo tên service trên mạng bridge; Kubernetes bằng DNS của Service — cả hai qua biến môi trường ghi đè các giá trị mặc định localhost.
3. Mỗi container có network namespace riêng, nên cả hai đều bind cổng 5432 “của mình”. Xung đột quay lại ở host dùng chung: cổng host được publish (vì thế mới có 5433/5434) — và với --network host, nơi không hề có sự tách biệt namespace.
4. depends_on thường chờ *đã khởi động*; service_healthy chờ healthcheck vượt qua — ở đây là config-server tự curl /actuator/health của chính nó, được mở bởi cấu hình management.

Chương 13 — Jib và Registry

1. Không cần daemon trong CI, chia layer theo độ biến động (push lại nhỏ), image không chạy bằng root và sẵn sàng cho OpenShift. Về bảo mật: không có daemon nghĩa là không có docker socket trong CI — socket đó là root trên host.
2. Tên image đồng thời là địa chỉ pull cho cụm. Ở đây chúng trùng nhau vì Jenkins dùng chung network namespace với host; nếu Jenkins chạy trên mạng bridge, localhost:5000 của nó sẽ là chính nó và push hỏng trong khi pull vẫn chạy.
3. Mỗi lần deploy phải import trên từng node, mất hàng phút tuần tự hóa, và không có lịch sử tag bất biến; lệnh set image của pipeline trở đến tên gắn tag SHA sẽ không có gì để trở vào — registry chính là thứ làm cho tag có địa chỉ.
4. Cùng tag, nội dung mới: với IfNotPresent image đã cache trên node âm thầm thắng. Cách sửa: imagePullPolicy: Always (lựa chọn của manifest), hoặc deploy bằng tag/digest bất biến (lựa chọn của pipeline) — dự án này làm cả hai.

Chương 14 — Giải phẫu cụm

1. Bạn viết trạng thái mong muốn; controller điều hòa trạng thái thực tế về phía đó mãi mãi. Apply lần thứ hai không tạo ra diff, nên không có hành động — lũy đẳng nhờ kiến trúc.
2. Thứ tự là khái niệm thủ tục; hội tụ thay thế nó: mọi thứ cùng khởi động, bên phụ thuộc sập rồi thử lại với backoff cho đến khi thứ nó cần tồn tại. CrashLoop trong lúc hội tụ là cơ chế, không phải bug.
3. kubectl gửi một lệnh REST delete đến API server; kho trạng thái cập nhật; vòng lặp của controller ReplicaSet thấy replica $0 < 1$ và tạo một object pod; scheduler gán nó; kubelet trên node khởi động container.
4. Apply hợp nhất trạng thái mong muốn; nó không bao giờ xóa những gì vắng mặt trong đầu vào. kubectl delete ingress course-service -n ts, hoặc apply với --prune (hoặc một controller GitOps) thì sẽ xóa.

Chương 15 — Workload

1. Mỗi thay đổi template sinh ra một ReplicaSet mới trong khi các ReplicaSet cũ được giữ ở scale 0 — cập nhật cuốn chiếu và rollback một lệnh là hệ quả của việc tách bộ đếm khỏi template có phiên bản.
2. Deployment: N replica thay thế được cho nhau, cái nào chết cũng được, thứ tự không quan trọng. StatefulSet: replica có tên với storage theo từng danh tính và thay thế có thứ tự. Eureka: Deployment (registry được dựng lại nhờ đăng ký lại). Redis cache: Deployment nếu thật sự chỉ là cache. Broker Kafka: StatefulSet. Gateway: Deployment.
3. Pod mới phải vượt qua readiness trước khi pod cũ bị kết thúc — probe của Chương 17 là cổng gác cho rollout của Chương 15.
4. Cập nhật Deployment chạy pod cũ và mới đồng thời; hai tiến trình postgres cùng mở một thư mục dữ liệu sẽ làm hỏng nó. StatefulSet thay thế từng cái một và gán lại claim cho pod kế nhiệm, không bao giờ hai cái cùng lúc.

Chương 16 — Cấu hình và Secret

1. RBAC về việc ai được đọc object Secret; tách vòng đời khỏi git; mã hóa khi lưu (tùy chọn). Dự án này: RBAC có (Role của Jenkins bỏ qua quyền đọc secret... nó cấp create/patch nhưng kiểm soát thực tế là giới hạn theo namespace), tách khỏi git có (script), mã hóa khi lưu không.

2. Image server đọc `POSTGRES_*`; Spring đọc `SPRING_DATASOURCE_*`. Tách Secret nghĩa là mỗi pod chỉ nhập những tên nó hiểu — không có biến lạ che mắt lỗi gõ hay rò rỉ phạm vi.
3. Cập nhật: `JWT_SECRET=new ./create-secrets.sh`; rồi ngay lập tức `kubectl -n ts rollout restart deployment/auth-service deployment/api-gateway`. Làm nửa chừng: bên ký và bên kiểm tra giữ khóa khác nhau — mọi lần đăng nhập đều 401, hoặc tệ hơn, chỉ sau lần restart không liên quan tiếp theo.
4. Cây `configurations/*.yaml` của `config-server`, mount dưới dạng file — và cho `config-server` nghĩ hữu luôn.

Chương 17 — Probe và tài nguyên

1. Startup thất bại quá ngân sách: restart. Readiness thất bại: bị gỡ khỏi endpoint, không restart — có thể mạnh tay. Liveness thất bại: restart — cái này phải dè dặt.
2. `/v3/api-docs` chỉ trả lời sau khi toàn bộ context (kể cả Flyway) đã lên và được `permitAll`; một cổng đã bind chỉ chứng minh Tomcat tồn tại. `/actuator/health` trả 401 ở nơi security không cho phép (`course/auth`) và trung thực ở nơi cho phép (`dictionary`, `gateway`).
3. OOM killer của kernel đã kết thúc cgroup vì vượt giới hạn bộ nhớ — JVM không hề tham gia, nên không có exception. Kiểm tra: phép tính giữa limit và `MaxRAMPercentage`, và xem tải hay rò rỉ đã làm heap phình lên.
4. Scheduler đọc requests (để xếp chỗ); kernel thực thi limits. Bộ nhớ không nén được — vượt là có kẻ bị giết, nên hãy chặn nó một cách dự đoán được; CPU nén được — limit sẽ bóp nghẹt lúc khởi động đúng khi JVM cần bứt tốc.

Chương 18 — Mạng

1. DNS phân giải tên Service thành một IP ảo ổn định; endpoint controller giữ danh sách pod Ready luôn cập nhật; kube-proxy (hoặc thứ tương đương của k3s) viết lại lưu lượng đến IP ảo sang IP của một pod được chọn; readiness quyết định tư cách thành viên.
2. Chỉ pod Ready mới là endpoint, nên cập nhật cuốn chiếu chuyển lưu lượng đúng lúc pod mới vượt probe và pod cũ kết thúc — không gián đoạn chính là tương tác giữa probe và endpoint.
3. Selector/label lệch nhau — Service khớp với không pod nào. Không có gì báo lỗi vì Service không có endpoint là một trạng thái hợp lệ; triệu chứng

chỉ nhìn thấy trong `get endpoints`.

4. `ts.local` không phải DNS thật; file hosts ghim nó vào IP của tailnet. Thay thế: một domain thật với DNS thực + TLS bằng cert-manager, hoặc một Cloudflare Tunnel công bố hostname đó.

Chương 19 — Lưu trữ

1. Pod mount PVC; PV (và StorageClass) có phạm vi toàn cụm; lớp gián tiếp này cho phép cùng một claim nhận local-path ở đây và EBS trên EKS — manifest không đổi.
2. Postgres chạy trên một node tại một thời điểm — RWO là đủ. Upload dùng chung đẩy người ta sang RWX; lưu trữ đối tượng thường mới là câu trả lời đúng.
3. Xóa StatefulSet không đụng đến PVC nên dữ liệu sống sót qua sai lầm điều phối; xóa PVC (reclaim Delete) gỡ luôn PV và dữ liệu — dữ liệu sống ở claim, không phải ở workload.
4. Bền vững (persistence): sống qua restart (có). Độ bền (durability): sống qua hỏng phần cứng (không — một SSD). Sao lưu: sống qua sai lầm của chính bạn (không — chưa có, khoảng trống thành thật).

Chương 20 — Kustomize

1. Một tập hợp chuẩn (không sót gì, một lệnh) và các chỉnh sửa xuyên suốt (namespace, label) được đóng dấu nhất quán.
2. Namespace: phạm vi toàn cụm — Role theo namespace của Jenkins không thể apply nó. Secret: giá trị tuyệt đối không được tồn tại trong git — hình dạng nằm trong script, giá trị nằm trong môi trường của người vận hành.
3. Kustomize: triển khai ứng dụng của chính bạn dưới dạng YAML dễ đọc. Helm: phân phối phần mềm có tham số đến cụm của người khác. Đến dưới dạng chart: Strimzi và kube-prometheus-stack.
4. Không gì cả — apply không dọn; mongodb vẫn chạy, giờ không còn ai quản lý. Đúng cách: `kubectl delete -f mongodb.yaml` (rồi gỡ khỏi danh sách), hoặc cơ chế prune/GitOps điều hòa cả việc xóa.

Chương 21 — Khái niệm CI/CD

1. CI: main luôn được xác nhận là build được — stage Build. CD: mọi thay đổi đã xác nhận tự đến production không cần ai đỡ — stage Deploy cộng với

cổng gác rollout của nó.

2. Jenkins nằm sau NAT, không có địa chỉ công khai — GitHub không gọi vào được, nên Jenkins phải poll. Cùng ràng buộc: CI bên trong mạng doanh nghiệp mà git host trên cloud không chạm tới được.
3. Build lại là một binary khác — phân giải dependency, timestamp, toolchain khác — nên test đã chứng nhận một thứ khác. Thứ tự ưu tiên biến môi trường của Chương 1 cho phép một artifact nhận cấu hình theo môi trường lúc chạy.
4. Test bị bỏ qua (bật lại, hoặc gác ở chỗ khác và biết rõ điều đó); deploy không được xác minh (rollout status/smoke test); thành công được định nghĩa bằng exit code chứ không bằng hành vi (xác minh dựa trên probe, alert).

Chương 22 — Jenkins

1. Job, credential, trạng thái plugin, lịch sử build. Image = binary+plugin tái tạo được; volume = trạng thái không thay thế được; gộp lại, server là gia súc với một thư mục thú cưng.
2. Jenkins chạy trên JDK 21 của image gốc; build export `JAVA_HOME=JDK25_HOME` trong khối environment của Jenkinsfile — theo từng pipeline, không phải toàn cục.
3. apply -k cần get/list + create/patch/update trên mọi kind được apply; rollout status cần get/list/watch trên deployment và replicaset. Bỏ được nếu không có set image: không gì mới — get+patch của set image đã được bao phủ sẵn; các quy tắc đọc pods/log chỉ tồn tại cho trình xử lý thất bại.
4. Jib push đến localhost:5000, và tên đó phải chỉ cùng một máy với cả bên push lẫn bên pull; host networking khiến localhost của Jenkins chính là localhost của host, đúng như tên image đã hứa với k3s.
5. ServiceAccount = danh tính; Role = danh sách năng lực; RoleBinding = mối nối. Role chứ không phải ClusterRole giới hạn một pipeline bị xâm nhập trong một namespace — phạm vi ảnh hưởng như một lựa chọn về kiểu.

Chương 23 — Pipeline

1. SNAPSHOT là khả biến — một node có thể phục vụ bất kỳ build lịch sử nào dưới cùng một tên; SHA ghim đúng commit và làm cho câu hỏi “cái gì đang chạy?” trả lời được từ `kubectl describe`.
2. Chỉ build các module được liệt kê cộng với dependency của chúng. Quên một mục MODULES: service mới không bao giờ có image; vòng lặp deploy

sau đó thất bại (hoặc deploy một image cũ nếu có) — điểm ghép nối thủ công duy nhất của pipeline.

3. `kubectl rollout status --timeout=300s` — nó chuyển kết quả probe (Chương 17) thành exit code của pipeline.
4. Mọi rollout nên tiến hành đồng thời; chờ theo từng service tuần tự hóa sự hội tụ mà Kubernetes sẵn sàng làm song song — cùng tổng công việc, thời gian thực dài hơn.
5. Thay đổi: cách kích hoạt (webhook), nơi build chạy (runner tạm thời), cơ chế secret (secret của repo). Không đổi: thứ tự stage, lệnh Maven, gắn tag SHA, bước xác minh rollout.

Chương 24 — Vận hành

1. `get pods` → `describe pod` → `logs` (`--previous` — thiết yếu với CrashLoop, khi container hiện tại chưa có lịch sử gì) → `exec` vào và thăm dò từ bên trong.
2. CrashLoop: tiến trình thoát — tăng ứng dụng. `Running 0/1`: tiến trình sống nhưng readiness thất bại — đích của probe (thường là một dependency) mới là tăng cần xem.
3. `Exec` vào pod và thử kết nối từ đó — DNS, đường mạng, và biến môi trường được bơm vào của pod khác với laptop của bạn; laptop chỉ chứng minh Postgres còn sống, không chứng minh pod chạm tới được nó.
4. Lưu lượng pod/service của k3s đi qua firewall của host; chặn `10.42.0.0/16` hoặc `10.43.0.0/16` làm hỏng DNS và các lời gọi pod-đến-pod một cách chập chờn — triệu chứng không phân biệt được với ứng dụng thiếu ổn định.

Chương 25 — Kafka trong cụm

1. Một controller có vòng lặp điều hòa nhắm đến các kind tài nguyên tùy chỉnh do CRD định nghĩa. Strimzi thêm `Kafka/KafkaNodePool` (cụm) và `KafkaTopic` (các hợp đồng) — cộng thêm các kind `user`, `connect` và `bridge` mà dự án này không dùng.
2. Gói operator chứa các object phạm vi toàn cụm (CRD, `ClusterRole`) mà Role theo namespace không thể cấp; `kafka.yaml` chỉ chứa tài nguyên tùy chỉnh theo namespace, thứ Role giờ đã cho phép. Role hay `ClusterRole` chính là ranh giới.
3. Advertised listener, các voter của controller quorum, bản đồ giao thức bảo mật theo từng listener (thêm cả cluster ID và Service cho từng broker). Operator tính chúng từ khai báo listener và node pool.

4. Trạng thái thực của pod broker (`get pods`) và các dòng điều hòa trong log của operator. Condition có thể là một lần ghi cũ từ một vòng điều hòa đã timeout trong lúc pull image kéo dài; vòng lặp định kỳ tiếp theo sẽ sửa lại.
5. Số partition được review, không có cuộc đua producer/topic ngay đầu, và một bảng kê các hợp đồng sự kiện dưới dạng object. Một partition: thứ tự toàn phần theo topic, đủ dùng chừng nào mỗi consumer group chỉ có một instance; partition có thể thêm về sau nhưng không bao giờ bớt được.

Chương 26 — Frontend và host đơn

1. Giai đoạn một (Node) biên dịch và đóng gói; giai đoạn hai (nginx) phục vụ. Chỉ `dist/` được chép bằng `COPY --from=build; Node, node_modules` và trình biên dịch bị bỏ đi, để lại một image ~15 MB với bề mặt tấn công nhỏ hơn.
2. Traefik so `/courses/7` với cả hai Ingress; `/` là tiền tố khớp dài nhất (không có quy tắc `/courses` nào), nên nginx nhận nó. Không có file như vậy; `try_files $uri $uri/ /index.html` rơi về `index.html`, ứng dụng khởi động và React Router hiển thị route từ URL.
3. Các quy tắc cho cùng một host được hợp nhất vào một bảng định tuyến và tiền tố đường dẫn khớp dài nhất thẳng: `/api/courses` khớp `/api` (dài hơn) thay vì `/; /assets/x.js` chỉ khớp `/`.
4. CORS chỉ áp dụng cho request xuyên origin. Trong cụm, trang và API dùng chung scheme, host và cổng (`http://ts.local`), nên trình duyệt không bao giờ gửi preflight; trên laptop trang là `localhost:3000` còn API là `localhost:8080` — cổng khác là origin khác.
5. Socket thuộc sở hữu của group docker trên host (GID 984 trên `ts-server`); user jenkins bên trong container không thuộc group nào có ID số đó, nên bị từ chối ghi. `--group-add` gán GID của host vào lúc chạy; tính toán nó giữ cho script đúng trên mọi host mà không cần cứng hóa một con số đặc thù của máy.

Chương 27 — Chặng đường phía trước

1. Độ trễ tiêu thụ (consumer-group lag): khoảng cách giữa offset mới nhất trong một partition và offset group đã commit — lag tăng nghĩa là notification-service đang tụt lại hoặc đã sập.
2. Đẩy: thông tin xác thực nằm ở CI (kubecfg của Jenkins). Kéo: nó không bao giờ rời khỏi cụm; CI chỉ ghi vào git. Prune giải quyết cái bẫy apply-không-bao-giờ-xóa của Chương 20.

-
3. Route của gateway đổi `lb://name` thành `http://name:port`; endpoint của Service thay cho đăng ký, readiness probe thay cho heartbeat; xóa discovery-server (và các dependency eureka client), giải phóng tài nguyên của nó.
 4. Chúng đã chạy không bằng root với quyền của group 0, nên SCC UID-tùy-ý của OpenShift chấp nhận chúng nguyên vẹn; chỉ còn Ingress → Route.

Chương 28 — Đào sâu: JWT ở quy mô lớn

1. Không. Config-server giờ chỉ giữ khóa công khai, thứ xác minh được nhưng không ký được. Giả mạo cần PEM riêng, chỉ nằm trong Secret của auth-service — Secret đó (hoặc pod auth-service) mới là thứ phải lộ.
2. Công bố khóa công khai mới trong JWKS dưới một `kid` mới trong khi giữ khóa cũ; đợi cache của các bên xác minh làm mới; chuyển ký sang khóa riêng mới; sau thời gian sống dài nhất của token, gỡ khóa cũ. Ở bước 3 hai thể hệ token lưu hành cùng lúc; `kid` cho mỗi bên xác minh biết *khóa công khai nào* cần áp dụng, nên token cũ vẫn xác minh được và khóa cũ về sau có thể nghỉ hưu một cách chắc chắn chứ không phải bằng cách thử.
3. User đã cầm access token không bị ảnh hưởng cho tới khi nó hết hạn (15 phút), vì các bên xác minh đã cache khóa công khai. Sau đó `/refresh` thất bại và họ bị đăng xuất; đăng nhập mới thất bại ngay từ phút đầu. Một bên xác minh khởi động lại trong lúc sự cố boot với cache JWKS rỗng và, khi fail closed, từ chối mọi token — đó là lý do khóa công khai cũng nên có sẵn trong cấu hình làm phương án dự phòng.
4. Lỗi thời: claim bị đóng băng lúc phát hành, nên một khóa học tạo sau khi đăng nhập là vô hình cho tới khi token hết hạn — token không thể biết dữ liệu nó không sở hữu. Quyền sở hữu: thành viên khóa học là dữ liệu của course-service; mã hóa nó vào token do auth-service phát hành là ghép schema của hai service qua hợp đồng token. (Kích thước là lý do thứ ba, và ít quan trọng nhất.)
5. Header được chuyển tiếp, nhưng không endpoint công khai nào đọc `X-User-Id`: `/refresh` xác thực bằng refresh token, `/actuator` không xác thực gì. Nên hôm nay không khai thác được, nhưng bất biến “chỉ gateway viết các header này” không được cưỡng chế. `default-filters: RemoveRequestHeader=X-User-Id` (và `X-User-Role`) trong `api-gateway.yml` cưỡng chế nó.
6. Tối đa 15 phút — thời gian sống của access token — vì gateway xác minh cục bộ và chỉ `/refresh` mới hỏi cơ sở dữ liệu. Deny-list theo `jti` thêm Redis (một lượt tra mỗi request, Redis chết thì fail-open hay fail-closed tùy bạn chọn); mẫu phantom token thêm Redis trên đường găng của mọi request

mà không có lựa chọn fail-open — Redis chết là sập toàn bộ.

Chương 29 — Đào sâu: Cấu hình

1. Secret thắng: nó tới dưới dạng biến môi trường `SPRING_DATASOURCE_PASSWORD`, và môi trường OS đứng trên tài liệu từ config-server. Nếu object Secret không tồn tại, `envFrom` làm pod thất bại; nếu chỉ thiếu khóa, pod khởi động, giá trị root nguyên văn trong file được phục vụ có hiệu lực, và service kết nối bằng root, lặng lẽ.
2. Ưu tiên chọn *tài liệu nào* định nghĩa `app.s3.secret-key`; tài liệu của config-server định nghĩa, và nó gửi nguyên văn `$APP_S3_SECRET_KEY:minioadmin` chưa phân giải. Client phân giải placeholder với Environment của chính nó sau đó, nên biến môi trường thắng vì văn bản yêu cầu thế, không phải vì thứ tự nguồn.
3. Cửa sổ một: giữa hai lần refresh, auth-service ký bằng khóa mới còn gateway xác minh bằng khóa cũ — mọi lần đăng nhập cho ra token gateway từ chối. Cửa sổ hai: sau khi cả hai refresh, mọi token phát trước thay đổi bị từ chối cho tới lúc đăng nào cũng hết hạn. Xoay nóng cần các khóa chồng lấn chọn bằng `kid`, thứ một secret HS256 dùng chung không diễn đạt được.
4. Không optional: lấy thất bại, JVM thoát, kubelet khởi động lại với back-off tăng dần cho tới khi server trở lại. optional:: boot từ mặc định đóng gói trở về localhost, qua readiness probe, nhận lưu lượng, thất bại mọi request. fail-fast cộng retry: thử lại trong tiến trình với back-off trong cửa sổ đã cấu hình, rồi thoát — không lãng phí lần khởi động JVM nào, và không có pod nói dối.
5. Generator của Kustomize gắn hash nội dung vào tên ConfigMap; file đổi là tên mới, Deployment tham chiếu nó là pod template mới, và Kubernetes roll các pod. Cấu hình, code và rollback thành một cơ chế có lịch sử — server không cho được điều đó nếu không có Bus hay khởi động lại.
6. Bỏ khi mọi runtime là Kubernetes, không giá trị nào phải đổi mà không khởi động lại pod, và secret đã sống trong nền tảng. Giữ khi một runtime ngoài Kubernetes (VM, Compose ở production) phải dùng chung cùng nguồn cấu hình.

Chương 30 — Đào sâu: Khám phá dịch vụ

1. Đăng ký tức thì khi web server khởi động (Spring ép việc đó). Rồi cache phản hồi chỉ đọc của server (30 s), chu kỳ tải registry của gateway (30 s),

-
- và TTL cache của LoadBalancer (35 s): khoảng 95 s xấu nhất, thường bằng một nửa.
2. Lease hết hạn 90 s, rồi lượt quét loại bỏ (tới 60 s), rồi cache server 30 s, tải của client 30 s, cache LoadBalancer 35 s — tới 245 s. Tắt êm gửi DELETE và bỏ qua hai bước đầu (150 s), chỉ còn lại các cache.
 3. Gia hạn kỳ vọng mỗi phút: $5 \times 2 = 10$; ngưỡng $\lfloor 10 \times 0,85 \rfloor = 8$; server chỉ khỏe khi gia hạn nhiều hơn 8. Một client chết cho đúng 8, nên tự bảo toàn kích hoạt và không gì bị loại bỏ. `eureka.server.enable-self-preservation: false` (hoặc `renewal-percent-threshold` thấp hơn) sửa việc đó trên đội tàu nhỏ.
 4. Trạng thái tự báo của Eureka gác đường của gateway, vì `lb://` phân giải thành IP pod đã đăng ký và không bao giờ chạm tới Service Kubernetes nơi readiness được cường chế. Định tuyến tới `http://course-service:8082` đưa kube-proxy, và do đó readiness, trở lại đường đi.
 5. Compose không kích hoạt profile nào, nên tài liệu gốc áp dụng: Eureka client bật, route `lb://`, và `url` của Feign phân giải thành chuỗi rỗng, thứ Feign hiểu là “dùng phân giải cân bằng tải theo tên”. Dưới `k8s` thuộc tính đó là URL Service và Feign bỏ qua LoadBalancer hoàn toàn.
 6. Registry biết một instance chỉ qua điều instance tự nói về mình, và biết đến sự im lặng chỉ sau một lease nó không quét ngay; endpoint của Service do kubelet duy trì, thứ chủ động probe pod vài giây một lần và gỡ nó ngay khoảnh khắc probe thất bại. Tự báo với thời gian ân hạn so với quan sát bên ngoài không có ân hạn.

Chương 31 — Đào sâu: Gateway dưới tải

1. Chưa có gì được gửi đi: `chain.filter` trả về một Mono mô tả phần còn lại của chuỗi; nó chỉ chạy khi framework subscribe, và thread gọi đã rảnh. `doFilter` của filter servlet chặn tới khi lời gọi phía sau hoàn tất, nên thread bị giữ suốt request — chính mô hình mà event loop tồn tại để tránh.
2. Mọi request có socket thuộc vòng lặp đang khựng đều dừng — kể cả các route không hề chạm Redis — vì vòng lặp dùng chung giữa các route, không phải theo route. Reactor Netty chạy `max(cores, 4)` vòng lặp; trên máy 2 lõi là bốn, nên một vòng lặp bị chặn là một phần tư năng lực gateway.
3. Mười sáu: pool mặc định theo host là `max(cores, 8) \times 2`. Không có timeout phản hồi, 16 request treo giữ cả 16 kết nối và request thứ 17 đợi 45 s trong hàng acquire. Đặt `spring.cloud.gateway.httpclient.response-timeout` (hoặc tăng `pool.max-connections`) đổi con số; timeout mới là thứ sửa nguyên nhân.
4. Filter bọc mỗi lời gọi trong một `TimeLimiter Resilience4j` có mặc định 1 s. Lấn tải lên 1,5 s bị cắt, tính là thất bại, và sau đủ số lần, breaker mở chống

lại một service khỏe mạnh. Đặt `timeout-duration` của `TimeLimiter` instance đó ít nhất dài bằng `timeout` phản hồi.

5. Với hai replica trở lên, bucket trong bộ nhớ cho mỗi replica giới hạn riêng, nên một user được $N \times$ hạn ngạch. Redis giữ một bucket cho mỗi khóa, cập nhật nguyên tử bằng script Lua. Đánh đổi: Redis là phụ thuộc trên mọi request; mặc định của filter khi Redis không với tới là cho request qua (fail-open), việc phải là lựa chọn có chủ ý.
6. Mọi request đều không đơn giản, nên khi dev mỗi lời gọi API đi sau một preflight `OPTIONS`. `setMaxAge` cho trình duyệt cache kết quả preflight (mặc định của Chrome là 5 s). Trong cụm, frontend và /api chung một origin phía sau Traefik, nên không request nào xuyên origin và filter CORS không bao giờ được hỏi tới.

Chương 32 — Đào sâu: Gọi đồng bộ

1. (a) Không code nào gọi `AuthServiceClient` — nối nó vào một service như `UserLookupService`. (b) Tích hợp breaker của Feign đang tắt — đặt `circuitbreaker.enabled: true` dưới `spring.cloud.openfeign`. (c) Không có bean `CircuitBreakerFactory` — thêm dependency `spring-cloud-starter-circuitbreaker-resilience4j`. Riêng nữa, không gì tham chiếu instance `authService` — gắn `@CircuitBreaker` với `name = "authService"` lên phương thức gọi.
2. Chín: `minimum-number-of-calls` mặc định bằng cỡ cửa sổ, nên breaker không đánh giá cho tới kết quả thứ mười. Lời gọi 30 s là thành công vì `slow-call-duration-threshold` mặc định 60 s; chỉ lời gọi chậm hơn thế mới tính là chậm, và chỉ khi đạt ngưỡng tỷ lệ chậm (mặc định 100 %).
3. Trình duyệt (dài nhất) > `timeout` phản hồi của gateway > Feign read `timeout` × số lần thử (cộng backoff). Mỗi chặng phải bỏ cuộc trước bên gọi của nó, nếu không nó làm việc cho request đã bị bỏ rơi; và số lần retry nhân với `timeout` trong cùng, nên phải nằm trong ngân sách của gateway — $2 s \times 2$ lần dưới `timeout` gateway 5 s, không phải 3.
4. Một `NullPointerException` ở mọi bên gọi quên kiểm tra, chỉ lộ ra khi có sự cố; và không phân biệt được 404 (user thực sự không có) với một lần breaker từ chối, nên cả hai bị đối xử như nhau. `degraded(Long id, Throwable cause)` — fallback hai tham số — nhận nguyên nhân và có thể trả giá trị đã cache hay giá trị thay thế cho lỗi hạ tầng trong khi để 4xx lan tiếp.
5. `SecurityConfig` của `auth-service` chỉ cho phép đường dẫn công khai; `/users/{id}` cần ngữ cảnh đã xác thực mà `GatewayHeaderAuthFilter` của nó dựng từ `X-User-Id`, thứ Feign không gửi. Mỗi 401 là một ngoại lệ, nên mười

lần mở breaker chống lại một service khỏe. Sửa: một `RequestInterceptor` của Feign chuyển tiếp header danh tính, và `ignore-exceptions` cho 4xx để lỗi phía client không bao giờ tính là lỗi phụ thuộc.

6. $0,999^3 \approx 99,7\%$. Nâng lên bằng cách bỏ phụ thuộc khỏi đường request — bản sao cục bộ nuôi bằng sự kiện, hoặc cache đặt trước lời gọi — và làm các phụ thuộc còn lại thành tùy chọn với fallback có kiểu để lỗi của chúng làm suy giảm response thay vì làm nó thất bại.

Chương 33 — Đào sâu: Dữ liệu xuyên service

1. Thứ tự A: transaction commit, rồi JVM chết trước khi bộ đệm producer flush — user đã kích hoạt, không có sự kiện, không bao giờ có học viên. Thứ tự B: bộ đệm flush và broker ack, rồi commit thất bại — course-service tạo học viên cho một user đã bị rollback. B tạo ra dòng ma.
2. Producer không lấy được metadata, chặn trong `max.block.ms` (60 000 ms), rồi ném ngoại lệ; ngoại lệ thoát khỏi `register()`, transaction rollback, và user không được tạo. Với outbox, request ghi một dòng user và một dòng outbox trong một commit và trả về sau vài mili giây; relay thử lại việc gửi ở nền.
3. Hai replica chạy cùng scheduler. Không có khóa, cả hai chọn cùng các dòng chưa phát và cả hai phát chúng — một bản sao cho mọi sự kiện, mỗi giây. FOR UPDATE bắt replica thứ hai chờ; SKIP LOCKED bắt nó lấy các dòng kế tiếp thay vào đó, nên chúng chia việc mà không chồng lấn.
4. Thứ tự theo key. Nếu dòng 1 của user 42 thất bại và vòng lặp continue sang dòng 2 cùng user, dòng 2 được phát trước; nhíp kế phát dòng 1 phía sau nó, và consumer thấy “updated” trước “activated”. Dừng lô giữ thứ tự partition bằng thứ tự outbox.
5. Không gì: `ts.user.activated` bắn một lần, và không có `ts.user.updated`, nên `student.email` đóng băng từ lúc kích hoạt. Thay đổi: phát một dòng outbox với `eventType=UserUpdated` ở bất cứ đâu lưu hồ sơ; làm `UserActivatedListener` upsert (tìm-hoặc-tạo, rồi đặt trường); cho nó đăng ký cả hai topic hoặc thêm listener thứ hai.
6. Trạng thái: ACTIVE → DELETING → đã xóa, hoặc về lại ACTIVE. Sự kiện: auth phát `UserDeletionRequested` qua outbox; course phát `StudentDataDeleted` hoặc, khi hỏng, `StudentDataDeletionFailed` sau khi xóa học viên, ghi danh, bài nộp và object MinIO. Bù: khi thất bại auth khôi phục ACTIVE và cảnh báo. DELETING là khóa ngữ nghĩa: nó từ chối đăng nhập và ghi danh mới trong lúc saga chạy, nên không bước nào thao tác trên dữ liệu bước khác đang xóa.

Chương 34 — Đào sâu: Bảo đảm giao nhận Kafka

1. `enable.idempotence=true` (broker khử trùng các lần thử lại của producer), `acks=all`, retries không giới hạn trong `delivery.timeout.ms` — toàn bộ là mặc định của client Kafka 3.x mà dự án chưa từng đặt. Trên Compose có một broker và replicas: 1, nên “mọi replica đồng bộ” là một cái đĩa; thiết lập này chỉ mua được độ bền khi `replication.factor=3` và `min.insync.replicas=2` tồn tại.
2. Một lô được poll; listener gửi mail từng cái trên luồng poll; xử lý vượt `max.poll.interval.ms` (300 000 ms); broker đuổi consumer và gán lại partition từ offset đã commit cuối; chủ mới xử lý cùng các record trong khi chủ cũ vẫn chạy nốt vòng lặp.
3. Nếu lượt giành commit một mình rồi lần ghi nghiệp vụ thất bại, lần thử lại thấy id đã được giành, trả về, và offset tiến lên — thay đổi mất vĩnh viễn trong khi bảng nói nó đã xảy ra. Trong một transaction, thất bại rollback cả lượt giành, nên lần thử lại xử lý được.
4. Chờ đợi chỉ giúp lỗi tạm thời: một SMTP relay quay lại, một nháy mạng. `NullPointerException` hay lỗi giải tuần tự là tất định; thử lại ba lần chỉ trì hoãn cùng thất bại. Nó bị loại trừ, nên đi thẳng tới DLT ngay lần thử đầu.
5. `KafkaAutoConfiguration` của Boot lùi lại khi có bean `ProducerFactory`, nên `spring.kafka.producer.*` trong `course-service.yml` bị `course-service` bỏ qua. Sửa: xóa `KafkaProducerConfig` (factory của Boot đọc cùng các serializer từ YAML) — hoặc giữ bean và xóa khối YAML.
6. Retention là 168 giờ; chín ngày dài hơn. Offset đã commit của group giờ trở trước record cũ nhất còn giữ, nên `auto-offset-reset: earliest` có hiệu lực và nó tiếp tục từ record còn sống cũ nhất — hai ngày sự kiện đã mất. Với group id mới không có offset commit nào; `earliest` khiến nó xử lý toàn bộ bảy ngày còn giữ, gửi lại mọi mail trong đó.

Chương 35 — Đào sâu: File ở quy mô lớn

1. `endpointOverride`: client dùng endpoint (`minio:9000`), presigner dùng `public-endpoint`. Nếu presigner ký với tên nội bộ, mọi link tải xuống sẽ nhúng một host trình duyệt không phân giải được — và viết lại sau đó làm hỏng chữ ký.
2. SigV4 ký một canonical request bao gồm các header được ký, và Host luôn nằm trong số đó (`X-Amz-SignedHeaders=host`). Link ký cho `minio:9000` rồi viết lại sang host khác trình ra một canonical request khác; MinIO tính lại chữ ký, không khớp, và trả về 403 `SignatureDoesNotMatch`.

-
3. Transaction chỉ bao phủ insert vào Postgres; putObject là một lời gọi HTTP bên ngoài nó. save thất bại thì dòng bị rollback trong khi object vẫn nằm trong bucket dưới một key không dòng nào ghi lại — một object mồ côi.
 4. Rê: liệt kê mọi lần nộp của một học sinh cho một bài tập (một ListObjectsV2 với tiền tố). Đắt: tìm mọi file của một học sinh qua các bài tập (không tiền tố nào khớp; quét toàn bộ hoặc truy vấn cơ sở dữ liệu). UUID khiến mọi key duy nhất, nên không upload nào ghi đè upload khác; versioning chỉ bảo vệ trước những lần ghi đè không thể xảy ra.
 5. Nộp text thành công (không gọi kho). Liệt kê thành công và trả về link (ký là tính toán cục bộ). Mở link thất bại (MinIO phải phục vụ nó). Khởi động lại course-service thất bại: kiểm tra bucket trong @PostConstruct ném lỗi khi connection refused.
 6. Key được dựng phía server từ assignmentId và studentId của chính người gọi, và lưu trên một dòng PENDING_FILE thuộc học sinh đó; completeUpload kiểm tra dòng thuộc về người gọi và HEAD trên key của *chính* dòng đó thành công. URL cho key của học sinh khác không gắn được vào dòng khác vì dòng không bao giờ nhận key do client cung cấp.

Chương 36 — Đào sâu: Từ điển trên MongoDB

1. filter của stage FETCH loại chúng. Index createdByUserId chỉ thu hẹp về lát của người dùng; một \$regex không neo, không phân biệt hoa thường không có ký tự đầu nào để seek, nên mọi document đã nạp đều bị thử. Neo pattern (^clou) với collation phù hợp cho phép index trên word phục vụ nó như một khoảng.
2. Mọi kết quả khớp — có thể là cả 3,000 — trở về từ Mongo; toResponse rồi chạy hai truy vấn word_links mỗi kết quả (khoảng 6,000); trang được dựng trong Java bằng subList sau khi sort toàn bộ danh sách.
3. Transaction nhiều document cần replica set; dự án chạy một mongod standalone. Crash sau khi xóa định nghĩa và trước khi xóa các liên kết để lại cạnh trở vào một từ đã mất. findByIdAndCreatedByUserId(...).orElse(null) rồi filter(Objects::nonNull) âm thầm bỏ các node như vậy.
4. BsonMaximumSizeExceededException (giới hạn 16 MB). Sáu ảnh khoảng 2–3 MB mỗi cái, phình thêm một phần ba khi base64, vượt 16,793,600 byte. Quy tắc của Chương 35: byte trong kho đối tượng, key trong cơ sở dữ liệu — lưu ảnh trong MinIO và giữ key của chúng trong images.
5. Text index tách token và rút gốc các từ trọn vẹn; “lou” không phải token của “cloud”, nên không có kết quả. Khớp giữa từ cần quét regex hoặc index n-gram/trigram (Atlas Search, hay pg_trgm trong Postgres).

6. dictionary-service hỏng trên mọi endpoint; mọi service và topic khác không bị ảnh hưởng (không kết nối chung, không bên gọi, không sự kiện). Thêm một lệnh ping Mongo vào readiness probe sẽ đưa pod ra khỏi danh sách endpoint của gateway, biến 500 thành một 503 sạch sẽ ở biên.

Chương 37 — Đào sâu: Khả năng quan sát

1. Phía producer: `spring.kafka.template.observation-enabled` (vắng mặt trong mọi tài liệu cấu hình); phía consumer: `spring.kafka.listener.observation-enabled`. Thuộc tính chỉ cấu hình `KafkaTemplate` *tự cấu hình*; `auth-service` dùng cái đó, nên thuộc tính sửa được nó. `Course-service` tự dựng template trong `KafkaProducerConfig`, thứ bỏ qua thuộc tính và cần `setObservationEnabled(true)` trong code.
2. Nó tiếp tục trace: mặc định của Boot 3 là *đọc* W3C, B3 và B3-multi và *phát* W3C, quyết định bởi các mặc định của `management.tracing.propagation.Header` rồi gateway là `transparent`, cùng trace id và một span id cha mới.
3. Exposure là danh sách tên; endpoint chỉ tồn tại khi một registry hiển thị định dạng đó nằm trên classpath. Không service nào phụ thuộc `micrometer-registry-prometheus`, nên Actuator không có gì để lộ ra dưới tên đó. Thêm đúng một artifact ấy (BOM Boot quản lý phiên bản) làm dòng exposure hiện có thành thật.
4. Độ trễ: không đổi — span được xếp hàng và gửi bởi thread nền. Heap: bị chặn — hàng đợi giữ 10.000 span và bỏ khi tràn kèm cảnh báo có đếm. Span: những cái sinh ra trong giờ đó mất; không có replay. Với reporter đồng bộ, *độ trễ* sẽ đổi: mọi request sẽ chặn trên một lời gọi HTTP thất bại tới Zipkin cho tới khi timeout.
5. Một phần trăm — lấy mẫu head-based quyết định tại gốc trước khi biết kết quả, nên lỗi được giữ cùng tỷ lệ với thành công. Nâng lên 1.0 chỉ cho lỗi đòi lấy mẫu tail-based, thứ cần một OpenTelemetry Collector đệm tròn trace trước kho; đường trực tiếp Brave-tới-Zipkin không làm được.
6. E-mail là giá trị tag không giới hạn; mỗi giá trị khác nhau tạo một chuỗi thời gian mới trong registry metric và một giá trị được đánh index mới trong kho trace. Bộ nhớ lớn theo số người dùng cho tới khi `/actuator/metrics` bỏ lê và pod bị `OOMKilled`. Quy tắc: tag mang *mẫu và enum* (mẫu route, status, outcome); định danh đi vào annotation của span hay dòng log, những thứ không được đánh index theo giá trị.

Chương 38 — Đào sâu: Trình duyệt gặp gateway

1. Khóa auth-storage trong localStorage, do persist của Zustand ghi. Một dòng: `JSON.parse(localStorage.getItem('auth-storage')).state.refreshToken`. Có nó, script (hay bất kỳ ai nó gửi tới) có thể gọi `/api/auth/refresh` suốt bảy ngày và đúc access token mới tùy ý, từ bất kỳ máy nào.
2. Một 401 từ `/refresh` đi qua `apiClient` sẽ vào `interceptor` phản hồi; `_retry` chưa đặt trên request mới đó, nên nó sẽ đọc refresh token và POST `/refresh` lần nữa qua `apiClient`; 401 đó lại vào `interceptor` — một chuỗi lời gọi refresh không giới hạn, mỗi cái sinh ra cái tiếp, cho tới khi đóng tab. `axios` trần không có `interceptor`, nên 401 của nó tới `catch` và đăng xuất.
3. Cả bốn nhận 401 và mỗi cái tự đặt `_retry`; bốn POST tới `/refresh` đua nhau. Cái đầu thành công và xoay token; cái thứ hai đưa ra token cũ giờ đã vô hiệu và nhận 401 từ auth-service; nó rơi vào `catch` — bước 4, `logout()` và chuyển hướng cứng — trong khi lần replay của request đầu thậm chí có thể thành công. Single-flight: một promise, khởi động bởi 401 đầu tiên, các cái còn lại `await` nó; một POST, một lần xoay, bốn lần replay cùng token mới.
4. (a) Một endpoint có lỗi hổng hay bị lãng quên ở nơi khác dưới `/api` không bao giờ nhận cookie, nên lỗi CSRF hay rò rỉ log ở đó không chạm được nó; (b) cookie không dính kèm các lời gọi API thường, nên nó không bao giờ xuất hiện trong access log của gateway, proxy hay lưu lượng trình duyệt cho `/api/courses`. Trình duyệt so Path với URL nó yêu cầu, tức `/api/auth/refresh` của gateway; `/refresh` đã bỏ tiền tố của auth-service không bao giờ là đường dẫn trình duyệt thấy.
5. Không mode nào: proxy dev của Vite và Traefik trong cụm đều giữ trang và API trên một origin, nên không request nào là cross-origin và không preflight nào xảy ra. Phục vụ bundle đã build từ một origin khác — một CDN, hay một static host ở cổng khác không qua proxy — sẽ làm mọi lời gọi API thành cross-origin và cấu hình đó thành quyết định.
6. CORS được trình duyệt cưỡng chế sau khi phản hồi tới: gateway đã xử lý request và trả 200, rồi trình duyệt từ chối trao body cho script vì phản hồi thiếu `Access-Control-Allow-Origin` chấp nhận được. So sánh: header `Origin` của request với danh sách `allowed-origins` (chuỗi chính xác, cả scheme và cổng); `Access-Control-Allow-Credentials` với việc request có `credentialed` hay không; và cho preflight, `Access-Control-Allow-Headers` với các header request thực sự gửi (`Authorization`, `Content-Type`).

Thuật ngữ

Actuator Các endpoint vận hành của Spring Boot (/actuator/health, /metrics, /prometheus); mở ra và phân quyền là hai quyết định riêng (Ch. 1).

Advertised listener Địa chỉ mà broker Kafka báo client dùng sau bước bootstrap — nguyên nhân thường gặp của hiện tượng kết nối được rồi timeout (Ch. 10).

Algorithm confusion (nhầm lẫn thuật toán) Tấn công sửa header alg của token để bên xác minh áp dụng HMAC với khóa công khai làm secret; bị chặn khi loại khóa, không phải header, chọn thuật toán (Ch. 28).

Artifact Bất cứ thứ gì một build sinh ra đáng giữ lại (jar, image, báo cáo); phải bất biến và truy nguyên được (Ch. 21).

Auto-configuration (tự cấu hình) Spring Boot áp dụng cấu hình có điều kiện dựa trên classpath, thuộc tính và bean của người dùng (Ch. 1).

Backend for frontend Service nhỏ ghép nhiều lời gọi service thành một response theo hình dạng client; chỗ đúng cho việc gộp mà gateway không được làm (Ch. 31).

Bootstrap servers Các địa chỉ Kafka ban đầu mà client liên hệ để lấy metadata; mọi lưu lượng sau đó dùng advertised listener (Ch. 10).

Bulkhead Giới hạn số lời gọi đồng thời tới một phụ thuộc (semaphore hoặc thread pool riêng) để một phụ thuộc chậm không tiêu hết thread của bên gọi (Ch. 32).

Cipher value (giá trị mã hóa) Giá trị config-server có tiền tố {cipher}, mã hóa bằng khóa của server để repository chỉ giữ bản mã; được giải mã lúc phục vụ (Ch. 29).

Circuit breaker (bộ ngắt mạch) Máy trạng thái trên cửa sổ trượt các kết quả gọi: CLOSED cho qua, OPEN từ chối tức thì trong một khoảng chờ, HALF_OPEN thăm dò bằng vài lời gọi (Ch. 32).

ClusterIP Kiểu Service mặc định: một IP ảo ổn định, chỉ tới được từ bên trong cụm (Ch. 18).

Consumer group (consumer group) Một nhóm consumer Kafka có tên, chia nhau các partition của một topic; mỗi nhóm giữ offset riêng (Ch. 10).

Consumer lag (độ trễ tiêu thụ) Khoảng cách giữa offset mới nhất của partition và offset đã commit của group; đo bằng giây, đặt cảnh báo trên nó (Ch. 34).

-
- Controller (k8s)** Một vòng lặp điều hòa hội tụ trạng thái thực tế về trạng thái mong muốn cho một kind object (Ch. 14).
- CORS / preflight** Cơ chế trình duyệt cưỡng chế để nói lòng chính sách same-origin; request cross-origin không đơn giản tốn thêm một vòng OPTIONS trừ khi được cache — và không bao giờ xảy ra khi trang và API chung một origin (Ch. 31, 38).
- CQRS** Tách lệnh/truy vấn xuyên service: ghi đi tới service sở hữu, đọc đi tới một mô hình dựng từ sự kiện của nó (Ch. 33).
- CrashLoopBackOff** Trạng thái pod: container liên tục thoát; độ trễ khởi động lại tăng gấp đôi mỗi lần (Ch. 24).
- CRD / Operator** Các kind resource tùy biến cộng một controller cho chúng — cách Strimzi quản lý Kafka theo kiểu khai báo (Ch. 25).
- Dead-letter topic** (dead-letter topic) Nơi consumer chuyển những thông điệp xử lý thất bại nhiều lần (Ch. 11).
- Deployment** Controller workload cho các replica thay thế được cho nhau; quản lý ReplicaSet; cho phép cập nhật cuốn chiếu và rollback (Ch. 15).
- Digest** Mã băm nội dung bất biến của một image (@sha256: ...); tag dịch chuyển, digest thì không (Ch. 13).
- Dual write** (ghi kép) Ghi vào hai hệ thống (cơ sở dữ liệu và Kafka) không có transaction chung; cái này có thể thành công trong khi cái kia thất bại (Ch. 33).
- Endpoint (k8s)** Một pod Ready đứng sau một Service; danh sách sống được duy trì theo label selector và readiness (Ch. 18).
- Eureka** Service registry của Netflix/Spring Cloud: đăng ký, heartbeat, cache phía client (Ch. 3).
- Event loop** (vòng lặp sự kiện) Nhóm thread nhỏ, cố định của Reactor Netty, mỗi thread ghép kênh nhiều socket bằng I/O không chặn; một lời gọi chặn làm khựng mọi request trên vòng lặp đó (Ch. 31).
- explain (Mongo)** find(...).explain("executionStats"): kế hoạch truy vấn với các stage IXSCAN/FETCH và totalDocsExamined SO với nReturned — thứ đầu tiên cần đọc khi một truy vấn chậm (Ch. 36).
- Fallback** Giá trị hay hành động thay thế khi lời gọi được bảo vệ thất bại hoặc bị từ chối; nên là giá trị cache, suy giảm-nhưng-có kiểu, rỗng, hoặc lỗi có kiểu — không bao giờ null (Ch. 32).
- Flyway** Các migration SQL có phiên bản, bất biến, áp dụng lúc khởi động và ghi vào một bảng lịch sử (Ch. 7).
- GitOps** Mô hình triển khai trong đó một agent trong cụm kéo trạng thái mong

muốn từ git và hội tụ cụm về đó (Ch. 26).

Head-based / tail-based sampling (lấy mẫu theo đầu / theo đuôi) Quyết định tại span gốc có giữ trace hay không (rẻ, mù với kết quả) so với quyết định sau khi biết trọn trace (giữ lỗi và request chậm; cần collector) (Ch. 37).

HS256 / RS256 Thuật toán ký JWT. HS256: một secret dùng chung vừa ký vừa xác minh. RS256: khóa riêng RSA ký, khóa công khai xác minh — một bên phát hành, bao nhiêu bên xác minh cũng được (Ch. 28).

HttpOnly cookie Cookie mà JavaScript không đọc được; cùng Secure, SameSite và một Path hẹp, nó là chỗ ở an toàn cho refresh token (Ch. 38).

Idempotent consumer (consumer lũy đẳng) Listener giành mỗi id sự kiện trong cùng transaction với hiệu ứng phụ, nên giao lại là no-op (Ch. 34).

Idempotent producer (producer lũy đẳng) Producer Kafka mà các lần thử lại được broker khử trùng theo số thứ tự; mặc định từ Kafka 3.0 (Ch. 34).

Ingress Object định tuyến HTTP tầng 7 (host/đường dẫn → Service), được thực thi bởi một Ingress controller như Traefik (Ch. 18).

Jib Plugin Maven build image OCI theo layer mà không cần Docker daemon (Ch. 13).

JWKS JSON Web Key Set: các khóa công khai mà bên phát hành công bố tại một URL quy ước để bên xác minh tải và cache (Ch. 28).

JWT Token danh tính tự chứa, có chữ ký: header.payload.signature; kiểm tra được mà không cần cơ sở dữ liệu (Ch. 5; ở quy mô lớn, Ch. 28).

Keyset pagination (phân trang theo khóa) Phân trang bằng “các dòng sau khóa này” trên cột sort có index thay vì skip, để trang sâu tồn bằng trang đầu (Ch. 36).

kid Key id trong header JWT, đặt tên khóa đã ký nó; thứ cho phép xoay khóa trong khi token cũ còn lưu hành (Ch. 28).

KRaft Cơ chế đồng thuận Raft tích hợp sẵn của Kafka thay cho ZooKeeper (Ch. 10).

kubelet Agent trên mỗi node, chạy pod và thực thi probe (Ch. 14).

Kustomize Ghép manifest không dùng template: base, overlay, patch; tích hợp sẵn trong kubectl (Ch. 20).

Lease (Eureka) Đăng ký của một instance, gia hạn bằng heartbeat mỗi 30 s và hết hạn sau 90 s im lặng; server quét mỗi 60 s, nên instance chết có thể lưu lại vài phút (Ch. 30).

Liveness probe “Khởi động lại tôi nếu cái này hỏng” — probe cần dè dặt nhất (Ch. 17).

Multipart upload Cơ chế S3 chia một object lớn thành các phần ít nhất 5 MB,

-
- upload song song và ráp phía server; phần bị bỏ dở cần quy tắc lifecycle hủy (Ch. 35).
- Namespace** Phân vùng của cụm; phạm vi cho tên và RBAC (Ch. 14).
- OOMKilled** Exit 137: kernel giết container vì vượt giới hạn bộ nhớ; không có trace nào từ phía ứng dụng (Ch. 17).
- Outbox pattern** Làm cho ghi DB + phát sự kiện trở nên nguyên tử trên thực tế bằng cách ghi sự kiện vào một bảng trong cùng giao dịch (Ch. 11; Ch. 33).
- Partition** (phân vùng) Đơn vị thứ tự và song song của Kafka; một chuỗi bản ghi bất biến, có thứ tự (Ch. 10).
- Path-style addressing** (địa chỉ kiểu đường dẫn) Dạng URL S3 với bucket nằm trong đường dẫn (host/bucket/key) thay vì là subdomain DNS; MinIO cần dạng này khi không có wildcard DNS (Ch. 35).
- Phantom token** Định danh mờ bên ngoài, JWT đầy đủ bên trong; gateway đổi cái này lấy cái kia qua một kho nhanh, cho token ngoài mỏng và thu hồi tức thì với cái giá là một phụ thuộc mới (Ch. 28).
- Poison pill** Một thông điệp giải tuần tự thất bại mãi mãi, làm kẹt partition của nó trừ khi xử lý lỗi bỏ qua nó (Ch. 11).
- Presigned URL** (presigned URL) Một link có chữ ký, giới hạn thời gian, cho phép trình duyệt truy cập trực tiếp một object trên S3/MinIO (Ch. 9).
- Probe (startup/readiness/liveness)** Các kiểm tra sức khỏe của kubelet, lần lượt gác việc khởi động lại, nhận lưu lượng, và thời gian ân hạn lúc boot (Ch. 17).
- Property source precedence** (thứ tự ưu tiên nguồn thuộc tính) Danh sách có thứ tự các nguồn cấu hình của Spring Boot; biến môi trường đứng trên tài liệu từ config-server, tài liệu đó đứng trên YAML đóng gói (Ch. 29).
- PV / PVC / StorageClass** Lưu trữ đã cấp phát, yêu cầu của một workload lên nó, và nhà máy tạo PV theo nhu cầu (Ch. 19).
- RBAC** Kiểm soát truy cập theo vai trò: ServiceAccount (danh tính) + Role (động từ trên resource) + RoleBinding (mối nối) (Ch. 22).
- Reconciliation** (điều hòa) Vòng lặp điều khiển: so sánh trạng thái mong muốn và thực tế, hành động trên phần chênh lệch, mãi mãi (Ch. 14).
- RED / USE** Rate, Errors, Duration cho service theo request; Utilization, Saturation, Errors cho tài nguyên — hai bộ từ vựng dashboard (Ch. 37).
- Refresh scope** Các bean được dựng lại khi /actuator/refresh để thay đổi cấu hình có hiệu lực không cần khởi động lại; theo từng instance trừ khi Spring Cloud Bus phát sóng (Ch. 29).
- Registry** Kho lưu image nói giao thức Docker/OCI; giao diện giữa bên build và cụm (Ch. 13).

- Relaxed binding** Cách Spring ánh xạ `SPRING_DATASOURCE_URL` thành `spring.datasource.url` (Ch. 1).
- ReplicaSet** Bộ đếm giữ N pod của một template luôn sống; do Deployment quản lý, giữ lại để rollback (Ch. 15).
- Retry storm** (bão retry) Các lần retry đồng bộ từ nhiều client nhân tải lên một phụ thuộc đang lỗi; ngăn bằng jitter, ngân sách retry và chỉ retry lời gọi lũy đẳng (Ch. 32).
- Retry topic** Topic theo độ trễ mà một record thất bại được phát lại vào (`-retry-2000, ...`), kết thúc ở dead-letter topic; `@RetryableTopic` của Spring (Ch. 34).
- Rolling update** (cập nhật cuốn chiếu) Thay pod từng phần, gác bằng readiness, để service không bao giờ dừng hẳn (Ch. 15).
- Saga** Một lần ghi xuyên service dưới dạng chuỗi transaction cục bộ, mỗi bước có hành động bù nếu bước sau thất bại (Ch. 33).
- Secret (k8s)** Object key/value theo namespace cho thông tin xác thực; mã hóa base64, được bảo vệ bởi RBAC chứ không phải mật mã học (Ch. 16).
- Self-preservation** (chế độ tự bảo toàn) Chế độ Eureka vào khi gia hạn rơi dưới 85 % kỳ vọng; loại bỏ dừng hoàn toàn — kích hoạt chỉ bởi một crash trên đội tàu năm instance (Ch. 30).
- Service (k8s)** Tên ổn định + IP ảo trên một tập pod chọn theo label, lọc theo readiness (Ch. 18).
- ServiceAccount** Một danh tính phi con người để truy cập API — thứ Jenkins dùng để deploy (Ch. 22).
- SigV4** AWS Signature Version 4: một chuỗi HMAC trên ngày, region và dịch vụ ký một canonical request luôn bao gồm header `Host` — presigned URL bị gắn chặt với host nó được ký cho (Ch. 35).
- Single-flight refresh** (refresh một chuyến) Một promise chờ dừng chung để các 401 đồng thời chỉ kích hoạt đúng một lần refresh token và tắt cả replay với kết quả của nó; bắt buộc khi refresh token có xoay vòng (Ch. 38).
- StatefulSet** Controller workload cho các replica có trạng thái: tên ổn định, PVC riêng từng replica, thay thế theo thứ tự (Ch. 15).
- Text index (Mongo)** Index tách token và rút gốc các trường chuỗi có trọng số, truy vấn qua `$text/TextCriteria`; khớp từ trọn vẹn, không khớp chuỗi con (Ch. 36).
- Token bucket** Thuật toán rate limit: token nạp lại đều dẫn tới một sức chứa burst và mỗi request tiêu một; Spring Cloud Gateway giữ bucket trong Redis (Ch. 31).

TOTP Mật khẩu một lần theo thời gian: bí mật chia sẻ + đồng hồ, RFC 6238 (Ch. 5).

traceparent Header W3C Trace Context mang trace id, span id cha và cờ lấy mẫu giữa các service; Boot 3 phát nó và cũng chấp nhận B3 (Ch. 37).

Trusted edge (biên tin cậy) Gateway kiểm tra JWT một lần và chuyển tiếp các header danh tính; các service tin chúng — chỉ đúng khi không có đường nào đi vòng qua gateway (Ch. 5).

Bảng tra nhanh

kubectl — hằng ngày

```
kubectl -n ts get pods           # the first command, always
kubectl -n ts get pods -w       # ...and watch it change
kubectl -n ts describe pod <pod> # platform's view; read Events
kubectl -n ts logs <pod>        # app's view
kubectl -n ts logs <pod> --previous # the container that just died
kubectl -n ts exec -it <pod> -- sh # see the world from inside
kubectl -n ts get endpoints <svc> # is anyone behind the Service?
kubectl -n ts get events --sort-by=.lastTimestamp | tail
```

kubectl — deploy và rollback

```
kubectl apply -k deploy/k8s/base # converge on the manifests
kubectl kustomize deploy/k8s/base | less # render without applying
kubectl -n ts set image deployment/<s> <s>=localhost:5000/<s>:<tag>
kubectl -n ts rollout status deployment/<s> --timeout=300s
kubectl -n ts rollout undo deployment/<s> # back to previous ReplicaSet
kubectl -n ts rollout restart deployment/<s> # re-read Secrets, re-pull
kubectl -n ts get rs -l app=<s> # revision history, live
```

kubectl — dung lượng và RBAC

```
kubectl top pods -n ts # actual vs requested
kubectl -n ts get pvc; kubectl get pv # storage and its backing
kubectl auth can-i <verb> <resource> -n ts \
  --as system:serviceaccount:ts:jenkins # test permissions dry
```

docker — các dịch vụ trên host ts-server

```

docker ps                                # jenkins, registry up?
docker logs -f jenkins
docker restart jenkins
curl -s localhost:5000/v2/_catalog      # what the registry holds
curl -s localhost:5000/v2/<name>/tags/list
docker system df; docker system prune  # disk hygiene (careful)

```

maven — máy local và CI

```

mvn -B -pl <modules> -am verify        # build listed + deps
mvn -B -pl course-service -am jib:build # image for one service
mvn -B jib:build -Djib.to.tags=<sha>    # extra tag, as CI does
mvn dependency:tree -pl course-service  # who pulled that jar in
mvn spring-boot:run -pl course-service  # run one service raw

```

compose — stack dev

```

docker compose up -d postgres-course kafka minio # infra only
docker compose up --no-deps course-service       # one service, own terminal
docker compose logs -f --tail=100 <svc>
docker compose down                             # keep volumes
docker compose down -v                          # DESTROY volumes too

```

k3s — bản thân cụm

```

sudo systemctl status k3s
sudo systemctl restart k3s          # rereads registries.yaml; pods bounce
sudo ls /var/lib/rancher/k3s/storage/ # PVs as directories
kubectl get nodes -o wide

```

kafka — broker trong cụm (Ch. 25)

```

kubectl -n ts get kafka,kafkanodepools,kafkatopics # operator's view
kubectl -n ts logs ts-kafka-dual-role-0 --tail=50  # the broker itself
# a throwaway pod with the Kafka CLI, same network position as services:

```

```
kubect1 -n ts run kcli --rm -it --restart=Never \  
  --image=quay.io/strimzi/kafka:1.2.0-kafka-4.3.1 -- sh  
B=ts-kafka-kafka-bootstrap:9092  
bin/kafka-topics.sh --bootstrap-server $B --list  
bin/kafka-console-consumer.sh --bootstrap-server $B \  
  --topic ts.user.registered --from-beginning  
bin/kafka-consumer-groups.sh --bootstrap-server $B \  
  --describe --group notification-group      # lag per partition  
kubect1 -n ts port-forward svc/maildev 1080:1080    # caught mail UI
```

Các file cấu hình

Các file đúng như chúng đang nằm trong repository tại thời điểm build — được chèn thẳng từ mã nguồn, nên phụ lục này không thể trôi khỏi thực tế. Bình luận từng dòng nằm ở các chương; đây là bản tham chiếu để đọc khi không có máy.

Jenkinsfile

```

pipeline {
    agent any

    options {
        timestamps()
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }

    triggers { pollSCM('H/5 * * * *') }

    environment {
        JAVA_HOME = "${env.JDK25_HOME}"
        PATH      = "${env.JDK25_HOME}/bin:${env.PATH}"
        TAG       = "${env.GIT_COMMIT.take(7)}"
        NS        = 'ts'
        REGISTRY  = 'localhost:5000'
        // frontend is in SERVICES (kubect1) but not MODULES (Maven): it is built by
        // the 'Frontend image' stage with docker, not by Jib.
        SERVICES = 'config-server discovery-server course-service api-gateway auth-service dictionary-
        ↪ service notification-service frontend'
        MODULES  = 'common,config-server,discovery-server,course-service,api-gateway,auth-service,
        ↪ dictionary-service,notification-service'
    }

    stages {
        stage('Build') {
            steps {
                dir('Project/TeacherSupporter') {
                    sh 'mvn -B -DskipTests -pl ${MODULES} -am verify'
                }
            }
        }

        stage('Push images') {
            steps {
                dir('Project/TeacherSupporter') {
                    // 'package' before jib:build is load-bearing: a bare goal runs no

```

```

    // lifecycle, so sibling modules (common) are never built in this
    // session and Jib cannot resolve com.ts:common from the reactor.
    sh 'mvn -B -DskipTests -pl ${MODULES} -am package jib:build -Djib.to.tags=${TAG}'
  }
}
}

stage('Frontend image') {
  steps {
    dir('Project/TeacherSupporter') {
      // Multi-stage Dockerfile: node builds dist/, nginx ships it. The
      // registry is localhost:5000 from the host's point of view and the
      // container shares the host network, so push needs no extra config.
      sh '''
        set -eu
        docker build -t ${REGISTRY}/frontend:${TAG} -t ${REGISTRY}/frontend:latest frontend
        docker push ${REGISTRY}/frontend:${TAG}
        docker push ${REGISTRY}/frontend:latest
      '''
    }
  }
}

stage('Deploy') {
  steps {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubeconfig-ts', variable: 'KUBECONFIG')]) {
        sh '''
          set -eu
          kubectl apply -k deploy/k8s/base
          for s in ${SERVICES}; do
            kubectl -n ${NS} set image deployment/${s} ${s}=${REGISTRY}/${s}:${TAG}
          done
          for s in ${SERVICES}; do
            kubectl -n ${NS} rollout status deployment/${s} --timeout=300s
          done
        '''
      }
    }
  }
}

post {
  success { echo "Deployed ${TAG} to namespace ${NS}." }
  failure {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubeconfig-ts', variable: 'KUBECONFIG')]) {
        sh 'kubectl -n ${NS} get pods || true'
      }
    }
  }
}
}

```

deploy/jenkins/Dockerfile

```
# Jenkins controller for ts-server, with the build toolchain baked in.
#
# Homelab compromise, stated openly: builds run ON the controller (2 executors),
# not on agent pods. On a 2-core box, agent pods would cost more CPU than they
# save. Migrating to the Kubernetes plugin is a later, self-contained exercise.
#
#   docker build -t ts-jenkins:1 deploy/jenkins
#   see run.sh for the docker run line
FROM jenkins/jenkins:lts-jdk21

USER root

# --- JDK 25 (Temurin) -- the project targets Java 25 -----
RUN apt-get update && apt-get install -y --no-install-recommends wget gpg apt-transport-https \
  && wget -qO- https://packages.adoptium.net/artifactory/api/gpg/key/public \
    | gpg --dearmor -o /usr/share/keyrings/adoptium.gpg \
  && echo "deb [signed-by=/usr/share/keyrings/adoptium.gpg] https://packages.adoptium.net/artifactory/
    ↪ deb $(. /etc/os-release && echo $VERSION_CODENAME) main" \
    > /etc/apt/sources.list.d/adoptium.list \
  && apt-get update && apt-get install -y --no-install-recommends temurin-25-jdk \
  && rm -rf /var/lib/apt/lists/*

# --- Maven 3.9 (Debian's apt maven is 3.8, too old for some plugins) -----
ARG MAVEN_VERSION=3.9.9
RUN wget -qO /tmp/mvn.tgz https://archive.apache.org/dist/maven/maven-3/${MAVEN_VERSION}/binaries/
  ↪ apache-maven-${MAVEN_VERSION}-bin.tar.gz \
  && tar -xzf /tmp/mvn.tgz -C /opt && ln -s /opt/apache-maven-${MAVEN_VERSION} /opt/maven \
  && rm /tmp/mvn.tgz

# --- kubectl: the deploy stage talks to k3s with this -----
RUN wget -qO /usr/local/bin/kubectl \
  "https://dl.k8s.io/release/$(wget -qO- https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/
  ↪ kubectl" \
  && chmod +x /usr/local/bin/kubectl

# --- Docker CLI (client only): the frontend image is a classic Dockerfile ---
# build, which Jib cannot do. Only the CLI is installed; it talks to the
# HOST's daemon through /var/run/docker.sock mounted by run.sh. No daemon
# inside the container, no docker-in-docker.
RUN install -m 0755 -d /etc/apt/keyrings \
  && wget -qO- https://download.docker.com/linux/debian/gpg | gpg --dearmor -o /etc/apt/keyrings/
  ↪ docker.gpg \
  && echo "deb [signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian $(. /
  ↪ etc/os-release && echo $VERSION_CODENAME) stable" \
  > /etc/apt/sources.list.d/docker.list \
  && apt-get update && apt-get install -y --no-install-recommends docker-ce-cli \
  && rm -rf /var/lib/apt/lists/*

# Jenkins itself keeps running on the bundled JDK 21; only *builds* see JDK 25.
# JAVA_HOME is therefore NOT changed here -- the Jenkinsfile sets it per stage.
ENV MAVEN_HOME=/opt/maven \
  JDK25_HOME=/usr/lib/jvm/temurin-25-jdk-amd64 \
  PATH=/opt/maven/bin:${PATH}
```

USER jenkins

Plugins pre-installed so the first boot needs no wizard clicks beyond unlock.

RUN jenkins-plugin-cli --plugins \
git workflow-aggregator pipeline-stage-view junit timestamp credentials-binding

deploy/k8s/jenkins-rbac.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins
  namespace: ts
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: jenkins-deployer
  namespace: ts
rules:
  # How to read a rule: "may perform <verbs> on <resources> in <apiGroups>".
  # Rules are additive; anything not listed is Forbidden. Namespace-scoped
  # (this is a Role, not a ClusterRole), so nothing here reaches kube-system.
  #
  # Verb cheat-sheet for the pipeline's commands:
  #   kubectl apply          -> get + create (new) or patch (existing)
  #   kubectl set image      -> get + patch
  #   kubectl rollout status -> get/list/watch on deployments AND replicaset
  #   kubectl logs / get pods -> get/list on pods, get on pods/log

  # -- What `kubectl apply -k deploy/k8s/base` creates, by API group -----
  # Core group is spelled "" (empty string). Namespace is deliberately absent:
  # it is cluster-scoped and cannot be granted by a Role, so the pipeline must
  # not try to apply namespace.yaml (ts already exists).
  - apiGroups: [""]
    resources: ["services", "persistentvolumeclaims", "secrets", "configmaps"]
    verbs: ["get", "list", "create", "patch", "update"]

  - apiGroups: ["apps"]
    resources: ["deployments", "statefulsets"]
    verbs: ["get", "list", "watch", "create", "patch", "update"]

  - apiGroups: ["networking.k8s.io"]
    resources: ["ingresses"]
    verbs: ["get", "list", "create", "patch", "update"]

  # Strimzi custom resources (milestone 6). Namespaced like everything
  # else here -- the pipeline may declare Kafka clusters and topics in ts,
  # but installing the operator/CRDs stays an admin act (install-strimzi.sh).
  - apiGroups: ["kafka.strimzi.io"]
    resources: ["kafkas", "kafkanodepools", "kafkatopics"]
```

```

    verbs: ["get", "list", "create", "patch", "update"]

# -- Read-only extras for `rollout status` and post-failure debugging -----
# A Deployment rolls out via ReplicaSets it owns; rollout status watches them.
- apiGroups: ["apps"]
  resources: ["replicasets"]
  verbs: ["get", "list", "watch"]

- apiGroups: [""]
  resources: ["pods", "pods/log", "events"]
  verbs: ["get", "list", "watch"]

# NOT granted, on purpose:
#   delete          -- CI should roll forward, never wipe; a human deletes.
#   nodes, namespaces -- cluster-scoped, out of a Role's reach anyway.
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins-deployer
  namespace: ts
subjects:
- kind: ServiceAccount
  name: jenkins
  namespace: ts
roleRef:
  kind: Role
  name: jenkins-deployer
  apiGroup: rbac.authorization.k8s.io

```

deploy/k8s/create-secrets.sh

```

#!/usr/bin/env bash
#
# Create the Secrets the ts namespace needs. Deliberately NOT a manifest:
# secrets do not belong in git. Re-runnable -- it replaces what is there.
#
#   ./deploy/k8s/create-secrets.sh
#
# Values match docker-compose.yml so behaviour is identical to local dev.
# Override any of them from the environment before running.

set -euo pipefail

NS="${NS:-ts}"
PG_USER="${PG_USER:-postgres}"
PG_PASSWORD="${PG_PASSWORD:-root}"
MINIO_USER="${MINIO_USER:-minioadmin}"
MINIO_PASSWORD="${MINIO_PASSWORD:-minioadmin}"
# Default matches the fallback baked into config-server's api-gateway.yml /
# auth-service.yml so dev behaviour is unchanged. Override in anything public.
JWT_SECRET="${JWT_SECRET:-mySecretKeyThatIsAtLeast256BitsLongForHs256Algorithm123456}"

```

```
kubectrl get namespace "$NS" >/dev/null 2>&1 || kubectrl create namespace "$NS"

# Four secrets, not two: the same credential is spelled differently by the
# server that owns it and the client that consumes it. Postgres wants
# POSTGRES_USER; Spring wants SPRING_DATASOURCE_USERNAME. Keeping them as
# separate secrets means each pod's envFrom pulls in only names it understands,
# rather than a grab-bag it has to ignore half of.
#
# --dry-run=client | apply is the idempotent-create idiom: `kubectrl create secret`
# alone fails if the secret exists, and there is no `kubectrl create --force`.

# --- postgres, server side ---
kubectrl -n "$NS" create secret generic postgres-course \
  --from-literal=POSTGRES_USER="$PG_USER" \
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_course \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- postgres, client side (course-service) ---
kubectrl -n "$NS" create secret generic postgres-course-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- minio, server side ---
kubectrl -n "$NS" create secret generic minio \
  --from-literal=MINIO_ROOT_USER="$MINIO_USER" \
  --from-literal=MINIO_ROOT_PASSWORD="$MINIO_PASSWORD" \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- minio, client side (course-service) ---
kubectrl -n "$NS" create secret generic minio-app \
  --from-literal=APP_S3_ACCESS_KEY="$MINIO_USER" \
  --from-literal=APP_S3_SECRET_KEY="$MINIO_PASSWORD" \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- postgres-auth, server side ---
kubectrl -n "$NS" create secret generic postgres-auth \
  --from-literal=POSTGRES_USER="$PG_USER" \
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_auth \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- postgres-auth, client side (auth-service) ---
kubectrl -n "$NS" create secret generic postgres-auth-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- google oauth (auth-service) ---
# Real values come from .env / the environment; the placeholders keep the pod
# bootable when they are absent -- Google login then fails at click time, not
# the whole service at startup. Mirrors the ${GOOGLE_CLIENT_ID:placeholder}
# defaults in config-server's auth-service.yml.
kubectrl -n "$NS" create secret generic google-oauth \
```



```

--from-literal=GOOGLE_CLIENT_ID="${GOOGLE_CLIENT_ID:-placeholder}" \
--from-literal=GOOGLE_CLIENT_SECRET="${GOOGLE_CLIENT_SECRET:-placeholder}" \
--dry-run=client -o yaml | kubectl apply -f -

# --- jwt, shared by api-gateway (validates) and auth-service (signs) ---
kubectl -n "$NS" create secret generic jwt \
  --from-literal=JWT_SECRET="${JWT_SECRET}" \
  --dry-run=client -o yaml | kubectl apply -f -

echo
kubectl -n "$NS" get secrets

```

deploy/k8s/install-strimzi.sh

```

#!/usr/bin/env bash
#
# One-time ADMIN step: install the Strimzi operator + CRDs into the ts
# namespace, watching that namespace only. Run before the first
# `kubectl apply -k deploy/k8s/base` that includes kafka.yaml.
#
# Same category as namespace creation: CRDs and the operator's RBAC are
# cluster-scoped, so the Jenkins ServiceAccount cannot (and should not)
# install them. Jenkins only applies the namespaced Kafka/KafkaTopic
# resources afterwards.
#
# ./deploy/k8s/install-strimzi.sh

set -euo pipefail

NS="${NS:-ts}"

kubectl get namespace "$NS" >/dev/null 2>&1 || kubectl create namespace "$NS"

# Server-side apply, because the Strimzi bundle's CRDs are too large for
# the client-side last-applied-configuration annotation. Idempotent: safe
# to re-run, also how you take operator upgrades later.
kubectl apply --server-side --force-conflicts \
  -f "https://strimzi.io/install/latest?namespace=${NS}" -n "$NS"

kubectl -n "$NS" rollout status deployment/strimzi-cluster-operator --timeout=300s

echo
echo "Operator ready. Kafka clusters in '${NS}' will now be reconciled."

```

deploy/k8s/base/kustomization.yaml

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

```

```
namespace: ts

# Applied in dependency order for readability only -- Kubernetes starts pods in
# parallel regardless. Ordering here is documentation, not orchestration.
resources:
  # namespace.yaml is intentionally NOT listed. Namespaces are cluster-scoped,
  # and the Jenkins ServiceAccount only holds a namespaced Role, so `apply`
  # would be Forbidden. The namespace is created once by create-secrets.sh
  # (or `kubectl apply -f namespace.yaml` by an admin).
  - postgres-course.yaml
  - postgres-auth.yaml
  - mongodb.yaml
  - minio.yaml
  # kafka.yaml requires the Strimzi operator (one-time admin step:
  # ../install-strimzi.sh) -- without its CRDs, apply fails on kind Kafka.
  - kafka.yaml
  - kafka-topics.yaml
  - maildev.yaml
  - config-server.yaml
  - discovery-server.yaml
  - course-service.yaml
  - auth-service.yaml
  - dictionary-service.yaml
  - notification-service.yaml
  - api-gateway.yaml
  # Non-JVM deployable: nginx + React build, claims / on ts.local.
  - frontend.yaml

# Stamped onto every object, so `kubectl delete -l part-of=teacher-supporter`
# cleans up the whole slice without touching kube-system.
commonLabels:
  app.kubernetes.io/part-of: teacher-supporter

# Secrets are NOT here on purpose -- see ../create-secrets.sh.
```

deploy/k8s/base/namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: ts
```

deploy/k8s/base/postgres-course.yaml

```
# StatefulSet rather than Deployment: a database needs a stable identity and a
# volume that is never handed to a second replica. volumeClaimTemplates gives
# each pod its own PVC, provisioned by k3s' local-path on the node's SSD.
```

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres-course
spec:
  serviceName: postgres-course
  replicas: 1
  selector:
    matchLabels: { app: postgres-course }
  template:
    metadata:
      labels: { app: postgres-course }
    spec:
      containers:
        - name: postgres
          image: postgres:17
          ports:
            - { name: postgres, containerPort: 5432 }
          envFrom:
            - secretRef: { name: postgres-course }
          env:
            # The official image refuses to initialise into a non-empty directory,
            # and local-path volumes arrive with a lost+found. A subdirectory
            # sidesteps it -- the standard fix, not a workaround.
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres", "-d", "ts_course"]
              initialDelaySeconds: 10
              periodSeconds: 5
              failureThreshold: 6
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: [ReadWriteOnce]
            resources:
              requests:
                storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-course
spec:
  selector: { app: postgres-course }
  ports:
    - { name: postgres, port: 5432, targetPort: 5432 }

```

deploy/k8s/base/postgres-auth.yaml

```
# Second Postgres, same shape as postgres-course.yaml -- see the comments
# there for why StatefulSet, PGDATA subdirectory, and volumeClaimTemplates.
#
# Deliberately a SEPARATE server rather than a second database in the same
# one: database-per-service is the microservice contract. auth and course
# must not be able to join each other's tables, today or by accident later.
# On this box it costs ~256Mi -- cheap insurance.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres-auth
spec:
  serviceName: postgres-auth
  replicas: 1
  selector:
    matchLabels: { app: postgres-auth }
  template:
    metadata:
      labels: { app: postgres-auth }
    spec:
      containers:
        - name: postgres
          image: postgres:17
          ports:
            - { name: postgres, containerPort: 5432 }
          envFrom:
            - secretRef: { name: postgres-auth }
          env:
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres", "-d", "ts_auth"]
              initialDelaySeconds: 10
              periodSeconds: 5
              failureThreshold: 6
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: [ReadWriteOnce]
            resources:
              requests:
                storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-auth
```

```
spec:
  selector: { app: postgres-auth }
  ports:
    - { name: postgres, port: 5432, targetPort: 5432 }
```

deploy/k8s/base/mongodb.yaml

```
# Same reasoning as postgres-course.yaml: databases get a StatefulSet for the
# stable identity + a volumeClaimTemplate PVC that is never shared.
#
# No credentials, matching docker-compose.yml -- the official mongo image only
# enables auth when MONGO_INITDB_ROOT_USERNAME/PASSWORD are set. Acceptable
# here because the Service is ClusterIP: only pods inside the cluster can
# reach it, and there is no Ingress to it. If this ever went multi-tenant or
# network policies were the goal, auth would come first.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongodb
spec:
  serviceName: mongodb
  replicas: 1
  selector:
    matchLabels: { app: mongodb }
  template:
    metadata:
      labels: { app: mongodb }
    spec:
      containers:
        - name: mongodb
          image: mongo:8
          ports:
            - { name: mongo, containerPort: 27017 }
          volumeMounts:
            # Unlike postgres, mongod is happy with a lost+found in its data
            # dir, so no PGDATA-style subdirectory trick is needed.
            - { name: data, mountPath: /data/db }
      readinessProbe:
        # mongosh ping: same idea as pg_isready -- "is the server
        # accepting commands", not just "is the port open".
        exec:
          command: ["mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
          initialDelaySeconds: 10
          periodSeconds: 5
          failureThreshold: 6
      resources:
        requests: { memory: 256Mi, cpu: 100m }
        limits: { memory: 512Mi }
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
```

```
    accessModes: [ReadWriteOnce]
    resources:
      requests:
        storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mongodb
spec:
  selector: { app: mongodb }
  ports:
    - { name: mongo, port: 27017, targetPort: 27017 }
```

deploy/k8s/base/minio.yaml

```
# MinIO is a hard startup dependency for course-service: S3StorageService's
# @PostConstruct calls headBucket and only catches NoSuchBucketException, so an
# unreachable MinIO fails bean creation. Kubernetes has no depends_on -- course
# service will CrashLoopBackOff until this pod is serving, then recover.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: minio
spec:
  replicas: 1
  strategy: { type: Recreate } # RWO volume: never two pods holding it at once
  selector:
    matchLabels: { app: minio }
  template:
    metadata:
      labels: { app: minio }
    spec:
      containers:
        - name: minio
          image: minio/minio:latest
          args: ["server", "/data", "--console-address", ":9001"]
          ports:
            - { name: s3, containerPort: 9000 }
            - { name: console, containerPort: 9001 }
          envFrom:
            - secretRef: { name: minio }
          volumeMounts:
            - { name: data, mountPath: /data }
          readinessProbe:
            httpGet: { path: /minio/health/ready, port: s3 }
            initialDelaySeconds: 10
            periodSeconds: 5
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
      volumes:
```

```

    - name: data
      persistentVolumeClaim: { claimName: minio-data }
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: minio-data
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: minio
spec:
  selector: { app: minio }
  ports:
    - { name: s3, port: 9000, targetPort: 9000 }
    - { name: console, port: 9001, targetPort: 9001 }

```

deploy/k8s/base/kafka.yaml

```

# Kafka via the Strimzi operator (KRaft mode -- no ZooKeeper).
#
# PREREQUISITE (one-time, admin): the operator + CRDs must be installed
# first -- ../install-strimzi.sh. Without it, applying this file fails with
# "no matches for kind Kafka". Same class of admin step as namespace.yaml.
#
# We do not create StatefulSets or Services for Kafka ourselves. We declare
# a Kafka cluster as data; the operator (Chapter 14's reconciliation idea,
# extended to custom kinds) computes the pods, Services, listener config,
# and rolling restarts from it. What we hand-tuned in docker-compose.yml --
# the advertised-listeners maze -- the operator now derives.
#
# Client address inside the cluster: ts-kafka-kafka-bootstrap:9092
# (<cluster-name>-kafka-bootstrap -- Strimzi's naming convention)
---
# Node pool: WHERE Kafka runs and with what storage/resources.
# One node holding both KRaft roles -- the k8s equivalent of the compose
# broker's process.roles=broker,controller. RF=1 everywhere: a homelab
# accepts zero fault tolerance; production starts at 3 nodes / RF=3.
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
metadata:
  name: dual-role
  labels:
    # Joins this pool to the Kafka resource below. The operator matches on
    # this label -- it must equal the Kafka resource's metadata.name.
    strimzi.io/cluster: ts-kafka

```

```
spec:
  replicas: 1
  roles: [controller, broker]
  storage:
    type: persistent-claim      # local-path PVC, like the databases
    size: 5Gi
    deleteClaim: false        # deleting the Kafka CR keeps the log data
  resources:
    requests: { memory: 768Mi, cpu: 200m }
    limits: { memory: 1Gi }
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: ts-kafka
  annotations:
    strimzi.io/kraft: enabled
    strimzi.io/node-pools: enabled
spec:
  kafka:
    # No version pinned: the operator's default (its newest supported
    # Kafka) applies, and operator upgrades roll the broker for us.
    listeners:
      # One internal plaintext listener. No TLS/auth for the same reason
      # Mongo has none: ClusterIP-only, nothing routes here from outside.
      - name: plain
        port: 9092
        type: internal
        tls: false
    config:
      # Single broker: every replication setting must be 1, or topic
      # creation fails asking for brokers that do not exist.
      offsets.topic.replication.factor: 1
      transaction.state.log.replication.factor: 1
      transaction.state.log.min.isr: 1
      default.replication.factor: 1
      min.insync.replicas: 1
      log.retention.hours: 168      # matches docker-compose.yml
    jvmOptions:
      # Kafka's default heap assumes a server; half a GiB serves a homelab
      # event trickle comfortably and leaves RAM for the Spring services.
      -Xms: 256m
      -Xmx: 512m
  entityOperator:
    # Reconciles KafkaTopic resources (kafka-topics.yaml) into real topics.
    # userOperator is omitted: no authentication, so no users to manage.
    topicOperator: {}
```

deploy/k8s/base/kafka-topics.yaml

```
# Topics as declared objects, reconciled by Strimzi's topic operator.
#
```

```

# Kafka would auto-create these on first use (auto.create.topics.enable
# defaults to true), but declared topics beat auto-created ones: partition
# count and retention are decisions, not defaults; they exist before the
# first producer runs (no race on day one); and 'kubectl get kafkatopics'
# becomes the inventory of every event contract in the system.
#
# partitions: 1 -- total order per topic, which trivially preserves the
# per-user ordering the publishers key for. Raise partitions later only
# with a consumer that scales past one instance; note partitions can be
# added but never removed.
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.registered
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.activated
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.admin-provisioned
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.assignment.created
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1

```

deploy/k8s/base/maildev.yaml

```
# Maildev: an SMTP server that delivers nothing -- it catches every mail
# and shows it in a web UI. The dev/homelab stand-in for a real relay;
# swapping to real mail later is only MAIL_HOST/credentials env changes
# on notification-service.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: maildev
spec:
  replicas: 1
  selector:
    matchLabels: { app: maildev }
  template:
    metadata:
      labels: { app: maildev }
    spec:
      containers:
        - name: maildev
          image: maildev/maildev:2.2.1
          ports:
            - { name: web, containerPort: 1080 }
            - { name: smtp, containerPort: 1025 }
          readinessProbe:
            tcpSocket: { port: smtp }
            periodSeconds: 5
          resources:
            requests: { memory: 64Mi, cpu: 25m }
            limits: { memory: 128Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: maildev
spec:
  selector: { app: maildev }
  ports:
    - { name: web, port: 1080, targetPort: 1080 }
    - { name: smtp, port: 1025, targetPort: 1025 }
# No Ingress on purpose: the UI is an operator tool, not part of the app.
# To read caught mail: kubectl -n ts port-forward svc/maildev 1080:1080
# then open http://localhost:1080 on Windows.
```

deploy/k8s/base/config-server.yaml

```
# Config Server holds no state: its YAML is baked into the jar under
# classpath:/configurations (spring.profiles.active=native). No PVC, no DB.
# Every other pod blocks on this one, so it starts first in practice by virtue
# of being the only thing with no dependencies of its own.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: config-server
```

```

spec:
  replicas: 1
  selector:
    matchLabels: { app: config-server }
  template:
    metadata:
      labels: { app: config-server }
    spec:
      containers:
        - name: config-server
          image: localhost:5000/config-server:1.0.0-SNAPSHOT
          # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
          # would keep serving whatever the node cached. Always re-check the
          # registry (cheap: only the manifest is fetched if digests match).
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8888 }
          env:
            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          # startupProbe buys a slow JVM time to boot without forcing a long
          # liveness initialDelay. Until it passes, liveness is not evaluated.
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 30      # up to 150s to start
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 640Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: config-server
spec:
  selector: { app: config-server }
  ports:
    - { name: http, port: 8888, targetPort: 8888 }

```

deploy/k8s/base/discovery-server.yaml

```

# Eureka. Kept deliberately -- this project is the classic Spring Cloud
# showcase, and running it on Kubernetes alongside Service DNS is the point.
#
# Note the two service-discovery systems now coexist: Kubernetes resolves
# `discovery-server` to a ClusterIP, and Eureka resolves `course-service` to a
# pod IP that the clients registered themselves. Only the first survives a pod
# reschedule instantly; Eureka takes up to 90s to evict a dead instance.
apiVersion: apps/v1
kind: Deployment

```

```
metadata:
  name: discovery-server
spec:
  replicas: 1
  selector:
    matchLabels: { app: discovery-server }
  template:
    metadata:
      labels: { app: discovery-server }
    spec:
      containers:
        - name: discovery-server
          image: localhost:5000/discovery-server:1.0.0-SNAPSHOT
          # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
          # would keep serving whatever the node cached. Always re-check the
          # registry (cheap: only the manifest is fetched if digests match).
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8761 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 30
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 640Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: discovery-server
spec:
  selector: { app: discovery-server }
  ports:
    - { name: http, port: 8761, targetPort: 8761 }
```

deploy/k8s/base/course-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: course-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: course-service }
```

```

template:
  metadata:
    labels: { app: course-service }
  spec:
    containers:
      - name: course-service
        image: localhost:5000/course-service:1.0.0-SNAPSHOT
        # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
        # would keep serving whatever the node cached. Always re-check the
        # registry (cheap: only the manifest is fetched if digests match).
        imagePullPolicy: Always
        ports:
          - { name: http, containerPort: 8082 }
        env:
          # Same values as docker-compose.yml, with compose service names
          # swapped for Kubernetes Service names. They happen to be identical,
          # because both resolve a name to an address on a private network.
          - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
          - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-server:8761/"
↪ eureka/" }
          - { name: SPRING_DATASOURCE_URL, value: "jdbc:postgresql://postgres-course:5432/ts_course
↪ " }
          - { name: APP_S3_ENDPOINT, value: "http://minio:9000" }

          # Eureka on Kubernetes needs this or nothing can call anything.
          # By default a client registers its *hostname*, which inside a pod is
          # the pod name -- a string no other pod can resolve. Pod IPs are
          # routable cluster-wide, so register the IP instead.
          - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

          # Strimzi's bootstrap Service (milestone 6). This service both
          # produces (ts.assignment.created) and consumes (ts.user.activated).
          - { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap:9092" }

          # Zipkin is not in the cluster yet (observability is milestone 8).
          - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

          - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
        envFrom:
          - secretRef: { name: postgres-course-app }
          - secretRef: { name: minio-app }

        # ---- Health probes -----
        # No actuator in this service, so /actuator/health is a 404. The one
        # unauthenticated HTTP path that exists is springdoc's /v3/api-docs
        # (permitAll in SecurityConfig). It only answers 200 once the whole
        # Spring context is up -- which is AFTER Flyway has migrated and the
        # MinIO bucket check has run. That makes it a true "ready" signal,
        # unlike a tcpSocket check that passes as soon as Tomcat binds.

        # startupProbe: "give the JVM time to boot, then hand over."
        # While this probe is failing, readiness/liveness are NOT evaluated.
        # 12 x 10s = up to 120s of grace for a cold start on two cores.
        # If it still isn't up after that, kubelet restarts the container.
        startupProbe:
          httpGet: { path: /v3/api-docs, port: http }

```

```
    periodSeconds: 10
    failureThreshold: 12

    # readinessProbe: "should the Service send traffic here right now?"
    # Failing readiness pulls the pod out of the Service endpoints; it
    # does NOT restart anything. Cheap and frequent.
    readinessProbe:
      httpGet: { path: /v3/api-docs, port: http }
      periodSeconds: 5
      failureThreshold: 3

    # No livenessProbe on purpose. Liveness restarts the container, and
    # the only thing it would catch beyond readiness is a wedged JVM --
    # rare, and a probe firing during a long GC pause on a 2-core box
    # would kill a healthy pod. Add one later if we ever see a real hang.
    # -----

    resources:
      requests: { memory: 512Mi, cpu: 200m }
      limits: { memory: 768Mi }
      # No CPU limit on purpose. A limit throttles the JVM exactly during
      # class loading, when it wants every core it can get -- turning a
      # 60s start into a 200s one and failing the startupProbe.
  ---
apiVersion: v1
kind: Service
metadata:
  name: course-service
spec:
  selector: { app: course-service }
  ports:
    - { name: http, port: 8082, targetPort: 8082 }
  # NOTE: the direct Ingress that used to live here (ts.local/courses ->
  # course-service) is gone. It was milestone-3 scaffolding to prove the chain
  # end-to-end, but it bypassed the api-gateway's JWT filter. All external
  # traffic now enters through the gateway's Ingress (api-gateway.yaml).
  # In-cluster callers still reach this Service directly -- that is what the
  # Service object above is for.
```

deploy/k8s/base/auth-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: auth-service }
  template:
    metadata:
      labels: { app: auth-service }
```

```

spec:
  containers:
    - name: auth-service
      image: localhost:5000/auth-service:1.0.0-SNAPSHOT
      imagePullPolicy: Always
      ports:
        - { name: http, containerPort: 8081 }
      env:
        - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
        - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-server:8761/" }
    ↪ eureka/" }
        - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }
        - { name: SPRING_DATASOURCE_URL, value: "jdbc:postgresql://postgres-auth:5432/ts_auth" }

    # Strimzi's bootstrap Service (milestone 6). Producer-only here:
    # registration/activation/admin-provisioned events.
    - { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap:9092" }

    # Google's OAuth callback must round-trip through the browser, so
    # it uses the PUBLIC name (the Tailscale MagicDNS host, via Traefik
    # -> gateway -> here), not a cluster-internal one. This exact URL
    # must also be registered under "Authorized redirect URIs" in the
    # Google Cloud console, or Google refuses with redirect_uri_mismatch.
    # (ts.local, the LAN-only name, was the value up to build #9.)
    - { name: SPRING_SECURITY_OAUTH2_CLIENT_REGISTRATION_GOOGLE_REDIRECT_URI,
      value: "http://ts-server.tailbfe002.ts.net/login/oauth2/code/google" }
    # After Google login, OAuth2LoginSuccessHandler redirects the
    # browser to the frontend with the tokens. config-server's default
    # says localhost:3000 (laptop Vite dev server); in the cluster the
    # frontend is the same origin as everything else (milestone 7).
    - { name: APP_OAUTH2_SUCCESS_REDIRECT_URI, value: "http://ts-server.tailbfe002.ts.net/" }
    ↪ oauth/callback" }

    # Zipkin arrives with observability (milestone 8).
    - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

    - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
  envFrom:
    - secretRef: { name: postgres-auth-app }
    # Signs JWTs with the same key the gateway validates with.
    - secretRef: { name: jwt }
    - secretRef: { name: google-oauth }

    # /actuator/health is NOT usable here: this service's SecurityConfig
    # permitAll list covers /register, /login, ... and /v3/api-docs, but
    # not /actuator/** -- so health answers 401 and the kubelet would
    # restart a perfectly healthy pod forever. Same fallback as
    # course-service: /v3/api-docs only turns 200 after the full context
    # (Flyway migration included) is up, which is an honest ready signal.
  startupProbe:
    httpGet: { path: /v3/api-docs, port: http }
    periodSeconds: 10
    failureThreshold: 12
  readinessProbe:
    httpGet: { path: /v3/api-docs, port: http }
    periodSeconds: 5

```

```
        failureThreshold: 3

    resources:
        requests: { memory: 512Mi, cpu: 200m }
        limits: { memory: 768Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: auth-service
spec:
  selector: { app: auth-service }
  ports:
    - { name: http, port: 8081, targetPort: 8081 }
```

deploy/k8s/base/dictionary-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dictionary-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: dictionary-service }
  template:
    metadata:
      labels: { app: dictionary-service }
    spec:
      containers:
        - name: dictionary-service
          image: localhost:5000/dictionary-service:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8083 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-server:8761/" }
            ↪ eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

            # Config-server's dictionary-service.yml says localhost:27017;
            # in the cluster mongo lives behind the `mongodb` Service.
            - { name: SPRING_DATA_MONGODB_URI, value: "mongodb://mongodb:27017/ts_dictionary" }

            # Zipkin arrives with observability (milestone 8).
            - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }

            # This service's SecurityConfig permits /actuator/** -- so unlike
            # course-service (which had to use /v3/api-docs), the probe can hit
```



```

# the purpose-built health endpoint. It aggregates a Mongo ping, so
# "ready" here really means "can serve dictionary lookups".
startupProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 10
  failureThreshold: 12
readinessProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 5
  failureThreshold: 3

resources:
  requests: { memory: 512Mi, cpu: 200m }
  limits: { memory: 768Mi }
  # No CPU limit -- same class-loading argument as course-service.
---
apiVersion: v1
kind: Service
metadata:
  name: dictionary-service
spec:
  selector: { app: dictionary-service }
  ports:
    - { name: http, port: 8083, targetPort: 8083 }

```

deploy/k8s/base/notification-service.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: notification-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: notification-service }
  template:
    metadata:
      labels: { app: notification-service }
    spec:
      containers:
        - name: notification-service
          image: localhost:5000/notification-service:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8084 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-server:8761/
↳ eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

          # Strimzi's bootstrap Service: <cluster>-kafka-bootstrap.

```

```
- { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap:9092" }

# The config's ${MAIL_HOST:localhost} default, repointed at the
# maildev Service. Overriding the spring.mail.* keys directly
# (relaxed binding) rather than the custom placeholder names --
# same result, and consistent with how compose does it.
- { name: SPRING_MAIL_HOST, value: "maildev" }
- { name: SPRING_MAIL_PORT, value: "1025" }

# Links inside emails (activate / invite / login) must work from
# whatever device opens the mail, so they carry the PUBLIC origin
# -- the Tailscale MagicDNS name every tailnet device resolves --
# never a Service name. Overrides app.frontend.base-url from
# config-server (relaxed binding), same trick as SPRING_MAIL_HOST.
- { name: APP_FRONTEND_BASE_URL, value: "http://ts-server.tailbfe002.ts.net" }

# Zipkin arrives with observability (milestone 8).
- { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

- { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }

# This service has no Spring Security at all -- no SecurityConfig,
# no security starter -- so /actuator/health is open and the probes
# can use the real endpoint (compare auth-service's workaround).
startupProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 10
  failureThreshold: 12
readinessProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 5
  failureThreshold: 3

resources:
  requests: { memory: 512Mi, cpu: 100m }
  limits: { memory: 768Mi }
---
# Even a pure consumer gets a Service: Eureka registration aside, the
# actuator endpoints (and later Prometheus scraping) need an address.
apiVersion: v1
kind: Service
metadata:
  name: notification-service
spec:
  selector: { app: notification-service }
  ports:
    - { name: http, port: 8084, targetPort: 8084 }
```

deploy/k8s/base/api-gateway.yaml

```
apiVersion: apps/v1
kind: Deployment
```

```

metadata:
  name: api-gateway
spec:
  replicas: 1
  selector:
    matchLabels: { app: api-gateway }
  template:
    metadata:
      labels: { app: api-gateway }
    spec:
      containers:
        - name: api-gateway
          image: localhost:5000/api-gateway:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8080 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-server:8761/" }
            ↪ eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

            # Zipkin is not in the cluster yet (observability is milestone 8).
            - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          envFrom:
            # JWT_SECRET -- shared with auth-service, which signs the tokens the
            # gateway validates. One secret, two consumers, or logins break in a
            # way that looks like a gateway bug.
            - secretRef: { name: jwt }

            # Unlike course-service, the gateway exposes actuator, so probes can
            # use the real health endpoint. Reactive stack (Netty) boots faster
            # than the MVC services: 6 x 10s of startup grace instead of 12.
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
            failureThreshold: 6
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 3

          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 512Mi }
      ---
    apiVersion: v1
    kind: Service
    metadata:
      name: api-gateway
    spec:
      selector: { app: api-gateway }
      ports:
        - { name: http, port: 8080, targetPort: 8080 }

```

```

---
# Edge routing decision: with the gateway in the cluster there are two routers.
# Traefik (this Ingress) is the front door on ts.local; Spring Cloud Gateway
# knows Eureka and validates JWTs. Decision made here:
#
#   /api           -> gateway   (all REST traffic, JWT-checked)
#   /oauth2        -> gateway   (Google login redirect start)
#   /login/oauth2  -> gateway   (Google login callback)
#   /              -> free      (reserved for the frontend, milestone 7)
#
# The old direct `courses` Ingress on course-service is deleted with this
# change: it reached the service without passing the gateway's JWT filter,
# i.e. an unauthenticated side door. From now on the ONLY way in is through
# the gateway -- which is the entire point of having one.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-gateway
spec:
  # Two hosts, identical paths. ts.local is the LAN name (hosts-file entry on
  # the laptop). The second is the machine's Tailscale MagicDNS name: every
  # device on the tailnet resolves it with no hosts-file edit, so the app is
  # reachable from anywhere Tailscale is -- the same Ingress, one more rule.
  # Traefik routes on the Host header, so a name it does not list gets 404.
  rules:
    - host: ts.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          - path: /oauth2
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          - path: /login/oauth2
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
    - host: ts-server.tailbfe002.ts.net
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          - path: /oauth2

```

```

    pathType: Prefix
    backend:
      service:
        name: api-gateway
        port: { number: 8080 }
  - path: /login/oauth2
    pathType: Prefix
    backend:
      service:
        name: api-gateway
        port: { number: 8080 }

```

deploy/k8s/base/frontend.yaml

```

# Frontend (milestone 7): nginx serving the built React app.
#
# Image comes from frontend/Dockerfile (docker build in Jenkins), not Jib --
# it is the one non-JVM deployable. Being static files it has no config
# server, no Eureka, no secrets: the app talks to "/api" on its own origin,
# and Traefik (Ingress below) splits that off to the gateway.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 1
  selector:
    matchLabels: { app: frontend }
  template:
    metadata:
      labels: { app: frontend }
    spec:
      containers:
        - name: frontend
          image: localhost:5000/frontend:latest
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 80 }

          # nginx is up in milliseconds; no startupProbe needed. /healthz is
          # a location in nginx.conf that answers without touching disk.
          readinessProbe:
            httpGet: { path: /healthz, port: http }
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /healthz, port: http }
            periodSeconds: 20

          # Static file serving: the cheapest pod in the namespace by far.
          resources:
            requests: { memory: 32Mi, cpu: 10m }
            limits: { memory: 64Mi }

```

```
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  selector: { app: frontend }
  ports:
    - { name: http, port: 80, targetPort: 80 }
---
# Claims "/" on ts.local -- the path api-gateway.yaml's Ingress left free.
# Traefik merges all Ingress rules for a host and picks the LONGEST matching
# prefix, so /api, /oauth2 and /login/oauth2 (declared on the gateway
# Ingress) still win over this catch-all. Net effect: one origin
# (http://ts.local) for pages and API alike -- no CORS preflights, and the
# Google-login redirect lands on the same host that started it.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend
spec:
  # Two hosts, identical paths. ts.local is the LAN name (hosts-file entry on
  # the laptop). The second is the machine's Tailscale MagicDNS name: every
  # device on the tailnet resolves it with no hosts-file edit, so the app is
  # reachable from anywhere Tailscale is -- the same Ingress, one more rule.
  # Traefik routes on the Host header, so a name it does not list gets 404.
  rules:
    - host: ts.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port: { number: 80 }
    - host: ts-server.tailbfe002.ts.net
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port: { number: 80 }
```

frontend/Dockerfile

```
# Frontend image: the ONLY Dockerfile the pipeline builds.
#
# The JVM services use Jib (no Dockerfile, no daemon). A React app is a pile
# of static files behind a web server, so it gets the classic two-stage
```

```
# build instead: stage 1 has Node and produces dist/, stage 2 is a plain
# nginx that ships only dist/ -- Node, node_modules (~300 MB) and the
# TypeScript compiler never reach the runtime image (~15 MB).
#
# docker build -t localhost:5000/frontend:dev frontend
# docker run --rm -p 3000:80 localhost:5000/frontend:dev -> http://localhost:3000

# ---- stage 1: build -----
FROM node:24-alpine AS build
WORKDIR /app

# Copy the lockfile first and install BEFORE copying sources: this layer is
# cached until package*.json changes, so a source edit does not re-download
# every dependency. `npm ci` (not `install`) refuses to touch the lockfile,
# i.e. the build gets exactly the versions committed in git.
COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund

COPY . .
# = `tsc -b && vite build` (package.json). tsc type-checks, Vite bundles and
# content-hashes the assets (index-<hash>.js) so they can be cached forever.
RUN npm run build

# ---- stage 2: serve -----
FROM nginx:1.27-alpine
# The stock image ships a default server block; replace it with ours.
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

frontend/nginx.conf

```
# nginx server block for the React frontend inside the cluster.
#
# Division of labour on ts.local (Traefik decides by longest path prefix):
# /api, /oauth2, /login/oauth2 -> api-gateway (never reach this nginx)
# everything else -> here (static files + SPA fallback)
# So this file does NOT proxy anything: in production the frontend's
# "/api" calls are same-origin and Traefik, not nginx, routes them. The
# Vite dev-server proxy in vite.config.ts plays that role only on a laptop.
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    # SPA fallback. React Router (BrowserRouter) owns URLs like /courses/42
    # purely in the browser -- no such file exists on disk. On a deep link or
    # a page refresh nginx must answer index.html and let the router take
    # over, otherwise the user gets a 404 for any URL except "/".
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

```
# Vite content-hashes everything under /assets (index-CqZING8N.js), so a
# changed file always has a new name: cache these forever. index.html is
# deliberately NOT covered -- it must be re-fetched so it can point at
# the new hashes after a deploy.
location /assets/ {
    add_header Cache-Control "public, max-age=31536000, immutable";
}

# Kubernetes probes hit this; keeps "is nginx up" separate from "/",
# which would also succeed but pollutes access logs less this way.
location = /healthz {
    access_log off;
    add_header Content-Type text/plain;
    return 200 "ok\n";
}

gzip on;
gzip_types text/css application/javascript application/json image/svg+xml;
}
```

docker-compose.yml

```
services:
  # ===== INFRASTRUCTURE =====
  postgres-auth:
    image: postgres:17
    environment:
      POSTGRES_DB: ts_auth
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: root
    ports:
      - "5433:5432"
    volumes:
      - postgres-auth-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 5

  postgres-course:
    image: postgres:17
    environment:
      POSTGRES_DB: ts_course
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: root
    ports:
      - "5434:5432"
    volumes:
      - postgres-course-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
```



```

    interval: 10s
    retries: 5

mongodb:
  image: mongo:8
  ports:
    - "27017:27017"
  volumes:
    - mongo-data:/data/db

kafka:
  image: apache/kafka:3.9.0
  hostname: kafka
  ports:
    - "9092:9092"
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    KAFKA_LISTENERS: PLAINTEXT://:19092,CONTROLLER://:9093,EXTERNAL://:9092
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:19092,EXTERNAL://localhost:9092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:
    ↪ PLAINTEXT
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_LOG_RETENTION_HOURS: 168
    CLUSTER_ID: MkU30EVbNTcwNTJENDM2Qk

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  ports:
    - "9090:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: local
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:19092
  depends_on:
    - kafka

zipkin:
  image: openzipkin/zipkin:latest
  ports:
    - "9411:9411"

maildev:
  image: maildev/maildev:2.2.1
  ports:
    - "1080:1080"
    - "1025:1025"

minio:
  image: minio/minio:latest
  command: server /data --console-address ":9001"
  ports:
    - "9000:9000"    # S3 API
    - "9001:9001"    # Web console

```

```

environment:
  MINIO_ROOT_USER: minioadmin
  MINIO_ROOT_PASSWORD: minioadmin
volumes:
  - minio-data:/data
healthcheck:
  test: ["CMD", "mc", "ready", "local"]
  interval: 10s
  retries: 5

# ===== SPRING CLOUD =====
config-server:
  build:
    context: ./config-server
    dockerfile: Dockerfile
  ports:
    - "8888:8888"
    - "5888:5888"
  environment:
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5888"
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"]
    interval: 15s
    timeout: 5s
    retries: 10
    start_period: 30s

discovery-server:
  build:
    context: ./discovery-server
    dockerfile: Dockerfile
  ports:
    - "8761:8761"
    - "5761:5761"
  depends_on:
    config-server:
      condition: service_healthy
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5761"

api-gateway:
  build:
    context: ./api-gateway
    dockerfile: Dockerfile
  ports:
    - "8080:8080"
    - "5080:5080"
  depends_on:
    - discovery-server
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5080"

```

```
# ===== APPLICATION SERVICES =====
auth-service:
  build:
    context: ./auth-service
    dockerfile: Dockerfile
  ports:
    - "8081:8081"
    - "5081:5081"
  depends_on:
    config-server:
      condition: service_healthy
    postgres-auth:
      condition: service_healthy
    kafka:
      condition: service_started
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EURKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-auth:5432/ts_auth
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    GOOGLE_CLIENT_ID: ${GOOGLE_CLIENT_ID}
    GOOGLE_CLIENT_SECRET: ${GOOGLE_CLIENT_SECRET}
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5081"

course-service:
  build:
    context: ./course-service
    dockerfile: Dockerfile
  ports:
    - "8082:8082"
    - "5082:5082"
  depends_on:
    config-server:
      condition: service_healthy
    postgres-course:
      condition: service_healthy
    kafka:
      condition: service_started
    minio:
      condition: service_started
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EURKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-course:5432/ts_course
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    APP_S3_ENDPOINT: http://minio:9000
    APP_S3_PUBLIC_ENDPOINT: http://localhost:9000
    APP_S3_ACCESS_KEY: minioadmin
    APP_S3_SECRET_KEY: minioadmin
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5082"

dictionary-service:
  build:
    context: ./dictionary-service
```

```
dockerfile: Dockerfile
ports:
  - "8083:8083"
  - "5083:5083"
depends_on:
  config-server:
    condition: service_healthy
  mongodb:
    condition: service_started
environment:
  SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
  EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
  SPRING_DATA_MONGODB_URI: mongodb://mongodb:27017/ts_dictionary
  MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
  JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5083"

notification-service:
  build:
    context: ./notification-service
    dockerfile: Dockerfile
  ports:
    - "8084:8084"
    - "5084:5084"
  depends_on:
    config-server:
      condition: service_healthy
    kafka:
      condition: service_started
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    SPRING_MAIL_HOST: maildev
    SPRING_MAIL_PORT: 1025
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5084"

volumes:
  postgres-auth-data:
  postgres-course-data:
  mongo-data:
  minio-data:
```

config-server: configurations/api-gateway.yml

```
server:
  port: 8080
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
```

```

    uri: lb://auth-service
    predicates:
      - Path=/api/auth/**
    filters:
      - StripPrefix=2
  - id: auth-oauth2
    uri: lb://auth-service
    predicates:
      - Path=/oauth2/**
  - id: auth-oauth2-callback
    uri: lb://auth-service
    predicates:
      - Path=/login/oauth2/**
  - id: course-service-courses
    uri: lb://course-service
    predicates:
      - Path=/api/courses/**
    filters:
      - StripPrefix=2
  - id: course-service-students
    uri: lb://course-service
    predicates:
      - Path=/api/students/**
    filters:
      - StripPrefix=2
  - id: course-service-assignments
    uri: lb://course-service
    predicates:
      - Path=/api/assignments/**
    filters:
      - StripPrefix=2
  - id: course-service-enrollments
    uri: lb://course-service
    predicates:
      - Path=/api/enrollments/**
    filters:
      - StripPrefix=2
  - id: course-service-submissions
    uri: lb://course-service
    predicates:
      - Path=/api/submissions/**
    filters:
      - StripPrefix=2
  - id: dictionary-service
    uri: lb://dictionary-service
    predicates:
      - Path=/api/dictionary/**
    filters:
      - StripPrefix=2
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456}
management:
  tracing:
    sampling:
      probability: 1.0

```

```
zipkin:
  tracing:
    endpoint: http://localhost:9411/api/v2/spans
endpoints:
  web:
    exposure:
      include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```

config-server: configurations/auth-service.yml

```
server:
  port: 8081
  error:
    include-message: always
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_auth
    username: postgres
    password: root
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: none
      show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
  flyway:
    enabled: true
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID:placeholder}
            client-secret: ${GOOGLE_CLIENT_SECRET:placeholder}
            scope: openid, profile, email
            redirect-uri: http://localhost:8080/login/oauth2/code/google
  app:
    jwt:
      secret: ${JWT_SECRET:mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456}
      access-token-expiry-ms: 900000
      refresh-token-expiry-ms: 604800000
```

```

totp:
  issuer: TeacherSupporter
activation:
  base-url: http://localhost:3000/activate
oauth2:
  success-redirect-uri: http://localhost:3000/oauth/callback
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"

```

config-server: configurations/course-service.yml

```

server:
  port: 8082
  error:
    include-message: always
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_course
    username: postgres
    password: root
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: none
      show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
  flyway:
    enabled: true
  servlet:
    multipart:
      max-file-size: 20MB
      max-request-size: 25MB
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer

```

```

consumer:
  group-id: course-group
  auto-offset-reset: earliest
  key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
  value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
  properties:
    spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.
    ↪ StringDeserializer
    spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.
    ↪ JsonDeserializer
  spring.json.trusted.packages: com.ts.common.dto
app:
  s3:
    endpoint: ${APP_S3_ENDPOINT:http://localhost:9000}
    # Browser-reachable endpoint used to sign download URLs. When the service runs
    # inside Docker the internal endpoint is minio:9000, which a browser cannot resolve.
    public-endpoint: ${APP_S3_PUBLIC_ENDPOINT:http://localhost:9000}
    region: ${APP_S3_REGION:us-east-1}
    access-key: ${APP_S3_ACCESS_KEY:minioadmin}
    secret-key: ${APP_S3_SECRET_KEY:minioadmin}
    bucket: ${APP_S3_BUCKET:ts-submissions}
    presigned-url-expiry-minutes: ${APP_S3_PREIGNED_EXPIRY_MIN:15}
  resilience4j:
    circuitbreaker:
      instances:
        authService:
          sliding-window-size: 10
          failure-rate-threshold: 50
          wait-duration-in-open-state: 10s
          permitted-number-of-calls-in-half-open-state: 3
  management:
    tracing:
      sampling:
        probability: 1.0
    zipkin:
      tracing:
        endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  logging:
    pattern:
      level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"

```

config-server: configurations/dictionary-service.yml

```

server:
  port: 8083
  error:
    include-message: always
spring:

```



```

data:
  mongodb:
    uri: mongodb://localhost:27017/ts_dictionary
management:
  tracing:
    sampling:
      probability: 1.0
zipkin:
  tracing:
    endpoint: http://localhost:9411/api/v2/spans
endpoints:
  web:
    exposure:
      include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"

```

config-server: configurations/notification-service.yml

```

server:
  port: 8084
  error:
    include-message: always
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: notification-group
      auto-offset-reset: earliest
      key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.
        ↪ StringDeserializer
        spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.
        ↪ JsonDeserializer
        spring.json.trusted.packages: com.ts.common.dto
  mail:
    host: ${MAIL_HOST:localhost}
    port: ${MAIL_PORT:1025}
    username: ${MAIL_USERNAME:}
    password: ${MAIL_PASSWORD:}
    properties:
      mail.smtp.auth: false
      mail.smtp.starttls.enable: false
# Where the links inside outgoing emails point. This is the URL a BROWSER
# uses to reach the frontend, so it differs per environment: the Vite dev
# server here and under docker-compose, the public Ingress host in the
# cluster (overridden there with the APP_FRONTEND_BASE_URL env var).
app:
  frontend:

```

```
    base-url: http://localhost:3000
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```